

PostgreSQL CRUD 입문

PostgreSQL 16으로 배우는 데이터 다루기 (Windows 11 기준)

PostgreSQL 16

목차

1장. PostgreSQL 시작하기: 소개·설치·접속 (Windows 11) [입문]

- 관계형 데이터베이스(relational database)와 PostgreSQL 소개
- Windows 11에 PostgreSQL 16 설치하기  native
docker
- psql 접속과 메타 명령(meta-command) 첫걸음
- 첫 데이터베이스 연결과 실습 환경 점검

2장. 데이터베이스·스키마·테이블 기본과 자료형 [입문]

- 데이터베이스·스키마·테이블의 계층 구조
- 자주 쓰는 자료형(data type) 익히기
- CREATE TABLE로 테이블 설계하기
- NULL과 기본값(DEFAULT) 이해하기
- 테이블 변경·삭제(ALTER·DROP)

3장. INSERT: 데이터 생성(Create) [입문]

- INSERT로 한 행 삽입하기와 CRUD 생명주기
- 여러 행 한 번에 삽입하기(multi-row insert)
- RETURNING으로 삽입 결과 돌려받기
- 다른 테이블 데이터로 삽입(INSERT ... SELECT)

4장. SELECT 기초: 데이터 조회(Read) [입문]

- SELECT 기본 구조와 컬럼 선택
- WHERE로 조건 필터링하기
- ORDER BY로 정렬하기

- LIMIT·OFFSET으로 행 수 제한하기

5장. SELECT 심화: 집계·그룹·서브쿼리 [입문]

- 집계 함수(aggregate function)로 요약하기
- GROUP BY로 묶어서 집계하기
- HAVING으로 그룹 결과 필터링하기
- 서브쿼리(subquery)로 조회 조립하기

6장. JOIN: 테이블을 결합해 읽기 확장하기 [입문]

- JOIN의 개념과 필요성
- INNER JOIN으로 일치하는 행 결합하기
- LEFT·RIGHT OUTER JOIN으로 빠짐없이 읽기
- 여러 테이블 JOIN과 테이블 별칭(alias)
- 자기 조인·교차 조인 살펴보기

7장. UPDATE: 데이터 수정(Update) [기초]

- UPDATE 기본과 조건부 갱신
- 여러 컬럼 동시에 갱신하기
- 다른 테이블을 참조해 갱신하기(UPDATE ... FROM)
- UPSERT: 있으면 수정, 없으면 삽입(INSERT ON CONFLICT)

8장. DELETE: 데이터 삭제(Delete) [기초]

- DELETE 기본과 안전한 삭제
- TRUNCATE와 DELETE의 차이
- 외래 키와 CASCADE 연쇄 삭제
- 삭제 전 안전 장치 - 트랜잭션과 백업 습관

9장. 제약조건과 인덱스: 무결성과 조회 성능 [기초]

- 제약조건(constraint)으로 잘못된 데이터 막기
- 기본 키·외래 키와 참조 무결성
- UNIQUE·CHECK·NOT NULL 활용
- 인덱스(index)와 조회 성능
- 인덱스 설계 시 주의점

10장. 트랜잭션: CRUD의 원자성과 ACID [기초]

- 트랜잭션(transaction)과 ACID

- BEGIN·COMMIT·ROLLBACK 사용하기
- 동시성과 격리 수준(isolation level)
- SAVEPOINT로 부분 롤백하기

11장. 뷰·함수로 CRUD 재사용하기 [기초]

- 뷰(view)로 복잡한 조회 재사용하기
- 뷰 갱신과 제약 이해하기
- 함수(function)로 로직 묶기
- 트리거(trigger)로 CRUD 자동화 맛보기

12장. 실무 CRUD 패턴과 마무리 [기초]

- 페이지네이션(pagination) 패턴
- 감사 컬럼과 소프트 삭제(soft delete)
- 안전한 운영 - 백업·권한·실수 방지
- 마무리 - CRUD 전체 흐름 복습과 다음 단계

13장. 부록 - 대용량 테스트 데이터 만들기 [실무]

- 왜 대용량 테스트 데이터가 필요한가
 - generate_series로 대량 생성하기
 - 규모 조정과 응용 - 날짜·분포·복합 인덱스
 - 실행·검증과 트러블슈팅
-

PostgreSQL 시작하기: 소개·설치·접속

(Windows 11)

학습 목표 - 관계형 데이터베이스에서 **테이블·행·열**이 무엇인지 설명할 수 있습니다. - Windows 11에 PostgreSQL 16을 설치하고 **서비스·포트를 확인**할 수 있습니다. - **psql**로 접속하고 `\l · \dt` 같은 기본 메타 명령을 사용할 수 있습니다. - **실습 전용 데이터베이스**를 만들고 접속 정보를 점검할 수 있습니다. - DBeaver 같은 화면(GUI) 도구로 PostgreSQL에 연결해 쿼리 결과를 표로 확인할 수 있습니다.

이 장은 SQL 문법을 본격적으로 배우기 전에, **"데이터베이스가 무엇이고, 내 PC에서 어떻게 켜고 들어가는가"**를 손에 익히는 장입니다. 처음 듣는 용어가 많이 나오므로 일상 비유를 충분히 들면서 천천히 진행합니다.

먼저 자주 나오는 낱말 몇 개를 짧게 풀어 두겠습니다. **데이터베이스(database)**는 데이터를 잘 정리해 담아 두고 빠르게 꺼내 쓰도록 만든 **창고**입니다. **서버(server)**는 그 창고를 24시간 지키며 요청을 처리하는 **창고지기 프로그램**입니다. **클라이언트(client)**는 창고지기에게 "이 데이터 좀 줘", "이거 넣어 줘" 라고 말을 거는 **손님 쪽 프로그램**입니다. 이 장에서 우리는 PostgreSQL이라는 창고지기를 내 PC에 들이고, `psql`이라는 손님 도구로 말을 거는 연습을 합니다.

또 하나, 이 책의 명령 화면에 자주 보이는 `PS C:\Users\사용자>`는 **PowerShell**(윈도우의 검은 명령 입력 창)에서 명령을 기다리는 중이라는 표시입니다. `>` 뒤에 우리가 명령을 입력한다고 생각하면 됩니다.

관계형 데이터베이스(relational database)와 PostgreSQL 소개

도입

엑셀에서 표 하나에 회원 명단을 정리해 본 적이 있을 것입니다. 한 줄에 한 사람, 칸마다 이름·이메일·가입일을 적는 그 표가 사실 데이터베이스의 출발점입니다. **관계형 데이터베이스(relational database)**는 이렇게 잘 정리된 표 여러 장을 서로 연결해 두고, 필요한 정보를 골라

Table ---- Table
연결

모아 보는 방식입니다. 이 절에서는 그 표의 구조와, 왜 백엔드 개발에서 PostgreSQL을 자주 고르는지를 살펴봅니다.

핵심 개념 설명

관계형 데이터베이스 (relational database)

관계형 데이터베이스(relational database)란, **데이터를 행과 열로 이루어진 테이블(table)에 저장하고, 테이블 사이의 관계(relation)로 데이터를 연결해 관리**하는 데이터베이스를 말합니다. 쉽게 말하면, 잘 정리된 엑셀 표 여러 장을 서로 연결해 두고 필요한 정보를 골라 모아 보는 것과 같습니다.

왜 표를 **여러 장으로 나누어** 둘까요. 한 장에 모든 것을 우겨 넣으면 같은 정보가 자주 중복됩니다. 예를 들어 한 회원이 주문을 100번 하면, 회원 이름과 주소를 100줄에 똑같이 반복해 적어야 합니다. 이름이 바뀌면 100줄을 모두 고쳐야 하고, 한 줄만 빠뜨려도 데이터가 어긋납니다. 그래서 **회원은 회원 표에, 주문은 주문 표에** 따로 적어 두고, 둘을 "이 주문은 몇 번 회원의 것"이라는 연결로 묶습니다. 이 "연결"이 바로 관계입니다.

- **왜(why) 나누는가**: 같은 데이터를 한 곳에만 적어 두면(중복 제거) 수정이 한 번으로 끝나고 데이터가 어긋나지 않습니다.
- **언제(when) 빛나는가**: 회원-주문-상품처럼 서로 얽힌 데이터가 많아질수록, 나눠 두고 필요할 때 합쳐 보는 방식이 강력합니다.
- **어떻게(how) 합치는가**: 나중에 배울 JOIN (6장)으로 흩어진 표를 키로 맞추려 한 화면처럼 읽습니다.

흔한 오해 두 가지 - "데이터베이스는 그냥 큰 엑셀 파일 하나 아닌가요?" → 아닙니다. 엑셀은 한두 사람이 보기 좋게 한 장에 모으는 데 강하고, 데이터베이스는 **여러 표를 나눠 저장하고 동시에 수많은 요청을 안전하게 처리**하는 데 강합니다. - "테이블끼리는 서로 무관하게 따로 있는 것 아닌가요?" → 아닙니다. 테이블은 **키로 연결**되어 서로를 가리킬 수 있고, 이 연결이 관계형 데이터베이스의 핵심입니다.

테이블·행·열 (table / row / column)

테이블 한 장을 확대해서 부품 이름을 붙여 보겠습니다.

- **열(column)**은 표의 **세로 칸**으로, "이 표가 무슨 속성을 담는가"를 정합니다. 예: 이름, 이메일, 가입일. 1개의 데이터를 저장하는 최소 단위
- **행(row)**은 표의 **가로 한 줄**로, **데이터 한 건**(예: 회원 한 명)을 뜻합니다. 실행의 기본 단위
- **셀(cell)**은 행과 열이 만나는 한 칸으로, 실제 **값(value)**이 들어가는 자리입니다.

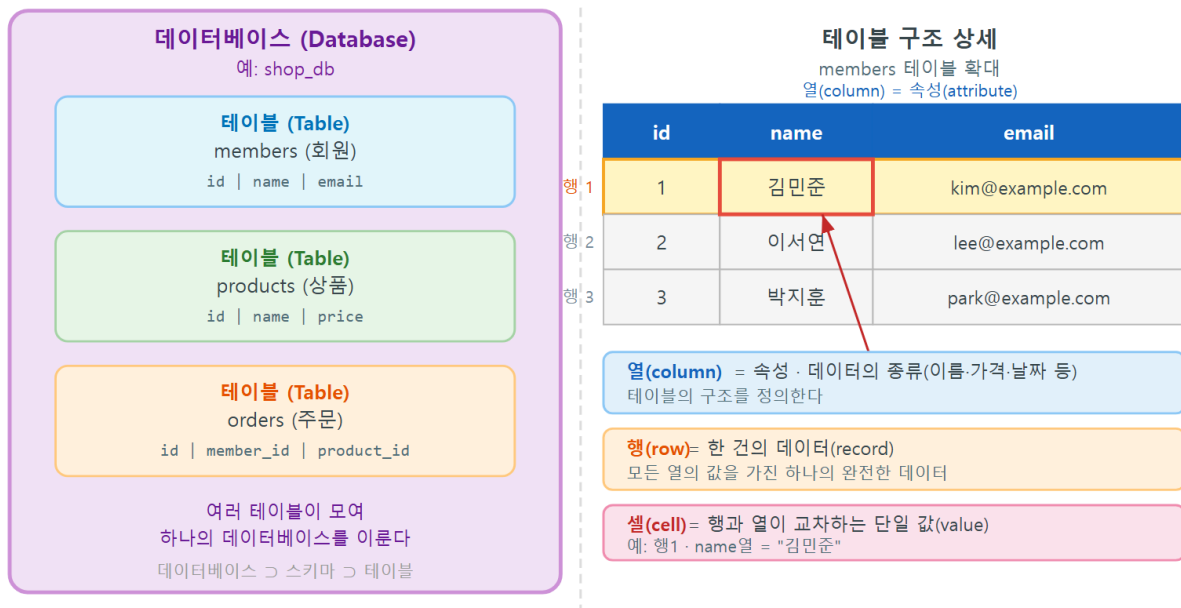
아래는 회원 테이블을 글자로 그려 본 모습입니다.

members 테이블

```

+---+-----+-----+-----+
| id | name  | email                | joined_at | <- 열(column): 속성 이름
+---+-----+-----+-----+
| 1  | 김민수 | minsu@example.com    | 2026-01-10 | <- 행(row): 회원 한 명
| 2  | 이서연 | seoyeon@example.com  | 2026-02-03 |
+---+-----+-----+-----+
^
+--- 셀(cell): 행과 열이 만나는 한 칸의 값
    
```

관계형 데이터베이스 구조: 데이터베이스 · 테이블 · 행 · 열



ch_01_s01 · 관계형 데이터베이스(relational database) · 테이블·행·열(table/row/column)

여러 개의 테이블(예: members, products, orders)이 모이면 하나의 **데이터베이스**가 됩니다. 창고(데이터베이스) 안에 여러 선반(테이블)이 있고, 선반마다 같은 양식의 물건(행)이 줄지어 놓인 모습을 떠올리면 됩니다.

PostgreSQL이 백엔드 개발에서 쓰이는 이유

PostgreSQL(흔히 "포스트그레스"라고 읽습니다)은 위와 같은 관계형 데이터베이스를 다루는 대표적인 오픈소스 데이터베이스입니다. 백엔드(서버 쪽) 개발에서 자주 고르는 이유는 다음과 같습니다. 무료

- **무료이고 제약이 적습니다.** 개인 학습부터 상용 서비스까지 비용 부담 없이 쓸 수 있습니다.

ANSI

- **표준 SQL을 충실히 따릅니다.** 여기서 배운 문법이 다른 데이터베이스에서도 대부분 통합니다.
- **기능이 풍부하고 튼튼합니다.** 트랜잭션(10장), 제약조건(9장) 같은 안전장치가 잘 갖춰져 데이터를 잃지 않고 다룰 수 있습니다.

팁: SQL(Structured Query Language)은 데이터베이스에게 일을 시키는 **공용어**입니다.

PostgreSQL은 그 공용어를 알아듣는 참고지기 중 하나입니다. 그래서 SQL을 배워 두면 다른 데이터베이스로 옮겨도 대부분 그대로 쓸 수 있습니다.

절 요약

- 관계형 데이터베이스는 데이터를 **테이블**에 담고 테이블 사이의 **관계**로 연결합니다.
- 테이블은 **열(속성)** 과 **행(데이터 한 건)** 으로 이루어지고, 값은 **셀**에 들어갑니다.
- PostgreSQL은 무료·표준 준수·튼튼함 덕분에 백엔드에서 널리 쓰입니다.

Windows 11에 PostgreSQL 16 설치하기

도입

참고지기(PostgreSQL 서버)를 부르려면 먼저 내 PC에 들여놓아야 합니다. 다행히 Windows 11에서는 **설치 관리자(installer)** 하나로 서버와 도구가 한 번에 깔립니다. 이 절에서는 설치 관리자를 내려받아 실행하고, 설치 중 묻는 **비밀번호와 포트** 설정을 어떻게 정할지 정리합니다.

핵심 개념 설명

설치 관리자 (installer)

설치 관리자(installer) 는 필요한 파일을 알맞은 위치에 풀어 놓고 초기 설정까지 안내해 주는 설치 도우미 프로그램입니다. PostgreSQL은 **EDB**라는 회사가 제공하는 Windows용 설치 관리자가 가장 널리 쓰입니다. 이 설치 관리자 하나에는 다음이 함께 들어 있습니다.

명령

- **PostgreSQL 서버** — 데이터를 실제로 보관하고 처리하는 참고지기 본체입니다.

client • **psql** — 명령으로 서버에 말을 거는 도구입니다(다음 절에서 사용합니다).

- **pgAdmin** — 마우스로 클릭해 다루는 화면(GUI) 도구입니다(이 책은 psql 위주로 진행합니다).

서비스와 포트 (service / port 5432)

설치가 끝나면 PostgreSQL 서버는 Windows의 **서비스(service)** 로 등록됩니다. 서비스란, PC를 켜면 **백그라운드에서 자동으로 켜져 조용히 돌아가는 프로그램**을 말합니다. 즉 우리가 매번 켜지 않아도 창고지기가 항상 대기하고 있습니다.

그 창고지기와 대화하려면 **포트(port)** 번호를 알아야 합니다. 포트는 한 PC 안에서 프로그램마다 가진 **출입문 번호**라고 생각하면 됩니다. PostgreSQL의 기본 출입문은 **5432번**입니다. 특별한 이유가 없으면 기본값 그대로 두는 것을 권합니다.

이렇게 설치합니다

다음은 설치 관리자를 내려받아 실행하는 흐름입니다(실제로는 화면의 "다음" 버튼을 눌러 진행합니다).

1. 웹 브라우저에서 PostgreSQL 공식 다운로드 페이지로 이동
(postgresql.org -> Download -> Windows -> EDB installer)
2. 16 버전 Windows x86-64 설치 파일을 내려받기
예: postgresql-16.x-windows-x64.exe
3. 내려받은 파일을 더블클릭해 설치 관리자 실행
4. 설치 경로(기본: C:\Program Files\PostgreSQL\16) 확인 후 다음
5. 설치할 구성요소(서버 / pgAdmin / Command Line Tools) 모두 체크
6. 슈퍼유저(postgres) 비밀번호 입력 <- 꼭 기억할 것
7. 포트 번호 5432 확인
8. 로캘(Locale)은 기본값으로 두고 설치 진행

설치가 끝났는지 PowerShell에서 다음처럼 버전을 확인할 수 있습니다. 아래 명령은 설치된 psql 도구에 버전을 물어보는 것입니다.

```
PS C:\Users\사용자> psql --version
```

예상 화면:

```
psql (PostgreSQL) 16.2
```

만약 psql 을 찾을 수 없다는 메시지가 나오면, psql이 있는 폴더가 **PATH(경로 목록)** 에 등록되지 않은 경우입니다. 이때는 아래처럼 전체 경로로 직접 실행하거나, 환경 변수 PATH에 C:\Program Files\PostgreSQL\16\bin 을 추가합니다.

```
PS C:\Users\사용자> & "C:\Program Files\PostgreSQL\16\bin\psql.exe" --version
```

설치 항목 설명

항목	의미
슈퍼유저 <code>postgres</code>	모든 권한을 가진 관리자 계정입니다. 비밀번호를 꼭 기억합니다
포트 <code>5432</code>	서버와 대화하는 출입문 번호입니다. 기본값을 권장합니다
<code>bin</code> 폴더	<code>psql.exe</code> 등 명령 도구가 들어 있는 폴더입니다
PATH 등록	어느 위치에서나 <code>psql</code> 만 입력해도 실행되게 해 줍니다

주의: 슈퍼유저(`postgres`) 비밀번호는 설치 중 한 번만 정합니다. 잊어버리면 접속이 막혀 재설정이 번거로우니, 안전한 곳에 메모해 둡니다. 또한 `5432` 포트를 이미 다른 프로그램이 쓰고 있으면 설치 관리자가 다른 번호를 제안하는데, 그 번호를 따로 적어 두어야 나중에 접속할 수 있습니다.

절 요약

- EDB 설치 관리자 하나로 서버·`psql`·`pgAdmin`이 함께 설치됩니다.
- 설치 후 서버는 Windows 서비스로 자동 실행되며, 기본 포트는 **5432**입니다.
- 설치 중 정한 슈퍼유저 비밀번호는 반드시 기억해 둡니다.

명령
대신 DBever 사용 <---> 서버

psql 접속과 메타 명령(meta-command) 첫걸음

도입

참고지기를 들었으니 이제 말을 걸어 볼 차례입니다. **psql**은 PostgreSQL과 대화하는 가장 기본적인 도구입니다. 마치 무전기처럼, 명령을 보내면 데이터베이스가 답을 돌려줍니다. 이 절에서는 PowerShell에서 `psql`로 접속하고, `\l · \dt` 같은 **메타 명령**으로 참고 안을 돌려봅니다.

핵심 개념 설명

psql 클라이언트 (psql client)

psql 클라이언트(psql client) 는 PostgreSQL 서버에 접속해 SQL과 메타 명령을 입력하고 결과를 확인하는 **명령줄 도구**입니다. 데이터베이스라는 참고와 대화하기 위한 무전기와 같아서, 명령을 보내면 참고가 답을 돌려줍니다.

여기서 두 가지를 구분하면 좋습니다. **PostgreSQL 서버**는 데이터를 실제로 보관하는 창고지기이고, **psql**은 그 창고지기에게 말을 거는 손님 도구일 뿐입니다.

흔한 오해 - "psql 자체가 데이터베이스 아닌가요?" → 아닙니다. psql은 **대화 창구**일 뿐, 데이터는 서버가 보관합니다. - "GUI 도구(pgAdmin)가 없으면 PostgreSQL을 못 쓰나요?" → 아닙니다. psql만으로 모든 작업이 가능하며, 오히려 명령에 익숙해지면 더 빠릅니다.

메타 명령 (meta-command)

메타 명령(meta-command) 은 SQL 문이 아니라 **psql 도구에게 직접 내리는 편의 명령**입니다. 모두 백슬래시(\)로 시작하는 것이 특징입니다. 데이터를 다루는 SQL(예: SELECT)과 달리, 메타 명령은 "지금 어떤 데이터베이스가 있지?", "이 테이블 구조가 어떻게 생겼지?" 처럼 **둘러보기**에 씁니다.

자주 쓰는 메타 명령은 다음과 같습니다.

메타 명령	하는 일
\l	서버에 있는 데이터베이스 목록 (list) 보기
\dt	현재 데이터베이스의 테이블 목록 (describe tables) 보기
\d 테이블명	특정 테이블의 구조 (열·자료형) 보기
\c 데이터베이스명	다른 데이터베이스로 접속 전환 (connect)
\q	psql 종료 (quit)

이렇게 접속합니다

PowerShell에서 슈퍼유저 postgres 로 접속하는 명령입니다. -u 는 접속할 사용자(User)를 지정하는 옵션입니다.

```
PS C:\Users\사용자> psql -U postgres
```

실행하면 설치 중 정한 비밀번호를 물어보고, 맞으면 다음과 같은 **psql 프롬프트**로 바뀝니다.

```

Password for user postgres:
psql (16.2)
Type "help" for help.

postgres=#

```

postgres=# 는 "지금 postgres 라는 데이터베이스에 접속해 명령을 기다리는 중"이라는 표시입니다. 끝의 # 은 슈퍼유저로 접속했다는 뜻입니다. 이제 메타 명령으로 둘러보겠습니다.

```

postgres=# \l
                                List of databases
  Name      | Owner   | Encoding | Collate |  Ctype  | Access ...
-----+-----+-----+-----+-----+-----
 postgres   | postgres | UTF8     | ...     | ...     |
 template0  | postgres | UTF8     | ...     | ...     |
 template1  | postgres | UTF8     | ...     | ...     |
(3 rows)

postgres=# \dt
Did not find any relations.

postgres=# \q
PS C:\Users\사용자>

```

화면 읽는 법

표시	의미
postgres=#	psql 프롬프트입니다. # 은 슈퍼유저로 접속한 상태를 뜻합니다
\l 결과	처음에는 postgres · template0 · template1 세 개가 기본으로 있습니다
Did not find any relations.	아직 테이블을 만들지 않아 테이블 목록이 비어 있다는 안내입니다
\q	psql을 종료하고 다시 PowerShell로 돌아옵니다

팁: SQL 문은 끝에 세미콜론(;)을 붙여야 실행되지만, 메타 명령(\l, \dt 등)은 세미콜론 없이 입력하고 엔터만 누르면 바로 동작합니다. 둘을 헷갈리지 않도록 "백슬래시로 시작하면 둘러보기 명령" 이라고 기억해 둡니다.

절 요약

- **psql**은 서버에 말을 거는 명령줄 도구이며, 서버 자체와는 다릅니다.
- `psql -U postgres` 로 접속하면 `postgres=#` 프롬프트로 바뀝니다.
- **메타 명령**(`\l · \dt · \d · \q`)은 백슬래시로 시작하는 둘러보기 전용 명령입니다.

첫 데이터베이스 연결과 실습 환경 점검

도입

접속에 성공했다면, 이제 학습 내내 마음껏 만들고 지울 **나만의 연습장**을 마련할 차례입니다. 기본 `postgres` 데이터베이스를 그대로 쓰기보다, **실습 전용 데이터베이스**를 따로 만들어 두면 실수해도 안전하고 정리가 깔끔합니다. 이 절에서는 접속에 필요한 정보 네 가지를 이해하고, 실습용 데이터베이스 `shop_lab` 을 만들어 봅니다.

핵심 개념 설명

접속 정보 (connection info)

서버에 접속하려면 보통 네 가지 정보가 필요합니다. 택배를 보낼 때 주소·받는 사람을 적는 것과 비슷합니다.

항목	의미	우리 실습 기본값
호스트(host)	서버가 있는 위치(주소)	<code>localhost</code> (내 PC)
포트(port)	서버의 출입문 번호	<code>5432</code>
사용자(user)	접속할 계정 이름	<code>postgres</code>
데이터베이스(database)	접속할 창고 이름	<code>shop_lab</code> (곧 만듭니다)

이 네 가지를 `psql` 옵션으로 한 번에 적을 수 있습니다. `-h` 는 호스트, `-p` 는 포트, `-u` 는 사용자, 그리고 맨 끝에 데이터베이스 이름을 적습니다.

```
PS C:\Users\사용자> psql -h localhost -p 5432 -U postgres -d postgres
```

옵션 설명

옵션	의미
-h localhost	내 PC에 있는 서버에 접속한다는 뜻입니다
-p 5432	5432번 출입문으로 접속합니다
-U postgres	postgres 계정으로 접속합니다
-d postgres	postgres 데이터베이스에 접속합니다

실습용 데이터베이스 (practice database)

실습용 데이터베이스(practice database) 는 학습 중 자유롭게 테이블을 만들고 지울 수 있는 격리된 연습 공간입니다. 진짜 서비스 데이터와 섞이지 않으므로, 무엇을 지우거나 망쳐도 다시 만들면 그만이라 마음 편히 실험할 수 있습니다.

데이터베이스를 새로 만드는 SQL은 `CREATE DATABASE` 입니다. 아래는 `shop_lab` 이라는 실습용 데이터베이스를 만드는 흐름입니다. SQL 문이므로 끝에 세미콜론(;)을 붙입니다.

```
-- 실습 전용 데이터베이스 만들기 (postgres=# 프롬프트에서 입력)
CREATE DATABASE shop_lab;
```

예상 화면:

```
postgres=# CREATE DATABASE shop_lab;
CREATE DATABASE
postgres=#
```

`CREATE DATABASE` 라는 응답이 돌아오면 성공입니다. 이제 만든 데이터베이스로 옮겨 가서 비어 있는지 확인합니다. 데이터베이스 전환은 메타 명령 `\c` 를 씁니다.

```
postgres=# \c shop_lab
You are now connected to database "shop_lab" as user "postgres".

shop_lab=# \dt
Did not find any relations.

shop_lab=#
```

프롬프트가 `postgres=#` 에서 `shop_lab=#` 으로 바뀐 것에 주목합니다. 지금 어느 데이터베이스에 들어와 있는지가 프롬프트에 항상 표시되므로, 명령을 내리기 전에 한 번씩 확인하는 습관을 들이면 실수를 줄일 수 있습니다.

연습이 끝나 데이터베이스를 통째로 정리하고 싶다면 `DROP DATABASE` 를 씁니다. 단, 그 데이터베이스에 **접속한 상태에서는 지울 수 없으므로** 먼저 다른 데이터베이스로 빠져나온 뒤 실행합니다.

```
-- 먼저 다른 데이터베이스로 이동한 뒤 삭제해야 합니다
-- (\c postgres 로 빠져나온 상태)
DROP DATABASE shop_lab;
```

복구X

위험: `DROP DATABASE` 는 그 안의 **모든 테이블과 데이터를 한 번에 영구 삭제**합니다. 되돌릴 수 없습니다. 실습용이 아닌 실제 데이터베이스에는 절대 함부로 쓰지 않으며, 이름을 한 번 더 확인하고 실행합니다.

권장 습관: 학습용과 실제 데이터를 **데이터베이스 단위로 분리**해 두면 사고를 크게 줄일 수 있습니다. 이 책의 모든 실습은 `shop_lab` 안에서만 진행하는 것을 권합니다.

절 요약

- 접속에는 **호스트·포트·사용자·데이터베이스** 네 정보가 필요합니다.
- `CREATE DATABASE` 로 **실습 전용 데이터베이스**를 만들고 `\c` 로 옮겨 갑니다.
- 프롬프트의 데이터베이스 이름을 확인하는 습관으로 실수를 예방합니다.

보완: DBeaver로 같은 작업을 화면에서 해 보기

도입

이 책은 명령으로 직접 다루는 `psql`을 기준으로 진행합니다. 다만 실무 백엔드 개발에서는 **화면(GUI) 도구**도 함께 쓰는 경우가 많습니다. 쿼리 결과를 표로 한눈에 보고, 테이블 목록을 트리로 펼쳐 보며, 자동완성의 도움을 받을 수 있기 때문입니다. 그중 가장 널리 쓰이는 무료 도구가 **DBeaver**입니다. 이 절은 선택 학습으로, `psql`과 **같은 작업을 화면에서 해 보는** 보완 과정입니다.

핵심 개념 설명

DBeaver (데이터베이스 GUI 클라이언트)

DBeaver는 PostgreSQL을 비롯한 여러 데이터베이스에 연결해 마우스로 다루는 **무료 GUI 클라이언트**입니다. 커뮤니티 에디션(Community Edition)은 무료이며 Windows·macOS·Linux에서 모두 동작합니다. 앞서 본 psql과 역할은 같습니다. 둘 다 서버에 말을 거는 **손님(클라이언트)** 도구이고, 다만 psql은 명령으로, DBeaver는 화면으로 말을 건다는 차이뿐입니다.

- **왜(why) 쓰는가:** 조회 결과를 **표(grid)**로 보여 주고, 컬럼 정렬·복사·편집이 클릭으로 됩니다. 입문 단계에서 데이터의 모양을 눈으로 익히기 좋습니다.
- **언제(when) 유리한가:** 테이블 구조를 둘러보거나, 결과를 표로 비교하거나, 여러 테이블 관계(ERD)를 그림으로 볼 때 편합니다.
- **언제(when) psql이 나은가:** 스크립트 자동화, 빠른 반복 작업, GUI가 없는 원격 서버 환경에서는 psql이 더 빠르고 확실합니다.

팁: DBeaver와 psql은 **둘 중 하나만** 골라야 하는 도구가 아닙니다. 같은 `shop_lab` 데이터베이스에 둘 다 접속할 수 있으므로, 평소에는 DBeaver로 둘러보고 반복 작업은 psql로 처리하는 식으로 함께 쓰면 됩니다.

이렇게 설치하고 연결합니다

설치는 psql과 마찬가지로 설치 파일을 내려받아 실행합니다(화면의 "다음" 버튼으로 진행).

1. 웹 브라우저에서 DBeaver 공식 페이지로 이동
(dbeaver.io -> Download -> Community Edition -> Windows Installer)
2. 내려받은 설치 파일(dbeaver-ce-...-x86_64-setup.exe)을 더블클릭해 설치
3. DBeaver 실행 후, 상단 메뉴에서 새 연결 추가
(Database -> New Database Connection -> PostgreSQL 선택)

연결 창에서는 앞 절에서 정리한 **접속 정보 네 가지**를 그대로 입력합니다. psql에서 `-h` · `-p` · `-u` · `-d` 옵션으로 적던 값과 동일합니다.

연결 창 입력값 (psql 옵션과 대응)

DBeaver 입력칸	값	psql에서의 같은 옵션
Host	localhost	-h localhost
Port	5432	-p 5432
Database	shop_lab	끝에 적던 데이터베이스 이름
Username	postgres	-U postgres
Password	설치 중 정한 비밀번호	접속 시 물어보던 비밀번호

값을 채운 뒤 **Test Connection**(연결 시험) 버튼을 누르면 접속 가능 여부를 미리 확인할 수 있습니다. 성공하면 **Finish**로 연결을 저장합니다. 왼쪽 **Database Navigator**(데이터베이스 탐색기) 트리에서 `shop_lab > Schemas > public > Tables` 를 펼치면 테이블 목록을 볼 수 있습니다(아직 테이블이 없으면 비어 있습니다).

쿼리는 **SQL Editor**(SQL 편집기)를 열어 입력하고 실행합니다.

```
-- DBeaver의 SQL Editor에 입력한 뒤 Ctrl+Enter 로 실행합니다
SELECT datname FROM pg_database;
```

psql에서는 결과가 글자 표로 나오지만, DBeaver에서는 같은 결과가 아래처럼 **표(grid)** 형태로 보입니다(편집기 아래 결과 패널).

```
+-----+
| datname |
+-----+
| postgres |
| template0 |
| template1 |
| shop_lab |
+-----+
```

주의: DBeaver로 연결할 때도 PostgreSQL **서버는 켜져 있어야** 합니다(Windows 서비스로 자동 실행 중). DBeaver는 어디까지나 손님 도구라, 서버가 꺼져 있으면 "연결할 수 없음" 오류가 납니다. 또한 회사·학습 환경에서 결과 표를 직접 수정하면 실제 데이터가 바뀔 수 있으므로, 실습은 `shop_lab` 에서만 진행합니다.

절 요약

- **DBeaver**는 PostgreSQL에 연결하는 무료 GUI 클라이언트로, **psql**과 역할은 같고 다루는 방식만 화면입니다.
- 연결에는 **psql**과 **똑같은 접속 정보**(호스트·포트·사용자·데이터베이스·비밀번호)를 입력합니다.
- 결과를 **표로 보기**에는 DBeaver가, **빠른 반복·자동화**에는 **psql**이 유리하므로 함께 쓰면 됩니다.

실습 과제

해보기: 내 PC에 PostgreSQL 설치하고 실습 DB 만들기

Windows 11에 PostgreSQL 16을 설치한 뒤 **psql**로 접속해 실습 전용 데이터베이스 **shop_lab**을 만들고, 메타 명령으로 상태를 점검해 봅니다.

- **단계 1:** EDB 설치 관리자로 PostgreSQL 16을 설치하고, PowerShell에서 **psql --version**으로 설치를 확인합니다.
- **단계 2:** **psql -U postgres** 로 접속하고, **\l** 로 기본 데이터베이스 세 개 (**postgres · template0 · template1**)가 보이는지 확인합니다.
- **단계 3:** **CREATE DATABASE shop_lab;** 으로 실습용 데이터베이스를 만들고, **\c shop_lab** 으로 옮겨 갑니다.
- **단계 4:** **\dt** 로 아직 테이블이 없음을 확인하고, **\q** 로 종료합니다.
- **단계 5(선택):** DBeaver를 설치하고 같은 접속 정보(**localhost · 5432 · postgres · shop_lab**)로 연결한 뒤, SQL Editor에서 **SELECT datname FROM pg_database;** 를 실행해 **shop_lab** 이 결과 표에 나오는지 확인합니다.

점검표(아래 항목에 직접 체크) - [] **psql --version** 결과에 **16** 이 보인다. - [] **\l** 결과에 방금 만든 **shop_lab** 이 목록에 나타난다. - [] 프롬프트가 **shop_lab=#** 으로 바뀌었다. - [] 슈퍼유저 비밀번호와 포트(5432)를 메모해 두었다. - [] (선택) DBeaver의 Test Connection이 성공하고, 결과 표에 **shop_lab** 이 보인다.

FAQ

Q1. **psql**을 입력하면 "psql: command not found" 또는 "용어가 cmdlet 이름으로 인식되지 않습니다" 라고 나옵니다. → **A1.** **psql**이 있는 폴더가 **PATH**에 등록되지 않은 경우입니다.

C:\Program Files\PostgreSQL\16\bin 을 환경 변수 PATH에 추가하거나, & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres 처럼 전체 경로로 실행하면 됩니다.

Q2. 접속할 때 비밀번호를 자꾸 틀린다고 합니다. → A2. 설치 중 정한 슈퍼유저(postgres) 비밀번호를 입력해야 합니다. 비밀번호를 입력할 때 화면에 글자가 보이지 않는 것은 정상이며(보안 상 숨김), 한영 입력 상태와 대소문자를 확인합니다. 비밀번호를 잊었다면 재설정 절차가 필요합니다.

Q3. \l 같은 메타 명령과 SELECT 같은 SQL은 무엇이 다른니까? → A3. 메타 명령은 백슬래시(\)로 시작하며 psql 도구에 직접 내리는 둘러보기 명령이라 세미콜론이 필요 없습니다. SQL 은 서버가 처리하는 데이터 명령이라 끝에 세미콜론(;)을 붙여야 실행됩니다.

Q4. pgAdmin이라는 화면 도구가 같이 깔렸는데, 꼭 psql을 써야 하나요? → A4. 꼭은 아닙니다. 다만 이 책은 명령으로 직접 다루는 psql을 기준으로 설명합니다. 명령에 익숙해지면 작업이 더 빠르고, 서버 환경(GUI가 없는 곳)에서도 동일하게 쓸 수 있어 학습 효과가 큼니다.

Q5. pgAdmin과 DBeaver는 무엇이 다른가요? 어느 것을 써야 하나요? → A5. 둘 다 PostgreSQL 을 화면으로 다루는 GUI 클라이언트입니다. pgAdmin은 PostgreSQL **전용** 도구로 설치 관리자에 함께 들어 있고, DBeaver는 PostgreSQL 외에 **여러 데이터베이스를 한 도구로** 다룰 수 있어 실무에서 널리 쓰입니다. 입문 단계에서는 이미 설치돼 있는 pgAdmin으로 시작해도 되고, 여러 데이터베이스를 다룰 계획이면 DBeaver가 편합니다. 어느 쪽이든 이 책의 SQL 학습 내용은 그대로 적용됩니다.

장 요약

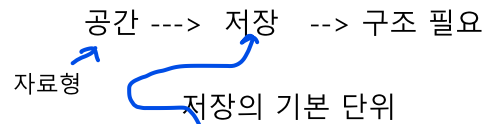
이번 장에서는 PostgreSQL을 시작하기 위한 토대를 다졌습니다. 관계형 데이터베이스가 데이터를 **테이블·행·열**로 저장하고 테이블 간 **관계**로 연결한다는 점을 살펴보았습니다. Windows 11에 EDB 설치 관리자로 PostgreSQL 16을 설치하면 서버는 **서비스**로 자동 실행되고 기본 **포트는 5432**임을 확인했습니다. 또한 **psql**로 접속해 **\l·\dt** 같은 **메타 명령**으로 내부를 둘러보고, 실습 전용 데이터베이스 **shop_lab** 을 만들어 안전한 연습 공간을 마련했습니다. 이로써 다음 장부터 본격적으로 테이블을 설계하고 데이터를 다룰 준비가 끝났습니다.

핵심 키워드

관계형 데이터베이스, 테이블·행·열, 설치 관리자, 포트 5432, psql, 메타 명령, 실습용 데이터베이스

다음 장 미리보기

다음 장에서는 데이터베이스 안의 **스키마와 테이블 계층 구조**를 이해하고, 자주 쓰는 **자료형**으로 첫 테이블을 `CREATE TABLE` 로 설계해 봅니다.



Row { insert
update
delete
select

데이터베이스·스키마·테이블 기본과 자료형

학습 목표 - 데이터베이스(database)·스키마(schema)·테이블(table)의 계층 관계를 설명할 수 있습니다. - 숫자·문자·날짜·불리언 등 자주 쓰는 **자료형(data type)**을 상황에 맞게 고를 수 있습니다. - **CREATE TABLE** 로 컬럼과 자동 증가 키를 갖춘 테이블을 만들 수 있습니다. - NULL 과 DEFAULT 를 이해하고 ALTER · DROP 으로 테이블을 변경·삭제할 수 있습니다.

1장에서는 PostgreSQL을 설치하고 psql로 접속해 실습용 데이터베이스 shop_lab 을 만들었습니다. 이번 장에서는 그 빈 데이터베이스 안에 **데이터를 담을 그릇**, 즉 테이블을 직접 설계합니다. 데이터를 넣고(Create) 읽고(Read) 고치고(Update) 지우는>Delete) CRUD를 배우기 전에, 그 데이터가 어디에 어떤 모양으로 저장되는지를 먼저 정해야 하기 때문입니다.

이 책의 SQL 화면에 자주 보이는 shop_lab=# 은 "지금 psql이 shop_lab 데이터베이스에 접속한 채 명령을 기다리는 중"이라는 표시입니다. # 은 관리자(슈퍼유저)로 접속했음을 뜻합니다. 직접 입력하는 부분은 이 프롬프트 표시 뒤에 오는 글자뿐이며, 프롬프트 자체는 따라 칠 필요가 없습니다. 셸 명령은 Windows 11의 기본 터미널인 PowerShell(파워셸) 기준으로 적었습니다.

이 장의 SQL은 직접 따라 입력하며 익히도록 구성했지만, 실행을 강제하지는 않습니다. 코드를 눈으로 읽고 예상 출력과 해설을 함께 보는 것만으로도 흐름을 충분히 이해할 수 있습니다.

데이터베이스·스키마·테이블의 계층 구조

도입

서류를 정리한다고 생각해 보겠습니다. 사무실(건물) 안에는 여러 캐비닛이 있고, 캐비닛 안에는 여러 서랍이 있으며, 서랍 안에 실제 서류철이 들어갑니다. PostgreSQL에서 데이터를 보관하는 방식도 똑같이 **층층이 담은 구조**입니다. 이 절에서는 그 3단계 계층을 그림처럼 머릿속에 그려 봅니다.

핵심 개념 설명

객체 계층 (object hierarchy)

PostgreSQL은 데이터를 다음 세 단계로 담습니다.

- **데이터베이스(database)**: 가장 바깥 그릇입니다. 서로 관련 있는 데이터를 하나로 묶는 큰 단위로, 1장에서 만든 `shop_lab` 이 여기에 해당합니다. 사무실 건물 한 채라고 보면 됩니다.
- **스키마(schema)**: 데이터베이스 안에서 테이블 같은 객체를 묶어 정리하는 이름 공간 (namespace) 입니다. 건물 안의 한 부서나 한 층에 해당합니다. 데이터베이스를 새로 만들면 `public` (퍼블릭)이라는 기본 스키마가 자동으로 하나 들어 있습니다.
- **테이블(table)**: 실제 데이터를 행(row)과 열(column)로 담는 표입니다. 서랍 안의 서류철 하나에 해당합니다.
행 단위로 담는다.

정리하면 데이터베이스 안에 스키마가 있고, 스키마 안에 테이블이 있습니다. 우리가 이름만 적고 쓰는 `members` 같은 테이블은 사실 `shop_lab` 데이터베이스의 `public` 스키마 안에 있는 `public.members` 입니다.

데이터베이스 · 스키마 · 테이블 계층 구조



하나의 데이터베이스는 여러 스키마를 포함하고, 각 스키마는 여러 테이블을 담습니다.
public 스키마는 PostgreSQL이 기본으로 제공하는 스키마입니다.

왜 스키마라는 중간 단계가 필요할까

테이블을 바로 데이터베이스에 넣으면 될 텐데 왜 스키마라는 층을 한 단계 더 두는지 궁금할 수 있습니다. 이유는 **정리와 충돌 방지** 때문입니다.

- **왜(why)**: 프로젝트가 커지면 테이블이 수십, 수백 개가 됩니다. 영업용 테이블과 회계용 테이블을 `sales` · `accounting` 스키마로 나눠 두면 서랍에 라벨을 붙여 둔 것처럼 찾기 쉽습니다.

니다.

- **언제(when):** 같은 이름의 테이블이 여러 용도로 필요할 때 유용합니다. `sales.report` 와 `accounting.report` 는 이름이 같아도 스키마가 달라 서로 충돌하지 않습니다.
- **어떻게(how):** 입문 단계에서는 기본 스키마인 `public` 하나만 써도 충분합니다. 이 책의 실습도 모두 `public` 스키마를 사용합니다.

이렇게 확인합니다

psql에 접속한 뒤, 지금 데이터베이스 안에 어떤 스키마가 있는지 메타 명령(meta-command) `\dn` 으로 살펴봅니다. 메타 명령은 SQL이 아니라 psql이 제공하는 단축 명령으로, 역슬래시(\)로 시작합니다.

관리자 로그인

```
shop_lab=# \dn
          목록 of schemas
  Name  | Owner
-----+-----
 public | pg_database_owner
(1개 행)
```

`shop_lab` 을 막 만든 직후에는 이렇게 `public` 스키마 하나만 보입니다. 이제 이 `public` 스키마 안에 테이블을 만들면, 테이블 목록은 메타 명령 `\dt` 로 확인할 수 있습니다.

```
shop_lab=# \dt
해당 릴레이션을 찾지 못했습니다.
```

아직 테이블을 하나도 만들지 않았으므로 "찾지 못했습니다"가 정상입니다. 이 장을 마치면 여기에 `members` · `products` · `orders` 세 테이블이 나타나게 됩니다.

계층 확인 메타 명령

메타 명령	확인하는 것	비유
<code>\l</code>	서버 안의 데이터베이스 목록	건물 목록
<code>\dn</code>	현재 데이터베이스의 스키마 목록	부서(층) 목록
<code>\dt</code>	현재 스키마의 테이블 목록	서랍 안 서류철 목록

public 스키마와 이름 충돌 피하기

테이블 이름 앞에 스키마를 붙이지 않으면 PostgreSQL은 기본적으로 `public` 스키마에서 테이블을 찾습니다. 그래서 `members` 라고만 적어도 `public.members` 로 해석됩니다. 다만 다음 두 가지는 흔히 오해하는 부분이라 짚고 갑니다.

```
-- 두 줄은 같은 테이블을 가리킵니다 (public 스키마가 기본값이기 때문)
SELECT * FROM members;
SELECT * FROM public.members;
```

주의: 스키마와 데이터베이스를 같은 말로 혼동하지 않도록 합니다. 데이터베이스는 psql 접속 대상 자체이고, 스키마는 그 데이터베이스 **안의** 정리 칸입니다. 또한 모든 테이블이 반드시 `public`에만 있어야 하는 것은 아닙니다. `public`은 단지 기본값일 뿐, 필요하면 다른 스키마를 만들어 나눌 수 있습니다.

절 요약

- 데이터는 **데이터베이스 > 스키마 > 테이블** 순서로 층층이 담깁니다.
- 데이터베이스를 만들면 `public` 스키마가 기본으로 들어 있고, 입문 단계에서는 이 하나면 충분합니다.
- `\dn`으로 스키마를, `\dt`로 테이블 목록을 확인합니다.

자주 쓰는 자료형(data type) 익히기

도입

택배 상자에는 보통 "깨지기 쉬움", "식품", "냉장" 같은 표시가 붙습니다. 내용물의 종류를 미리 정해 두면 잘못된 물건이 섞이거나 상하는 일을 막을 수 있습니다. 테이블의 각 컬럼(column)에도 똑같이 **무엇을 담을지** 미리 정하는데, 그 규칙이 바로 **자료형(data type)**입니다.

공간 생성 명령

핵심 개념 설명

종류 지정
크기

자료형 (data type)

자료형(data type)이란 컬럼에 저장할 값의 **종류(숫자·문자·날짜·불리언 등)**와 **허용 범위**를 정하는 규칙입니다. 자료형을 정해 두면 PostgreSQL이 **맞지 않는 값을 거부**해 주므로, 가격 칸에 "사고" 같은 글자가 잘못 들어가는 사고를 데이터베이스 차원에서 막을 수 있습니다.

자료형을 고를 때는 다음을 생각합니다.

- **왜(why) 중요한가:** 자료형이 맞아야 정렬·계산·검색이 의도대로 동작합니다. 숫자를 글자로 저장하면 9가 100보다 크게 정렬되는 황당한 일이 생깁니다.
- **언제(when) 무엇을 쓰나:** 값의 성격에 맞춥니다. 개수는 정수형, 금액은 정확한 소수형, 이름은 문자형, 가입일은 날짜형, 활성 여부는 불리언형이 자연스럽습니다.
- **어떻게(how) 정하나:** CREATE TABLE 에서 컬럼 이름 옆에 자료형 이름을 적습니다(다음 절에서 실습합니다).

입문 단계에서 꼭 아는 자료형

자주 쓰는 자료형 한눈에 보기

분류	자료형	답는 값	예시
정수	integer (int)	소수점 없는 정수	재고 수량 120
정수(큰 값)	bigint	아주 큰 정수	누적 조회수
소수(정확)	numeric(p, s)	반올림 오차 없는 소수	금액 19900.00
문자(가변)	varchar(n)	최대 길이 n 까지 글자	이름 홍길동
문자(무제한)	text	길이 제한 없는 글자	상품 설명
날짜	date	날짜만	가입일 2026-06-16
날짜+시간	timestamp	날짜와 시각	주문 시각
참/거짓	boolean 다른 db에는 없음.	true / false	활성 회원 여부

숫자형 고르기 - integer vs numeric

개수처럼 소수점이 필요 없는 값은 integer 를, 금액처럼 소수점이 정확해야 하는 값은 numeric 을 씁니다. numeric(p, s) 에서 p 는 전체 자릿수, s 는 소수점 아래 자릿수입니다. 예를 들어 numeric(10, 2) 는 전체 10자리 중 소수점 아래 2자리를 보장합니다.

```
-- 가격을 numeric(10, 2)로 저장하면 19900.00 처럼 소수점 2자리가 보존됩니다
SELECT 19900.00::numeric(10, 2) AS 가격;
```

예상 출력:

```
가격
-----
19900.00
(1개 행)
```

주의: 금액 계산에는 `real` · `double precision` 같은 부동소수점형을 피하고 `numeric` 을 씁니다. 부동소수점형은 빠르지만 `0.1 + 0.2` 가 정확히 `0.3` 이 되지 않는 미세한 오차가 생길 수 있어, 돈을 다룰 때는 위험합니다.

문자형 고르기 - `varchar` vs `text`

이름처럼 길이가 어느 정도 정해진 값은 `varchar(n)` 으로 최대 길이를 두고, 상품 설명처럼 길이를 예측하기 어려운 값은 `text` 를 씁니다. 흔한 오해 하나를 짚자면, PostgreSQL에서 `varchar` 와 `text` 는 내부 저장 방식이 같아 **성능 차이가 사실상 없습니다**. `varchar(n)` 의 `n` 은 성능이 아니라 "이 길이를 넘기는 값은 받지 않겠다"는 규칙으로 이해하면 됩니다.

```
-- varchar(5)는 5글자를 넘는 값을 거부합니다
SELECT '홍길동'::varchar(5) AS 짧은_이름;
```

가변: 들어오는 갯수에 맞춰진다.
예. "고길" varchar 공간 : 2개

예상 출력:

```
짧은_이름
-----
홍길동
(1개 행)
```

날짜·시각·불리언

날짜 공간 생성

날짜&시간 공간 생성

가입일은 `date`, 주문 시각처럼 **시각**까지 필요하면 `timestamp`, **활성 회원**인지 같은 **예/아니오** 값은 `boolean` 을 씁니다. `boolean` 에는 `true` / `false` 를 넣으며, `'t' · 'f' · 'yes' · 'no'` 같은 표현도 받아 줍니다.

```
-- 현재 날짜와 시각, 그리고 불리언 값을 확인합니다
SELECT CURRENT_DATE AS 오늘, now()::timestamp AS 지금, true AS 활성화여부;
```

키워드(명령): 현재 날짜 함수: 현재 날짜&시간

예상 출력:

```
오늘 |          | 지금 |          | 활성화여부
-----+-----+-----+-----+-----
2026-06-16 | 2026-06-16 14:30:05.123456 | t
(1개 행)
```

팁: 자료형이 헛갈릴 때는 "이 값으로 나중에 계산이나 정렬을 할까?"를 먼저 떠올립니다. 계산·정렬·범위 검색이 필요하면 숫자형·날짜형을, 단순히 보관만 하는 식별 문자열(전화번호, 우편번호)이면 문자형을 고르는 편이 안전합니다.

절 요약

- 자료형은 컬럼에 담을 값의 종류와 범위를 정하는 규칙으로, 잘못된 값을 막아 줍니다.
- 개수는 `integer`, 금액은 `numeric`, 이름은 `varchar`, 긴 글은 `text`, 날짜는 `date / timestamp`, 예/아니오는 `boolean` 을 씁니다.
- 금액에는 부동소수점형 대신 `numeric` 을 써서 오차를 피합니다.

CREATE TABLE로 테이블 설계하기

도입

자료형이라는 재료를 골랐으니, 이제 그 재료로 실제 서류철(테이블)을 만들 차례입니다. 새 노트의 표 머리글에 "번호 / 이름 / 가입일"이라고 칸 이름과 형식을 미리 적어 두는 것처럼, `CREATE TABLE` 로 컬럼 이름과 자료형을 선언합니다.

핵심 개념 설명

테이블 생성 (CREATE TABLE)

`CREATE TABLE` 은 새 테이블을 만드는 명령입니다. 테이블 이름을 정하고, 괄호 안에 **컬럼 정의 (column definition)** 를 쉼표로 나열합니다. 컬럼 정의는 보통 컬럼이름 자료형 [제약] 형태입니다.

`CREATE TABLE` 테이블이름 (컬럼이름 자료형 [제약조건], ...);

첫 테이블 만들기

쇼핑몰의 회원 정보를 담은 `members` 테이블을 만들어 보겠습니다.

-- 회원 정보를 담은 members 테이블을 만듭니다

```
CREATE TABLE members (
  id          integer,
  name        varchar(50),
  email       varchar(100),
  joined_at   date
);
```

예상 출력:

```
CREATE TABLE
```

CREATE TABLE 이라는 한 줄만 출력되면 성공입니다. 만든 테이블의 구조는 메타 명령 `\d` 테이블 이름 으로 확인합니다.

```
shop_lab=# \d members
```

"public.members" 테이블

필드	형식	정렬규칙	NULL허용	기본값
id	integer			
name	character varying(50)			
email	character varying(100)			
joined_at	date			

컬럼 정의 읽는 법

줄	정의	의미
id integer	id 컬럼을 정수형으로	회원 번호를 담습니다
name varchar(50)	name 컬럼을 최대 50글자 문자형으로	회원 이름을 담습니다
email varchar(100)	email 컬럼을 최대 100글자 문자형으로	이메일을 담습니다
joined_at date	joined_at 컬럼을 날짜형으로	가입 날짜를 담습니다

serial-identity로 자동 증가 키 만들기

회원마다 id 를 1, 2, 3처럼 겹치지 않게 직접 매기는 일은 번거롭고 실수도 잦습니다.

PostgreSQL은 새 **행이 들어올 때마다** 번호를 **자동으로 하나씩 늘려 주는** 기능을 제공합니다. 두 가지 방식이 있습니다. insert

- **serial**: 예전부터 쓰던 간편한 방식입니다. 내부적으로 시퀀스(sequence)라는 번호표 발급기를 만들어 자동으로 값을 채웁니다.
- **GENERATED ALWAYS AS IDENTITY (identity)**: SQL 표준을 따르는 최신 방식으로, PostgreSQL 16에서 권장됩니다. 동작은 비슷하지만 표준에 더 가깝고 관리가 깔끔합니다.

```
-- 기존 members를 지우고, 자동 증가 id를 가진 형태로 다시 만듭니다
DROP TABLE IF EXISTS members;

CREATE TABLE members (      일련번호 부여
  id      integer GENERATED ALWAYS AS IDENTITY,
  name    varchar(50),
  email   varchar(100),
  joined_at date
);
```

예상 출력:

```
DROP TABLE
CREATE TABLE
```

이렇게 만들면 데이터를 넣을 때 `id`를 적지 않아도 PostgreSQL이 1, 2, 3 순서로 자동으로 채웁니다(데이터 삽입은 3장에서 다룹니다).

자동 증가 키 두 방식 비교

방식	작성 예	특징
serial	id serial	짧고 익숙함, 오래된 코드에서 흔함
identity	id integer GENERATED ALWAYS AS IDENTITY	SQL 표준, PostgreSQL 16 권장

팁: 새로 시작하는 프로젝트라면 **GENERATED ALWAYS AS IDENTITY**를 권장합니다. 다만 기존 자료나 다른 책에서 `serial`을 자주 보게 되므로, 두 표현이 "자동 증가 번호"라는 같은 목적을 가진다는 점만 기억하면 충분합니다.

절 요약

- `CREATE TABLE 이름 ( 컬럼 자료형, ... )`으로 테이블을 만듭니다.
- 만든 구조는 `\d 테이블이름`으로 확인합니다.

- 겹치지 않는 식별 번호는 `GENERATED ALWAYS AS IDENTITY` (권장) 또는 `serial` 로 자동 증가시킵니다.

NULL과 기본값(DEFAULT) 이해하기

도입

설문지를 받았는데 "나이" 칸이 비어 있다고 해서 그 사람의 나이가 0살인 것은 아닙니다. 그저 **아직 적지 않았을 뿐**입니다. 데이터베이스에도 "값이 없음"을 나타내는 특별한 표시가 있는데, 그것이 바로 `NULL` (널 값)입니다. 이 절에서는 `NULL` 을 올바르게 이해하고, 빈 값을 다루는 `NOT NULL` · `DEFAULT` 를 배웁니다.

핵심 개념 설명

NULL (널 값)

명령

`NULL` (~~널 값~~)은 값이 아직 없거나 알 수 없음을 나타내는 특별한 표시입니다. 가장 중요한 점은 `NULL` 이 숫자 `0`이나 빈 문자열(' ')과 다르다는 것입니다. `0`은 "0이라는 값이 있다", 빈 문자열은 "비어 있는 글자가 있다"는 뜻이지만, `NULL` 은 "값 자체가 없다"는 뜻입니다.

그래서 `NULL` 은 비교 방식도 특별합니다. "값이 없는 것"끼리는 같은지조차 알 수 없으므로, `NULL = NULL` 은 참(true)이 아니라 `NULL` (알 수 없음)이 됩니다. `NULL` 여부는 `=` 가 아니라 `IS` `NULL` · `IS NOT NULL` 로 확인합니다. 비교연산자

-- NULL은 = 로 비교되지 않습니다. IS NULL을 써야 합니다

```
SELECT
  NULL = NULL      AS 등호비교,
  NULL IS NULL     AS is_null비교;
```

예상 출력:

```
등호비교 | is_null비교
-----+-----
          | t
(1개 행)
```

등호비교 칸이 비어 있는 것은 결과가 `true` 도 `false` 도 아닌 `NULL` 이기 때문이고, `is_null비교` 는 `t` (true)로 제대로 나옵니다.

주의: WHERE 나이 = NULL 처럼 쓰면 아무 행도 걸러지지 않습니다. "값이 없는 행"을 찾으려면 반드시 WHERE 나이 IS NULL 로 적어야 합니다. 입문자가 가장 자주 빠지는 함정입니다.

저장공간 (column) 제약조건 **NOT NULL**과 **DEFAULT**로 빈 값 다루기

예.
default =5
값 저장 x -> 값에 5 들어감.

값이 비는 것을 막거나, 비었을 때 자동으로 채울 값을 정할 수 있습니다.

- **NOT NULL**: 이 컬럼은 반드시 값이 있어야 한다는 규칙입니다. 회원 이름처럼 비면 안 되는 칸에 붙입니다. insert
- **DEFAULT 값**: 데이터를 넣을 때 그 컬럼을 적지 않으면 자동으로 채울 기본값(DEFAULT)입니다. 가입일을 안 적으면 오늘 날짜로, 활성 여부를 안 적으면 true 로 채우는 식입니다.

```
-- members를 다시 만들면서 NOT NULL과 DEFAULT를 적용합니다
DROP TABLE IF EXISTS members;

CREATE TABLE members (
  id          integer GENERATED ALWAYS AS IDENTITY,
  name        varchar(50) NOT NULL, 제약. "이름 반드시 있어야 함."
  email       varchar(100),
  joined_at   date        NOT NULL DEFAULT CURRENT_DATE,
  is_active   boolean     NOT NULL DEFAULT true
);      활성여부
```

예상 출력:

```
DROP TABLE
CREATE TABLE
```

제약·기본값 읽는 법

컬럼 정의	의미
name varchar(50) NOT NULL	이름은 비워 둘 수 없습니다
joined_at date NOT NULL DEFAULT CURRENT_DATE	가입일을 안 적으면 오늘 날짜로 채웁니다
is_active boolean NOT NULL DEFAULT true	활성 여부를 안 적으면 true 로 채웁니다

베스트 프랙티스: "이 칸이 비어 있어도 의미가 있는가?"를 기준으로 NOT NULL 여부를 정합니다. 이름·가격처럼 반드시 있어야 하는 값은 NOT NULL 로 강제하고, 자주 생략되는 값은 DEFAULT 로

합리적인 초깃값을 정해 두면 데이터가 훨씬 깔끔해집니다.

절 요약

- `NULL` 은 "값이 없음"으로, 0이나 빈 문자열과 다릅니다.
- `NULL` 여부는 `=` 이 아니라 `IS NULL` · `IS NOT NULL` 로 확인합니다.
- `NOT NULL` 로 빈 값을 막고, `DEFAULT` 로 생략 시 채울 기본값을 정합니다.

테이블 구조 수정 삭제

테이블 변경·삭제(`ALTER`·`DROP`)

도입

처음 설계한 표가 끝까지 그대로인 경우는 드뭅니다. "전화번호 칸을 추가하자", "컬럼 이름을 바꾸자" 같은 요구가 늘 생깁니다. 이미 만든 테이블을 새로 갈아엎지 않고도 구조를 고치는 명령이 `ALTER TABLE` 이고, 더 이상 필요 없는 테이블을 치우는 명령이 `DROP TABLE` 입니다.

핵심 개념 설명

테이블 변경 (`ALTER TABLE`)

`ALTER TABLE` 은 이미 만든 테이블의 구조를 바꾸는 명령입니다. 컬럼을 더하고, 이름을 바꾸고, 자료형을 고치는 일을 데이터를 유지한 채 할 수 있습니다.

```
-- members에 전화번호 컬럼을 추가합니다
ALTER TABLE members ADD COLUMN phone varchar(20);

-- phone 컬럼 이름을 phone_number로 바꿉니다
ALTER TABLE members RENAME COLUMN phone TO phone_number;

-- name 컬럼의 최대 길이를 100으로 늘립니다
ALTER TABLE members ALTER COLUMN name TYPE varchar(100);
```

예상 출력:

```
ALTER TABLE
ALTER TABLE
ALTER TABLE
```

자주 쓰는 `ALTER TABLE` 동작

동작	구문	설명
컬럼 추가	ADD COLUMN 이름 자료형	컬럼 추가 새 칸 을 더합니다
컬럼 이름 변경	RENAME COLUMN 옛이름 TO 새이름	컬럼 칸 이름을 바꿉니다
자료형 변경	ALTER COLUMN 이름 TYPE 새자료형	컬럼 데이터타입 칸 의 형식을 바꿉니다
컬럼 삭제	DROP COLUMN 이름	컬럼 칸 을 없앱니다

주의: 자료형 변경(ALTER COLUMN ... TYPE)은 기존 값이 새 자료형으로 변환될 수 있을 때만 성공합니다. 예를 들어 '사과' 같은 글자가 들어 있는 문자 컬럼을 integer 로 바꾸려 하면 오류가 납니다. 또 DROP COLUMN 은 그 칸의 데이터까지 함께 사라지므로, 실행 전 정말 필요 없는 컬럼인지 확인합니다.

테이블 삭제 (DROP TABLE)와 IF EXISTS

DROP TABLE 은 테이블 자체를 통째로 없앱니다. 안에 들어 있던 데이터도 함께 사라지므로 신중해야 합니다. 이때 유용한 것이 IF EXISTS 입니다. 테이블이 없을 때 DROP TABLE 을 그냥 실행하면 오류가 나지만, IF EXISTS 를 붙이면 "있으면 지우고, 없으면 조용히 넘어가라"는 뜻이 되어 스크립트가 멈추지 않습니다.

```
-- 테이블이 없어도 오류 없이 넘어갑니다
DROP TABLE IF EXISTS temp_members;
```

if x Table x X -> Error
O -> 정상 삭제
if o Table X -> ErrorX
O -> 정상 삭제

예상 출력(테이블이 없을 때):

주의: "temp_members" 테이블이 없습니다. 건너뛰니다
DROP TABLE

같은 차이를 IF EXISTS 없이 실행하면 다음처럼 오류로 멈춥니다.

```
shop_lab=# DROP TABLE temp_members;
오류: "temp_members" 테이블이 없습니다
```

위험: DROP TABLE 은 되돌릴 수 없는 삭제입니다(트랜잭션 밖에서 실행한 경우). 실무 데이터베이스에서 테이블 이름을 착각해 DROP 하는 사고가 드물지 않습니다. 실습용 shop_lab에서는 마음껏 연습하되, 실제 운영 데이터베이스에서는 항상 대상 이름을 두 번 확인하는 습관을 들입니다.

팁: 실습 스크립트를 반복 실행할 때 맨 위에 `DROP TABLE IF EXISTS 테이블이름;` 을 적어 두면, 이전에 만든 테이블이 남아 있어도 깨끗이 지우고 새로 시작할 수 있어 편리합니다. 이 장의 예제 도 이 방식을 활용했습니다.

절 요약

- `ALTER TABLE` 로 데이터를 유지한 채 컬럼을 추가·이름변경·자료형변경·삭제할 수 있습니다.
- `DROP TABLE` 은 테이블과 데이터를 통째로 없애므로 신중히 사용합니다.
- `DROP TABLE IF EXISTS` 는 테이블이 없어도 오류 없이 넘어가, 반복 실습 스크립트에 유용합니다.

실습 과제

해보기: 쇼핑몰 도메인 테이블 3종 설계하기

이번 장에서 배운 자료형·자동 증가 키·`NOT NULL`·`DEFAULT`·`ALTER` 를 한 흐름으로 적용해 봅니다. `psql`에서 `shop_lab` 에 접속한 뒤 진행합니다.

- 단계 1 - 회원 테이블: `members` 를 자동 증가 `id`, `NOT NULL` 이름, 기본값을 가진 `joined_at`·`is_active` 로 만듭니다.
- 단계 2 - 상품 테이블: `products` 를 만들되, 가격은 `numeric(10, 2)` `NOT NULL`, 재고는 `integer NOT NULL DEFAULT 0`, 설명은 `text` 로 둡니다.
- 단계 3 - 주문 테이블: `orders` 를 자동 증가 `id`, 회원 번호 `member_id integer`, 상품 번호 `product_id integer`, 주문 시각 `ordered_at timestamp NOT NULL DEFAULT now()` 로 만듭니다.
- 단계 4 - 변경: `ALTER TABLE products ADD COLUMN category varchar(30);` 으로 카테고리 컬럼을 추가합니다.
- 점검: `\dt` 로 세 테이블이 모두 보이는지, `\d products` 로 `category` 컬럼이 추가됐는지 확인하고 결과를 기록합니다.

이 세 테이블은 3장 이후의 `INSERT·SELECT` 실습에서 계속 사용하므로, 잘 만들어 두면 이후 장이 한결 수월합니다.

FAQ

Q1. `varchar(50)` 과 `text` 중 무엇을 써야 하나요? → A1. PostgreSQL에서 둘은 저장 방식이 같아 성능 차이가 사실상 없습니다. 길이에 명확한 상한이 있다면(예: 이름 50자) `varchar(n)` 으로 규칙을 두고, 상한을 예측하기 어려운 긴 글(상품 설명, 메모)이라면 `text` 를 쓰면 됩니다. 길이 제한 자체가 의미 없는 값이라면 `text` 가 더 단순하고 안전합니다.

Q2. `serial` 과 `GENERATED ALWAYS AS IDENTITY` 는 무엇이 다른가요? → A2. 둘 다 새 행마다 번호를 자동으로 1씩 늘려 주는 자동 증가 키입니다. `serial` 은 오래 쓰여 온 PostgreSQL 고유 표현이고, `GENERATED ALWAYS AS IDENTITY` 는 SQL 표준을 따르는 최신 방식으로 PostgreSQL 16에서 권장됩니다. 새 프로젝트라면 `IDENTITY` 를, 기존 코드를 읽을 때는 `serial` 도 같은 목적임을 기억하면 됩니다.

Q3. 컬럼을 `NOT NULL` 로 만들었는데 `DEFAULT` 도 꼭 같이 줘야 하나요? → A3. 반드시 함께 줄 필요는 없습니다. `NOT NULL` 만 있으면 데이터를 넣을 때 그 컬럼 값을 **직접 적어야** 하고, 안 적으면 오류가 납니다. `DEFAULT` 까지 있으면 값을 생략해도 기본값이 자동으로 채워져 오류가 나지 않습니다. 가입일·생성시각처럼 거의 항상 같은 초깃값을 쓰는 컬럼에 `DEFAULT` 를 함께 주면 편리합니다.

Q4. 실수로 만든 테이블을 지우려는데 `DROP TABLE` 에서 오류가 납니다. → A4. 지우려는 테이블 이름이 정확한지, 그리고 그 테이블이 정말 존재하는지 `\dt` 로 먼저 확인합니다. 이름이 없는 테이블을 `DROP` 하면 오류가 나므로, `DROP TABLE IF EXISTS 이름;` 을 쓰면 없을 때도 조용히 넘어갑니다. 또 다른 테이블이 외래 키로 이 테이블을 참조하고 있으면 삭제가 막히는데, 이 경우는 8장에서 다룹니다.

장 요약

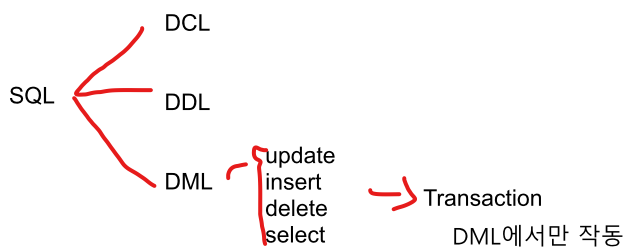
이번 장에서는 데이터를 담을 그릇을 직접 설계했습니다. 먼저 데이터가 **데이터베이스 > 스키마 > 테이블** 순서로 층층이 담기며, 입문 단계에서는 기본 `public` 스키마 하나로 충분하다는 것을 확인했습니다. 이어 `integer · numeric · varchar · text · date · timestamp · boolean` 같은 자주 쓰는 자료형을 상황별로 고르는 기준을 배우고, `CREATE TABLE` 로 컬럼과 자동 증가 키(`GENERATED ALWAYS AS IDENTITY`)를 갖춘 테이블을 만들었습니다. 또한 "값이 없음"을 뜻하는 `NULL` 을 0·빈 문자열과 구분하고, `NOT NULL · DEFAULT` 로 빈 값을 다뤘으며, `ALTER TABLE` 로 구조를 바꾸고 `DROP TABLE IF EXISTS` 로 안전하게 정리하는 법까지 익혔습니다. 이제 데이터를 담을 테이블이 준비됐습니다.

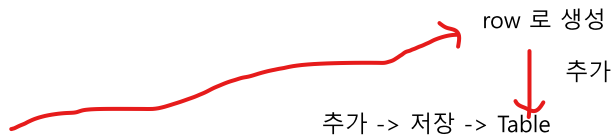
핵심 키워드

스키마(schema), 자료형(data type), numeric, varchar, 테이블 생성(CREATE TABLE), 자동 증가 키(IDENTITY), NULL(널 값), 기본값(DEFAULT), 테이블 변경(ALTER TABLE), 테이블 삭제(DROP TABLE)

다음 장 미리보기

다음 장에서는 이렇게 만든 테이블에 실제 데이터를 채워 넣습니다. INSERT 로 한 행과 여러 행을 삽입하고, RETURNING 으로 방금 생성된 자동 증가 id를 돌려받는 방법까지 CRUD의 첫 글자인 생성(Create)을 본격적으로 다룹니다.





INSERT: 데이터 생성(Create)

학습 목표 - INSERT INTO ... VALUES로 한 행을 삽입하고 CRUD 생명주기에서 위치를 설명할 수 있습니다. - 여러 행을 한 번에 삽입하고 컬럼 목록을 지정할 수 있습니다. - RETURNING으로 삽입과 동시에 생성된 키를 받을 수 있습니다. - INSERT ... SELECT로 다른 테이블의 데이터를 옮길 수 있습니다.



앞 장에서 우리는 빈 테이블을 설계했습니다. 그런데 테이블만 만들어 두면, ~~칸~~은 잘 그려졌지만 아직 아무것도 적히지 않은 장부와 같습니다. 이 장에서는 그 빈 장부에 **첫 줄을 적어 넣는 일**, 즉 데이터를 새로 만들어 채우는 INSERT를 배웁니다.

INSERT는 데이터의 한살이가 시작되는 출발점입니다. 데이터는 일단 생겨야(Create) 그다음에 읽고(Read), 고치고(Update), 지울>Delete) 수 있습니다. 그래서 이 장은 짧지만 CRUD 전체의 첫 단추에 해당합니다.

이 책은 Windows 11의 **PowerShell(파워셸)**에서 psql로 PostgreSQL에 접속한 상태를 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, psql은 그 안에서 데이터베이스와 대화하는 도구입니다. 화면 앞의 shop_lab=# 표시는 "지금 shop_lab 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 모두 아래 두 테이블을 기준으로 합니다. 앞 장에서 만든 것과 같은 모양이라고 생각하면 됩니다.

```
-- 이 장 예시가 기준으로 삼는 두 테이블 (참고용)
CREATE TABLE members (
    id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  회원번호
    name        varchar(50) NOT NULL,
    email       varchar(100) NOT NULL,
    points      integer DEFAULT 0,   포인트
    created_at  timestamp DEFAULT now()
);

CREATE TABLE products (
    id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        varchar(100) NOT NULL,
    category    varchar(30),
    price       numeric(10, 2) NOT NULL,
    stock       integer DEFAULT 0
);
```

여기서 `id` 컬럼은 `GENERATED ALWAYS AS IDENTITY` 로 정의되어 있어, 우리가 값을 넣지 않아도 PostgreSQL이 1, 2, 3...처럼 자동으로 번호를 채워 줍니다. 이 점이 뒤에서 `RETURNING` 을 배울 때 중요한 역할을 합니다.

INSERT로 한 행 삽입하기와 CRUD 생명주기

도입

장부에 거래 한 줄을 적는다고 떠올려 봅시다. "누가, 무엇을, 얼마에"를 정해진 칸에 맞춰 적으면 한 건이 기록됩니다. `INSERT` 가 하는 일이 바로 이것입니다. 이 절에서는 한 행을 넣는 가장 기본형을 익히고, 그 한 건이 앞으로 어떤 흐름(생성-조회-수정-삭제)을 거치는지 큰 그림으로 살펴봅시다.

핵심 개념 설명

행 삽입 (INSERT)

행 삽입(INSERT) 이란, 테이블에 새로운 행(데이터 한 건)을 추가하는 SQL 명령입니다. CRUD에서 생성(Create)에 해당하며, 장부에 새 거래 한 줄을 적어 넣는 일과 같습니다.

기본 형태는 다음과 같습니다. "어느 테이블에(`INTO`), 어떤 컬럼에(`(...)`), 어떤 값을(`VALUES`)" 넣을지 세 가지를 알려 주는 구조입니다.

- **왜(why) 필요한가:** 테이블은 만들어도 데이터가 없으면 아무 의미가 없습니다. 조회·수정·삭제는 모두 "이미 들어 있는 데이터"를 전제로 하므로, 가장 먼저 데이터를 만들어 넣는 `INSERT` 가 출발점입니다.
- **언제(when) 쓰나:** 회원이 새로 가입할 때, 상품을 새로 등록할 때, 주문이 새로 들어올 때처럼 "없던 한 건이 새로 생기는" 모든 순간입니다.
- **어떻게(how) 쓰나:** 컬럼 목록과 값 목록의 **개수와 순서를 1:1로 맞춰** 적습니다. 첫 번째 컬럼에는 첫 번째 값이, 두 번째 컬럼에는 두 번째 값이 들어갑니다.

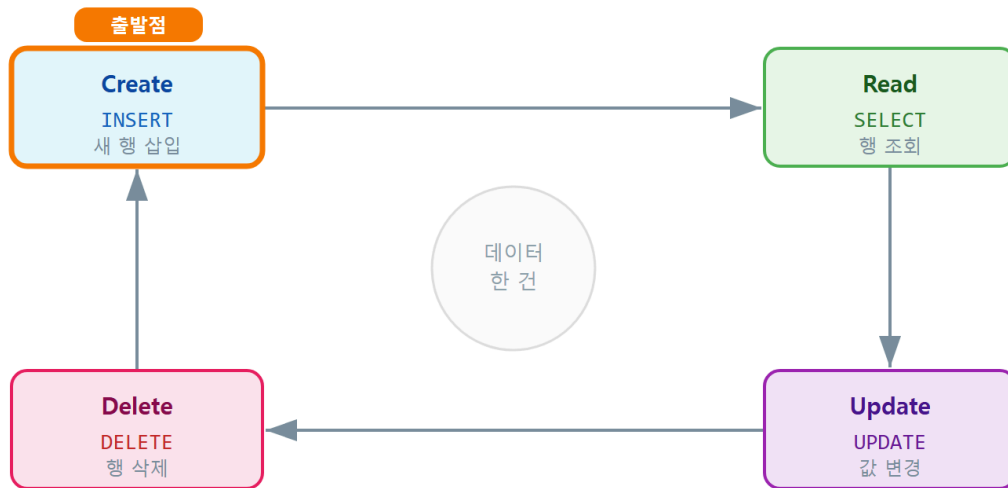
여기서 입문자가 자주 하는 오해 하나를 짚겠습니다. `INSERT` 가 기존 행을 덮어쓴다고 생각하는 경우입니다. `INSERT` 는 언제나 **새 줄을 한 줄 더 추가할 뿐**, 기존 줄을 건드리지 않습니다. 기존 값을 바꾸는 일은 7장의 `UPDATE` 가 맡습니다.

CRUD 생명주기 (CRUD lifecycle)

CRUD 생명주기(CRUD lifecycle)란, 데이터가 생성(Create)·조회(Read)·수정(Update)·삭제(Delete)되는 네 가지 기본 작업의 흐름을 가리킵니다. 물건을 사서(생성) 쓰다가(조회) 고치고(수정) 결국 버리는(삭제) 한살이와 같습니다.

이 네 작업은 각각 SQL 키워드와 짝을 이룹니다.

CRUD 생명주기 — 데이터 한 건의 여정



INSERT가 출발점 — 새 행을 삽입해야 조회·수정·삭제가 가능합니다

ch_03_s01 · INSERT로 데이터 생성(Create)

CRUD와 SQL 키워드 대응

작업	뜻	SQL 키워드	이 책에서 배우는 곳
Create	생성	INSERT	이 장(ch_03)
Read	조회	SELECT	ch_04~ch_06
Update	수정	UPDATE	ch_07
Delete	삭제	DELETE	ch_08

CRUD가 특정 프로그래밍 언어의 기능이라고 생각하기 쉽지만, 그렇지 않습니다. CRUD는 데이터를 다루는 모든 시스템에 공통으로 나타나는 **개념의 묶음**이며, 데이터베이스에서는 SQL 키워드로 표현됩니다.

이렇게 작성합니다

다음은 `members` 테이블에 회원 한 명을 새로 추가하는 가장 기본적인 `INSERT` 입니다.

```
-- members 테이블에 회원 한 명 추가하기
INSERT INTO members (name, email)
VALUES ('김민수', 'minsu@example.com');
```

예상 화면:

```
shop_lab=# INSERT INTO members (name, email)
shop_lab=# VALUES ('김민수', 'minsu@example.com');
INSERT 0 1
```

마지막 줄의 `INSERT 0 1` 은 "삽입 명령이 성공했고, 영향받은 행이 1건"이라는 뜻입니다. 뒤의 숫자 `1` 이 실제로 추가된 행 수입니다.

구문 요소 설명

구문	의미
<code>INSERT INTO members</code>	<code>members</code> 테이블에 새 행을 넣겠다고 지정합니다
<code>(name, email)</code>	값을 채울 컬럼 목록입니다
<code>VALUES ('김민수', ...)</code>	컬럼 목록과 순서가 같은 실제 값입니다
따옴표 <code>'...'</code>	문자열 값은 작은따옴표로 감쌉니다(큰따옴표 아님)

`id` 와 `created_at` 은 값 목록에 넣지 않았습니다. `id` 는 `IDENTITY`라서 자동으로 번호가 매겨지고, `created_at` 은 `DEFAULT now()` 라서 현재 시각이 저장됩니다. 잘 들어갔는지 4장에서 배울 `SELECT` 로 미리 한 번 확인해 보겠습니다.

```
-- 방금 넣은 행 확인하기 (조화는 ch_04에서 자세히 배웁니다)
SELECT id, name, email, points FROM members;
```

예상 화면:

```
id | name | email | points
---+-----+-----+-----
 1 | 김민수 | minsu@example.com |      0
(1 row)
```

id 에 1 이 자동으로 들어갔고, 값을 주지 않은 points 에는 기본값 0 이 채워진 것을 볼 수 있습니다.

팁: 컬럼 목록 (name, email) 을 생략하고 INSERT INTO members VALUES (...) 처럼 쓸 수도 있지만, 그러면 테이블에 정의된 **모든 컬럼 순서대로** 값을 빠짐없이 줘야 합니다. 입문 단계에서는 컬럼 목록을 항상 적는 습관을 권합니다. 컬럼 순서가 바뀌어도 안전하고, 코드만 봐도 무엇을 넣는지 명확합니다.

흔한 실수 두 가지 - 컬럼 개수와 값 개수가 다름 → INSERT INTO members (name, email) VALUES ('김민수') 처럼 개수가 안 맞으면 오류가 납니다. 컬럼과 값의 개수를 반드시 1:1로 맞추십시오. - 문자열에 큰따옴표 사용 → PostgreSQL에서 큰따옴표 "..." 는 컬럼·테이블 이름을 가리킵니다. 문자열 값은 반드시 작은따옴표 '...' 로 감쌉니다.

절 요약

- INSERT INTO 테이블 (컬럼들) VALUES (값들) 로 새 행을 한 건 추가합니다.
- 컬럼과 값의 개수·순서를 1:1로 맞추고, 문자열은 작은따옴표로 감쌉니다.
- IDENTITY·DEFAULT 컬럼은 값을 주지 않으면 자동으로 채워집니다.
- INSERT 는 CRUD 생명주기의 생성(Create) 단계로, 모든 데이터의 출발점입니다.

여러 행 한 번에 삽입하기(multi-row insert)

도입

회원 한 명을 넣는 법을 배웠으니, 이제 명단 전체를 한 번에 옮겨 적는 법이 궁금해집니다. 회원 100명을 등록한다고 INSERT 를 100번 적는 것은 번거롭고 느립니다. 이 절에서는 한 번의 INSERT 로 **여러 행을 동시에** 넣는 방법을 배웁니다.

핵심 개념 설명

다중 행 삽입 (multi-row INSERT)

다중 행 삽입(multi-row INSERT) 이란, 하나의 INSERT 문에서 VALUES 뒤에 여러 묶음을 쉼표로 나열해 여러 행을 한 번에 추가하는 방법입니다. 명단을 한 줄씩 따로 보내는 대신, 한 봉투에 모아 한 번에 부치는 것과 같습니다.

- **왜(why) 쓰나:** 행마다 따로 INSERT 를 보내면 데이터베이스와 여러 번 왕복해 느립니다. 한 번에 묶어 보내면 훨씬 빠르고, 같은 시도로 묶이므로 관리도 쉽습니다.
- **언제(when) 쓰나:** 초기 샘플 데이터를 채울 때, 여러 건을 한꺼번에 등록할 때 유용합니다.
- **어떻게(how) 쓰나:** VALUES (행1), (행2), (행3) 처럼 행 묶음을 쉼표로 이어 적습니다.

컬럼 목록 지정 (column list)

컬럼 목록 지정(column list) 이란, INSERT INTO 테이블 (컬럼A, 컬럼B) 처럼 어떤 컬럼에 값을 넣을지 명시하는 것입니다. 목록에 적지 않은 컬럼은 기본값(DEFAULT)이나 NULL로 채워지므로, 꼭 필요한 컬럼만 골라 넣을 수 있습니다.

컬럼 순서를 테이블 정의 순서와 똑같이 맞춰야만 한다고 오해하기 쉽지만, 그렇지 않습니다. 컬럼 목록에 적은 순서대로 값을 주면 되므로, (email, name) 처럼 순서를 바꿔 적어도 값만 그 순서에 맞으면 됩니다.

이렇게 작성합니다

다음은 products 테이블에 상품 4건을 한 번에 등록하는 예시입니다.

```
-- 상품 4건을 한 번의 INSERT로 등록하기
INSERT INTO products (name, category, price, stock)
VALUES
  ('무선 마우스', '전자기기', 19900, 50),
  ('기계식 키보드', '전자기기', 89000, 20),
  ('머그컵', '주방용품', 8900, 100),
  ('노트', '문구', 3000, 200);
```

예상 화면:

```
INSERT 0 4
```

INSERT 0 4 는 한 번의 명령으로 4건이 삽입되었다는 뜻입니다. 행마다 따로 보냈다면 INSERT 0 1 이 네 번 나왔을 것입니다.

구문 요소 설명

구문	의미
(name, category, price, stock)	값을 채울 컬럼 4개를 지정합니다
VALUES (...), (...), ...	행 묶음을 심표로 이어 여러 행을 나열합니다
각 (...)	한 행에 해당하며, 컬럼 목록과 순서가 같아야 합니다
id 미포함	IDENTITY 컬럼이라 1, 2, 3, 4가 자동 부여됩니다

컬럼을 일부만 지정하면 나머지는 기본값으로 채워집니다. 다음은 `stock` 을 생략해 기본값 `0` 이 들어가게 한 예시입니다.

```
-- stock을 생략하면 DEFAULT 0이 채워집니다
INSERT INTO products (name, category, price)
VALUES ('텀블러', '주방용품', 15000);
```

예상 화면:

```
shop_lab=# SELECT name, stock FROM products WHERE name = '텀블러';
 name | stock
-----+-----
 텀블러 |      0
(1 row)
```

값을 주지 않은 `stock` 에 기본값 `0` 이 자동으로 들어간 것을 확인할 수 있습니다.

팁: 여러 행을 나열할 때는 예시처럼 행마다 줄을 바꾸고 값을 세로로 맞춰 정렬하면 읽기 쉽습니다. 어느 행에서 값 개수가 틀렸는지도 눈으로 금방 찾을 수 있습니다.

주의: 여러 행 중 한 행이라도 규칙을 어기면 전체가 실패합니다. 예를 들어 `NOT NULL` 컬럼에 값을 빠뜨린 행이 하나라도 있으면, 나머지 멀쩡한 행도 함께 삽입되지 않습니다. 다중 행 삽입은 한 묶음으로 처리되기 때문입니다.

절 요약

- VALUES (행1), (행2), ... 로 한 번에 여러 행을 삽입하면 빠르고 관리가 쉽습니다.
- 컬럼 목록으로 넣을 컬럼만 골라 지정하고, 생략한 컬럼은 기본값으로 채웁니다.
- 여러 행 중 하나라도 규칙을 어기면 전체가 함께 실패합니다.

RETURNING으로 삽입 결과 돌려받기

도입

방금 회원을 추가했는데, 자동으로 매겨진 `id` 가 몇 번인지 알고 싶을 때가 있습니다. 예를 들어 회원을 만들자마자 그 회원의 `id` 로 첫 주문을 연결해야 하는 경우입니다. 다시 `SELECT` 로 찾아볼 수도 있지만, 더 깔끔한 방법이 있습니다. 이 절에서 배우는 **RETURNING** 입니다.

핵심 개념 설명

RETURNING 절 (RETURNING clause) 예. 이미 delete된 레코드의 제목을 가져오고 싶음.

RETURNING 절(RETURNING clause) 이란, `INSERT` (또는 `UPDATE` · `DELETE`)를 실행하면서 **영향받은 행의 값을 곧바로 돌려받게 해 주는 절**입니다. 주문을 넣자마자 발급된 주문 번호를 영수증으로 바로 받아 보는 것과 같습니다.

- **왜(why) 쓰나:** IDENTITY로 자동 생성된 `id` 나 DEFAULT로 채워진 값은 삽입하기 전에는 알 수 없습니다. RETURNING 을 쓰면 삽입과 동시에 그 값을 받아, 다시 조회하러 가는 왕복을 줄일 수 있습니다.
- **언제(when) 쓰나:** 생성된 키를 바로 다음 작업에 써야 할 때, 삽입 결과를 화면에 즉시 보여 줘야 할 때 유용합니다.
- **어떻게(how) 쓰나:** `INSERT ... VALUES ...` 뒤에 RETURNING 컬럼들 을 붙입니다. RETURNING * 을 쓰면 삽입된 행의 모든 컬럼을 돌려받습니다.

삽입 후에는 반드시 다시 `SELECT` 해야만 생성된 `id` 를 알 수 있다고 오해하기 쉽지만, RETURNING 이 있으면 그럴 필요가 없습니다.

생성된 키 확인 (generated key)

생성된 키 확인(generated key) 이란, 데이터베이스가 자동으로 만들어 준 식별자(주로 IDENTITY 컬럼의 `id`)를 삽입 직후에 알아내는 것을 말합니다. 백엔드 개발에서는 "방금 만든 회원의 `id`로 다음 처리를 잇는" 상황이 매우 흔하므로, RETURNING 은 실무에서 자주 쓰입니다.

이렇게 작성합니다

다음은 회원을 추가하면서 자동 생성된 `id` 와 가입 시각을 곧바로 돌려받는 예시입니다.

```
-- 삽입과 동시에 생성된 id와 created_at 돌려받기
INSERT INTO members (name, email)
VALUES ('이서연', 'seoyeon@example.com')
RETURNING id, created_at;
```

예상 화면:

```
id |          created_at
---+-----
  2 | 2026-06-16 14:05:32.118233
(1 row)
INSERT 0 1
```

INSERT 인데도 마치 SELECT 처럼 결과 표가 함께 나옵니다. 자동으로 매겨진 id 가 2 이고, created_at 에 현재 시각이 채워진 것을 삽입 즉시 확인할 수 있습니다.

구문 요소 설명

구문	의미
RETURNING id, created_at	삽입된 행에서 이 두 컬럼 값을 돌려받습니다
RETURNING *	삽입된 행의 모든 컬럼을 돌려받습니다
결과 표	삽입된 행의 값으로 채워진 결과입니다
INSERT 0 1	삽입이 1건 성공했음을 함께 알려 줍니다

RETURNING 은 다중 행 삽입과도 함께 쓸 수 있습니다. 여러 행을 넣으면서 각 행의 생성된 id 를 모두 돌려받습니다.

```
-- 여러 행을 넣으면서 생성된 id를 모두 받기
INSERT INTO products (name, category, price)
VALUES
  ('손목 받침대', '전자기기', 12000),
  ('USB 허브', '전자기기', 23000)
RETURNING id, name;
```

예상 화면:

```

id |      name
---+-----
 6 | 손목 받침대
 7 | USB 허브
(2 rows)
INSERT 0 2

```

팁: 애플리케이션(예: 백엔드 서버) 코드에서 회원을 만든 직후 그 `id`가 필요할 때, `RETURNING id`는 가장 간단하고 안전한 방법입니다. "방금 넣은 행의 `id`"를 다른 방법으로 추측하면 동시에 여러 요청이 들어올 때 엉뚱한 값을 가져올 위험이 있습니다.

절 요약

- `RETURNING` 컬럼들 을 붙이면 삽입과 동시에 그 행의 값을 돌려받습니다.
- `IDENTITY-DEFAULT`로 자동 생성된 값을 다시 조회하지 않고 바로 알 수 있습니다.
- 다중 행 삽입에서도 각 행의 결과를 함께 돌려받습니다.

다른 테이블 데이터로 삽입(INSERT ... SELECT)

도입

```

insert into table name
select ....

```

지금까지는 값을 직접 손으로 적어 넣었습니다. 그런데 "기존 상품 중 전자기기만 골라 별도의 테이블에 옮기고 싶다"면 어떻게 할까요. 값을 일일이 다시 타이핑하는 대신, 조회 결과를 그대로 삽입에 흘려보낼 수 있습니다. 이 절에서 배우는 `INSERT ... SELECT`입니다.

핵심 개념 설명

조회 결과 삽입 (INSERT ... SELECT)

조회 결과 삽입(INSERT ... SELECT) 이란, `VALUES`로 값을 직접 적는 대신 `SELECT`로 조회한 결과를 그대로 다른 테이블에 넣는 방법입니다. 명부에서 조건에 맞는 사람만 골라 새 명단으로 베껴 적는 것과 같습니다.

- **왜(why) 쓰나:** 이미 데이터베이스 안에 있는 데이터를 옮기거나 복사할 때, 사람이 값을 다시 입력할 필요가 없습니다. 빠르고 정확합니다.
- **언제(when) 쓰나:** 특정 조건의 데이터를 백업용·이력용 테이블에 보관할 때, 한 테이블의 데이터를 다른 구조로 옮길 때 유용합니다.

- **어떻게(how) 쓰나:** INSERT INTO 대상 (컬럼들) 뒤에 VALUES 대신 SELECT ... 를 적습니다. SELECT 가 돌려주는 컬럼 순서·개수가 대상 컬럼 목록과 맞아야 합니다.

데이터 복사 (data copy)

데이터 복사(data copy)란, 한 테이블의 행을 골라 다른 테이블로 옮겨 적는 것을 말합니다. 원본은 그대로 남고, 대상 테이블에 같은 내용이 새 행으로 추가됩니다. WHERE 로 조건을 걸면 **일부만 골라** 복사할 수 있습니다.

이렇게 작성합니다

먼저 전자기기 상품만 따로 보관할 대상 테이블을 만들어 둡니다.

```
-- 전자기기 상품을 모아 둘 대상 테이블 (참고용)
CREATE TABLE electronics (
  id    integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name  varchar(100) NOT NULL,
  price numeric(10, 2) NOT NULL
);
```

이제 products 에서 카테고리가 전자기기인 행만 골라 electronics 로 복사합니다.

```
-- products에서 전자기기만 골라 electronics로 복사하기
INSERT INTO electronics (name, price)
SELECT name, price
FROM products
WHERE category = '전자기기';
```

예상 화면:

```
INSERT 0 4
```

INSERT 0 4 는 조건에 맞는 4건이 복사되어 들어갔다는 뜻입니다. 잘 옮겨졌는지 확인합니다.

```
-- 복사된 결과 확인하기
SELECT id, name, price FROM electronics;
```

예상 화면:

```

id | name      | price
---+-----+-----
1  | 무선 마우스 | 19900.00
2  | 기계식 키보드 | 89000.00
3  | 손목 받침대 | 12000.00
4  | USB 허브   | 23000.00
(4 rows)

```

구문 요소 설명

구문	의미
<code>INSERT INTO electronics (name, price)</code>	값을 받을 대상 테이블과 컬럼입니다
<code>SELECT name, price FROM products</code>	넣을 값을 만들어 내는 조회입니다
<code>WHERE category = '전자기기'</code>	복사할 행을 조건으로 좁힙니다
<code>VALUES</code> 없음	값 목록 대신 SELECT 결과가 그 자리를 채웁니다

대상 테이블의 `id`는 `electronics` 쪽에서 IDENTITY로 새로 매겨집니다. 즉, 원본 `products`의 `id`를 그대로 가져오는 것이 아니라, 새 테이블 기준으로 1부터 다시 부여됩니다.

주의: `SELECT`가 돌려주는 컬럼의 **개수·순서·자료형**이 `INSERT INTO ...` 뒤의 컬럼 목록과 맞아야 합니다. 예를 들어 `SELECT price, name` 처럼 순서가 뒤바뀌면, 가격이 이름 칸에 들어가려다 자료형이 안 맞아 오류가 나거나, 순 나쁘게 둘 다 문자열이면 엉뚱한 값이 조용히 들어갈 수 있습니다. 순서를 항상 확인합니다.

팁: 큰 테이블을 복사하기 전에는 `INSERT INTO ...` 부분을 떼고 `SELECT ...` 부분만 먼저 실행해 봅니다. 어떤 행이 몇 건 복사될지 눈으로 확인한 뒤 삽입하면, 의도와 다른 대량 복사를 막을 수 있습니다.

절 요약

- `INSERT INTO` 대상 (컬럼들) `SELECT ...` 로 조회 결과를 그대로 삽입합니다.
- `WHERE` 로 조건을 걸어 일부 행만 골라 복사할 수 있습니다.
- `SELECT`의 컬럼 개수·순서·자료형이 대상 컬럼 목록과 맞아야 합니다.

실습 과제

해보기: 회원·상품 샘플 데이터 채워 넣기

실습 데이터베이스 `shop_lab` 에서 이 장의 `INSERT` 를 직접 다뤄 봅니다.

- 단계 1: `members` 테이블에 다중 행 `INSERT` 로 회원 5명을 한 번에 등록합니다. `name` 과 `email` 만 지정하고 `points` 는 기본값에 맡깁니다.
- 단계 2: `products` 테이블에 상품 6건을 등록하되, 그중 한 건은 `RETURNING id, name` 을 붙여 생성된 `id` 를 화면에서 확인합니다.
- 단계 3: `electronics` 테이블을 만들고, `INSERT ... SELECT` 로 `products` 에서 가격이 20000 원 이상인 상품만 골라 복사합니다.
- 점검: `SELECT count(*) FROM members;` 와 `SELECT count(*) FROM electronics;` 로 들어간 행 수가 의도와 맞는지 확인합니다. 복사 전에 `SELECT` 부분만 먼저 실행해 대상 건수를 미리 세어 봅니다.

FAQ

Q1. `id` 컬럼에 값을 직접 넣어도 되나요? → A1. `GENERATED ALWAYS AS IDENTITY` 로 만든 컬럼은 사용자가 값을 직접 넣으면 오류가 납니다(자동 부여가 원칙이기 때문입니다). 굳이 직접 넣어야 한다면 `OVERRIDING SYSTEM VALUE` 를 써야 하지만, 입문 단계에서는 `id` 는 데이터베이스에 맡기고 값을 주지 않는 것이 안전합니다.

Q2. 삽입할 때 따옴표는 작은따옴표인가요 큰따옴표인가요? → A2. 문자열 값은 항상 **작은따옴표** `'...'` 로 감쌉니다. 큰따옴표 `"..."` 는 PostgreSQL에서 컬럼이나 테이블 같은 식별자 이름을 가리키므로, 값에 쓰면 "그런 컬럼이 없다"는 오류가 납니다. 값 안에 작은따옴표가 들어가야 하면 `''` 처럼 두 번 적습니다(예: `'O''Brien'`).

Q3. 같은 데이터를 또 `INSERT` 하면 어떻게 되나요? → A3. `INSERT` 는 기존 행을 확인하지 않고 무조건 새 행을 추가하므로, 같은 내용이 한 줄 더 생겨 중복됩니다. 중복을 막으려면 9장에서 배울 `UNIQUE` 제약을 걸거나, 7장에서 배울 `INSERT ... ON CONFLICT (UPSERT)` 로 "있으면 넘어가거나 수정"하도록 처리합니다.

Q4. `INSERT 0 1` 에서 앞의 `0` 은 무슨 뜻인가요? → A4. 과거 PostgreSQL이 행의 내부 위치(OID)를 표시하던 자리로, 지금은 거의 항상 `0` 입니다. 우리가 의미 있게 봐야 할 값은 뒤의 숫자, 즉 **실제로 삽입된 행 수**입니다.

장 요약

이 장에서는 CRUD의 첫 단계인 데이터 생성(Create)을 INSERT 로 배웠습니다. INSERT INTO ... VALUES 로 한 행을 넣고, VALUES 에 여러 묶음을 나열해 다중 행을 한 번에 삽입하는 법을 익혔습니다. RETURNING 으로 삽입과 동시에 자동 생성된 id 를 돌려받았고, INSERT ... SELECT 로 다른 테이블의 데이터를 조건에 맞게 골라 복사하는 법까지 다루었습니다. INSERT 는 모든 데이터의 출발점이므로, 이후 배울 조회·수정·삭제는 모두 이 장에서 채워 넣은 데이터를 대상으로 합니다.

핵심 키워드

INSERT INTO, VALUES, 다중 행 삽입(multi-row INSERT), 컬럼 목록(column list), RETURNING, INSERT ... SELECT, CRUD 생명주기(CRUD lifecycle)

다음 장 미리보기

다음 장에서는 이렇게 채워 넣은 데이터를 다시 꺼내 읽는 SELECT 를 배웁니다. 원하는 컬럼만 고르고, WHERE 로 조건에 맞는 행만 추리며, 정렬과 개수 제한까지 다루는 조회(Read)의 기본기를 익힙니다.

SELECT 기초: 데이터 조회(Read)

학습 목표 - **SELECT**의 기본 구조와 논리적 실행 순서를 설명할 수 있습니다. - **WHERE**와 비교·논리 연산자, **BETWEEN·IN·LIKE**로 행을 필터링할 수 있습니다. - **ORDER BY**로 여러 기준 정렬과 **NULL** 정렬을 다룰 수 있습니다. - **LIMIT·OFFSET**으로 가져올 행 수를 제한할 수 있습니다.

앞 장에서 우리는 테이블에 데이터를 채워 넣었습니다(생성, Create). 그런데 데이터는 넣어 두기만 해서는 쓸모가 없습니다. 필요할 때 원하는 부분만 골라 **꺼내 읽을 수 있어야** 비로소 가치가 생깁니다. 이 장에서는 CRUD의 두 번째 단계인 조회(Read), 즉 **SELECT** 를 배웁니다.

SELECT 는 도서관 서가에서 조건에 맞는 책만 골라 뽑아 오는 일과 같습니다. "어느 서가에서 (**FROM**), 어떤 조건의 책을(**WHERE**), 어떤 정보만(**SELECT** 컬럼), 어떤 순서로(**ORDER BY**), 몇 권까지(**LIMIT**)" 가져올지 정하는 것이 이 장의 전부입니다. 백엔드 개발에서 가장 자주 쓰는 명령이 바로 **SELECT** 이므로, 여기서 기본기를 단단히 다져 두면 이후 모든 장이 한결 쉬워집니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 에서 **psql** 로 PostgreSQL에 접속한 상태를 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, **psql** 은 그 안에서 데이터베이스와 대화하는 도구입니다. 화면 앞의 **shop_lab=#** 표시는 "지금 **shop_lab** 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 모두 아래 **products** 테이블과, 거기에 채워진 샘플 데이터를 기준으로 합니다. 앞 장에서 만들고 채운 것과 같은 모양이라고 생각하면 됩니다.

```
-- 이 장 예시가 기준으로 삼는 상품 테이블 (참고용)
CREATE TABLE products (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name    varchar(100) NOT NULL,
  category varchar(30),
  price   numeric(10, 2) NOT NULL,
  stock   integer DEFAULT 0
);
```

채워진 샘플 데이터는 다음과 같다고 가정합니다. 이 표를 옆에 두고 각 조회 결과와 비교하면 이해가 빠릅니다.

id	name	category	price	stock
1	무선 마우스	전자기기	19900.00	50
2	기계식 키보드	전자기기	89000.00	20
3	머그컵	주방용품	8900.00	100
4	노트	문구	3000.00	200
5	텀블러	주방용품	15000.00	0
6	USB 허브	전자기기	23000.00	35
7	볼펜 세트	문구	4500.00	0

(7 rows)

SELECT 기본 구조와 컬럼 선택

도입

서가에서 책을 찾을 때 우리는 보통 "어느 책장에서, 어떤 정보를 볼지"부터 정합니다. `SELECT` 도 똑같습니다. 이 절에서는 가장 기본이 되는 "어느 테이블에서(`FROM`) 어떤 컬럼을(`SELECT`) 가져올지"를 익히고, 컬럼에 별명을 붙이거나 값을 계산해 새 컬럼을 만드는 법까지 살펴봅니다. 또한 작성하는 순서와 데이터베이스가 실제로 처리하는 순서가 다르다는 중요한 사실도 짚습니다.

핵심 개념 설명

조회 (SELECT) 메모리에 테이블 올림 => 결과 화면

조회(SELECT) 란, 테이블에서 원하는 컬럼과 행을 골라 읽어 오는 SQL 명령입니다. CRUD에서 읽기(Read)에 해당하며, 도서관 서가에서 조건에 맞는 책만 골라 뽑아 오는 일과 같습니다.

가장 기본 형태는 `SELECT 컬럼들 FROM 테이블` 입니다. "무엇을(컬럼) 어디에서(테이블) 가져올까"를 알려 주는 구조입니다.

- **왜(why) 필요한가:** 데이터는 넣어 두는 것 자체가 목적이 아니라, 필요할 때 꺼내 보고 활용하기 위해 저장합니다. 화면에 상품 목록을 보여 주는 일, 통계를 내는 일, 다른 작업의 입력값을 얻는 일 모두 `SELECT` 에서 출발합니다.
- **언제(when) 쓰나:** 데이터를 화면에 표시할 때, 조건에 맞는 데이터가 있는지 확인할 때, 다른 명령(수정·삭제·복사)의 대상을 미리 점검할 때 등 거의 모든 순간입니다.
- **어떻게(how) 쓰나:** `SELECT` 뒤에 보고 싶은 컬럼을 쉼표로 나열하고, `FROM` 뒤에 테이블 이름을 적습니다. 모든 컬럼을 보려면 `*` 를 씁니다.

입문자가 자주 하는 오해 하나를 짚겠습니다. `SELECT` 가 데이터를 바꾼다고 생각하는 경우입니다. `SELECT` 는 데이터를 읽기만 할 뿐, 테이블의 내용을 절대 변경하지 않습니다. 몇 번을 조회해도 원본은 그대로입니다.

컬럼 투영 (column projection)

컬럼 투영(column projection) 이란, 테이블의 여러 컬럼 중에서 **필요한 컬럼만 골라** 결과로 가져오는 것을 말합니다. 큰 표에서 보고 싶은 세로줄(열)만 오려 내는 것과 같습니다.

`SELECT *` 로 모든 컬럼을 가져올 수도 있지만, 화면에 꼭 필요한 컬럼만 골라 적으면 결과가 깔끔하고 데이터 전송량도 줄어듭니다. `SELECT *` 가 항상 최선이라고 오해하기 쉽지만, 실무에서는 필요한 컬럼만 명시하는 편이 더 안전하고 빠릅니다.

작성 순서와 논리적 실행 순서

`SELECT` 문은 우리가 **적는 순서**와 데이터베이스가 **처리하는 순서**가 다릅니다. 우리는 `SELECT` 를 맨 앞에 적지만, 데이터베이스는 먼저 테이블을 정하고(`FROM`), 행을 거르고(`WHERE`), 그다음에 컬럼을 고르고(`SELECT`), 정렬하고(`ORDER BY`), 마지막으로 개수를 제한합니다(`LIMIT`). 이 순서를 알아 두면 "왜 `WHERE`에서 별칭을 못 쓰지?" 같은 의문이 자연스럽게 풀립니다.



ch_04_s01 · SELECT 기본 구조와 컬럼 선택

작성 순서 vs 논리적 실행 순서

단계	절	하는 일
1	FROM	어느 테이블에서 가져올지 정합니다
2	WHERE	조건에 맞는 행만 거릅니다
3	SELECT	보여 줄 컬럼을 고르고 별칭을 붙입니다
4	ORDER BY	결과를 정렬합니다
5	LIMIT / OFFSET	가져올 행 수를 제한합니다

이렇게 작성합니다

다음은 `products` 테이블에서 상품 이름과 가격 두 컬럼만 골라 조회하는 예시입니다.

```
-- 상품 이름과 가격만 골라 조회하기
SELECT name, price
FROM products;
```

예상 화면:

```

name      | price
-----+-----
무선 마우스 | 19900.00
기계식 키보드 | 89000.00
머그컵      |  8900.00
노트        |  3000.00
텀블러      | 15000.00
USB 허브    | 23000.00
볼펜 세트   |  4500.00
(7 rows)
```

`id` · `category` · `stock` 은 결과에 나오지 않습니다. `SELECT` 에 적은 `name` 과 `price` 두 컬럼만 골라 (투영) 가져왔기 때문입니다. 맨 아래 `(7 rows)` 는 가져온 행이 7건이라는 안내입니다.

구문 요소 설명

구문	의미
<code>SELECT name, price</code>	결과에 포함할 컬럼을 심표로 나열합니다
<code>FROM products</code>	데이터를 가져올 테이블을 지정합니다
<code>SELECT *</code>	(대안) 모든 컬럼을 가져옵니다
끝의 <code>;</code>	한 SQL 문이 끝났음을 알리는 세미콜론입니다

별칭(AS)을 쓰면 결과 컬럼의 머리글을 보기 좋은 이름으로 바꾸거나, 계산으로 만든 새 컬럼에 이름을 붙일 수 있습니다. 다음은 재고 가치를 즉석에서 계산해 새 컬럼으로 보여 주는 예시입니다.

```
-- 별칭(AS)으로 머리글 바꾸기 + 계산 컬럼 만들기
SELECT name      AS 상품명,
       price * stock AS 재고가치
FROM products;
```

예상 화면:

상품명	재고가치
무선 마우스	995000.00
기계식 키보드	1780000.00
머그컵	890000.00
노트	600000.00
텀블러	0.00
USB 허브	805000.00
볼펜 세트	0.00

(7 rows)

`price * stock` 은 테이블에 실제로 저장된 컬럼이 아니라, 조회하는 순간 계산해 만든 값입니다. 이렇게 만든 컬럼에 `AS 재고가치` 로 이름을 붙이면 결과 머리글이 읽기 쉬워집니다.

구문 요소 설명

구문	의미
<code>name AS 상품명</code>	<code>name</code> 컬럼을 결과에서 <code>상품명</code> 이라는 머리글로 보여 줍니다
<code>price * stock</code>	두 컬럼을 곱해 만든 계산 컬럼입니다
<code>AS 재고가치</code>	계산 컬럼에 붙인 별칭(머리글)입니다

팁: AS 는 생략할 수 있어 `name` 상품명 처럼만 적어도 같은 결과가 나옵니다. 다만 입문 단계에서는 AS 를 적어 두면 "이건 별칭이다"가 한눈에 보여 읽기 쉽습니다.

주의: SELECT 에서 붙인 별칭은 같은 문장의 WHERE 에서는 쓸 수 없습니다. 앞에서 본 실행 순서 때문입니다. WHERE (2단계)가 SELECT (3단계)보다 먼저 처리되므로, WHERE 시점에는 별칭이 아직 만들어지지 않았습니니다. 조건에는 별칭 대신 원래 식(`price * stock`)을 그대로 적어야 합니다.

절 요약

- SELECT 컬럼들 FROM 테이블 로 원하는 컬럼만 골라(투영) 조회합니다.
- AS 로 결과 머리글을 바꾸거나 계산 컬럼에 이름을 붙일 수 있습니다.
- 작성 순서(SELECT 먼저)와 실행 순서(FROM → WHERE → SELECT ...)는 다릅니다.
- SELECT 는 데이터를 읽기만 하고 원본을 바꾸지 않습니다.

WHERE로 조건 필터링하기

도입

상품이 7건뿐이면 전체를 봐도 되지만, 수만 건이라면 이야기가 다릅니다. 보통은 "가격이 2만원 이하인 것", "전자기기 카테고리인 것"처럼 **조건에 맞는 일부만** 보고 싶습니다. 쇼핑몰에서 '5만 원 이하' 필터를 켜는 것과 같은 일을, SQL에서는 WHERE 로 합니다. 이 절에서는 비교·논리 연산자부터 BETWEEN · IN · LIKE · IS NULL 같은 자주 쓰는 패턴까지 익힙니다.

핵심 개념 설명

조건 필터 (WHERE)

조건 필터(WHERE) 란, 조건식을 만족하는 행만 골라내도록 결과를 거르는 절입니다. 쇼핑몰에서 '5만 원 이하' 같은 필터를 켜서 조건에 맞는 상품만 남기는 것과 같습니다.

- **왜(why) 쓰나:** 필요한 행만 추리면 결과를 빠르게 파악할 수 있고, 데이터가 많을수록 전송량과 처리 시간도 줄어듭니다. 수정·삭제에서 **WHERE** 는 더욱 중요해, "어떤 행을 바꿀지" 정확히 좁히는 안전장치가 됩니다.
- **언제(when) 쓰나:** 특정 카테고리·가격대·기간의 데이터만 볼 때, 조건에 맞는 데이터가 존재하는지 확인할 때 씁니다.
- **어떻게(how) 쓰나:** **FROM** 테이블 뒤에 **WHERE** 조건식 을 적습니다. 조건식이 참(true)인 행만 결과에 남습니다.

비교·논리 연산자 (comparison / logical operator)

비교 연산자는 두 값을 견주어 참·거짓을 가립니다. 같음 **=**, 다름 **<>** (또는 **!=**), 큼 **>**, 작음 **<**, 크거나 같음 **>=**, 작거나 같음 **<=** 가 있습니다. **논리 연산자**는 여러 조건을 엮습니다. 둘 다 참이어야 하면 **AND**, 하나만 참이면 되면 **OR**, 조건을 뒤집으면 **NOT** 입니다.

여기에 자주 쓰는 패턴 연산자가 더해집니다.

자주 쓰는 조건 패턴

패턴	의미	예시
BETWEEN a AND b	a 이상 b 이하(양 끝 포함)	<code>price BETWEEN 5000 AND 20000</code>
IN (...)	목록 안의 값 중 하나와 일치	<code>category IN ('전자기기', '문구')</code>
LIKE	문자열 패턴 일치(%는 임의 글자들)	<code>name LIKE '무선%'</code>
IS NULL	값이 없음(NULL)인지 검사	<code>category IS NULL</code>

입문자가 자주 하는 오해 하나를 짚겠습니다. **NULL**을 **= NULL** 로 비교하려는 경우입니다. **NULL**은 "값이 없음"이라 같다·다르다를 따질 수 없어, **= NULL** 은 참도 거짓도 아닌 알 수 없음으로 처리되어 **아무 행도 걸리지 않습니다**. **NULL**은 반드시 **IS NULL** 또는 **IS NOT NULL** 로 검사합니다.

이렇게 작성합니다

먼저 가장 단순한 비교 조건입니다. 가격이 2만 원 이하인 상품만 골라 봅니다.

```
-- 가격이 20000원 이하인 상품만 조회하기
SELECT name, price
FROM products
WHERE price <= 20000;
```

예상 화면:

name	price
무선 마우스	19900.00
머그컵	8900.00
노트	3000.00
텀블러	15000.00
볼펜 세트	4500.00

(5 rows)

조건을 만족하는 5건만 남았습니다. price 가 20000을 넘는 기계식 키보드(89000)와 USB 허브(23000)는 걸러졌습니다.

여러 조건은 AND · OR 로 엮습니다. 다음은 "전자기기이면서 가격이 2만 원 이상"인 상품입니다.

```
-- 전자기기 카테고리이면서 가격이 20000원 이상인 상품
SELECT name, category, price
FROM products
WHERE category = '전자기기' AND price >= 20000;
```

예상 화면:

name	category	price
기계식 키보드	전자기기	89000.00
USB 허브	전자기기	23000.00

(2 rows)

두 조건을 **모두** 만족하는 행만 남습니다. 무선 마우스는 전자기기지만 가격이 19900원이라 두 번째 조건에서 걸러졌습니다.

구문 요소 설명

구문	의미
<code>WHERE category = '전자기기'</code>	카테고리가 정확히 '전자기기'인 행
<code>AND price >= 20000</code>	그리고 가격이 20000 이상인 행
<code>=</code>	같음 비교(대입이 아님)
<code>>=</code>	크거나 같음

이제 패턴 연산자를 봅니다. `BETWEEN` 은 범위를, `IN` 은 여러 후보 값을, `LIKE` 는 문자열 패턴을 다룹니다.

```
-- 가격이 5000~20000원이고, 주방용품 또는 문구인 상품
SELECT name, category, price
FROM products
WHERE price BETWEEN 5000 AND 20000
      AND category IN ('주방용품', '문구');
```

예상 화면:

```
name | category | price
-----+-----+-----
머그컵 | 주방용품 | 8900.00
텀블러 | 주방용품 | 15000.00
(2 rows)
```

`BETWEEN 5000 AND 20000` 은 5000 이상 20000 이하(양 끝 포함)를 뜻합니다. `IN ('주방용품', '문구')` 는 둘 중 하나와 일치하면 통과입니다. 노트(3000원)와 볼펜 세트(4500원)는 문구지만 가격이 5000원 미만이라 빠졌습니다.

다음은 `LIKE` 로 이름이 특정 글자로 시작하는 상품을 찾는 예시입니다.

```
-- 이름이 '무선'으로 시작하는 상품 찾기
SELECT name, price
FROM products
WHERE name LIKE '무선%';
```

예상 화면:

```

name      | price
-----+-----
무선 마우스 | 19900.00
(1 row)

```

%는 "임의의 글자 0개 이상"을 뜻하는 자리표시입니다. '무선%'는 "무선으로 시작하고 뒤에 무엇이 오든 상관없음"이라는 의미입니다. 한 글자만 대신하려면 밑줄 `_`를 씁니다.

팁: 대소문자를 가리지 않고 문자열을 찾고 싶으면 `ILIKE`를 씁니다. 예를 들어 `name ILIKE 'usb%'`는 'USB 허브'도 찾아냅니다. `LIKE`는 대소문자를 구분하지만 `ILIKE`는 구분하지 않습니다(PostgreSQL 전용 기능입니다).

주의: `AND`와 `OR`를 섞어 쓸 때는 괄호로 묶어 의도를 분명히 합니다. `OR`보다 `AND`가 먼저 묶이므로, `category = '문구' OR category = '주방용품' AND price < 5000`은 "문구 전체, 또는 (주방용품이면서 5000원 미만)"으로 해석되어 의도와 달라질 수 있습니다. (`category = '문구' OR category = '주방용품' AND price < 5000`처럼 괄호로 묶어야 안전합니다).

절 요약

- `WHERE` 조건식 으로 조건을 만족하는 행만 골라냅니다.
- 비교 연산자(`=`, `<>`, `>`, `<` 등)와 논리 연산자(`AND`, `OR`, `NOT`)를 조합합니다.
- 범위는 `BETWEEN`, 여러 후보는 `IN`, 문자열 패턴은 `LIKE` / `ILIKE`로 다룹니다.
- `NULL`은 `= NULL`이 아니라 `IS NULL` / `IS NOT NULL`로 검사합니다.

ORDER BY로 정렬하기

도입

정렬

조건에 맞는 상품을 골랐다면, 다음 관심사는 "어떤 순서로 보여 줄까"입니다. **비싼 순서**, **이름 가나다순**, **재고가 많은 순**처럼 보기 좋은 차례로 줄을 세우는 일을 `ORDER BY`가 맡습니다. 이 절에서는 오름·내림차순 지정, 여러 컬럼을 기준으로 한 정렬, 그리고 `NULL`이 섞였을 때 그 위치를 다루는 법을 익힙니다.

핵심 개념 설명

정렬 (ORDER BY)

정렬(ORDER BY) 이란, **조회 결과의 행을 지정한 컬럼 값 기준으로 일정한 순서대로 줄 세우는** 작업입니다. 책을 제목순이나 출간일순으로 서가에 꽂는 것과 같습니다.

- **왜(why) 쓰나:** 데이터베이스에 저장된 행에는 정해진 순서가 없습니다. ORDER BY 를 쓰지 않으면 결과 순서가 그때그때 달라질 수 있어, 사람이 보기에나 프로그램이 처리하기에도 일정한 순서가 필요합니다.
- **언제(when) 쓰나:** 목록을 가격순·최신순으로 보여 줄 때, "가장 비싼 상품"처럼 순위를 매길 때 씁니다.
- **어떻게(how) 쓰나:** ORDER BY 컬럼 뒤에 **오름차순은 ASC**, **내림차순은 DESC** 를 붙입니다. 생략하면 **기본은 오름차순(ASC)**입니다.

오름·내림차순 (ASC / DESC)

오름차순(ASC) 은 작은 값에서 큰 값으로(1, 2, 3 / 가, 나, 다), **내림차순(DESC)** 은 큰 값에서 작은 값으로(3, 2, 1) 정렬합니다. 여러 컬럼을 적으면 **앞 컬럼이 우선**이고, 앞 컬럼 값이 같을 때만 뒤 컬럼으로 순서를 가립니다. 동점자를 가리는 2차 기준과 같습니다.

이렇게 작성합니다

먼저 가격이 비싼 순서(내림차순)로 정렬해 봅니다.

```
-- 가격이 비싼 순서로 정렬하기
SELECT name, price
FROM products
ORDER BY price DESC;
```

예상 화면:

name	price
기계식 키보드	89000.00
USB 허브	23000.00
무선 마우스	19900.00
텀블러	15000.00
머그컵	8900.00
볼펜 세트	4500.00
노트	3000.00

(7 rows)

price DESC 라서 가장 비싼 기계식 키보드가 맨 위에, 가장 싼 노트가 맨 아래에 옵니다.

여러 컬럼으로 정렬하면 동점 상황을 깔끔하게 정리할 수 있습니다. 다음은 카테고리를 가나다 순(오름차순)으로 묶고, 같은 카테고리 안에서는 가격이 비싼 순으로 정렬합니다.

```
-- 카테고리 오름차순, 같은 카테고리 안에서는 가격 내림차순
SELECT category, name, price
FROM products
ORDER BY category ASC, price DESC;
```

예상 화면:

category	name	price
문구	볼펜 세트	4500.00
문구	노트	3000.00
전자기기	기계식 키보드	89000.00
전자기기	USB 허브	23000.00
전자기기	무선 마우스	19900.00
주방용품	텀블러	15000.00
주방용품	머그컵	8900.00

(7 rows)

먼저 카테고리가 문구→전자기기→주방용품 순으로 묶이고(1차 기준), 같은 카테고리 안에서만 가격이 비싼 순으로 정렬됩니다(2차 기준).

구문 요소 설명

구문	의미
ORDER BY category ASC	카테고리를 오름차순으로 정렬(1차 기준)
, price DESC	카테고리가 같을 때 가격 내림차순(2차 기준)
ASC	오름차순(생략 시 기본값)
DESC	내림차순

NULL이 섞이면 정렬에서 어디에 놓일지가 문제입니다. PostgreSQL에서는 오름차순(ASC)일 때 NULL이 **맨 뒤**, 내림차순(DESC)일 때 **맨 앞**에 옵니다. 이를 직접 지정하려면 **NULLS FIRST** 또는 **NULLS LAST**를 씁니다.

① 값이 없다

② 의미적으로
무한대 의미?

③ 연산에서 제외

```
-- 카테고리 정렬에서 NULL을 항상 맨 뒤로 보내기
SELECT name, category
FROM products
ORDER BY category ASC NULLS LAST;
```

예상 화면:

name	category
노트	문구
볼펜 세트	문구
무선 마우스	전자기기
기계식 키보드	전자기기
USB 허브	전자기기
머그컵	주방용품
텀블러	주방용품

(7 rows)

지금 샘플에는 `category` 가 NULL인 행이 없어 결과 위치 변화는 보이지 않지만, NULL인 행이 있었다면 `NULLS LAST` 덕분에 항상 목록의 맨 끝에 모였을 것입니다.

팁: 정렬 기준으로 컬럼 이름 대신 `SELECT` 목록에서의 순번을 쓸 수도 있습니다. 예를 들어 `ORDER BY 2 DESC` 는 "두 번째로 적은 컬럼 기준 내림차순"을 뜻합니다. 다만 순번은 나중에 컬럼 순서가 바뀌면 엉뚱한 기준이 되므로, 입문 단계에서는 컬럼 이름을 그대로 적는 편을 권합니다.

주의: 정렬 기준이 같은 행들(동점)의 내부 순서는 보장되지 않습니다. 매번 같은 차례를 원한다면 마지막에 `id` 처럼 값이 겹치지 않는 컬럼을 정렬 기준에 추가해 "동점을 가리는 최종 기준"을 두는 것이 안전합니다.

절 요약

- `ORDER BY` 컬럼 `ASC|DESC` 로 결과를 일정한 순서로 줄 세웁니다.
- 여러 컬럼을 적으면 앞 컬럼이 우선이고, 같을 때만 뒤 컬럼으로 가립니다.
- NULL 위치는 `NULLS FIRST / NULLS LAST` 로 지정할 수 있습니다.
- 정렬 없이는 결과 순서가 보장되지 않습니다.

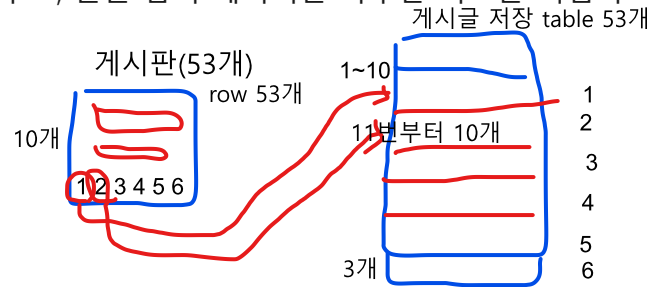
LIMIT·OFFSET으로 행 수 제한하기

도입

상품이 수천 건이라면 한 화면에 다 보여 줄 수 없습니다. 검색 결과를 "한 페이지에 10개씩" 끊어 보여 주듯, 가져올 행 수를 제한하는 일이 필요합니다. 이 절에서는 **상위 N건만 가져오는 LIMIT** 과, **앞쪽 몇 건을 건너뛰는 OFFSET** 을 배우고, 둘을 합쳐 페이지를 나누는 기초를 익힙니다.

핵심 개념 설명

행 제한 (LIMIT)



행 제한(LIMIT) 이란, 조회 결과에서 가져올 행의 **최대 개수를 제한**하는 절입니다. 검색 결과를 '한 페이지에 10개씩'으로 끊어 보여 주는 것과 같습니다.

- **왜(why) 쓰나:** 전체를 다 가져오면 화면도 메모리도 부담됩니다. "가장 비싼 상품 3개"처럼 상위 일부만 필요할 때, **LIMIT** 으로 딱 그만큼만 가져옵니다.
- **언제(when) 쓰나:** 순위 상위 N건을 볼 때, 목록을 페이지로 나눠 보여 줄 때 씁니다.
- **어떻게(how) 쓰나:** 문장 맨 뒤에 **LIMIT** 개수 를 붙입니다. 보통 **ORDER BY** 와 함께 써서 "정렬한 결과의 위에서 N건"을 가져옵니다.

건너뛰기 (OFFSET)

건너뛰기(OFFSET) 란, **결과의 앞쪽 몇 건을 건너뛰고 그다음부터 가져오게 하는 절**입니다.

OFFSET 10 은 "앞 10건은 빼고 11번째부터"라는 뜻입니다. **LIMIT** 과 함께 쓰면 "몇 건을 건너뛰 뒤 몇 건을 가져올지"가 정해져, 페이지 나누기가 됩니다.

입문자가 자주 하는 오해 하나를 짚겠습니다. **ORDER BY** 없이 **LIMIT** 만 써도 순서가 항상 같다고 생각하는 경우입니다. 정렬이 없으면 결과 순서 자체가 보장되지 않으므로, "상위 N건"이 매번 달라질 수 있습니다. **LIMIT** 은 거의 항상 **ORDER BY** 와 짝지어 써야 의미가 분명해집니다.

이렇게 작성합니다

다음은 가격이 비싼 상위 3개 상품만 가져오는 예시입니다.

```
-- 가격이 비싼 상위 3개 상품만 가져오기
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 3;
```

예상 화면:

```

      name      | price
-----+-----
기계식 키보드   | 89000.00
USB 허브       | 23000.00
무선 마우스    | 19900.00
(3 rows)

```

ORDER BY price DESC 로 비싼 순으로 줄 세운 뒤, LIMIT 3 으로 위에서 3건만 가져왔습니다. 정렬이 먼저, 개수 제한이 나중임에 주목합니다(앞 절의 실행 순서 그대로입니다).

이제 OFFSET 을 더해 "페이지"를 나눠 봅니다. 한 페이지에 3건씩 보여 준다고 할 때, 2페이지 (4~6번째)는 앞 3건을 건너뛰고 3건을 가져옵니다.

```

-- 가격 내림차순으로 2페이지(4~6번째) 가져오기 (페이지당 3건)
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 3 OFFSET 3;

```

예상 화면:

```

      name      | price
-----+-----
텀블러        | 15000.00
머그컵        | 8900.00
볼펜 세트     | 4500.00
(3 rows)

```

OFFSET 3 으로 비싼 순 상위 3건(키보드·허브·마우스)을 건너뛰고, 그다음 3건을 가져왔습니다. 이렇게 OFFSET 을 0, 3, 6...으로 늘리면 1페이지, 2페이지, 3페이지가 됩니다.

구문 요소 설명

구문	의미
LIMIT 3	최대 3건만 가져옵니다
OFFSET 3	앞 3건을 건너뛸니다
LIMIT 3 OFFSET 3	4~6번째 행, 즉 페이지당 3건일 때 2페이지
(페이지 공식)	OFFSET = (페이지번호 - 1) × 페이지당건수

팁: 페이지를 나눌 때 `OFFSET` 은 "(페이지 번호 - 1) × 페이지당 건수"로 계산합니다. 페이지당 10건이면 1페이지는 `OFFSET 0`, 2페이지는 `OFFSET 10`, 3페이지는 `OFFSET 20` 입니다.

주의: `OFFSET` 은 건너뛴 행을 일단 세고 버리는 방식이라, 뒤쪽 페이지로 갈수록(예: `OFFSET 100000`) 느려집니다. 데이터가 아주 많을 때는 이 방식이 비효율적이며, 12장에서 배울 키셋 페이지네이션(keyset pagination)이 더 나은 대안입니다. 입문 단계의 작은 데이터에서는 `LIMIT · OFFSET` 만으로 충분합니다.

절 요약

- `LIMIT` 개수 로 가져올 행의 최대 수를 제한합니다.
- `OFFSET` 개수 로 앞쪽 행을 건너뛰며, `LIMIT` 과 함께 페이지를 나눕니다.
- `LIMIT` 은 거의 항상 `ORDER BY` 와 함께 써야 순서가 일정합니다.
- `OFFSET` 은 뒤쪽 페이지일수록 느려지므로 큰 데이터에선 주의합니다.

실습 과제

해보기: 상품 카탈로그 조회 쿼리 모음 만들기

실습 데이터베이스 `shop_lab` 의 `products` 테이블에서 이 장의 `SELECT` 를 직접 다뤄 봅니다. 결과를 표로 기록해 두고 조건을 바꿔 가며 차이를 비교합니다.

- 단계 1(투영): `name · category · price` 세 컬럼만 골라 전체를 조회합니다. 이어 `price` 를 `price * 1.1 AS` 인상이 처럼 계산 컬럼으로 바꿔 봅니다.
- 단계 2(필터): 가격이 5000원 이상 30000원 이하(`BETWEEN`)이면서 카테고리가 전자기기 또는 주방용품(`IN`)인 상품을 조회합니다.
- 단계 3(패턴): 이름에 특정 글자가 들어가는 상품을 `LIKE '%세트%'` 처럼 찾아 봅니다.
- 단계 4(정렬): 카테고리 오름차순, 같은 카테고리 안에서는 재고(`stock`) 내림차순으로 정렬합니다.
- 단계 5(제한): 가격이 비싼 상위 3건을 `ORDER BY price DESC LIMIT 3` 으로 가져오고, `OFFSET 3` 을 더해 다음 3건과 비교합니다.
- 점검: 각 쿼리의 결과 행 수가 손으로 센 예상과 맞는지 확인합니다. `WHERE` 를 뺐을 때와 넣었을 때 행 수가 어떻게 달라지는지 적어 둡니다.

FAQ

Q1. `SELECT *` 와 컬럼을 직접 적는 것 중 무엇이 더 좋나요? → A1. 빠르게 데이터를 둘러볼 때는 `SELECT *` 가 편하지만, 실제 서비스 코드에서는 필요한 컬럼만 적는 편이 좋습니다. 전송량이 줄고, 테이블에 컬럼이 추가되어도 결과 구조가 흔들리지 않으며, 쿼리만 봐도 무엇을 쓰는지 분명합니다. 입문 실습에서는 `*` 로 살펴본 뒤, 정리할 때 필요한 컬럼만 남기는 습관을 권합니다.

Q2. `WHERE name = '머그컵'` 인데 결과가 안 나옵니다. 왜 그런가요? → A2. 흔한 원인은 두 가지입니다. 첫째, 앞뒤 공백처럼 보이지 않는 글자가 값에 섞여 정확히 일치하지 않는 경우입니다. 둘째, 대소문자나 철자가 실제 저장 값과 다른 경우입니다. 정확히 일치하지 않아도 찾고 싶다면 `LIKE '%머그%'` 처럼 부분 일치를 쓰고, `NULL`을 찾을 때는 `=` 이 아니라 `IS NULL` 을 씁니다.

Q3. `ORDER BY` 와 `LIMIT` 은 어느 쪽을 먼저 적나요? → A3. 작성 순서는 `WHERE` → `ORDER BY` → `LIMIT` 순으로 적고, 데이터베이스도 정렬을 먼저 한 뒤 개수를 제한합니다. 그래서 `ORDER BY price DESC LIMIT 3` 은 "비싼 순으로 정렬한 결과의 위에서 3건"이 됩니다. `LIMIT` 을 `ORDER BY` 보다 앞에 적으면 문법 오류가 납니다.

Q4. `BETWEEN 5000 AND 20000` 은 5000과 20000을 포함하나요? → A4. 네, `BETWEEN` 은 양 끝 값을 모두 포함합니다(5000 이상 20000 이하). 끝값을 빼고 싶다면 `price > 5000 AND price < 20000` 처럼 비교 연산자로 직접 적어야 합니다. 끝값 포함 여부는 실수가 잦은 부분이니 항상 확인합니다.

장 요약

이 장에서는 CRUD의 읽기(Read)에 해당하는 `SELECT` 의 기본기를 익혔습니다. `SELECT` 컬럼 `FROM` 테이블 로 원하는 컬럼만 골라(투영) 가져오고, 별칭과 계산 컬럼을 만들었습니다. `WHERE` 와 비교·논리 연산자, `BETWEEN` · `IN` · `LIKE` · `IS NULL` 로 조건에 맞는 행만 추렸으며, `ORDER BY` 로 여러 기준 정렬과 `NULL` 위치를 다루었습니다. 끝으로 `LIMIT` · `OFFSET` 으로 가져올 행 수를 제한하고 페이지를 나누는 법까지 배웠습니다. 무엇보다 작성 순서(`SELECT` 먼저)와 논리적 실행 순서(`FROM` → `WHERE` → `SELECT` → `ORDER BY` → `LIMIT`)가 다르다는 점을 기억하면, 이후 더 복잡한 조회도 차분히 풀어 갈 수 있습니다.

핵심 키워드

SELECT, FROM, 컬럼 투영(column projection), 별칭(AS), WHERE, BETWEEN, IN, LIKE, IS NULL, ORDER BY, ASC/DESC, LIMIT, OFFSET

다음 장 미리보기

다음 장에서는 조회를 한 단계 끌어올립니다. 여러 행을 하나의 값으로 요약하는 집계 함수 (COUNT · SUM · AVG 등)와, 같은 값끼리 묶어 집계하는 GROUP BY, 그룹 결과를 거르는 HAVING, 그리고 쿼리 안에 쿼리를 넣는 서브쿼리(subquery)까지 다루며 데이터를 요약해 읽는 법을 배웁니다.

SELECT 심화: 집계·그룹·서브쿼리

학습 목표 - COUNT·SUM·AVG·MAX·MIN 집계 함수로 데이터를 요약할 수 있습니다. - GROUP BY로 그룹별 집계를 수행할 수 있습니다. - HAVING과 WHERE의 적용 대상 차이를 구분해 그룹을 필터링할 수 있습니다. - IN·EXISTS·스칼라 서브쿼리로 조회를 조립할 수 있습니다.

앞 장에서 우리는 `SELECT` 로 행을 그대로 꺼내 보는 법을 배웠습니다. 그런데 실무에서 자주 받는 질문은 "한 줄 한 줄 보여 줘"가 아니라 "전부 합치면 얼마야?", "카테고리별 평균은?", "평균보다 많이 산 고객이 누구야?" 같은 것입니다. 이런 질문은 행을 나열하는 것만으로는 답할 수 없습니다. 여러 행을 모아 **하나의 요약 값으로 계산**해야 합니다.

이 장에서는 `SELECT`를 한 단계 끌어올립니다. 여러 행을 한 값으로 줄이는 **집계 함수 (aggregate function)**, 같은 값끼리 묶어 그룹마다 집계하는 **GROUP BY**, 그룹 결과를 걸러 내는 **HAVING**, 그리고 쿼리 안에 또 다른 쿼리를 넣는 **서브쿼리(subquery)** 까지 차례로 익힙니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 에서 `psql` 로 PostgreSQL에 접속한 상태를 기준으로 합니다. 화면 앞의 `shop_lab=#` 표시는 "지금 `shop_lab` 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 모두 아래 세 테이블과 데이터를 기준으로 합니다. 앞 장들에서 만든 것과 같은 모양이라고 생각하면 됩니다.

```

-- 이 장 예시가 기준으로 삼는 데이터 (참고용)
-- members: 회원 4명 (한지민은 아직 주문 없음)
INSERT INTO members (name, email) VALUES
  ('김민수', 'minsu@example.com'),
  ('이서연', 'seoyeon@example.com'),
  ('박지훈', 'jihun@example.com'),
  ('한지민', 'jimin@example.com');

-- products: 상품 8건 (마지막 한 건은 category가 NULL)
INSERT INTO products (name, category, price, stock) VALUES
  ('무선 마우스', '전자기기', 19900, 50),
  ('기계식 키보드', '전자기기', 89000, 20),
  ('머그컵', '주방용품', 8900, 100),
  ('노트', '문구', 3000, 200),
  ('텀블러', '주방용품', 15000, 30),
  ('USB 허브', '전자기기', 23000, 40),
  ('볼펜', '문구', 1500, 500),
  ('미분류 상품', NULL, 5000, 10);

-- orders: 주문 8건 (member_id로 회원과 연결, category와 amount 보관)
INSERT INTO orders (member_id, category, amount, ordered_at) VALUES
  (1, '전자기기', 19900, '2026-05-03'),
  (1, '주방용품', 8900, '2026-05-15'),
  (2, '전자기기', 89000, '2026-05-20'),
  (2, '전자기기', 23000, '2026-06-02'),
  (3, '문구', 3000, '2026-06-05'),
  (1, '전자기기', 12000, '2026-06-10'),
  (3, '주방용품', 15000, '2026-06-12'),
  (2, '문구', 1500, '2026-06-18');

```

products 의 마지막 행은 category 가 NULL (값 없음)입니다. 이 한 건이 뒤에서 집계와 NULL의 관계를 설명할 때 중요한 역할을 합니다.

집계 함수(aggregate function)로 요약하기

도입

영수증 여러 장을 책상에 펼쳐 두고 "전부 몇 장이지? 합계는 얼마고 평균은 얼마지?"를 계산기로 두드린다고 떠올려 봅시다. 집계 함수가 하는 일이 정확히 이것입니다. 이 절에서는 여러 행을 하나의 요약 값으로 줄이는 다섯 가지 기본 함수를 익히고, DISTINCT 와 NULL 이 그 계산에 어떤 영향을 주는지 살펴봅시다.

핵심 개념 설명

집계 함수 (aggregate function)

집계 함수(aggregate function)란, 여러 행의 값을 모아 **하나의 요약 값으로 계산**하는 함수입니다. 여러 영수증 금액을 모아 합계나 평균을 내는 계산기와 같습니다. 가장 자주 쓰는 다섯 가지는 다음과 같습니다.

- **왜(why) 필요한가:** "데이터가 총 몇 건인지", "매출 합계가 얼마인지", "가장 비싼 상품 가격이 얼마인지" 같은 질문은 행을 나열해서는 답할 수 없습니다. 여러 행을 한 값으로 줄여야 답이 나옵니다.
- **언제(when) 쓰나:** 건수·합계·평균·최댓값·최솟값처럼 **요약 숫자 하나**가 필요할 때 씁니다.
- **어떻게(how) 쓰나:** `SELECT 함수(컬럼) FROM 테이블` 처럼 `SELECT` 자리에 집계 함수를 적습니다.

기본 집계 함수 다섯 가지

함수	하는 일	예시 질문
<code>COUNT()</code>	행(또는 값)의 개수를 셉니다	상품이 몇 개 있나요
<code>SUM()</code>	숫자 값을 모두 더합니다	가격 합계는 얼마인가요
<code>AVG()</code>	숫자 값의 평균을 냅니다	평균 가격은 얼마인가요
<code>MAX()</code>	가장 큰 값을 찾습니다	가장 비싼 가격은 얼마인가요
<code>MIN()</code>	가장 작은 값을 찾습니다	가장 싼 가격은 얼마인가요

DISTINCT와 NULL이 집계에 미치는 영향

집계 함수를 쓸 때 입문자가 가장 많이 헷갈리는 두 가지가 **NULL(값 없음)** 과 **DISTINCT(중복 제거)** 입니다.

- `COUNT(*)` 는 **행 자체의 개수**를 세므로 NULL이 있어도 모두 셉니다.
- `COUNT(컬럼)` 은 그 컬럼이 **NULL이 아닌 행만** 셉니다. 즉 NULL은 빠집니다.
- `SUM` · `AVG` · `MAX` · `MIN` 도 마찬가지로 **NULL은 계산에서 제외**합니다. 특히 `AVG` 는 NULL을 0으로 치지 않고 아예 빼고 평균을 내므로 결과가 달라집니다.
- `COUNT(DISTINCT 컬럼)` 처럼 `DISTINCT` 를 넣으면 **중복을 뺀 고유한 값의 개수**만 셉니다.

집계 함수가 NULL도 세어 평균에 넣는다고 생각하기 쉽지만, 그렇지 않습니다. NULL은 "값이 없음"이라 더할 대상도 셀 대상도 아니기 때문에 조용히 제외됩니다.

이렇게 작성합니다

먼저 `products` 테이블 전체를 한 값씩으로 요약해 보겠습니다.

```
-- 상품 테이블을 요약하기: 건수, 합계, 평균, 최대, 최소
SELECT
  count(*)          AS 상품수,
  sum(price)        AS 가격합계,
  round(avg(price), 2) AS 평균가격,
  max(price)        AS 최고가,
  min(price)        AS 최저가
FROM products;
```

예상 화면:

```
상품수 | 가격합계 | 평균가격 | 최고가 | 최저가
-----+-----+-----+-----+-----
      8 | 165300.00 | 20662.50 | 89000.00 | 1500.00
(1 row)
```

여덟 개 행이 단 한 줄의 요약으로 줄었습니다. `avg(price)` 는 소수점이 길게 나오므로 `round(..., 2)` 로 소수 둘째 자리까지 반올림했습니다.

구문 요소 설명

구문	의미
<code>count(*)</code>	테이블의 전체 행 수를 셉니다(NULL 포함)
<code>sum(price)</code>	<code>price</code> 값을 모두 더합니다
<code>round(avg(price), 2)</code>	평균을 구한 뒤 소수 둘째 자리까지 반올림합니다
<code>AS 상품수</code>	결과 컬럼에 보기 좋은 별칭(이름)을 붙입니다

이번에는 NULL과 DISTINCT의 차이를 직접 확인해 보겠습니다. `products` 의 `category` 에는 NULL인 행이 한 건 있었습니다.

```
-- COUNT(*) 와 COUNT(category) 와 COUNT(DISTINCT category) 비교
SELECT
    count(*)           AS 전체행수,
    count(category)    AS 카테고리있는행,
    count(DISTINCT category) AS 고유카테고리수
FROM products;
```

예상 화면:

전체행수	카테고리있는행	고유카테고리수
8	7	3

(1 row)

같은 컬럼을 세는데도 결과가 다릅니다. `count(*)` 는 8건 모두 세지만, `count(category)` 는 NULL 인 한 건을 빼고 7건만 셉니다. `count(DISTINCT category)` 는 NULL을 제외하고 남은 값에서 중복을 없앤 결과, 전자기기·주방용품·문구 세 종류만 셉니다.

팁: "데이터가 몇 건인가"를 셀 때는 거의 항상 `count(*)` 를 씁니다. 특정 컬럼에 값이 실제로 채워진 건수가 궁금할 때만 `count(컬럼)` 을 쓰면 됩니다. 둘은 NULL이 있을 때 결과가 달라지므로 의도에 맞게 골라야 합니다.

흔한 실수: 집계 함수와 일반 컬럼을 함께 적는 경우입니다. 예를 들어 `SELECT name, count(*) FROM products;` 는 오류가 납니다. `count(*)` 는 모든 행을 한 줄로 줄이는데 `name` 은 행마다 다르므로, 둘을 어떻게 한 줄에 담을지 PostgreSQL이 결정할 수 없기 때문입니다. 일반 컬럼과 집계를 함께 보려면 다음 절의 `GROUP BY` 가 필요합니다.

절 요약

- `COUNT` · `SUM` · `AVG` · `MAX` · `MIN` 은 여러 행을 하나의 요약 값으로 줄입니다.
- `count(*)` 는 NULL을 포함한 행 수를, `count(컬럼)` 은 NULL이 아닌 값만 셉니다.
- `SUM` · `AVG` 등은 NULL을 계산에서 제외하며, `DISTINCT` 로 고유 값만 셀 수 있습니다.
- 집계 함수와 일반 컬럼을 그냥 섞어 쓰면 오류가 나며, 그때는 `GROUP BY` 가 필요합니다.

GROUP BY로 묶어서 집계하기

도입

앞 절에서는 테이블 **전체**를 한 값으로 요약했습니다. 그런데 보통은 "전체 평균"보다 "**카테고리별 평균**", "**회원별 합계**"가 더 궁금합니다. 동전을 한 무더기로 세지 않고 100원·500원으로 **무더기를 나눠** 각 무더기의 합을 세는 것처럼, **GROUP BY** 는 같은 값끼리 묶어 그룹마다 따로 집계합니다.

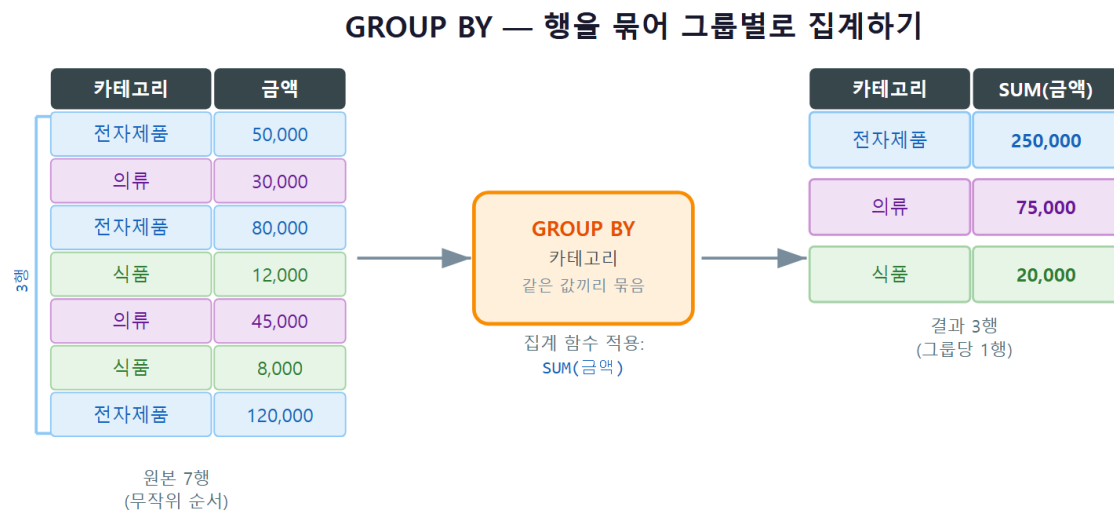
핵심 개념 설명

그룹화 (GROUP BY)

그룹화(GROUP BY) 란, 지정한 컬럼의 **값이 같은 행끼리 한 묶음으로 모은 뒤**, 그 묶음마다 집계 값을 계산하게 하는 절입니다. 동전을 액면가별로 무더기로 나눈 뒤 각 무더기의 합계를 세는 것과 같습니다.

- **왜(why) 쓰나**: "카테고리별 매출", "회원별 주문 수"처럼 **어떤 기준으로 나눈 뒤 각각 요약** 하고 싶을 때 씁니다. 그룹마다 집계 함수를 다시 적용한다고 생각하면 됩니다.
- **언제(when) 쓰나**: 결과를 "**~별로**" 보고 싶을 때입니다. "**~별**"이라는 말이 나오면 대개 GROUP BY가 필요합니다.
- **어떻게(how) 쓰나**: **GROUP BY** 기준컬럼 을 적고, **SELECT** 에는 **그 기준 컬럼과 집계 함수만** 적습니다.

여러 행이 그룹으로 묶이고, 각 그룹이 집계를 거쳐 한 줄의 요약 행으로 줄어드는 과정을 그림으로 나타내면 다음과 같습니다.



SQL 예시:

```
SELECT  카테고리, SUM(금액) AS 합계
FROM    주문 GROUP BY  카테고리;
```

ch_05_s02 - GROUP BY로 묶어서 집계하기

그룹별 집계 (grouped aggregation)

핵심 규칙이 하나 있습니다. **SELECT 에 적은 일반 컬럼은 반드시 GROUP BY 에도 적어야 합니다.** 그렇지 않으면 PostgreSQL이 "이 컬럼은 그룹 안에서 값이 여러 개인데 어느 것을 보여 줄 까?"를 판단할 수 없어 오류가 납니다. 집계 함수만은 예외로, 그룹 전체를 한 값으로 줄이므로 GROUP BY에 넣지 않아도 됩니다.

GROUP BY 없이도 그룹별 집계가 된다고 생각하기 쉽지만, 그렇지 않습니다. 기준을 GROUP BY 로 알려 줘야 비로소 그룹이 나뉩니다.

이렇게 작성합니다

`orders` 테이블에서 **카테고리별** 주문 건수와 매출 합계를 구해 보겠습니다.

```
-- 카테고리별 주문 건수와 매출 합계
SELECT
  category      AS 카테고리,
  count(*)      AS 주문수,
  sum(amount)   AS 매출합계
FROM orders
GROUP BY category
ORDER BY 매출합계 DESC;
```

예상 화면:

카테고리	주문수	매출합계
전자기기	4	144800.00
주방용품	2	23900.00
문구	2	4500.00

(3 rows)

여덟 건의 주문이 카테고리 세 그룹으로 묶였고, 그룹마다 건수와 합계가 따로 계산되었습니다. `ORDER BY 매출합계 DESC` 로 매출이 큰 그룹부터 정렬했습니다.

구문 요소 설명

구문	의미
GROUP BY category	category 값이 같은 행끼리 한 그룹으로 묶습니다
count(*)	각 그룹에 속한 행 수를 셉니다
sum(amount)	각 그룹의 amount 를 더합니다
category (SELECT)	그룹 기준 컬럼이라 SELECT에 적을 수 있습니다
ORDER BY 매출합계 DESC	집계 결과(별칭)를 기준으로 정렬합니다

기준 컬럼을 두 개 이상 적으면 더 잘게 나눌 수 있습니다. 다음은 회원별, 그 안에서 다시 카테고리별 매출을 구하는 예시입니다.

```
-- 회원별 + 카테고리별 매출 (두 컬럼으로 그룹 나누기)
```

```
SELECT
  member_id AS 회원,
  category AS 카테고리,
  sum(amount) AS 매출합계
FROM orders
GROUP BY member_id, category
ORDER BY member_id, 매출합계 DESC;
```

예상 화면:

```
회원 | 카테고리 | 매출합계
-----+-----+-----
1 | 전자기기 | 31900.00
1 | 주방용품 | 8900.00
2 | 전자기기 | 112000.00
2 | 문구 | 1500.00
3 | 주방용품 | 15000.00
3 | 문구 | 3000.00
(6 rows)
```

GROUP BY member_id, category 는 "회원과 카테고리 조합이 같은 행끼리" 묶습니다. 그래서 회원 1번의 전자기기 주문 두 건(19900 + 12000)이 한 줄로 합쳐져 31900이 됩니다.

주의: SELECT 에 적은 비집계 컬럼은 빠짐없이 GROUP BY 에 넣어야 합니다. 예를 들어 SELECT member_id, category, sum(amount) ... GROUP BY member_id 처럼 category 를 빠뜨리면 column "orders.category" must appear in the GROUP BY clause 오류가 납니다. "SELECT의 일반 컬럼 = GROUP BY의 컬럼"이라고 외워 두면 편합니다.

팁: NULL도 하나의 그룹이 됩니다. `category` 가 NULL인 행끼리는 "NULL 그룹"으로 묶여 한 줄로 집계됩니다. 위 `orders` 예시에는 NULL이 없었지만, `products` 를 카테고리로 그룹화하면 NULL 그룹이 빈칸으로 한 줄 나타납니다.

DBeaver 활용: 집계 결과는 DBeaver(1장)의 결과 표(grid)에서 보기 편합니다. 컬럼 머리글을 클릭하면 합계·평균 같은 결과를 **정렬**할 수 있고, 결과를 CSV 등으로 **내보내기**(Export) 해 엑셀에서 다시 볼 수도 있습니다. 그룹별 수치를 비교·공유할 때 유용합니다.

절 요약

- `GROUP BY` 기준컬럼 은 같은 값끼리 묶어 그룹마다 집계 함수를 적용합니다.
- 기준 컬럼을 여러 개 적으면 그 조합으로 더 잘게 그룹을 나눕니다.
- `SELECT` 의 비집계 컬럼은 반드시 `GROUP BY` 에도 적어야 합니다.
- NULL 값도 하나의 그룹으로 묶입니다.

HAVING으로 그룹 결과 필터링하기

도입

"카테고리별 매출을 구했더니, 그중 **매출이 3만 원을 넘는 카테고리만** 보고 싶다"면 어떻게 할까요. 여기서 막히는 입문자가 많습니다. `WHERE` 를 써 보면 오류가 나기 때문입니다. 그룹으로 묶고 난 **뒤의 결과**를 거르는 일은 `WHERE` 가 아니라 `HAVING` 이 말합니다. 이 절에서 둘의 차이를 확실히 구분합니다.

핵심 개념 설명

그룹 필터 (HAVING)

그룹 필터(HAVING) 란, `GROUP BY` 로 묶어 집계한 **그룹 결과에 조건을 거는** 절입니다. `WHERE` 가 "날개 행"을 거른다면, `HAVING` 은 "묶인 그룹"을 거릅니다.

- **왜(why) 쓰나:** `WHERE` 에는 `sum` · `count` 같은 집계 함수를 쓸 수 없습니다. 집계는 그룹이 만들어진 뒤에야 계산되는데, `WHERE` 는 그보다 먼저 동작하기 때문입니다. 그래서 "합계가 얼마 이상인 그룹"을 거르려면 `HAVING` 이 필요합니다.
- **언제(when) 쓰나:** "건수가 3건 이상인 그룹만", "매출 합계가 10만 원을 넘는 그룹만"처럼 **집계 결과를 기준으로** 그룹을 추릴 때 씁니다.

- 어떻게(how) 쓰나: GROUP BY 뒤에 HAVING 집계조건 을 적습니다.

WHERE와 HAVING 차이

이 절에서 가장 중요한 한 가지입니다. WHERE 는 그룹으로 묶기 전에 행을 거르고, HAVING 은 묶은 뒤에 그룹을 거릅니다. 처리 순서로 보면 다음과 같습니다.

WHERE와 HAVING의 차이

구분	WHERE	HAVING
거르는 대상	날개 행(row)	묶인 그룹(group)
적용 시점	GROUP BY로 묶기 전	묶고 집계한 후
집계 함수 사용	불가(sum() 등 못 씀)	가능(sum() 등 사용)
예시 조건	WHERE ordered_at >= '2026-06-01'	HAVING sum(amount) >= 30000

WHERE와 HAVING이 같은 것이고 위치만 다르다고 오해하기 쉽지만, 둘은 거르는 대상도 시점도 다릅니다. 하나의 쿼리에서 함께 쓰는 일도 흔합니다.

이렇게 작성합니다

먼저 카테고리별 매출 합계 중 3만 원 이상인 그룹만 추려 보겠습니다.

```
-- 매출 합계가 30000 이상인 카테고리만 보기
SELECT
  category AS 카테고리,
  sum(amount) AS 매출합계
FROM orders
GROUP BY category
HAVING sum(amount) >= 30000
ORDER BY 매출합계 DESC;
```

예상 화면:

```
카테고리 | 매출합계
-----+-----
전자기기 | 144800.00
(1 row)
```

전체 세 그룹(전자기기 144800, 주방용품 23900, 문구 4500) 중에서 `HAVING` 조건을 통과한 전자기기 한 그룹만 남았습니다.

주의: 위 조건을 `WHERE sum(amount) >= 30000` 으로 쓰면 `aggregate functions are not allowed in WHERE` 오류가 납니다. 합계는 그룹이 만들어진 뒤에 계산되는데 `WHERE` 는 그 전에 동작하기 때문입니다. 집계 결과를 거를 때는 반드시 `HAVING` 을 씁니다.

이번에는 `WHERE` 와 `HAVING` 을 **함께** 써 보겠습니다. "6월 주문만 대상으로, 카테고리별 매출이 3만 원 이상인 그룹"을 찾습니다.

```
-- WHERE로 6월 주문만 추린 뒤, HAVING으로 그룹을 거르기
SELECT
  category    AS 카테고리,
  count(*)    AS 주문수,
  sum(amount) AS 매출합계
FROM orders
WHERE ordered_at >= '2026-06-01'
GROUP BY category
HAVING sum(amount) >= 30000;
```

예상 화면:

```
카테고리 | 주문수 | 매출합계
-----+-----+-----
전자기기 |      2 | 35000.00
(1 row)
```

처리 순서를 따라가 보면 이해가 쉽습니다. (1) `WHERE` 가 6월 주문 다섯 건만 먼저 남기고, (2) `GROUP BY` 가 카테고리별로 묶고, (3) `HAVING` 이 그중 합계 3만 원 이상인 그룹만 통과시켰습니다. 6월 전자기기 주문은 $23000 + 12000 = 35000$ 으로 조건을 통과했습니다.

구문 요소 설명

구문	의미
<code>WHERE ordered_at >= '2026-06-01'</code>	그룹으로 묶기 전, 6월 이후 행만 남깁니다
<code>GROUP BY category</code>	남은 행을 카테고리별로 묶습니다
<code>HAVING sum(amount) >= 30000</code>	합계 3만 원 이상인 그룹만 남깁니다

절 요약

- `HAVING` 은 `GROUP BY` 로 묶어 집계한 **그룹 결과에** 조건을 겁니다.
- `WHERE` 는 묶기 전 낱개 행을, `HAVING` 은 묶은 뒤 그룹을 거릅니다.
- 집계 함수(`sum` · `count` 등)는 `WHERE` 에 못 쓰고 `HAVING` 에 씁니다.
- 처리 순서는 `WHERE` -> `GROUP BY` -> `HAVING`이며, 둘을 함께 쓰는 일도 흔합니다.

서브쿼리(subquery)로 조회 조립하기

도입

"평균 주문액보다 비싼 주문만 보고 싶다"는 질문을 떠올려 봅시다. 평균을 알아야 비교할 수 있는데, 그 평균 자체가 또 하나의 조회 결과입니다. 큰 질문에 답하려면 먼저 작은 질문을 풀어 그 답을 가져다 써야 합니다. 이렇게 **쿼리 안에 또 다른 쿼리를 넣는** 것이 서브쿼리입니다. 이 절에 서는 가장 자주 쓰는 세 가지 형태를 익힙니다.

핵심 개념 설명

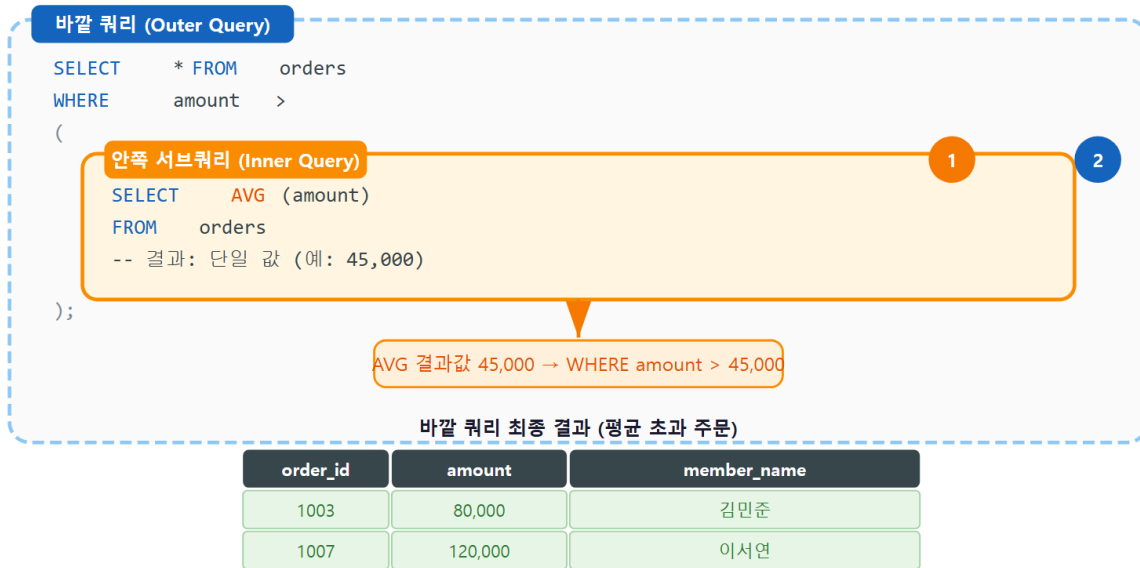
서브쿼리 (subquery)

서브쿼리(subquery) 란, 다른 쿼리 안에 **중첩된 `SELECT` 문**으로, 그 결과를 바깥 쿼리의 조건이나 값으로 사용하는 것입니다. 큰 질문에 답하기 위해 먼저 작은 질문을 풀어 그 답을 쓰는 것과 같습니다.

안쪽 서브쿼리(inner query)가 먼저 계산되고, 그 결과가 바깥 쿼리(outer query)의 조건으로 전달되는 흐름은 다음과 같습니다.

서브쿼리 — 안쪽이 먼저, 결과가 바깥으로 전달

실행 순서: 안쪽 서브쿼리 먼저 계산 → 결과를 바깥 쿼리 WHERE 조건으로 전달



ch_05_s04 · 서브쿼리(subquery)로 조회 조립하기

서브쿼리는 돌려주는 결과의 모양에 따라 쓰임이 나뉩니다.

- **스칼라 서브쿼리(scalar subquery)**: 값을 딱 하나만 돌려줍니다(예: 평균값 하나). `=`, `>`, `<` 같은 비교에 바로 씁니다.
- **IN 서브쿼리**: 값을 여러 개(목록) 돌려줍니다. "이 목록 안에 있으면"이라는 조건에 씁니다.
- **EXISTS 서브쿼리**: 값이 아니라 "조건에 맞는 행이 있는지 없는지(참/거짓)"만 봅니다.

IN·EXISTS 서브쿼리

서브쿼리가 한 번에 여러 값을 돌려주는데도 `=` 로 비교할 수 있다고 오해하기 쉽습니다. 그렇지 않습니다. 여러 값과 비교할 때는 `=` 이 아니라 `IN` 을 써야 합니다. `= (SELECT ...)` 에 여러 행이 돌아오면 `more than one row returned by a subquery` 오류가 납니다.

- 값 하나와 비교 -> `=` (스칼라 서브쿼리)
- 값 목록 안에 포함 여부 -> `IN` (목록 서브쿼리)
- 행의 존재 여부만 확인 -> `EXISTS` (서브쿼리)

이렇게 작성합니다

먼저 **스칼라 서브쿼리**입니다. "전체 주문 평균액보다 비싼 주문"을 찾습니다. 평균을 구하는 작은 쿼리를 `WHERE` 조건 안에 그대로 넣습니다.

```
-- 평균 주문액보다 비싼 주문만 조회 (스칼라 서브쿼리)
SELECT id, category, amount
FROM orders
WHERE amount > (SELECT avg(amount) FROM orders)
ORDER BY amount DESC;
```

예상 화면:

```
id | category | amount
---+-----+-----
 3 | 전자기기 | 89000.00
 4 | 전자기기 | 23000.00
(2 rows)
```

안쪽 (SELECT avg(amount) FROM orders) 가 먼저 평균 21537.50을 계산하고, 바깥 쿼리가 그 값보다 큰 주문만 골랐습니다. 평균값을 손으로 적어 넣지 않아도 데이터가 바뀌면 평균도 자동으로 다시 계산됩니다.

구문 요소 설명

구문	의미
(SELECT avg(amount) FROM orders)	값 하나(평균)를 돌려주는 스칼라 서브쿼리입니다
WHERE amount > (...)	그 평균값과 각 행의 amount 를 비교합니다
실행 순서	안쪽 평균이 먼저, 그 다음 바깥 비교가 동작합니다

다음은 **IN 서브쿼리**입니다. "전자기기를 주문한 적이 있는 회원"의 이름을 찾습니다. 안쪽 쿼리가 회원 id 목록을 돌려주면, 바깥 쿼리가 그 목록에 든 회원만 고릅니다.

```
-- 전자기기를 주문한 적 있는 회원 이름 (IN 서브쿼리)
SELECT name
FROM members
WHERE id IN (SELECT member_id FROM orders WHERE category = '전자기기');
```

예상 화면:

```
name
-----
김민수
이서연
(2 rows)
```

안쪽 쿼리가 전자기기 주문자의 `member_id` 목록(1, 2)을 돌려주고, 바깥 쿼리가 그 목록에 든 회원만 골랐습니다.

마지막은 **EXISTS 서브쿼리**입니다. EXISTS 는 값을 비교하지 않고 "맞는 행이 하나라도 있는지"만 봅니다. "주문이 한 건이라도 있는 회원"과, NOT EXISTS 로 "주문이 한 건도 없는 회원"을 찾아보겠습니다.

```
-- 주문이 한 건도 없는 회원 찾기 (NOT EXISTS)
SELECT name
FROM members m
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.member_id = m.id
);
```

예상 화면:

```
name
-----
한지민
(1 row)
```

바깥 회원 한 명마다, 안쪽에서 "이 회원의 주문이 있는지"를 확인합니다. 한지민은 주문이 한 건도 없으므로 NOT EXISTS 가 참이 되어 결과에 남았습니다. 안쪽 SELECT 1 은 "존재 여부만 보므로 무엇을 고르든 상관없다"는 관용 표현입니다.

팁: IN 과 EXISTS 는 비슷한 일을 하지만, "없는 것을 찾을 때"는 NOT EXISTS 가 더 안전합니다. NOT IN 은 안쪽 목록에 NULL이 섞이면 결과가 통째로 비어 버리는 함정이 있기 때문입니다. 입문 단계에서는 "포함 여부는 IN, 존재 여부는 EXISTS"로 기억해 두면 충분합니다.

주의: WHERE amount = (SELECT ...) 처럼 = 뒤에 둔 서브쿼리가 여러 행을 돌려주면 오류가 납니다. 값 하나가 확실할 때만 = 을 쓰고, 여러 개일 가능성이 있으면 IN 을 씁니다. 평균·최대값처럼 본래 한 값인 집계 서브쿼리는 = > 로 비교해도 안전합니다.

절 요약

- 서브쿼리는 쿼리 안에 중첩된 SELECT 로, 안쪽이 먼저 계산되어 바깥 조건에 쓰입니다.
- 값 하나는 스칼라 서브쿼리로 = > 비교에, 값 목록은 IN 에 씁니다.
- EXISTS 는 값이 아니라 행의 존재 여부(참/거짓)를 확인합니다.
- "없는 것"을 찾을 때는 NOT IN 보다 NOT EXISTS 가 안전합니다.

실습 과제

해보기: 매출 요약 리포트 쿼리 작성하기

실습 데이터베이스 `shop_lab` 의 `orders` 테이블로 이 장의 집계·그룹·서브쿼리를 직접 다뤄 봅니다.

- 단계 1: `orders` 전체에 대해 `count(*)` · `sum(amount)` · `round(avg(amount), 2)` 로 총 주문 수, 매출 합계, 평균 주문액을 한 줄로 요약합니다.
- 단계 2: `GROUP BY category` 로 카테고리별 매출 합계와 주문 수를 구하고, `ORDER BY` 로 매출이 큰 순서로 정렬합니다.
- 단계 3: 같은 그룹 집계에 `HAVING sum(amount) >= 20000` 을 붙여 일정 매출 이상 카테고리만 남깁니다. 이어서 `WHERE ordered_at >= '2026-06-01'` 을 함께 걸어 "6월 이후, 매출 2만원 이상 카테고리"를 구해 봅니다.
- 단계 4: 서브쿼리로 "전체 평균 주문액보다 비싼 주문"을 조회하고, `IN` 서브쿼리로 "주방 용품을 주문한 회원 이름"을 찾습니다.
- 점검: 단계 3에서 `WHERE` 를 뺐을 때와 넣었을 때 결과 행 수가 어떻게 달라지는지 비교하고, 그 차이를 `WHERE`와 `HAVING`의 적용 시점으로 설명해 봅니다.

FAQ

Q1. `WHERE` 와 `HAVING` 은 무엇이 다른가요? → A1. 거르는 대상과 시점이 다릅니다. `WHERE` 는 `GROUP BY` 로 묶기 **전에 날개 행**을 거르고, `HAVING` 은 묶어서 집계한 **뒤에 그룹**을 거릅니다. 그래서 `sum()` · `count()` 같은 집계 함수는 `WHERE` 에는 못 쓰고 `HAVING` 에만 쓸 수 있습니다. "행을 거르면 `WHERE`, 집계 결과를 거르면 `HAVING`"으로 기억하면 됩니다.

Q2. `count(*)` 와 `count(컬럼)` 은 왜 결과가 다른가요? → A2. `count(*)` 는 행 자체의 개수를 세므로 `NULL`이 있어도 모두 셉니다. 반면 `count(컬럼)` 은 그 컬럼이 **`NULL`이 아닌 행만** 셉니다. 그래서 `NULL`이 섞인 컬럼에서는 `count(*)` 가 더 큰 값이 나옵니다. 전체 건수가 궁금하면 `count(*)`, 값이 실제로 채워진 건수가 궁금하면 `count(컬럼)` 을 씁니다.

Q3. `GROUP BY` 를 쓰면 왜 "must appear in the GROUP BY clause" 오류가 나나요? → A3. `SELECT` 에 적은 일반 컬럼을 `GROUP BY` 에 넣지 않았기 때문입니다. 그룹 안에는 그 컬럼 값이 여러 개일 수 있어 PostgreSQL이 어느 값을 보여 줄지 정할 수 없습니다. 해결책은 그 컬럼을 `GROUP BY` 에도 적거나, 아니면 집계 함수로 감싸는 것입니다("SELECT의 일반 컬럼 = GROUP BY의 컬럼").

Q4. 서브쿼리와 JOIN 중 무엇을 써야 하나요? → A4. 둘 다 여러 테이블을 함께 다루지만 쓰임이 조금 다릅니다. "다른 테이블의 값으로 조건을 거는" 경우는 서브쿼리가 읽기 쉽고, "여러 테이블의 컬럼을 한 화면에 나란히 보여 주는" 경우는 JOIN(6장)이 적합합니다. 서브쿼리가 항상 JOIN보다 느리다는 말도 흔하지만, PostgreSQL이 내부적으로 최적화하므로 입문 단계에서는 읽기 쉬운 쪽을 고르면 됩니다.

장 요약

이 장에서는 SELECT를 요약과 분석 도구로 확장했습니다. COUNT · SUM · AVG · MAX · MIN 집계 함수로 여러 행을 한 값으로 줄이고, NULL과 DISTINCT가 그 계산에 주는 영향을 확인했습니다. GROUP BY로 같은 값끼리 묶어 "~별" 집계를 수행하고, HAVING으로 그룹 결과를 걸러 냈으며, WHERE (행 필터)와 HAVING (그룹 필터)의 적용 시점 차이를 분명히 했습니다. 마지막으로 스칼라 · IN · EXISTS 서브쿼리로 한 쿼리 안에 또 다른 쿼리를 끼워 넣어 복잡한 질문에 답하는 법을 배웠습니다. 이제 한 테이블 안에서 데이터를 요약하고 분석하는 기본기를 갖추었습니다.

핵심 키워드

집계 함수(aggregate function), COUNT, SUM, AVG, GROUP BY, HAVING, WHERE vs HAVING, 서브쿼리(subquery), IN, EXISTS

다음 장 미리보기

다음 장에서는 흩어진 여러 테이블을 키로 연결해 한 결과로 합쳐 읽는 JOIN을 배웁니다. 회원·주문·상품 테이블을 묶어 "누가 무엇을 언제 주문했는지"를 한 화면에 모아 보는, 조회의 폭을 크게 넓히는 기법입니다.

JOIN: 테이블을 결합해 읽기 확장하기

학습 목표 - 데이터를 나눠 저장하고 JOIN으로 합쳐 읽는 이유를 설명할 수 있습니다. - INNER JOIN과 ON 절로 일치하는 행을 결합할 수 있습니다. - LEFT·RIGHT OUTER JOIN의 차이를 벤다이어그램 수준으로 구분할 수 있습니다. - 여러 테이블을 별칭과 함께 조인하고 자기 조인·교차 조인을 이해합니다.

앞 장까지 우리는 한 테이블에서 데이터를 넣고(INSERT) 읽는(SELECT) 법을 배웠습니다. 그런데 현실의 데이터는 한 테이블에 다 담기지 않습니다. 회원 정보는 회원 테이블에, 주문 정보는 주문 테이블에 나뉘어 있습니다. 이 장에서는 이렇게 **나뉘어 있는 테이블을 공통 키로 다시 합쳐 읽는 방법**, 즉 **JOIN (조인)**을 배웁니다.

조인은 SQL에서 가장 자주 쓰이고, 가장 강력한 조회 기술입니다. "누가 무엇을 주문했는가" 같은 질문은 회원 테이블 하나로도, 주문 테이블 하나로도 답할 수 없고, 두 테이블을 맞물려야 답할 수 있기 때문입니다. 조인을 익히면 조회의 폭이 단숨에 넓어집니다.

이 책은 Windows 11의 **PowerShell(파워셸)**에서 `psql`로 PostgreSQL에 접속한 상태를 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, `psql`은 그 안에서 데이터베이스와 대화하는 도구입니다. 화면 앞의 `shop_lab=#` 표시는 "지금 `shop_lab` 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 모두 아래 세 테이블을 기준으로 합니다. `members` (회원)와 `products` (상품)는 앞에서 본 것과 같고, 여기에 주문을 기록하는 `orders` (주문) 테이블이 새로 등장합니다.

```
-- 이 장 예시가 기준으로 삼는 세 테이블 (참고용)
CREATE TABLE members (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name    varchar(50) NOT NULL,
  email   varchar(100) NOT NULL
);

CREATE TABLE products (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name    varchar(100) NOT NULL,
  price   numeric(10, 2) NOT NULL
);

CREATE TABLE orders (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  member_id   integer REFERENCES members (id),
  product_id  integer REFERENCES products (id),
  quantity    integer NOT NULL DEFAULT 1,
  ordered_at  date     NOT NULL DEFAULT current_date
);
```

여기서 `orders` 테이블의 `member_id`와 `product_id`는 **외래 키(foreign key)**입니다. `member_id`는 "이 주문을 한 회원이 `members` 테이블의 몇 번 회원인지"를 가리키는 번호이고, `product_id`는 "주문한 상품이 `products` 테이블의 몇 번 상품인지"를 가리키는 번호입니다. 이 가리키는 번호가 바로 조인이 두 테이블을 맞물리는 연결 고리입니다.

JOIN의 개념과 필요성

도입

주문서에는 보통 "회원 번호 3번이 상품 번호 5번을 2개 주문"처럼 번호만 적혀 있습니다. 사람이 보기엔 불친절합니다. 회원 번호 3번이 누구인지, 상품 5번이 무엇인지 알려면 회원 명부와 상품 목록을 따로 뒤져야 하기 때문입니다. 이 절에서는 왜 데이터를 굳이 나눠 두는지, 그리고 나뉜 데이터를 어떻게 다시 합쳐 읽는지 큰 그림을 잡습니다.

핵심 개념 설명

조인 (JOIN)

조인(JOIN)이란, 둘 이상의 테이블을 공통 키로 연결해 하나의 결과 집합으로 결합하는 조회 방법입니다. 주문서에 적힌 회원 번호를 회원 명부와 맞춰 이름을 채워 넣는 것과 같습니다.

조인을 이해하려면 먼저 "왜 데이터를 한 테이블에 다 담지 않고 나눠 두는가"를 알아야 합니다.

- **왜(why) 나눠 두나:** 만약 주문할 때마다 회원 이름·이메일·주소를 주문 테이블에 통째로 적어 넣는다면, 같은 회원이 100번 주문하면 같은 이름이 100번 중복 저장됩니다. 회원이 이메일을 바꾸면 100군데를 모두 고쳐야 하고, 한 군데라도 놓치면 데이터가 어긋납니다. 그래서 회원 정보는 `members` 에 한 번만 두고, 주문에는 **회원 번호(member_id)**만 적어 가리키게 합니다. 이렇게 중복을 줄이는 설계를 정규화(normalization)라고 합니다.
- **언제(when) 합쳐 읽나:** "누가 무엇을 주문했는지"처럼 한 테이블만으로는 답할 수 없는 질문을 할 때, 조인으로 나뉜 정보를 다시 모읍니다.
- **어떻게(how) 합치나:** `FROM 테이블A JOIN 테이블B ON A.키 = B.키` 처럼, 어느 컬럼끼리 같은 값을 가질 때 두 행을 한 줄로 묶을지 **연결 조건**을 알려 줍니다.

여기서 입문자가 자주 하는 오해 두 가지를 짚겠습니다. 첫째, 조인이 두 테이블을 영구히 하나로 합쳐 버린다고 생각하는 경우입니다. 조인은 **조회하는 그 순간에만** 결과를 임시로 합쳐 보여 줄 뿐, 원래 테이블은 그대로 남습니다. 둘째, 연결 조건(ON)을 빼도 알아서 맞춰 준다고 생각하는 경우입니다. 조건을 빼면 엉뚱하게 모든 조합이 만들어지므로(뒤의 교차 조인 참고), ON은 반드시 적어야 합니다.



위 그림처럼 `orders` 테이블의 `member_id` 값이 `members` 테이블의 `id` 를 가리키고, 두 값이 같은 행끼리 맞물려 하나의 결과 행으로 합쳐지는 것이 조인의 핵심 동작입니다.

이렇게 작성합니다

먼저 조인 없이 주문 테이블만 조회하면 어떤 한계가 있는지 봅니다.

```
-- 주문 테이블만 조회하면 번호만 보입니다
SELECT id, member_id, product_id, quantity
FROM orders;
```

예상 화면:

id	member_id	product_id	quantity
1	1	2	1
2	1	3	2
3	3	1	1

(3 rows)

member_id 가 1, 3 이라고만 나올 뿐, 누구인지 알 수 없습니다. 이제 같은 조회를 조인으로 확장해 회원 이름을 함께 가져옵니다. 자세한 문법은 다음 절에서 다루므로, 여기서는 "번호가 이름으로 채워진다"는 결과에 집중합니다.

```
-- 주문에 회원 이름을 덧붙여 함께 읽기
SELECT orders.id, members.name, orders.product_id, orders.quantity
FROM orders
JOIN members ON orders.member_id = members.id;
```

예상 화면:

id	name	product_id	quantity
1	김민수	2	1
2	김민수	3	2
3	박지훈	1	1

(3 rows)

member_id 였던 자리에 이제 실제 회원 이름이 채워졌습니다. orders 의 member_id (1, 1, 3)가 members 의 id 와 맞물려, 해당 회원의 name 을 함께 끌어온 결과입니다.

팁: 조인은 외래 키(foreign key)를 따라가는 조회라고 생각하면 쉽습니다. orders.member_id 가 members.id 를 "가리키도록" 설계해 두었기 때문에, 조인에서도 바로 그 두 컬럼을 ON 으로 연결합니다. 테이블을 잘 설계해 두면 조인 조건은 자연스럽게 따라옵니다.

절 요약

- 데이터는 중복을 줄이기 위해 여러 테이블로 나눠 저장하고, 주문에는 회원 번호 같은 가리키는 키만 둡니다.

- 조인은 그 키를 맞추려 나뉜 데이터를 **조회 순간에만** 하나로 합쳐 읽는 방법입니다.
- 연결 조건(ON)은 반드시 적어야 하며, 원래 테이블은 조인 후에도 그대로 남습니다.

INNER JOIN으로 일치하는 행 결합하기

도입

조인의 가장 기본이자 가장 많이 쓰는 형태가 **INNER JOIN** (내부 조인)입니다. 두 명단을 비교해 **양쪽에 모두 이름이 있는 사람만** 골라내는 것과 같습니다. 이 절에서는 **ON** 으로 연결 조건을 지정하고, 양쪽에 짝이 있는 행만 결합하는 방법을 익힙니다.

핵심 개념 설명

내부 조인 (INNER JOIN)

내부 조인(INNER JOIN) 이란, 조인 조건을 양쪽 테이블에서 **모두 만족하는 행만** 결합해 가져오는 조인입니다. 회원 명부와 주문서를 맞춰 볼 때, 주문이 있는 회원과 회원이 확인되는 주문만 짝지어 보여 줍니다.

- **왜(why) 쓰나:** "실제로 연결되는 데이터만" 보고 싶을 때 씁니다. 주문한 회원과 그 주문 내역처럼, 양쪽이 분명히 맞추리는 행만 결과로 얻습니다.
- **언제(when) 쓰나:** 가장 일반적인 조인 상황 대부분에 씁니다. 짝이 없는 행(주문이 없는 회원 등)을 굳이 포함할 필요가 없을 때 적합합니다.
- **어떻게(how) 쓰나:** `FROM A INNER JOIN B ON A.키 = B.키` 형태로 적습니다. **INNER** 는 생략할 수 있어 그냥 **JOIN** 이라고만 써도 내부 조인입니다.

조인 조건 (ON 절)

조인 조건(ON 절) 이란, 두 테이블의 어떤 컬럼이 같은 값일 때 두 행을 한 줄로 묶을지 지정하는 부분입니다. `ON orders.member_id = members.id` 는 "주문의 회원 번호와 회원의 번호가 같은 행끼리 묶어라"는 뜻입니다.

INNER JOIN 이 짝 없는 행도 NULL로 채워 포함한다고 오해하기 쉽지만, 그렇지 않습니다. 내부 조인은 짝이 있는 행만 남기고, **짝이 없는 행은 결과에서 빠집니다**. 짝 없는 행까지 보고 싶다면 다음 절의 **OUTER JOIN**을 써야 합니다.

이렇게 작성합니다

다음은 주문과 회원을 내부 조인해, 주문이 확인된 회원의 이름과 주문 수량을 함께 조회하는 예시입니다.

```
-- 주문이 있는 회원과 그 주문 내역만 결합하기
SELECT orders.id, members.name, orders.quantity
FROM orders
INNER JOIN members ON orders.member_id = members.id;
```

예상 화면:

```
id | name  | quantity
---+-----+-----
 1 | 김민수 |        1
 2 | 김민수 |        2
 3 | 박지훈 |        1
(3 rows)
```

orders 의 세 행 모두 짝이 되는 회원이 있어 그대로 결합되었습니다. 만약 회원 명단에 이서연 이 있어도 그가 주문한 적이 없다면, 내부 조인 결과에는 이서연 이 나타나지 않습니다. 주문 쪽에 짝이 없기 때문입니다.

구문 요소 설명

구문	의미
FROM orders	조인의 기준이 되는 첫 번째 테이블입니다
INNER JOIN members	members 를 결합 대상으로 추가합니다(INNER 는 생략 가능)
ON orders.member_id = members.id	두 행을 묶는 연결 조건입니다
orders.id , members.name	어느 테이블의 컬럼인지 테이블.컬럼 으로 밝힙니다

여기서 컬럼 앞에 orders. , members. 처럼 테이블 이름을 붙인 점에 주목합니다. 두 테이블에 같은 이름의 컬럼(id)이 있을 때, 어느 쪽 id 인지 밝히지 않으면 PostgreSQL이 "어느 id인지 모호하다"는 오류를 냅니다.

흔한 실수: ON 절 빠뜨리기

`FROM orders JOIN members` 라고만 쓰고 `ON` 조건을 빠뜨리면 문법 오류가 나거나, 조건이 잘못되면 의미 없는 결과가 나옵니다. 특히 `ON orders.member_id = members.id` 를 써야 할 자리에 `ON orders.id = members.id` 처럼 엉뚱한 컬럼을 맞추면, 우연히 번호가 겹친 무관한 행이 묶여 **틀린 결과가 조용히 나옵니다**. 조인 조건은 반드시 "가리키는 키 = 가리켜지는 키"인지 확인합니다.

팁: 결과 행 수가 예상보다 적다면 내부 조인 때문에 짝 없는 행이 빠진 것은 아닌지 의심해 봅니다. 예를 들어 회원은 10명인데 조인 결과가 3행뿐이라면, 주문한 회원이 3명뿐이라는 뜻일 수 있습니다.

절 요약

- `INNER JOIN ... ON` 은 양쪽 조건을 모두 만족하는 행만 결합합니다(`INNER` 는 생략 가능).
- 짝이 없는 행은 결과에서 빠지므로, 결과 행 수가 줄어든 수 있습니다.
- 같은 이름의 컬럼은 `테이블.컬럼` 으로 어느 쪽인지 밝혀야 모호함 오류를 피합니다.

LEFT·RIGHT OUTER JOIN으로 빠짐없이 읽기

도입

내부 조인은 짝이 있는 행만 보여 줍니다. 하지만 "주문이 한 번도 없는 회원까지 포함해 전체 회원을 보고 싶다"면 어떻게 할까요. 참석 명단을 기준으로, 짝지을 발표 주제가 없는 사람도 빈 칸으로 남겨 모두 표시하는 것과 같습니다. 이 절에서는 한쪽 테이블의 행을 짝이 없어도 모두 포함하는 `OUTER JOIN` (외부 조인)을 배웁니다.

핵심 개념 설명

외부 조인 (OUTER JOIN)

외부 조인(OUTER JOIN) 이란, 한쪽(LEFT/RIGHT) 또는 양쪽(FULL) 테이블의 행을 짝이 없어도 모두 포함하고, 없는 값은 `NULL` 로 채우는 조인입니다. 기준이 되는 쪽의 모든 행을 빠짐없이 살리는 것이 핵심입니다.

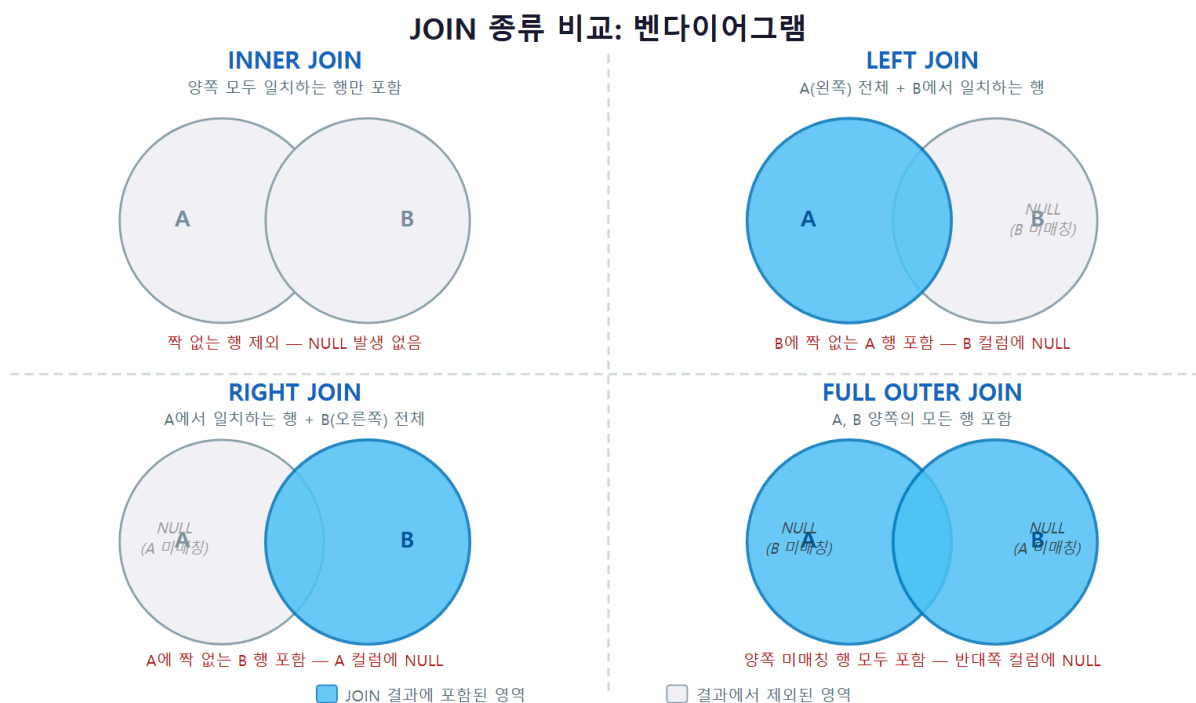
- **LEFT OUTER JOIN:** `FROM A LEFT JOIN B` 라고 쓰면 **왼쪽 테이블 A의 모든 행**을 살리고, B에 짝이 없으면 B 쪽 컬럼을 `NULL`로 채웁니다.
- **RIGHT OUTER JOIN:** 반대로 **오른쪽 테이블 B의 모든 행**을 살립니다.
- **FULL OUTER JOIN:** 양쪽 테이블의 모든 행을 살리고, 짝이 없는 쪽을 `NULL`로 채웁니다.

OUTER 라는 단어는 생략할 수 있어, LEFT JOIN 은 LEFT OUTER JOIN 과 같습니다.

LEFT/RIGHT JOIN의 방향

LEFT와 RIGHT는 어느 테이블을 기준으로 모두 살릴지 방향만 다릅니다. A LEFT JOIN B 는 B RIGHT JOIN A 와 같은 결과를 냅니다. 실무에서는 대부분 LEFT JOIN 을 쓰고 RIGHT JOIN은 잘 쓰지 않는데, "기준 테이블을 먼저 적고 그 왼쪽을 모두 살린다"는 흐름이 사람이 읽기에 더 자연스럽기 때문입니다.

LEFT와 RIGHT가 항상 같은 결과라고 오해하거나, OUTER JOIN에서 NULL이 생기지 않는다고 오해하기 쉽습니다. 기준 쪽에 짝이 없는 행이 있으면 반대편 컬럼은 반드시 NULL로 채워집니다.



위 벤다이어그램처럼, INNER는 두 집합이 겹치는 가운데만, LEFT는 왼쪽 전체, RIGHT는 오른쪽 전체, FULL은 양쪽 전체를 결과로 가져온다고 기억하면 네 조인의 차이가 한눈에 들어옵니다.

이렇게 작성합니다

다음은 회원을 기준으로 왼쪽 외부 조인을 해, **주문이 없는 회원까지 모두** 조회하는 예시입니다. 회원 명단에 이서연 이 있지만 주문은 없다고 가정합니다.

```
-- 주문이 없는 회원도 포함해 전체 회원 읽기
SELECT members.name, orders.id AS order_id, orders.quantity
FROM members
LEFT JOIN orders ON members.id = orders.member_id;
```

예상 화면:

name	order_id	quantity
김민수	1	1
김민수	2	2
박지훈	3	1
이서연		

(4 rows)

주문이 두 건인 김민수 는 두 줄로, 주문이 없는 이서연 도 한 줄로 나타났습니다. 이서연 의 order_id 와 quantity 가 비어 있는데, 이 빈칸이 바로 짝이 없어 채워진 NULL 입니다. 내부 조인 이었다면 이서연 은 아예 빠졌을 것입니다.

구문 요소 설명

구문	의미
FROM members	모두 살릴 기준(왼쪽) 테이블입니다
LEFT JOIN orders	오른쪽 orders 를 결합하되, 짝 없는 왼쪽 행도 유지합니다
orders.id AS order_id	결과 컬럼 이름을 order_id 로 바꿔 표시합니다
빈칸(NULL)	짝이 없어 오른쪽 컬럼이 채워지지 않은 자리입니다

"주문을 한 번도 안 한 회원만" 골라내고 싶다면, LEFT JOIN 결과에서 orders.id 가 NULL인 행만 추리면 됩니다.

```
-- 주문 이력이 전혀 없는 회원만 추리기
SELECT members.name
FROM members
LEFT JOIN orders ON members.id = orders.member_id
WHERE orders.id IS NULL;
```

예상 화면:

name
이서연

(1 row)

`WHERE orders.id IS NULL`은 "짜이 없어 NULL이 된 행만"이라는 뜻이므로, 결과적으로 주문이 없는 회원만 남습니다. 이는 LEFT JOIN의 대표적인 활용 패턴입니다.

주의: NULL은 `=`로 비교하지 않습니다

위에서 `WHERE orders.id IS NULL`이라고 썼지, `WHERE orders.id = NULL`이라고 쓰지 않은 점에 유의합니다. SQL에서 NULL은 "값이 없음"이라 같다·다르다를 판단할 수 없어, `= NULL`은 항상 거짓이 되어 한 행도 걸리지 않습니다. NULL 여부는 반드시 `IS NULL` 또는 `IS NOT NULL`로 확인합니다.

팁: 헛갈릴 때는 LEFT JOIN 하나만 외워도 충분합니다. "모두 살리고 싶은 테이블을 FROM에 먼저 적고 LEFT JOIN을 붙인다"고 기억하면, RIGHT JOIN이 필요한 상황도 테이블 순서만 바꿔 LEFT JOIN으로 표현할 수 있습니다.

절 요약

- 외부 조인은 기준 쪽 행을 짜이 없어도 모두 살리고, 빈 자리를 NULL로 채웁니다.
- LEFT JOIN은 왼쪽을, RIGHT JOIN은 오른쪽을 모두 살리며, 둘은 테이블 순서를 바꾼 관계입니다.
- WHERE 짜이 IS NULL로 "짜이 없는 행만" 골라내는 패턴이 자주 쓰입니다.

여러 테이블 JOIN과 테이블 별칭(alias)

도입

지금까지는 두 테이블만 묶었습니다. 그런데 "어떤 회원이, 어떤 상품을, 몇 개 주문했는가"를 한 화면에 보려면 `members` · `orders` · `products` 세 테이블을 모두 연결해야 합니다. 이 절에서는 세 개 이상의 테이블을 잇는 다중 조인과, 긴 쿼리를 짧고 명확하게 만드는 테이블 별칭(alias)을 배웁니다.

핵심 개념 설명

다중 조인 (multiple JOIN)

다중 조인(multiple JOIN)이란, `JOIN ... ON`을 여러 번 이어 붙여 세 개 이상의 테이블을 연결하는 것입니다. 주문(`orders`)을 가운데 두고, 한쪽으로는 회원(`members`)을, 다른 쪽으로는 상품

(products)을 연결하는 식입니다.

- **왜(why) 쓰나:** 한 질문에 필요한 정보가 세 테이블에 흩어져 있을 때, 하나의 조회로 모두 모아 보기 위해서입니다.
- **어떻게(how) 쓰나:** FROM orders JOIN members ON ... JOIN products ON ... 처럼 조인을 차례로 덧붙입니다. 각 JOIN 마다 그에 맞는 ON 조건을 따로 적습니다.

테이블 별칭 (table alias)

테이블 별칭(table alias)이란, 테이블에 짧은 이름을 붙여 쿼리에서 대신 부르는 것입니다.

FROM orders AS o 처럼 쓰면 이후 orders.member_id 를 o.member_id 로 짧게 적을 수 있습니다.

AS 는 생략해 FROM orders o 라고만 써도 됩니다.

별칭은 단순히 타이핑을 줄이는 것을 넘어, 같은 테이블을 두 번 조인하는 자기 조인(다음 절)에서는 **꼭 필요**합니다. 또 컬럼 앞에 o., m., p. 처럼 짧은 접두어를 붙이면 어느 테이블의 컬럼인지 한눈에 들어와 쿼리가 읽기 쉬워집니다.

이렇게 작성합니다

다음은 세 테이블을 모두 연결해 "누가, 무엇을, 몇 개" 주문했는지 한 줄로 보여 주는 다중 조인입니다. 별칭 m·o·p 를 사용했습니다.

```
-- 회원·주문·상품을 모두 연결해 주문 상세 보기
SELECT m.name      AS 회원,
       p.name      AS 상품,
       o.quantity  AS 수량,
       o.ordered_at AS 주문일
FROM orders AS o
JOIN members AS m ON o.member_id = m.id
JOIN products AS p ON o.product_id = p.id
ORDER BY o.ordered_at;
```

예상 화면:

회원	상품	수량	주문일
김민수	기계식 키보드	1	2026-06-10
김민수	머그컵	2	2026-06-11
박지훈	무선 마우스	1	2026-06-12

(3 rows)

번호만 적혀 있던 주문이, 회원 이름·상품 이름·수량·주문일이 모두 채워진 읽기 쉬운 표로 바뀌었습니다. orders 를 기준(o)으로 두고, member_id 로 회원, product_id 로 상품을 각각 연결한

결과입니다.

구문 요소 설명

구문	의미
FROM orders AS o	기준 테이블 orders 에 별칭 o 를 붙입니다
JOIN members AS m ON o.member_id = m.id	회원을 연결하는 첫 번째 조인입니다
JOIN products AS p ON o.product_id = p.id	상품을 연결하는 두 번째 조인입니다
m.name AS 회원	컬럼에도 별칭을 붙여 결과 머리글을 보기 좋게 바꿉니다
ORDER BY o.ordered_at	결과를 주문일 순으로 정렬합니다

조인을 추가할 때마다 결과의 의미가 어떻게 좁혀지는지 생각하면 좋습니다. 위 쿼리는 모두 INNER JOIN (내부 조인)이므로, 회원 정보나 상품 정보가 짝지어지지 않는 주문은 결과에서 빠집니다. 만약 상품이 삭제되어 product_id 의 짝이 없는 주문까지 보고 싶다면, 해당 조인을 LEFT JOIN 으로 바꾸면 됩니다.

팁: 별칭은 짧고 의미가 통하게 짓습니다. members 는 m, orders 는 o, products 는 p 처럼 첫 글자를 쓰는 관례가 흔합니다. 다만 별칭을 한번 정하면 그 쿼리 안에서는 원래 이름(members) 대신 별칭(m)으로만 불러야 합니다. 별칭을 정해 놓고 members.id 라고 다시 쓰면 오류가 납니다.

주의: 조인이 늘수록 조건도 늘어야 합니다

테이블을 N개 조인하면 보통 ON 조건이 N-1개 필요합니다(세 테이블이면 조건 두 개). 조인은 추가했는데 그에 맞는 ON 조건을 빠뜨리면, 짝짓지 못한 모든 행이 마구 곱해지는 의도치 않은 교차 조인이 일어나 결과가 폭발적으로 늘어납니다. 조인 개수와 조건 개수가 맞는지 늘 점검합니다.

절 요약

- JOIN ... ON 을 여러 번 이어 세 개 이상의 테이블을 연결할 수 있습니다.
- 테이블 별칭(AS m)으로 쿼리를 짧고 읽기 쉽게 만들며, 컬럼 별칭(AS 회원)으로 머리글도 다듬습니다.

- 조인 N개에는 보통 ON 조건 N-1개가 필요하며, 빠뜨리면 결과가 잘못 부풀려집니다.

자기 조인·교차 조인 살펴보기

도입

조인은 꼭 서로 다른 두 테이블에만 쓰는 것이 아닙니다. **같은 테이블을 두 번 참조**해 자기 자신과 조인할 수도 있고(자기 조인), 조건 없이 **모든 조합**을 만드는 조인도 있습니다(교차 조인). 이 절에서는 이 두 가지 특수한 조인을 살펴보고, 특히 교차 조인의 주의점을 짚습니다.

핵심 개념 설명

자기 조인 (self join)

자기 조인(self join)이란, 하나의 테이블을 서로 다른 두 별칭으로 두 번 불러 자기 자신과 조인하는 것입니다. 같은 명부 안에서 "이 사람의 추천인은 누구인가"처럼, **한 행이 같은 테이블의 다른 행을 가리킬 때** 사용합니다.

예를 들어 `members` 테이블에 "나를 추천한 회원의 번호"를 담는 `referrer_id` 컬럼이 있다고 하면, 그 번호 역시 같은 `members` 테이블의 `id`를 가리킵니다. 회원과 추천인이 모두 같은 회원 테이블에 있으므로, 테이블을 두 번 불러 한쪽은 "본인", 다른 쪽은 "추천인"으로 삼아 조인합니다. 이때 두 별칭이 없으면 어느 쪽이 본인이고 어느 쪽이 추천인인지 구분할 수 없으므로, **별칭은 자기 조인에서 필수**입니다.

교차 조인 (CROSS JOIN)

교차 조인(CROSS JOIN)이란, 조인 조건 없이 한쪽 테이블의 모든 행을 다른 쪽 테이블의 모든 행과 짝지어, 가능한 **모든 조합**을 만드는 조인입니다. 이를 곱집합(cartesian product)이라고도 합니다.

교차 조인은 결과 행 수가 두 테이블 행 수의 **곱**이 됩니다. 10행과 10행이면 100행, 1000행과 1000행이면 100만 행이 됩니다. 그래서 의도하지 않은 교차 조인은 흔히 성능 사고의 원인이 됩니다. 앞 절에서 "ON 조건을 빠뜨리면 결과가 폭발한다"고 한 것이 바로 이 교차 조인이 일어났기 때문입니다. 반대로 "모든 사이즈 x 모든 색상의 조합표"처럼 일부러 모든 조합이 필요할 때는 유용하게 씁니다.

이렇게 작성합니다

먼저 자기 조인 예시입니다. `members` 에 `referrer_id` (추천인 번호) 컬럼이 있다고 가정하고, 각 회원과 그를 추천한 회원의 이름을 함께 조회합니다.

```
-- 회원과 그 추천인을 같은 테이블에서 짝지어 읽기
SELECT m.name AS 회원,
       r.name AS 추천인
FROM members AS m
LEFT JOIN members AS r ON m.referrer_id = r.id;
```

예상 화면:

```
회원 | 추천인
-----+-----
김민수 |
박지훈 | 김민수
이서연 | 김민수
(3 rows)
```

`members` 를 `m` (본인)과 `r` (추천인) 두 별칭으로 불러 자기 자신과 조인했습니다. 박지훈 과 이서연 은 김민수 의 추천으로 가입했고, 김민수 는 추천인이 없어(`referrer_id` 가 NULL) 추천인 칸이 비어 있습니다. 추천인이 없는 회원도 보이도록 `LEFT JOIN` 을 썼습니다.

구문 요소 설명

구문	의미
<code>FROM members AS m</code>	같은 테이블을 "본인" 역할로 부릅니다
<code>LEFT JOIN members AS r</code>	같은 테이블을 다시 "추천인" 역할로 부릅니다
<code>ON m.referrer_id = r.id</code>	본인의 추천인 번호가 가리키는 회원을 찾습니다
별칭 <code>m, r</code>	같은 테이블을 구분하므로 자기 조인에 필수입니다

다음은 교차 조인 예시입니다. 티셔츠의 모든 사이즈와 모든 색상 조합표를 만들어 봅니다(설명용 작은 임시 테이블을 가정합니다).

```
-- 모든 사이즈 x 모든 색상 조합 만들기
SELECT s.label AS 사이즈, c.label AS 색상
FROM sizes AS s
CROSS JOIN colors AS c;
```

예상 화면:

사이즈	색상
S	검정
S	흰색
M	검정
M	흰색
L	검정
L	흰색

(6 rows)

사이즈 3개와 색상 2개의 모든 조합인 $3 \times 2 = 6$ 행이 만들어졌습니다. `ON` 조건이 아예 없는 점이 다른 조인과의 결정적 차이입니다.

주의: 실수로 만든 교차 조인 알아채기

`FROM a, b` 처럼 콤마로 테이블을 나열하고 `WHERE` 로 연결 조건을 깜빡 빠뜨려도 교차 조인이 됩니다. 결과 행 수가 비정상적으로 많다면(예: 두 테이블 행 수의 곱에 가까우면) 교차 조인을 의심합니다. 의도한 교차 조인이 아니라면 `JOIN ... ON` 으로 명확히 조건을 적는 습관이 안전합니다.

팁: 자기 조인은 추천인-피추천인, 직원-상사, 카테고리-상위 카테고리처럼 **한 테이블 안에서 행끼리 계층·참조 관계**가 있을 때 자주 등장합니다. "같은 테이블을 역할만 나눠 두 번 본다"고 생각하면 어렵지 않습니다.

절 요약

- 자기 조인은 같은 테이블을 두 별칭으로 불러 행끼리 연결하며, 별칭이 반드시 필요합니다.
- 교차 조인은 조건 없이 모든 조합을 만들어 결과가 두 테이블 행 수의 곱이 됩니다.
- 의도치 않은 교차 조인은 성능 사고의 원인이므로, 조인 조건을 빠뜨리지 않도록 주의합니다.

실습 과제

해보기: 주문 상세 조회 화면용 조인 쿼리 만들기

실습 데이터베이스 `shop_lab` 에서 `members` · `orders` · `products` 세 테이블을 직접 조인해 봅니다. 데이터가 없다면 3장의 `INSERT` 로 회원·상품을 채우고, `orders` 에 주문 몇 건을 넣어 둡니다.

- 단계 1: `members` 와 `orders` 를 `INNER JOIN` 으로 묶어 "주문이 있는 회원의 이름과 주문 수량"을 조회합니다. 별칭 `m`, `o` 를 사용합니다.
- 단계 2: 세 테이블을 모두 `JOIN` 해 "누가(회원 이름), 무엇을(상품 이름), 몇 개(수량), 언제(주문일)" 주문했는지 한 화면으로 보여 주는 쿼리를 만들고, `ORDER BY` 로 주문일 순으로 정렬합니다.
- 단계 3: 같은 쿼리에서 `members` 와 `orders` 의 조인을 `LEFT JOIN` 으로 바꿔, **주문이 없는 회원까지 포함**하는 결과와 비교합니다. 주문 없는 회원의 상품·수량 칸이 `NULL`로 비는지 확인합니다.
- 점검: `WHERE o.id IS NULL` 을 덧붙여 "주문을 한 번도 안 한 회원"만 추려 봅니다. 그 회원 수가 전체 회원 수에서 주문한 회원 수를 뺀 값과 맞는지 확인합니다.

FAQ

Q1. `INNER JOIN` 과 그냥 `JOIN` 은 다른가요? → A1. 같습니다. `JOIN` 이라고만 쓰면 PostgreSQL은 `INNER JOIN` 으로 해석합니다. `INNER` 는 "양쪽에 짝이 있는 행만"이라는 의미를 분명히 드러내기 위한 선택적 단어입니다. 반대로 `LEFT` · `RIGHT` · `FULL` 을 붙이면 외부 조인이 되어 짝 없는 행도 포함됩니다.

Q2. `LEFT JOIN` 과 `RIGHT JOIN` 중 무엇을 써야 하나요? → A2. 둘은 기준 테이블의 방향만 다를 뿐 표현력은 같습니다. `A LEFT JOIN B` 와 `B RIGHT JOIN A` 는 같은 결과를 냅니다. 실무에서는 "모두 살릴 테이블을 `FROM` 에 먼저 적고 `LEFT JOIN` 을 붙인다"는 한 가지 흐름으로 통일하는 경우가 많아, 대개 `LEFT JOIN` 만 써도 충분합니다.

Q3. 조인 결과의 행 수가 원래 테이블보다 많아졌습니다. 정상인가요? → A3. 정상일 수 있습니다. 한 회원이 주문을 세 번 했다면, 회원-주문 조인에서 그 회원은 세 줄로 나타납니다. 조인 결과 행 수는 "짝이 맞물린 횟수"만큼 늘어납니다. 다만 예상보다 지나치게 많다면(거의 두 테이블 행 수의 곱이라면) `ON` 조건을 빠뜨려 교차 조인이 일어난 것은 아닌지 점검합니다.

Q4. 컬럼 앞에 테이블.컬럼 을 꼭 붙여야 하나요? → A4. 두 테이블에 같은 이름의 컬럼(예: `id`) 이 있을 때는 반드시 붙여야 합니다. 안 붙이면 "어느 `id`인지 모호하다"는 오류가 납니다. 한쪽에만 있는 컬럼은 생략해도 되지만, 조인 쿼리에서는 어느 테이블 컬럼인지 늘 밝히는 편이 읽기에도 좋고 실수도 줄여 줍니다.

장 요약

이 장에서는 나뉘어 저장된 테이블을 공통 키로 다시 합쳐 읽는 JOIN 을 배웠습니다. 데이터를 나눠 두는 이유(중복 제거)와 조인의 기본 동작을 이해하고, 양쪽에 짝이 있는 행만 결합하는 INNER JOIN, 기준 쪽을 모두 살리고 빈 자리를 NULL로 채우는 LEFT · RIGHT OUTER JOIN 을 익혔습니다. 이어서 세 개 이상의 테이블을 잇는 다중 조인과 쿼리를 간결하게 하는 테이블 별칭, 그리고 같은 테이블을 두 번 참조하는 자기 조인과 모든 조합을 만드는 교차 조인까지 살펴보았습니다. 조인은 조회의 폭을 넓혀 주는 핵심 기술이므로, 어느 컬럼을 ON으로 맞물릴지 정확히 짚는 연습이 가장 중요합니다.

핵심 키워드

JOIN, INNER JOIN, ON, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, 다중 조인(multiple JOIN), 테이블 별칭(table alias), 자기 조인(self join), 교차 조인(CROSS JOIN)

다음 장 미리보기

다음 장에서는 이미 저장된 데이터를 고치는 UPDATE 를 배웁니다. WHERE 로 바꿀 행을 정확히 좁히고, 여러 컬럼을 한 번에 수정하며, 잘못된 일괄 수정을 막는 안전한 습관까지 다루는 수정(Update)의 기본기를 익힙니다.

UPDATE: 데이터 수정(Update)

학습 목표 - UPDATE ... SET ... WHERE로 조건에 맞는 행만 갱신할 수 있습니다. - WHERE 누락이 전체 갱신을 일으키는 위험을 설명하고 예방할 수 있습니다. - 여러 컬럼을 동시에, 또 다른 테이블을 참조해 갱신할 수 있습니다. - INSERT ON CONFLICT로 UPSERT를 구현할 수 있습니다.

3장에서 데이터를 새로 넣었고(INSERT), 4~6장에서 그 데이터를 읽는 법(SELECT)을 익혔습니다. 그런데 현실의 데이터는 한 번 적고 끝나지 않습니다. 회원이 비밀번호를 바꾸고, 상품 가격이 오르고, 재고가 줄어듭니다. 이렇게 **이미 들어 있는 값을 고쳐 쓰는 일**, 즉 CRUD의 수정(Update)을 맡는 명령이 이 장에서 배울 UPDATE입니다.

UPDATE는 장부에 적힌 값을 지우고 새 값으로 고쳐 쓰는 일과 같습니다. 새 줄을 추가하는 INSERT와 달리, 기존 줄의 내용을 바꿉니다. 강력한 만큼 위험도 큼니다. 조건을 빠뜨리면 단 한 줄로 테이블 전체가 한꺼번에 바뀔 수 있기 때문입니다. 이 장은 그 힘을 안전하게 다루는 법에 초점을 둡니다.

이 책은 Windows 11의 **PowerShell(파워셸)**에서 psql로 PostgreSQL에 접속한 상태를 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, psql은 그 안에서 데이터베이스와 대화하는 도구입니다. 화면 앞의 shop_lab=# 표시는 "지금 shop_lab 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 모두 아래 두 테이블을 기준으로 합니다. 앞 장에서 쓰던 것과 같은 모양이라고 생각하면 됩니다.

```
-- 이 장 예시가 기준으로 삼는 두 테이블 (참고용)
CREATE TABLE members (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        varchar(50) NOT NULL,
  email       varchar(100) NOT NULL UNIQUE,
  grade       varchar(20) DEFAULT '일반',
  points      integer DEFAULT 0,
  updated_at  timestamp DEFAULT now()
);

CREATE TABLE products (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        varchar(100) NOT NULL,
  category    varchar(30),
  price       numeric(10, 2) NOT NULL,
  stock       integer DEFAULT 0
);
```

email 에 UNIQUE (고유 제약)가 붙어 있다는 점을 기억해 둡니다. 같은 이메일이 두 번 들어올 수 없다는 규칙인데, 마지막 절에서 배울 UPSERT에서 핵심 역할을 합니다.

UPDATE 기본과 조건부 갱신

도입

장부에서 특정 한 줄을 찾아 값을 고쳐 적는다고 떠올려 봅시다. 먼저 "어느 줄인지"를 정하고, 그다음 "무엇을 어떻게 바꿀지"를 정합니다. UPDATE 도 똑같습니다. 다만 "어느 줄인지"를 빠뜨리면 모든 줄을 한꺼번에 고치는 사고가 납니다. 이 절에서는 UPDATE 의 기본형과, 입문자가 가장 많이 겪는 사고인 조건 누락을 막는 법을 배웁니다.

핵심 개념 설명

갱신 (UPDATE)

갱신(UPDATE) 이란, 테이블의 기존 행 값을 바꾸는 SQL 명령입니다. CRUD에서 수정(Update)에 해당하며, 장부에 적힌 값을 지우고 새 값으로 고쳐 쓰는 일과 같습니다.

기본 형태는 "어느 테이블을(UPDATE), 어떻게 바꾸고(SET), 어떤 행을 대상으로(WHERE)" 세 가지로 이루어집니다.

- **왜(why) 필요한가:** 데이터는 시간이 지나면 변합니다. 회원 등급이 올라가고, 가격이 조정 되고, 상태가 바뀝니다. 새로 넣고(INSERT) 지우는(DELETE) 대신, 있던 값을 그대로 두고 일부만 고치는 가장 자연스러운 방법이 UPDATE 입니다.
- **언제(when) 쓰나:** 한 회원의 등급을 바꿀 때, 특정 상품의 가격을 올릴 때, 주문 상태를 '배송중'으로 바꿀 때처럼 "이미 있는 행의 값을 바꾸는" 모든 순간입니다.
- **어떻게(how) 쓰나:** UPDATE 테이블 SET 컬럼 = 새값 WHERE 조건 순서로 적습니다. WHERE 로 대상 행을 좁히는 것이 핵심입니다.

여기서 입문자가 자주 하는 오해 두 가지를 짚겠습니다. 첫째, UPDATE 가 새 행을 추가한다고 생각하는 경우입니다. UPDATE 는 절대 행 수를 늘리지 않습니다. 기존 행의 값만 바꿉니다. 둘째, WHERE 없는 UPDATE 가 한 행만 바꾼다고 생각하는 경우입니다. 사실은 정반대로, **모든 행**이 바뀝니다.

WHERE 없는 UPDATE 위험 (unfiltered update)

WHERE 없는 UPDATE 위험(unfiltered update)이란, WHERE 조건을 빠뜨린 UPDATE 가 테이블의 **모든 행**을 한꺼번에 바꿔 버리는 사고를 말합니다. 한 줄짜리 명령으로 수만 건의 데이터가 같은 값으로 덮어써질 수 있어, 실무에서 가장 무서운 실수 중 하나로 꼽힙니다.

UPDATE — WHERE 유무에 따른 결과 비교

WHERE 조건 있음 — 안전

```
UPDATE products
SET price = 1200
WHERE id = 3;
```

id = 3인 행만 갱신 대상

id	name	price
1	상품A	1000
2	상품B	1500
3	상품C	1200
4	상품D	800
5	상품E	2000

1건만 변경, 4건 유지됨

id=3인 행만 price가 1200으로 변경
 나머지 행은 영향 없음
 의도한 대로 정확히 수정됩니다

WHERE 생략 — 전체 변경 위험!

```
UPDATE products
SET price = 1200;
-- WHERE 없음! 전체 갱신 주의
```

조건 없음 — 모든 행이 갱신 대상!

id	name	price
1	상품A	1200
2	상품B	1200
3	상품C	1200
4	상품D	1200
5	상품E	1200

5건 전체 변경 — 의도치 않은 덮어쓰기!

모든 행의 price가 1200으로 덮어쓰워짐
 UPDATE 전 WHERE 절을 반드시 확인하세요
 BEGIN 후 확인하고 COMMIT하는 습관 권장

VS

ch_07_s01 · UPDATE 기본과 조건부 갱신

같은 UPDATE 문이라도 WHERE 가 있느냐 없느냐에 따라 결과가 완전히 달라집니다. 아래 표로 두 경우를 비교합니다.

WHERE 유무에 따른 UPDATE 결과

구분	WHERE 있음	WHERE 없음
대상 행	조건을 만족하는 일부 행	테이블의 모든 행
결과 메시지	UPDATE 1 등 일부 건수	UPDATE 1000 등 전체 건수
사용 의도	특정 행만 고치기	정말 전부 바꿀 때만(드물)
위험도	낮음	높음(되돌리기 어려움)

이렇게 작성합니다

다음은 `id` 가 3인 상품 한 건의 가격을 25000원으로 고치는 기본적인 `UPDATE` 입니다.

```
-- id가 3인 상품의 가격을 25000으로 수정하기
UPDATE products
SET price = 25000
WHERE id = 3;
```

예상 화면:

```
shop_lab=# UPDATE products
shop_lab=# SET price = 25000
shop_lab=# WHERE id = 3;
UPDATE 1
```

마지막 줄의 `UPDATE 1` 은 "갱신 명령이 성공했고, 실제로 바뀐 행이 1건"이라는 뜻입니다. 이 숫자가 의도한 건수와 같은지 확인하는 습관이 중요합니다.

구문 요소 설명

구문	의미
UPDATE products	products 테이블의 행을 고치겠다고 지정합니다
SET price = 25000	price 컬럼을 새 값으로 바꿉니다
WHERE id = 3	바꿀 대상 행을 조건으로 좁힙니다
UPDATE 1	실제로 바뀐 행이 1건임을 알려 줍니다

이제 WHERE 를 빠뜨리면 어떤 일이 벌어지는지 봅시다. 다음은 의도와 달리 **모든 상품**의 가격이 0이 되어 버리는 위험한 예시입니다.

```
-- 위험 예시: WHERE를 빠뜨려 모든 상품 가격이 바뀐다
UPDATE products
SET price = 0;
```

예상 화면:

```
UPDATE 6
```

UPDATE 1 이 아니라 UPDATE 6 이 나왔습니다. 한 건만 고치려 했는데 테이블의 6건 전부가 0원이 된 것입니다. 이미 COMMIT 된 뒤라면 되돌리기 어렵습니다.

위험: UPDATE 와 DELETE 는 WHERE 를 빠뜨려도 오류 없이 그냥 실행됩니다. PostgreSQL은 "정말 전부 바꿀 생각인가요?"라고 묻지 않습니다. 따라서 WHERE 누락은 전적으로 작성자의 책임입니다. 갱신 전에는 항상 WHERE 절이 있는지 눈으로 확인합니다.

팁: 갱신하기 전에 같은 WHERE 로 SELECT 를 먼저 실행해 봅시다. 예를 들어 SELECT * FROM products WHERE id = 3; 을 돌려 대상이 정확히 1건인지 확인한 뒤, SELECT * FROM products WHERE id = 3 부분을 UPDATE products SET ... WHERE id = 3 으로 바꿔 실행하면 사고를 크게 줄일 수 있습니다. "조회로 먼저 겨냥하고, 같은 조건으로 갱신"하는 순서를 습관으로 만듭니다.

절 요약

- UPDATE 테이블 SET 컬럼 = 값 WHERE 조건 으로 조건에 맞는 행만 고칩니다.
- WHERE 를 빠뜨리면 테이블의 모든 행이 바뀌므로 항상 확인합니다.
- 결과 메시지의 건수(UPDATE N)가 의도한 수와 같은지 점검합니다.
- 갱신 전 같은 WHERE 로 SELECT 를 먼저 돌려 대상을 확인합니다.

여러 컬럼 동시에 갱신하기

도입

상품 가격을 올리면서 동시에 카테고리도 바꿔야 할 때가 있습니다. 컬럼마다 `UPDATE` 를 따로 실행할 수도 있지만, 한 번에 처리하는 편이 빠르고 안전합니다. 또 "현재 재고에서 10을 뺀 값"처럼 **기존 값을 이용해** 새 값을 계산하는 갱신도 자주 필요합니다. 이 절에서는 여러 컬럼을 한 번에 바꾸고, 기존 값을 계산에 활용하는 법을 배웁니다.

핵심 개념 설명

다중 컬럼 갱신 (multi-column update)

다중 컬럼 갱신(multi-column update) 이란, 하나의 `UPDATE` 문에서 `SET a = ..., b = ...` 처럼 여러 컬럼을 쉼표로 나열해 동시에 바꾸는 것입니다. 같은 행을 두 번 찾아갈 필요 없이 한 번에 여러 값을 고칩니다.

- **왜(why) 쓰나:** 컬럼마다 `UPDATE` 를 따로 실행하면 같은 행을 여러 번 찾아가 느리고, 중간에 하나만 실패하면 데이터가 어긋날 수 있습니다. 한 문장으로 묶으면 함께 적용됩니다.
- **언제(when) 쓰나:** 등급과 적립 포인트를 함께 조정할 때, 가격과 재고를 동시에 바꿀 때처럼 한 행의 여러 값을 같이 바꿀 때입니다.
- **어떻게(how) 쓰나:** `SET 컬럼1 = 값1, 컬럼2 = 값2` 형태로 쉼표로 이어 적습니다.

표현식 갱신 (expression update)

표현식 갱신(expression update) 이란, `SET` 오른쪽에 고정된 값 대신 **계산식**을 적어, 기존 값을 바탕으로 새 값을 만드는 것입니다. 예를 들어 `price = price * 1.1` 은 "지금 가격에서 10% 올린 값"을 뜻합니다.

여기서 `SET price = price * 1.1` 의 오른쪽 `price` 는 **갱신되기 전의 현재 값**을 가리킵니다. 같은 행의 기존 값을 읽어 계산한 뒤 새 값으로 덮어쓰는 것이라, "값을 올리거나 줄이는" 상대적 갱신에 매우 유용합니다.

이렇게 작성합니다

다음은 `id` 가 5인 회원의 등급과 포인트를 한 번에 바꾸는 예시입니다.

```
-- 회원 한 명의 등급과 포인트를 동시에 갱신하기
UPDATE members
SET grade = 'VIP',
    points = 1000,
    updated_at = now()
WHERE id = 5;
```

예상 화면:

UPDATE 1

SET 뒤에 세 컬럼을 쉼표로 나열했고, 한 번의 명령으로 모두 바뀌었습니다. `updated_at = now()` 처럼 "수정 시각"을 함께 기록해 두면 언제 바뀌었는지 추적하기 좋습니다.

구문 요소 설명

구문	의미
SET grade = 'VIP'	첫 번째 컬럼을 새 값으로 바꿉니다
, points = 1000	쉼표로 이어 두 번째 컬럼도 바꿉니다
, updated_at = now()	현재 시각 함수로 수정 시각을 기록합니다
WHERE id = 5	대상 행을 한 건으로 좁힙니다

다음은 표현식 갱신 예시입니다. 전자기기 카테고리 상품의 가격을 10% 인상하면서 재고는 5씩 줄입니다.

```
-- 전자기기 가격은 10% 인상, 재고는 5 감소
UPDATE products
SET price = price * 1.1,
    stock = stock - 5
WHERE category = '전자기기';
```

예상 화면:

UPDATE 4

`price * 1.1` 과 `stock - 5` 의 왼쪽 기준값은 각 행의 **기존 값**입니다. 따라서 가격이 19900원이던 행은 21890원이 되고, 89000원이던 행은 97900원이 됩니다. 조건에 맞는 4건이 각자 자기 값을

기준으로 따로 계산됩니다.

```
-- 갱신 결과 확인하기
SELECT name, price, stock FROM products WHERE category = '전자기기';
```

예상 화면:

name	price	stock
무선 마우스	21890.00	45
기계식 키보드	97900.00	15

(2 rows)

주의: 표현식 갱신에서 `stock = stock - 5` 처럼 빼는 경우, 재고가 5보다 적은 행은 음수가 될 수 있습니다. 음수 재고를 막으려면 `WHERE category = '전자기기' AND stock >= 5` 처럼 조건을 더 좁히거나, 9장에서 배울 `CHECK` 제약으로 `stock >= 0` 을 강제합니다. 계산 갱신은 항상 "경계값(0, 음수)"을 함께 떠올립니다.

팁: 갱신할 컬럼이 많아도 `SET` 뒤에 순서는 자유입니다. 다만 한 `UPDATE` 안에서 같은 컬럼을 두 번 지정할 수는 없습니다. 또 `SET a = b, b = a` 처럼 두 컬럼 값을 맞바꾸려 해도, 오른쪽은 모두 갱신 전 값을 기준으로 계산되므로 의도대로 서로 바뀝니다(임시 변수가 필요 없습니다).

절 요약

- `SET 컬럼1 = 값1, 컬럼2 = 값2` 로 여러 컬럼을 한 번에 갱신합니다.
- `SET price = price * 1.1` 처럼 기존 값을 이용한 계산식을 쓸 수 있습니다.
- 오른쪽 표현식의 컬럼 값은 모두 갱신 전 값을 기준으로 계산됩니다.
- 빼기 계산은 음수 같은 경계값을 함께 고려합니다.

다른 테이블을 참조해 갱신하기(UPDATE ... FROM)

도입

지금까지는 고정값이나 같은 행의 값으로만 갱신했습니다. 그런데 "입고 테이블에 들어온 수량 만큼 상품 재고를 늘려라"처럼, **다른 테이블의 값**을 가져와 갱신해야 할 때가 있습니다. 값을 일일이 찾아 손으로 적는 대신, 두 테이블을 연결해 한 번에 반영할 수 있습니다. 이 절에서 배우는 `UPDATE ... FROM` 입니다.

핵심 개념 설명

참조 갱신 (UPDATE ... FROM)

참조 갱신(UPDATE ... FROM) 이란, 갱신할 대상 테이블 외에 **FROM** 으로 다른 테이블을 끌어와, 그 테이블의 값을 이용해 대상을 고치는 방법입니다. PostgreSQL 고유의 편리한 확장으로, 6장에서 배운 조인(JOIN)과 비슷한 원리로 두 테이블을 맞물립니다.

- **왜(why) 쓰나:** 갱신할 값이 다른 테이블에 들어 있을 때, 그 값을 사람이 옮겨 적으면 느리고 실수가 생깁니다. 데이터베이스가 직접 연결해 채우면 빠르고 정확합니다.
- **언제(when) 쓰나:** 입고 데이터로 재고를 갱신할 때, 집계 결과를 요약 테이블에 반영할 때, 한 테이블의 값을 기준으로 다른 테이블을 일괄 보정할 때입니다.
- **어떻게(how) 쓰나:** UPDATE 대상 SET ... FROM 참조테이블 WHERE 대상.키 = 참조테이블.키 형태로, **FROM** 으로 가져온 테이블과 **WHERE** 로 연결 조건을 맞춥니다.

조인 기반 갱신 (join update)

조인 기반 갱신(join update) 이란, **WHERE** 의 연결 조건으로 두 테이블의 행을 짝지어, 짝이 맞는 대상 행만 참조 테이블의 값으로 갱신하는 것을 말합니다. **WHERE** 에는 연결 조건과 갱신 대상을 좁히는 조건을 함께 적을 수 있습니다.

여기서 가장 중요한 점은 **FROM** 절의 테이블과 대상 테이블을 잇는 **연결 조건을 반드시 WHERE 에 적어야 한다**는 것입니다. 연결 조건을 빠뜨리면 모든 조합이 짝지어져, 의도와 전혀 다른 값으로 전체가 갱신됩니다. 앞 절의 "WHERE 누락"만큼 위험합니다.

이렇게 작성합니다

먼저 입고 수량을 담은 참조 테이블이 있다고 가정합니다.

```
-- 입고 데이터를 담은 참조 테이블 (참고용)
CREATE TABLE restock (
    product_id integer NOT NULL,
    quantity   integer NOT NULL
);

INSERT INTO restock (product_id, quantity)
VALUES (1, 30), (3, 50);
```

이제 **restock** 의 입고 수량만큼 **products** 의 재고를 늘립니다.

```
-- restock의 입고 수량만큼 products의 재고를 더하기
UPDATE products AS p
SET stock = p.stock + r.quantity
FROM restock AS r
WHERE p.id = r.product_id;
```

예상 화면:

UPDATE 2

restock 에 두 건(상품 1번, 3번)이 있으므로 products 에서 짝이 맞는 2건만 갱신됩니다. 1번 상품 재고에는 30이, 3번 상품 재고에는 50이 더해집니다.

구문 요소 설명

구문	의미
UPDATE products AS p	갱신 대상 테이블에 별칭 p 를 붙입니다
SET stock = p.stock + r.quantity	대상의 기존 재고에 참조 테이블 수량을 더합니다
FROM restock AS r	값을 끌어올 참조 테이블을 별칭 r 로 지정합니다
WHERE p.id = r.product_id	두 테이블을 잇는 연결 조건(필수)입니다

별칭(p, r)을 쓰면 어느 테이블의 컬럼인지 분명해져 헷갈리지 않습니다. 6장에서 조인을 배울 때 익힌 테이블 별칭과 같은 방식입니다.

위험: WHERE p.id = r.product_id 같은 **연결 조건을 빠뜨리면** 대상 테이블의 모든 행이 참조 테이블의 첫 행 또는 임의의 행 값으로 갱신될 수 있습니다. 이는 조인에서 연결 조건을 빠뜨려 교차 조합(CROSS JOIN)이 되는 것과 같은 사고입니다. UPDATE ... FROM 에서는 WHERE 의 연결 조건이 있는지 가장 먼저 확인합니다.

팁: 참조 갱신도 실행 전에 SELECT 로 미리 확인할 수 있습니다. SELECT p.id, p.stock, r.quantity FROM products p JOIN restock r ON p.id = r.product_id; 처럼 조인으로 먼저 조회해, 어떤 행이 어떤 값으로 바뀔지 눈으로 점검한 뒤 갱신합니다.

절 요약

- `UPDATE` 대상 `SET ... FROM 참조 WHERE 대상.키 = 참조.키` 로 다른 테이블 값을 가져와 갱신합니다.
- `WHERE` 의 연결 조건은 필수이며, 빠뜨리면 전체가 잘못 갱신됩니다.
- 테이블 별칭을 쓰면 어느 테이블의 컬럼인지 분명해집니다.
- 실행 전 조인 `SELECT` 로 바뀔 결과를 미리 확인합니다.

UPSERT: 있으면 수정, 없으면 삽입(INSERT ON CONFLICT)

도입

회원을 등록하는데 "이미 가입한 이메일이면 정보를 갱신하고, 처음이면 새로 추가"하고 싶을 때가 있습니다. 먼저 조회해서 있는지 확인하고, 있으면 `UPDATE`, 없으면 `INSERT` 를 따로 실행할 수도 있습니다. 하지만 그 사이에 다른 요청이 끼어들면 꼬일 수 있고, 코드도 번거롭습니다. 이 일을 한 문장으로 안전하게 처리하는 것이 이 절에서 배우는 UPSERT입니다.

핵심 개념 설명

업서트 (UPSERT)

업서트(UPSERT)란, `INSERT` 를 시도하되 키가 충돌하면 기존 행을 `UPDATE` 하는 동작입니다.

"Update"와 "Insert"를 합친 말로, PostgreSQL에서는 `INSERT ... ON CONFLICT` 로 구현합니다. 출석부에 이름이 없으면 새로 적고, 이미 있으면 그 줄을 갱신하는 것과 같습니다.

- **왜(why) 쓰나:** "있으면 수정, 없으면 삽입"을 조회 후 분기로 처리하면 코드가 길고, 동시에 여러 요청이 들어올 때 같은 키가 중복 삽입되거나 충돌할 수 있습니다. UPSERT는 이 판단을 데이터베이스가 한 번에 안전하게 처리합니다.
- **언제(when) 쓰나:** 같은 키로 들어오는 데이터를 받아 최신 값으로 유지할 때(설정값, 일별 집계, 외부에서 동기화되는 데이터 등)입니다.
- **어떻게(how) 쓰나:** `INSERT ... VALUES ...` 뒤에 `ON CONFLICT (충돌컬럼)` 을 적고, 충돌 시 동작으로 `DO UPDATE SET ...` 또는 `DO NOTHING` 을 고릅니다.

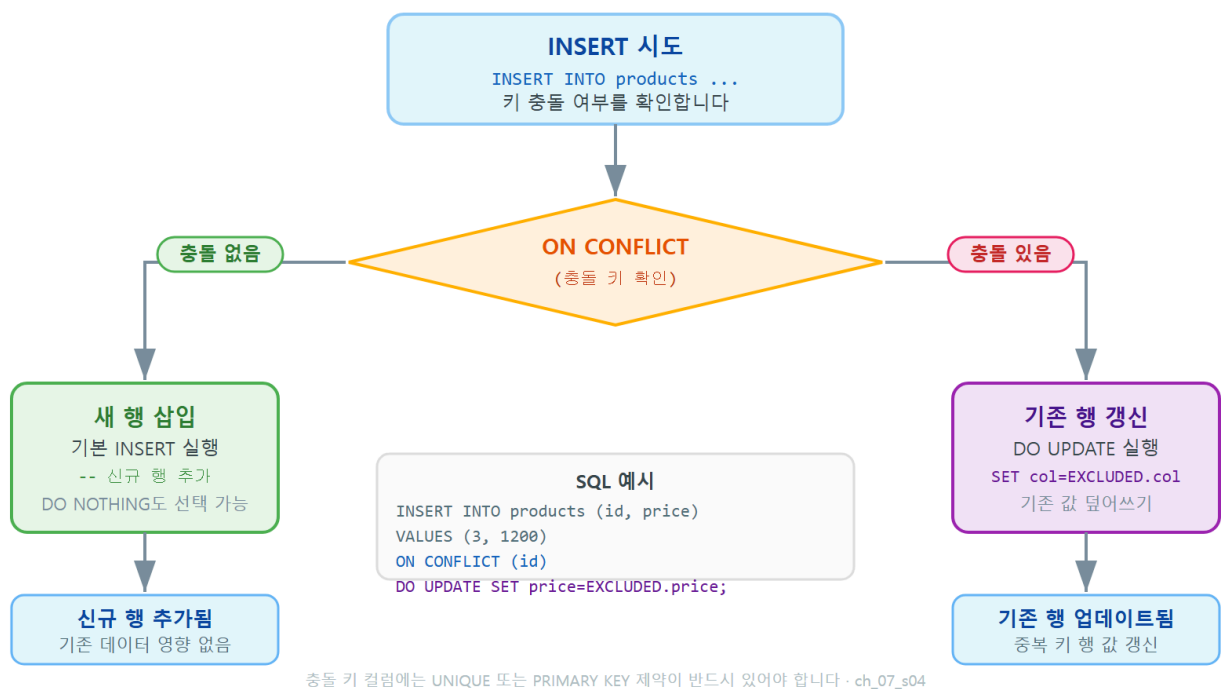
여기서 두 가지 오해를 짚겠습니다. 첫째, UPSERT가 별도의 SQL 키워드라고 생각하는 경우입니다. UPSERT는 개념의 이름일 뿐이고, 실제 문법은 `INSERT ... ON CONFLICT` 입니다. 둘째, 충돌 대상 컬럼에 고유 제약이 없어도 동작한다고 오해하는 경우입니다. `ON CONFLICT (컬럼)` 이 작동하

려면 그 컬럼에 **UNIQUE 제약**이나 **기본 키**가 반드시 있어야 합니다. 충돌을 판단할 기준이 바로 그 제약이기 때문입니다.

ON CONFLICT DO UPDATE

ON CONFLICT DO UPDATE 란, 충돌이 났을 때(같은 키가 이미 있을 때) 기존 행을 어떻게 갱신할지 지정하는 부분입니다. 여기서 새로 넣으려던 값은 **EXCLUDED** 라는 특별한 이름으로 참조합니다. 예를 들어 `SET points = EXCLUDED.points` 는 "이번에 삽입하려던 `points` 값으로 기존 행을 덮어쓰라"는 뜻입니다.

UPSERT — INSERT ON CONFLICT 분기 흐름도



DO UPDATE와 DO NOTHING 비교

동작	충돌이 없을 때	충돌이 있을 때	언제 쓰나
<code>DO UPDATE SET ...</code>	새 행 삽입	기존 행을 새 값으로 갱신	최신 값으로 유지하고 싶을 때
<code>DO NOTHING</code>	새 행 삽입	아무 일도 하지 않음(무시)	중복은 그냥 건너뛰고 싶을 때

이렇게 작성합니다

다음은 회원을 등록하되, 같은 email 이 이미 있으면 이름과 등급을 갱신하는 UPSERT입니다.
email 에 UNIQUE 제약이 있어야 충돌을 판단할 수 있다는 점을 기억합니다.

```
-- email이 같으면 갱신, 없으면 삽입하는 UPSERT
INSERT INTO members (name, email, grade)
VALUES ('박지훈', 'jihoon@example.com', 'VIP')
ON CONFLICT (email)
DO UPDATE SET name = EXCLUDED.name,
               grade = EXCLUDED.grade,
               updated_at = now();
```

예상 화면(처음 넣는 경우):

```
INSERT 0 1
```

같은 이메일로 한 번 더 실행하면, 이번에는 삽입 대신 기존 행이 갱신됩니다.

```
-- 같은 email로 다시 실행 (이번엔 갱신됨)
INSERT INTO members (name, email, grade)
VALUES ('박지훈(수정)', 'jihoon@example.com', '일반')
ON CONFLICT (email)
DO UPDATE SET name = EXCLUDED.name,
               grade = EXCLUDED.grade,
               updated_at = now();
```

예상 화면(충돌이 나 갱신된 경우):

```
INSERT 0 1
```

두 경우 모두 결과 메시지는 INSERT 0 1로 같지만, 첫 번째는 새 행이 생긴 것이고 두 번째는 기존 행이 갱신된 것입니다. 행 수가 늘지 않고 같은 이메일의 행 하나만 최신 값으로 유지됩니다.

구문 요소 설명

구문	의미
<code>INSERT INTO members (...) VALUES (...)</code>	평소처럼 삽입을 시도합니다
<code>ON CONFLICT (email)</code>	<code>email</code> 이 충돌(중복)하면 분기 처리합니다
<code>DO UPDATE SET ...</code>	충돌 시 기존 행을 새 값으로 갱신합니다
<code>EXCLUDED.name</code>	이번에 삽입하려던(충돌로 빠진) 값입니다

충돌을 그냥 무시하고 싶다면 `DO NOTHING` 을 씁니다. 다음은 같은 이메일이 있으면 조용히 넘어가는 예시입니다.

```
-- 이미 있는 email이면 무시하고 넘어가기
INSERT INTO members (name, email)
VALUES ('박지훈', 'jihoon@example.com')
ON CONFLICT (email) DO NOTHING;
```

예상 화면:

```
INSERT 0 0
```

`INSERT 0 0` 은 "삽입된 행이 0건", 즉 이미 있어서 아무것도 하지 않았다는 뜻입니다.

주의: `ON CONFLICT (email)` 의 괄호 안 컬럼에는 반드시 `UNIQUE` 제약이나 기본 키가 있어야 합니다. 제약이 없는 컬럼을 적으면 "그 컬럼에는 충돌을 판단할 제약이 없다"는 오류가 납니다. UPSERT를 쓰려면 먼저 9장에서 배울 고유 제약을 그 컬럼에 걸어 둡니다.

팁: `DO UPDATE` 에서 일부 컬럼만 새 값으로 덮고 싶을 때는 `SET` 뒤에 원하는 컬럼만 적습니다. 예를 들어 가입 시각(`created_at`)은 그대로 두고 등급만 갱신하려면 `created_at` 은 `SET` 에서 빼면 됩니다. `EXCLUDED` 는 "이번에 넣으려던 값", 컬럼 이름 그대로는 "기존 행의 값"을 의미한다는 점을 활용합니다.

절 요약

- UPSERT는 `INSERT ... ON CONFLICT` 로 "있으면 수정, 없으면 삽입"을 한 문장으로 처리합니다.
- `ON CONFLICT (컬럼)` 의 컬럼에는 `UNIQUE` 나 기본 키 제약이 있어야 합니다.
- `DO UPDATE SET ...` 은 기존 행을 갱신하고, `DO NOTHING` 은 충돌을 무시합니다.
- `EXCLUDED.컬럼` 은 이번에 삽입하려던 값을 가리킵니다.

실습 과제

해보기: 재고·가격 일괄 갱신과 UPSERT 적용하기

실습 데이터베이스 `shop_lab` 에서 이 장의 `UPDATE` 와 `UPSERT`를 직접 다뤄 봅니다.

- 단계 1: `products` 에서 카테고리가 '전자기기'인 상품의 가격을 15% 인상합니다. 먼저 같은 `WHERE` 로 `SELECT` 를 돌려 대상 건수를 확인한 뒤 `UPDATE` 를 실행하고, 결과 메시지의 건수가 일치하는지 점검합니다.
- 단계 2: 특정 회원 한 명의 `grade` 와 `points`, `updated_at` 을 한 번의 `UPDATE` 로 동시에 갱신합니다.
- 단계 3: `restock` 테이블을 만들어 입고 데이터 두 건을 넣고, `UPDATE products ... FROM restock` 으로 입고 수량만큼 재고를 늘립니다. 연결 조건(`WHERE p.id = r.product_id`)을 꼭 넣었는지 확인합니다.
- 단계 4: `members` 에 `INSERT ... ON CONFLICT (email) DO UPDATE` 로 회원 한 명을 등록하고, 같은 이메일로 한 번 더 실행해 행 수가 늘지 않고 값만 갱신되는지 확인합니다.
- 점검: 각 단계 전후로 `SELECT` 를 실행해 값이 의도대로 바뀌었는지, 행 수가 늘거나 줄지 않았는지 기록합니다.

FAQ

Q1. 실수로 `WHERE` 없이 `UPDATE` 를 실행했는데 되돌릴 수 있나요? → A1. 아직 `COMMIT` 전이라면 `ROLLBACK` 으로 되돌릴 수 있습니다. 그래서 실무에서는 위험한 갱신을 `BEGIN;` 으로 트랜잭션을 연 뒤 실행하고, 결과를 확인한 다음 `COMMIT` 하거나 `ROLLBACK` 합니다. 이미 `COMMIT` 되었다면 백업 복원밖에 방법이 없으므로, 트랜잭션과 백업 습관이 중요합니다(자세한 내용은 10장 트랜잭션에서 배웁니다).

Q2. `UPDATE` 결과가 `UPDATE 0` 이면 오류인가요? → A2. 오류가 아닙니다. `WHERE` 조건에 맞는 행이 하나도 없어서 바뀐 행이 0건이라는 뜻입니다. 명령 자체는 정상 실행된 것입니다. 만약 분명히 있어야 할 행이 0건으로 나온다면, `WHERE` 조건의 값이나 철자가 맞는지 `SELECT` 로 확인합니다.

Q3. `SET price = price * 1.1` 에서 오른쪽 `price` 는 바뀐 값인가요, 원래 값인가요? → A3. **갱신 전의 원래 값**입니다. PostgreSQL은 각 행에서 기존 값을 읽어 계산한 뒤 그 결과로 덮어씁니다.

그래서 한 `UPDATE` 안에서 `SET a = b, b = a` 로 두 컬럼을 맞바꿔도, 양쪽 모두 갱신 전 값을 기준으로 하므로 의도대로 서로 바뀝니다.

Q4. UPSERT의 `ON CONFLICT` 에 아무 컬럼이나 적어도 되나요? → A4. 안 됩니다. `ON CONFLICT` (컬럼) 의 컬럼에는 `UNIQUE` 제약이나 기본 키가 걸려 있어야 합니다. 충돌 여부를 판단할 기준이 바로 그 제약이기 때문입니다. 제약이 없는 컬럼을 적으면 오류가 납니다. 충돌 기준을 따로 정하지 않고 "어떤 충돌이든 무시"하려면 `ON CONFLICT DO NOTHING` 처럼 컬럼을 생략할 수도 있습니다.

장 요약

이 장에서는 CRUD의 수정(Update)을 `UPDATE` 로 배웠습니다. `UPDATE ... SET ... WHERE` 로 조건에 맞는 행만 고치되, `WHERE` 를 빠뜨리면 테이블 전체가 바뀌는 위험을 항상 경계해야 함을 익혔습니다. `SET` 에 여러 컬럼을 나열해 동시에 갱신하고, `price * 1.1` 같은 표현식으로 기존 값을 이용한 계산 갱신도 다루었습니다. `UPDATE ... FROM` 으로 다른 테이블의 값을 끌어와 갱신했고, `INSERT ... ON CONFLICT` 로 "있으면 수정, 없으면 삽입"하는 UPSERT까지 구현했습니다. 갱신은 강력한 만큼, 항상 같은 조건으로 먼저 `SELECT` 해 대상을 확인하는 습관이 안전한 사용의 핵심입니다.

핵심 키워드

`UPDATE`, `SET`, `WHERE`, 다중 컬럼 갱신(multi-column update), 표현식 갱신(expression update), `UPDATE ... FROM`, 업서트(UPSERT), `ON CONFLICT`, `EXCLUDED`

다음 장 미리보기

다음 장에서는 CRUD의 마지막 단계인 삭제(Delete)를 `DELETE` 로 배웁니다. 조건에 맞는 행만 안전하게 지우는 법, `DELETE` 와 `TRUNCATE` 의 차이, 외래 키로 연결된 데이터를 함께 지우는 `ON DELETE CASCADE`, 그리고 삭제 사고를 막는 트랜잭션·백업 습관을 다룹니다.

DELETE: 데이터 삭제(Delete)

학습 목표 - DELETE ... WHERE로 조건에 맞는 행만 안전하게 삭제할 수 있습니다. - DELETE와 TRUNCATE의 동작·속도·롤백 차이를 구분할 수 있습니다. - 외래 키의 ON DELETE CASCADE·RESTRICT·SET NULL 동작을 설명할 수 있습니다. - 트랜잭션과 백업으로 삭제 사고를 예방하는 습관을 적용할 수 있습니다.

앞 장에서 우리는 이미 있는 데이터를 고치는 UPDATE 를 배웠습니다. 이 장은 데이터의 한살이에 서 마지막 단계, 즉 더 이상 필요 없는 데이터를 **지우는 일**을 다룹니다. CRUD에서 삭제(Delete) 에 해당하는 DELETE 입니다.

삭제는 네 가지 작업 중 가장 조심스러운 작업입니다. 잘못 넣은 데이터는 다시 고치면 되고, 잘못 읽은 결과는 다시 읽으면 됩니다. 그러나 잘못 지운 데이터는 백업이 없으면 되돌리기 어렵 습니다. 그래서 이 장은 단순히 "지우는 문법"만이 아니라 **안전하게 지우는 습관**까지 함께 배웁 니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 에서 `psql` 로 PostgreSQL에 접속한 상태를 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, `psql` 은 그 안에서 데이터베이스와 대화하는 도구입니다. 화면 앞의 `shop_lab=#` 표시는 "지금 `shop_lab` 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 아래 세 테이블을 기준으로 합니다. `orders` (주문)는 `members` (회원)를 가리키는 **외래 키(foreign key)** 를 가지고 있는데, 이 관계가 뒤에서 연쇄 삭제를 배울 때 핵심이 됩니다.

```

-- 이 장 예시가 기준으로 삼는 세 테이블 (참고용)
CREATE TABLE members (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        varchar(50) NOT NULL,
  grade       varchar(20) DEFAULT 'normal',
  created_at  timestamp DEFAULT now()
);

CREATE TABLE products (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        varchar(100) NOT NULL,
  stock       integer DEFAULT 0
);

-- orders.member_id는 members.id를 가리키는 외래 키입니다
CREATE TABLE orders (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  member_id   integer REFERENCES members(id),
  amount      numeric(10, 2) NOT NULL,
  ordered_at  timestamp DEFAULT now()
);

```

여기서 `orders`의 `member_id REFERENCES members(id)` 부분이 외래 키입니다. "이 주문은 어느 회원의 것인가"를 회원 `id`로 연결해 둔 것입니다. 이 연결 때문에 회원을 함부로 지울 수 없게 되는데, 그 동작을 3절에서 자세히 다룹니다.

DELETE 기본과 안전한 삭제

도입

장부에서 특정 줄을 골라 지우는 장면을 떠올려 봅시다. "이 줄과 이 줄을 지운다"라고 대상을 정확히 가리켜야 의도한 것만 사라집니다. `DELETE`도 똑같습니다. 이 절에서는 조건에 맞는 행만 골라 지우는 기본형을 익히고, 지우기 전에 **무엇이 지워질지 먼저 확인하는** 안전 습관을 배웁니다.

핵심 개념 설명

삭제 (DELETE)

삭제(DELETE)란, 조건에 맞는 행을 테이블에서 제거하는 SQL 명령입니다. CRUD에서 삭제(Delete)에 해당하며, 장부에서 특정 줄을 골라 지우는 일과 같습니다.

기본 형태는 `DELETE FROM 테이블 WHERE 조건` 입니다. 여기서 가장 중요한 부분은 `WHERE` 입니다. `WHERE` 가 "어떤 행을 지울지"를 정하기 때문입니다.

- **왜(why) 필요한가:** 더 이상 유효하지 않은 데이터(탈퇴한 회원, 취소된 주문, 단종된 상품)는 그대로 두면 조화·집계 결과를 흐립니다. 깨끗한 데이터를 유지하려면 불필요한 행을 지울 수 있어야 합니다.
- **언제(when) 쓰나:** 특정 한 건을 지울 때(예: 잘못 들어간 주문 1건), 조건을 만족하는 여러 건을 한꺼번에 지울 때(예: 오래된 임시 데이터) 사용합니다.
- **어떻게(how) 쓰나:** `DELETE FROM 테이블` 로 대상 테이블을 정하고, `WHERE` 로 지울 행을 좁힙니다. `WHERE` 를 빠뜨리면 **테이블의 모든 행**이 지워집니다.

여기서 입문자가 자주 하는 두 가지 오해를 짚겠습니다. 첫째, `WHERE` 없는 `DELETE` 가 한 행만 지운다고 생각하는 경우입니다. 그렇지 않습니다. `WHERE` 가 없으면 **전체 행**이 사라집니다. 둘째, `DELETE` 가 테이블 자체를 없앤다고 오해하는 경우입니다. `DELETE` 는 행(내용)만 지우고 테이블(틀)은 그대로 남깁니다. 테이블 자체를 없애는 것은 2장에서 배운 `DROP TABLE` 입니다.

조건부 삭제 (filtered delete)

조건부 삭제(filtered delete)란, `WHERE` 조건을 붙여 "지정한 행만" 골라 지우는 것을 말합니다. `WHERE` 에는 4장에서 배운 비교·논리 연산자(`=`, `<>`, `>`, `AND`, `OR`, `IN`, `LIKE` 등)를 그대로 쓸 수 있습니다. 즉, **조회로 고를 수 있는 행이면 삭제로도 고를 수 있습니다.**

바로 이 점에서 안전한 삭제의 핵심 습관이 나옵니다. 지우기 전에 같은 `WHERE` 조건으로 먼저 `SELECT` 해 보면, 무엇이 몇 건 지워질지 눈으로 확인할 수 있습니다.

이렇게 작성합니다

다음은 `orders` 테이블에서 주문 한 건(`id` 가 3인 주문)을 지우는 가장 기본적인 `DELETE` 입니다. 먼저 지울 대상을 `SELECT` 로 확인하는 습관부터 보입니다.

```
-- 1단계: 지울 대상을 같은 조건으로 먼저 확인하기
SELECT id, member_id, amount FROM orders WHERE id = 3;
```

예상 화면:

```
id | member_id | amount
---+-----+-----
 3 |          2 | 15000.00
(1 row)
```


확인했으니 이제 같은 조건으로 지웁니다.

```
-- 2단계: 확인한 조건 그대로 삭제하기
DELETE FROM orders WHERE id = 3;
```

예상 화면:

```
DELETE 1
```

DELETE 1 은 "삭제 명령이 성공했고, 지워진 행이 1건"이라는 뜻입니다. 뒤의 숫자가 실제로 지워진 행 수입니다.

구문 요소 설명

구문	의미
DELETE FROM orders	orders 테이블에서 행을 지우겠다고 지정합니다
WHERE id = 3	지울 행을 조건으로 좁힙니다(이 조건이 없으면 전체 삭제)
DELETE 1	실제로 지워진 행 수가 1건임을 알려 줍니다

조건을 만족하는 여러 행을 한 번에 지울 수도 있습니다. 다음은 금액이 1000원 미만인 주문을 모두 지우는 예시입니다.

```
-- 조건을 만족하는 여러 행을 한 번에 삭제하기
DELETE FROM orders WHERE amount < 1000;
```

예상 화면:

```
DELETE 4
```

DELETE 4 는 조건에 맞는 4건이 지워졌다는 뜻입니다. RETURNING 을 붙이면 지워진 행의 내용을 돌려받아 무엇이 사라졌는지 확인할 수도 있습니다.

```
-- 무엇이 지워졌는지 RETURNING으로 확인하면서 삭제하기
DELETE FROM orders WHERE amount < 1000
RETURNING id, member_id, amount;
```

예상 화면:

```

id | member_id | amount
---+-----+-----
 7 |          1 | 500.00
 9 |          3 | 900.00
(2 rows)
DELETE 2

```

위험: WHERE 없는 DELETE는 전체 삭제 `DELETE FROM orders;` 처럼 `WHERE` 를 빠뜨리면 **테이블의 모든 행**이 지워집니다. 한 행만 지우려다 전 직원·전 주문을 날리는 사고가 여기서 납니다.

`DELETE` 를 적을 때는 `WHERE` 를 먼저 적고 조건을 채운 뒤, 그 앞에 `DELETE FROM ...` 을 붙이는 순서로 작성하는 습관이 안전합니다.

팁: 지우기 전 SELECT는 가장 싼 보험 삭제하려는 `WHERE` 조건을 그대로 `SELECT` 에 붙여 먼저 실행해 보면, 몇 건이 어떤 내용으로 지워질지 미리 알 수 있습니다. 한 줄 더 실행하는 수고로 대형 사고를 막는, 가장 값싼 보험입니다.

절 요약

- `DELETE FROM` 테이블 `WHERE` 조건 으로 조건에 맞는 행만 지웁니다.
- `WHERE` 를 빠뜨리면 테이블의 모든 행이 지워지므로 반드시 확인합니다.
- 지우기 전에 같은 조건으로 `SELECT` 해 대상을 미리 확인하는 습관을 들입니다.
- `DELETE` 는 행만 지우고 테이블 틀은 남기며, 지워진 건수는 `DELETE N` 으로 알려 줍니다.

TRUNCATE와 DELETE의 차이

도입

테이블을 통째로 비우고 싶을 때가 있습니다. 예를 들어 테스트가 끝난 임시 데이터를 싹 지우고 처음부터 다시 시작하고 싶을 때입니다. `DELETE FROM` 테이블; 로도 비울 수 있지만, 이런 "전체 비우기"에는 더 빠른 전용 명령 `TRUNCATE` 가 있습니다. 이 절에서는 둘의 차이를 정확히 구분합니다.

핵심 개념 설명

전체 비우기 (TRUNCATE)

전체 비우기(TRUNCATE) 란, 테이블의 모든 행을 빠르게 제거해 비우는 명령입니다. 노트의 한 장 전체를 한 번에 찢어 버리는 것과 같습니다. 조건을 걸 수 없고(WHERE 불가), 자동 증가 번호(시퀀스)를 처음으로 되돌릴 수도 있습니다.

- **왜(why) 쓰나:** 큰 테이블을 통째로 비울 때 DELETE 보다 훨씬 빠릅니다. DELETE 는 행을 하나하나 지우며 기록을 남기지만, TRUNCATE 는 저장 공간을 통째로 해제하는 방식이라 행 수가 많아도 거의 즉시 끝납니다.
- **언제(when) 쓰나:** 테스트·임시 테이블을 초기화할 때, 데이터를 모두 비우고 자동 증가 id 도 1부터 다시 시작하고 싶을 때 적합합니다.
- **어떻게(how) 쓰나:** TRUNCATE TABLE 테이블; 형태로 씁니다. 시퀀스까지 초기화하려면 RESTART IDENTITY 를 붙입니다.

입문자가 자주 하는 오해 두 가지가 있습니다. 첫째, TRUNCATE 에 WHERE 조건을 걸 수 있다고 생각하는 경우입니다. TRUNCATE 는 **전체 비우기 전용**이라 조건을 걸 수 없습니다. 일부만 지우려면 DELETE ... WHERE 를 써야 합니다. 둘째, TRUNCATE 는 트랜잭션에서 롤백되지 않는다고 오해하는 경우입니다. PostgreSQL에서는 TRUNCATE 도 트랜잭션 안에서 실행하면 ROLLBACK 으로 되돌릴 수 있습니다(일부 다른 데이터베이스와 다른 점입니다).

DELETE vs TRUNCATE

두 명령 모두 "행을 없앤다"는 결과는 같지만, 동작 방식과 부가 효과가 다릅니다. 아래 비교로 정리합니다.

DELETE vs TRUNCATE — 차이점 비교

DELETE DELETE FROM 테이블 WHERE ...	TRUNCATE TRUNCATE TABLE 테이블
삭제 방식 행을 하나씩 순서대로 제거	삭제 방식 테이블 데이터 페이지를 통째로 해제
WHERE 조건 사용 가능 — 일부 행만 삭제	WHERE 조건 사용 불가 — 전체 행만 삭제
트랜잭션 롤백 ROLLBACK으로 복구 가능	트랜잭션 롤백 PostgreSQL에서는 롤백 가능
속도 행이 많을수록 느림	속도 행 수와 무관하게 빠름
시퀀스(AUTO INCREMENT) 초기화되지 않음	시퀀스(AUTO INCREMENT) RESTART IDENTITY로 초기화 가능

ch_08_s02 · TRUNCATE와 DELETE의 차이

DELETE와 TRUNCATE 비교

항목	DELETE	TRUNCATE
조건(WHERE)	가능(일부만 삭제)	불가능(전체만)
속도	행 수에 비례해 느려짐	행 수와 무관하게 빠름
트랜잭션 롤백	가능	가능(PostgreSQL)
시퀀스 초기화	안 됨	RESTART IDENTITY 로 가능
삭제 건수 표시	DELETE N	건수 표시 없음
트리거 발동	행마다 발동	기본적으로 발동 안 함

이렇게 작성합니다

다음은 `orders` 테이블을 통째로 비우는 `TRUNCATE` 입니다.

```
-- orders 테이블의 모든 행을 빠르게 비우기
TRUNCATE TABLE orders;
```

예상 화면:

```
TRUNCATE TABLE
```

`DELETE` 와 달리 지워진 건수가 표시되지 않고, "비우기를 마쳤다"는 의미로 `TRUNCATE TABLE` 만 나옵니다.

자동 증가 `id` 까지 1부터 다시 시작하고 싶다면 `RESTART IDENTITY` 를 붙입니다.

```
-- 비우면서 id 자동 증가 번호도 1부터 다시 시작하기
TRUNCATE TABLE orders RESTART IDENTITY;
```

예상 화면:

```
TRUNCATE TABLE
```

이 차이를 눈으로 비교해 봅시다. 그냥 TRUNCATE 만 하면 다음 삽입의 id 가 이전 번호에 이어지지만, RESTART IDENTITY 를 붙이면 1부터 다시 매겨집니다.

```
-- 비운 뒤 새 주문을 넣고 id 확인하기
INSERT INTO orders (member_id, amount) VALUES (1, 5000)
RETURNING id;
```

예상 화면:

```
id
----
1
(1 row)
INSERT 0 1
```

RESTART IDENTITY 로 비웠기 때문에 새 주문의 id 가 1 부터 시작한 것을 확인할 수 있습니다.

주의: 외래 키로 참조되는 테이블의 TRUNCATE 다른 테이블이 외래 키로 참조하는 테이블(예: orders 가 가리키는 members)을 TRUNCATE 하려 하면, PostgreSQL이 참조 무결성을 지키기 위해 막거나 CASCADE 를 요구합니다. TRUNCATE members CASCADE; 처럼 쓰면 참조하는 orders 까지 함께 비워지므로, 그 파급 범위를 반드시 이해하고 사용합니다.

팁: 언제 무엇을 쓸까 "일부만 지운다" 또는 "지운 내용을 RETURNING 으로 확인하고 싶다"면 DELETE 를, "테이블을 통째로 비우고 빠르게 초기화한다"면 TRUNCATE 를 고릅니다. 운영 중인 실제 데이터 테이블을 통째로 비우는 일은 드물고 위험하므로, TRUNCATE 는 주로 테스트·임시 데이터에 씁니다.

절 요약

- TRUNCATE 는 테이블 전체를 빠르게 비우는 전용 명령으로, WHERE 조건을 걸 수 없습니다.
- RESTART IDENTITY 를 붙이면 자동 증가 id 를 1부터 다시 시작합니다.
- PostgreSQL에서는 TRUNCATE 도 트랜잭션 안에서 롤백할 수 있습니다.
- 일부 삭제·삭제 확인은 DELETE , 전체 초기화는 TRUNCATE 로 나누어 씁니다.

외래 키와 CASCADE 연쇄 삭제

도입

회원을 지우려 했더니 "이 회원을 참조하는 주문이 있어서 지울 수 없다"는 오류가 났습니다. 처음 만나면 당황스럽지만, 사실 이것은 데이터베이스가 데이터를 지켜 주는 안전장치입니다. 이 절에서는 외래 키가 삭제를 어떻게 막는지, 그리고 함께 지우거나 연결만 끊는 방법은 무엇인지 배웁니다.

핵심 개념 설명

외래 키 (foreign key)

외래 키(foreign key)란, 한 테이블의 컬럼이 다른 테이블의 행을 가리키도록 연결해 둔 약속입니다. 우리 예시에서 `orders.member_id`는 `members.id`를 가리키는 외래 키입니다. "이 주문은 이 회원의 것"이라는 관계를 데이터베이스가 보증하게 만드는 장치입니다.

여기서 가리키는 쪽(`members`)을 **부모(parent)**, 가리키는 값을 가진 쪽(`orders`)을 **자식(child)**이라고 부릅니다. 자식 주문이 존재하는 한, 부모 회원을 함부로 지우면 그 주문은 "주인 없는 주문"이 되어 버립니다. 이것을 막는 것이 참조 무결성입니다.

연쇄 삭제 (ON DELETE CASCADE)

연쇄 삭제(ON DELETE CASCADE)란, 외래 키로 연결된 부모 행을 삭제하면 그를 참조하는 자식 행도 함께 자동으로 삭제되도록 하는 옵션입니다. 회원 계정을 지우면 그 사람이 남긴 글도 함께 지워지는 것과 같습니다.

외래 키를 정의할 때 `ON DELETE` 동작을 지정할 수 있고, 대표적으로 세 가지가 있습니다.

- **CASCADE** : 부모를 지우면 자식도 함께 지웁니다(연쇄 삭제).
- **RESTRICT** (기본값에 가까움): 자식이 있으면 부모 삭제를 막습니다.
- **SET NULL** : 부모를 지우면 자식의 외래 키 값을 `NULL`로 바꿔 연결만 끊습니다(자식 행은 남습니다).

입문자가 자주 하는 오해 두 가지가 있습니다. 첫째, `CASCADE`가 기본 동작이라고 생각하는 경우입니다. 아무 옵션도 지정하지 않으면 기본은 `NO ACTION` (사실상 `RESTRICT` 처럼 막음)이라 연쇄 삭제는 일어나지 않습니다. 둘째, 참조하는 자식 행이 있어도 부모를 그냥 지울 수 있다고 오해하는 경우입니다. 자식이 있으면 기본적으로 막히며, 함께 지우려면 명시적으로 `CASCADE`를 설정해야 합니다.

이렇게 작성합니다

먼저 옵션 없는 외래 키일 때 부모 삭제가 어떻게 막히는지 봅시다. 회원 `id` 가 2인 회원에게 주문이 남아 있는 상태에서 그 회원을 지워 봅시다.

```
-- 자식(주문)이 남아 있는 회원을 지우려 시도하기
DELETE FROM members WHERE id = 2;
```

예상 화면:

```
ERROR: update or delete on table "members" violates foreign key
constraint "orders_member_id_fkey" on table "orders"
DETAIL: Key (id)=(2) is still referenced from table "orders".
```

오류 메시지는 "그 회원(`id=2`)이 `orders` 테이블에서 아직 참조되고 있어 지울 수 없다"는 뜻입니다. 데이터베이스가 주인 없는 주문이 생기는 것을 막아 준 것입니다.

오류 메시지 읽기

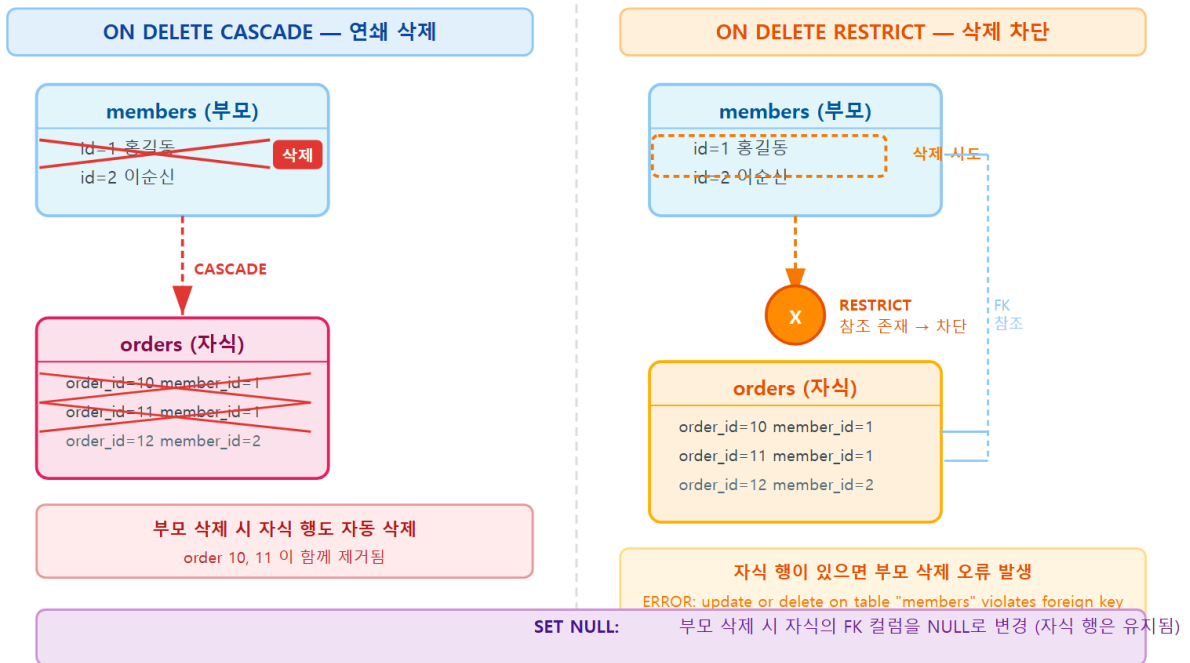
메시지 조각	의미
violates foreign key constraint	외래 키 약속을 어겼습니다
orders_member_id_fkey	위반된 외래 키 제약의 이름입니다
Key (id)=(2) is still referenced	<code>id=2</code> 가 자식 테이블에서 아직 참조 중입니다

이 상황을 처리하는 방법은 외래 키에 어떤 `ON DELETE` 동작을 걸어 두었느냐에 따라 갈립니다. 다음은 연쇄 삭제(`CASCADE`)로 정의한 외래 키 예시입니다.

```
-- orders를 만들 때 외래 키에 ON DELETE CASCADE를 지정한 경우 (참고용)
CREATE TABLE orders (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  member_id  integer REFERENCES members(id) ON DELETE CASCADE,
  amount      numeric(10, 2) NOT NULL,
  ordered_at timestamp DEFAULT now()
);
```

이렇게 `ON DELETE CASCADE` 가 걸려 있으면, 부모 회원을 지울 때 그 회원의 주문도 함께 사라집니다.

외래 키 ON DELETE 옵션 — CASCADE vs RESTRICT



ch_08_s03 · 외래 키와 CASCADE 연쇄 삭제

```
-- CASCADE가 걸린 상태에서 회원 삭제하기 (주문도 함께 삭제됨)
DELETE FROM members WHERE id = 2;
```

예상 화면:

```
DELETE 1
```

회원 1건이 지워졌다고 표시되지만, 화면 뒤에서는 그 회원이 가진 주문들도 함께 사라졌습니다. 확인해 봅니다.

```
-- 그 회원의 주문이 함께 사라졌는지 확인하기
SELECT count(*) FROM orders WHERE member_id = 2;
```

예상 화면:

```
count
-----
      0
(1 row)
```

member_id 가 2인 주문이 0건으로, 연쇄 삭제로 함께 지워진 것을 확인할 수 있습니다.

연결만 끊고 자식 행은 남기고 싶다면 `ON DELETE SET NULL` 을 씁니다. 이 경우 부모를 지우면 자식의 `member_id` 가 `NULL` 이 되고, 주문 자체는 남습니다.

```
-- ON DELETE SET NULL로 정의한 경우 (참고용)
CREATE TABLE orders (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  member_id   integer REFERENCES members(id) ON DELETE SET NULL,
  amount      numeric(10, 2) NOT NULL,
  ordered_at  timestamp DEFAULT now()
);
```

주의: CASCADE는 편리하지만 파급이 큽니다 `ON DELETE CASCADE` 는 부모 한 줄을 지우는 명령이 자식 수백 줄을 조용히 함께 지울 수 있습니다. 화면에는 `DELETE 1` 만 보여 실제 파급을 가늠하기 어렵습니다. 중요한 데이터(주문·결제 같은)는 함부로 `CASCADE` 로 묶기보다, 막아 주는 `RESTRICT` 로 두고 삭제 절차를 명시적으로 밟는 편이 더 안전한 경우가 많습니다.

팁: 삭제 전 자식 건수 세어 보기 부모를 지우기 전에 `SELECT count(*) FROM orders WHERE member_id = 2;` 처럼 자식 건수를 먼저 세어 보면, `CASCADE` 로 몇 건이 함께 사라질지 미리 알 수 있습니다. 예상보다 많으면 멈추고 다시 검토합니다.

절 요약

- 외래 키는 부모-자식 관계를 보증하며, 자식이 있으면 부모 삭제를 기본적으로 막습니다.
- `ON DELETE CASCADE` 는 부모를 지울 때 자식도 함께 지웁니다.
- `RESTRICT` 는 막고, `SET NULL` 은 자식을 남기고 연결만 끊습니다.
- `CASCADE` 는 파급이 크므로, 삭제 전에 함께 사라질 자식 건수를 확인합니다.

삭제 전 안전 장치 - 트랜잭션과 백업 습관

도입

지금까지 배운 삭제 방법은 모두 "실행하는 순간 바로 확정"됩니다. 그런데 사람은 실수합니다. 조건을 잘못 적거나, 엉뚱한 테이블을 지정할 수 있습니다. 이 절에서는 삭제를 **즉시 확정하지 않고 한 번 더 확인할 기회**를 주는 트랜잭션과, 최후의 보루인 백업 습관을 배웁니다.

핵심 개념 설명

삭제 안전망 (deletion safety)

삭제 안전망(deletion safety) 이란, 삭제가 사고로 이어지지 않도록 미리 마련해 두는 점검·대비 장치를 통틀어 말합니다. 핵심은 두 가지입니다. 하나는 "지우기 전에 무엇이 지워질지 확인하는 것", 다른 하나는 "확정하기 전에 되돌릴 수 있게 해 두는 것"입니다.

앞 절들에서 배운 "지우기 전 `SELECT`"가 첫 번째 장치라면, 이 절의 트랜잭션이 두 번째 장치입니다.

롤백 대비 (rollback safety)

롤백 대비(rollback safety)란, 삭제를 트랜잭션 안에서 실행해 결과를 확인한 뒤, 잘못되었으면 `ROLLBACK`으로 통째로 되돌릴 수 있게 하는 것입니다. 트랜잭션은 10장에서 자세히 배우지만, 삭제와 직접 관련되므로 여기서 핵심만 익힙니다.

- `BEGIN` : 트랜잭션을 시작합니다. 이후의 작업은 아직 "임시"이며 확정되지 않습니다.
- `COMMIT` : 임시 상태를 실제로 확정합니다.
- `ROLLBACK` : 임시 상태를 모두 취소하고 `BEGIN` 직전으로 되돌립니다.

즉, `BEGIN`으로 시작해 `DELETE`를 실행하고, `SELECT`로 결과가 맞는지 확인한 다음, 맞으면 `COMMIT`, 틀리면 `ROLLBACK`하면 됩니다.

이렇게 작성합니다

다음은 트랜잭션으로 감싸 삭제를 확인 후 확정하는 흐름입니다. 먼저 잘못된 경우를 가정해 `ROLLBACK`으로 되돌립니다.

```
-- 트랜잭션 시작 후 삭제하고, 결과를 확인한 뒤 결정하기
BEGIN;

DELETE FROM orders WHERE amount < 1000;

-- 지운 뒤 남은 행을 확인합니다 (아직 확정 전)
SELECT count(*) FROM orders;
```

예상 화면:

```
BEGIN
DELETE 2
count
-----
      18
(1 row)
```

확인해 보니 지울 의도가 없던 데이터까지 영향을 받았다고 가정합니다. 아직 `COMMIT` 전이므로 되돌릴 수 있습니다.

```
-- 결과가 잘못됐으니 통째로 되돌리기
ROLLBACK;
```

예상 화면:

```
ROLLBACK
```

`ROLLBACK` 후에는 `DELETE` 가 없던 일이 됩니다. 반대로 결과가 의도와 맞으면 `COMMIT` 으로 확정합니다.

```
-- 결과가 맞으면 확정하기
BEGIN;
DELETE FROM orders WHERE id = 50;
COMMIT;
```

예상 화면:

```
BEGIN
DELETE 1
COMMIT
```

`COMMIT` 까지 마쳐야 삭제가 실제로 확정되어 다른 사용자에게도 반영됩니다.

트랜잭션 안전 삭제 흐름

단계	명령	의미
1	<code>BEGIN</code>	트랜잭션 시작(이후 작업은 임시)
2	<code>DELETE ... WHERE ...</code>	삭제 실행(아직 확정 전)
3	<code>SELECT ...</code>	결과가 의도와 맞는지 확인
4a	<code>COMMIT</code>	맞으면 확정
4b	<code>ROLLBACK</code>	틀리면 전부 취소

대량 삭제 전이나 운영 데이터를 다룰 때는 트랜잭션만으로 부족합니다. 명령을 실행하기 전에 백업을 떠 두는 습관이 최후의 보루입니다. PostgreSQL은 `pg_dump` 라는 백업 도구를 제공하며, PowerShell에서 다음과 같이 실행합니다(백업은 12장에서 자세히 다룹니다).

```
# PowerShell에서 shop_lab 데이터베이스를 파일로 백업하기
pg_dump -U postgres -d shop_lab -f C:\Users\me\backup\shop_lab_20260616.sql
```

예상 화면:

```
PS C:\Users\me> pg_dump -U postgres -d shop_lab -f C:\Users\me\backup\shop_lab_20260616.sql
Password:
PS C:\Users\me>
```

오류 없이 프롬프트로 돌아오면 백업 파일이 만들어진 것입니다. 이 파일이 있으면 삭제가 잘못 되어도 복구할 수 있습니다.

베스트 프랙티스: 운영 데이터 삭제 3단계 1. **확인**: 지울 `WHERE` 조건을 그대로 `SELECT` 해 대상 건수·내용을 본다. 2. **보호**: `BEGIN` 으로 트랜잭션을 열고, 대량·중요 삭제라면 먼저 `pg_dump` 로 백업한다. 3. **확정**: 결과가 맞으면 `COMMIT`, 조금이라도 이상하면 `ROLLBACK` 한다.

팁: 정말 지워야 할까 - 소프트 삭제 탈퇴 회원처럼 "보이지 않게만 하면 되는" 데이터는 실제로 지우는 대신 `is_deleted` 같은 컬럼을 `true` 로 바꾸는 **소프트 삭제(soft delete)** 도 고려합니다. 데이터를 남겨 두므로 복구와 이력 추적이 쉽습니다. 이 패턴은 12장에서 자세히 다룹니다.

절 요약

- `BEGIN ... DELETE ... 확인 ... COMMIT/ROLLBACK` 흐름으로 삭제를 되돌릴 여지를 남깁니다.
- `COMMIT` 전까지는 `ROLLBACK` 으로 삭제를 취소할 수 있습니다.
- 대량·운영 데이터 삭제 전에는 `pg_dump` 로 백업해 두는 습관을 들입니다.
- 지우지 않아도 되는 경우 소프트 삭제로 데이터를 남겨 두는 방법도 있습니다.

실습 과제

해보기: 안전한 삭제 절차 설계하기

실습 데이터베이스 `shop_lab` 에서 이 장의 삭제 방법을 안전 습관과 함께 직접 다뤄 봅니다.

- 단계 1: `orders` 에서 금액이 일정 기준 미만인 주문을 지우기 전에, 같은 `WHERE` 조건으로 `SELECT count(*)` 를 먼저 실행해 지워질 건수를 확인합니다.
- 단계 2: `BEGIN` 으로 트랜잭션을 연 뒤 같은 조건으로 `DELETE` 를 실행하고, 남은 행을 `SELECT` 로 확인한 다음 결과가 맞으면 `COMMIT` , 아니면 `ROLLBACK` 합니다.
- 단계 3: 외래 키가 `RESTRICT` 인 상태에서 주문이 있는 회원을 지워 보고, 막히는 오류 메시지를 직접 확인합니다. 그다음 `ON DELETE CASCADE` 로 정의한 표를 만들어 부모 삭제 시 자식이 함께 사라지는지 비교합니다.
- 점검: `TRUNCATE TABLE` 임시테이블 `RESTART IDENTITY;` 로 임시 테이블을 비운 뒤 새 행을 넣어 `id` 가 1부터 다시 시작하는지 확인합니다. 대량 삭제 전에는 `pg_dump` 로 백업 파일을 한 번 만들어 봅니다.

FAQ

Q1. 실수로 `WHERE` 없이 `DELETE FROM orders;` 를 실행해 전부 지웠습니다. 되돌릴 수 있나요? → **A1.** `BEGIN` 으로 트랜잭션을 열어 둔 상태였다면 `ROLLBACK` 으로 즉시 되돌릴 수 있습니다. 그러나 트랜잭션 밖에서 실행해 이미 확정(자동 `COMMIT`)되었다면 SQL만으로는 되돌릴 수 없고, `pg_dump` 같은 백업에서 복구해야 합니다. 그래서 평소에 "삭제는 `BEGIN` 부터" 습관과 정기 백업이 중요합니다.

Q2. `DELETE`와 `TRUNCATE` 중 무엇이 더 빠른가요? → **A2.** 전체를 비우는 경우 `TRUNCATE` 가 훨씬 빠릅니다. `DELETE` 는 행을 하나씩 지우며 기록을 남기지만, `TRUNCATE` 는 저장 공간을 통째로 해제하기 때문에 행 수와 거의 무관하게 즉시 끝납니다. 다만 `TRUNCATE` 는 조건을 걸 수 없으니, 일부만 지울 때는 `DELETE ... WHERE` 를 써야 합니다.

Q3. 회원을 지우려는데 외래 키 오류가 납니다. 어떻게 해야 하나요? → **A3.** 그 회원을 참조하는 자식 행(주문 등)이 남아 있기 때문입니다. 선택지는 세 가지입니다. (1) 자식 행을 먼저 지운 뒤 부모를 지운다, (2) 외래 키를 `ON DELETE CASCADE` 로 정의해 함께 지운다, (3) `ON DELETE SET NULL` 로 연결만 끊고 자식은 남긴다. 중요한 데이터라면 무턱대고 `CASCADE` 로 뭉기보다 절차를 명시적으로 밟는 편이 안전합니다.

Q4. `DELETE 0` 이 나왔는데 오류인가요? → **A4.** 오류가 아닙니다. "조건에 맞는 행이 0건이라 아무것도 지우지 않았다"는 정상 결과입니다. 지우려던 행이 분명히 있다고 생각했다면, `WHERE` 조건이 실제 데이터와 맞는지 같은 조건으로 `SELECT` 해 다시 확인합니다.

장 요약

이 장에서는 CRUD의 마지막 단계인 데이터 삭제(Delete)를 배웠습니다. `DELETE ... WHERE` 로 조건에 맞는 행만 골라 지우되, 지우기 전에 같은 조건으로 `SELECT` 해 대상을 확인하는 습관을 익혔습니다. 전체를 빠르게 비우는 `TRUNCATE` 와 행 단위로 지우는 `DELETE` 의 차이를 속도·롤백·시퀀스 초기화 관점에서 구분했고, 외래 키의 `ON DELETE CASCADE · RESTRICT · SET NULL` 동작으로 부모-자식 관계에서 삭제가 어떻게 처리되는지 살펴보았습니다. 마지막으로 트랜잭션 (`BEGIN · COMMIT · ROLLBACK`)과 `pg_dump` 백업으로 삭제 사고를 예방하는 안전 습관을 다루었습니다. 삭제는 가장 조심스러운 작업이므로, 문법보다 안전 절차를 몸에 익히는 것이 더 중요합니다.

핵심 키워드

`DELETE`, `WHERE`, 조건부 삭제(filtered delete), `TRUNCATE`, `RESTART IDENTITY`, 외래 키(foreign key), `ON DELETE CASCADE`, `RESTRICT`, `SET NULL`, `BEGIN/COMMIT/ROLLBACK`, `pg_dump`

다음 장 미리보기

다음 장에서는 지금까지 다룬 데이터를 처음부터 잘못 들어오지 않도록 막는 **제약조건 (constraint)** 과, 조회를 빠르게 해 주는 **인덱스(index)** 를 배웁니다. 기본 키·외래 키로 무결성을 지키고, `UNIQUE · CHECK` 로 잘못된 값을 거르며, 인덱스가 검색 속도를 어떻게 높이는지 그 원리를 살펴봅니다.

제약조건과 인덱스: 무결성과 조회 성능

학습 목표 - 제약조건이 데이터 무결성을 지키는 역할을 설명할 수 있습니다. - 기본 키·외래 키로 행 식별과 참조 무결성을 구성할 수 있습니다. - UNIQUE·CHECK·NOT NULL로 잘못된 데이터를 막을 수 있습니다. - 인덱스가 조회를 빠르게 하는 원리와 비용·설계 주의점을 이해합니다.

지금까지 우리는 테이블에 데이터를 넣고(Create), 읽고(Read), 고치고(Update), 지우는>Delete) 기본기를 익혔습니다. 그런데 한 가지 빠진 것이 있습니다. 바로 **"애초에 잘못된 데이터가 들어오지 못하게 막는 일"**입니다. 이메일이 빈칸인 회원, 존재하지도 않는 회원이 넣은 주문, 가격이 음수인 상품처럼 말이 안 되는 데이터는 처음부터 거절되어야 합니다.

이 장에서는 데이터베이스 스스로가 규칙을 지키게 하는 **제약조건(constraint)** 과, 쌓인 데이터를 빠르게 찾아 주는 **인덱스(index)** 를 배웁니다. 앞쪽 절(제약조건)은 "올바른 데이터를 지키는 일", 뒤쪽 절(인덱스)은 "그 데이터를 빠르게 꺼내는 일"이라고 생각하면 됩니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 에서 `psql` 로 PostgreSQL에 접속한 상태를 기준으로 합니다. 화면 앞의 `shop_lab=#` 표시는 "지금 `shop_lab` 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 아래 세 테이블을 기준으로 합니다. 회원(`members`), 상품(`products`), 주문(`orders`)이며, 제약조건과 인덱스를 하나씩 입혀 가며 살펴보겠습니다.


```
-- 이 장 예시가 기준으로 삼는 세 테이블 (참고용)
CREATE TABLE members (
  id    integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name  varchar(50) NOT NULL,
  email varchar(100) NOT NULL,
  age   integer
);

CREATE TABLE products (
  id    integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name  varchar(100) NOT NULL,
  price numeric(10, 2) NOT NULL
);

CREATE TABLE orders (
  id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  member_id    integer NOT NULL,
  product_id   integer NOT NULL,
  quantity     integer NOT NULL
);
```

제약조건으로 잘못된 데이터 막기

도입

식당 입구에 "신발 벗고 입장"이라는 안내가 붙어 있으면, 손님은 들어오기 전에 그 규칙을 지킵니다. 제약조건도 똑같습니다. 데이터가 테이블에 들어오기 전에 "이런 값은 안 된다"는 규칙을 통과해야만 저장됩니다. 이 절에서는 제약조건이 왜 필요하고 어떤 종류가 있는지 큰 그림을 먼저 잡습니다.

핵심 개념 설명

제약조건 (constraint)

제약조건(constraint)이란, 테이블에 들어올 수 있는 값에 규칙을 두어 잘못된 데이터를 막는 장치입니다. 입력 양식에서 "필수 항목"이나 "숫자만 입력" 같은 검증 규칙을 미리 걸어 두는 것과 같습니다. 규칙을 어기는 값이 들어오려 하면 PostgreSQL이 그 작업을 거절하고 오류를 냅니다.

- **왜(why) 필요한가:** 데이터는 시간이 지날수록 쌓이고 여러 사람·여러 프로그램이 함께 다룹니다. 한 곳에서만 검증하면 다른 경로로 들어온 잘못된 값을 놓칩니다. 제약조건은 데

이터베이스 자체가 마지막 문지기 역할을 해, 어떤 경로로 들어와도 규칙을 지키게 합니다.

- **언제(when) 쓰나:** 테이블을 설계할 때 "이 컬럼은 비면 안 된다", "이 값은 겹치면 안 된다", "이 숫자는 0보다 커야 한다" 같은 규칙이 떠오르는 모든 순간입니다.
- **어떻게(how) 쓰나:** CREATE TABLE 로 테이블을 만들 때 컬럼 옆에 규칙을 적거나, 이미 있는 테이블에 ALTER TABLE 로 나중에 추가합니다.

여기서 입문자가 자주 하는 오해 두 가지를 짚겠습니다.

첫째, "검증은 애플리케이션(백엔드 서버) 코드에서만 하면 된다"는 생각입니다. 애플리케이션 검증은 물론 필요하지만, 그것만으로는 부족합니다. 데이터를 직접 손보는 관리자 도구, 데이터 이관 스크립트, 다른 서비스의 접근 등 애플리케이션을 거치지 않는 경로가 늘 존재하기 때문입니다. 제약조건은 그 모든 경로의 **공통 안전망**입니다.

둘째, "제약조건은 성능만 떨어뜨린다"는 오해입니다. 제약조건을 검사하는 데 약간의 비용이 들기는 합니다. 하지만 잘못된 데이터가 쌓인 뒤 그것을 찾아 고치는 비용에 비하면 훨씬 작습니다. 게다가 기본 키·외래 키 같은 제약은 오히려 조회를 빠르게 하는 인덱스를 함께 만들어 주기도 합니다.

데이터 무결성 (data integrity)

데이터 무결성(data integrity)이란, 데이터가 정확하고 일관되며 신뢰할 수 있는 상태로 유지되는 것을 말합니다. "장부에 적힌 숫자를 그대로 믿어도 된다"는 안심이 곧 무결성입니다. 제약조건은 이 무결성을 **기술적으로 강제하는 수단**입니다.

예를 들어 회원 명부에 같은 이메일이 두 번 등록되어 있다면, 어느 쪽이 진짜인지 알 수 없어 데이터를 믿을 수 없게 됩니다. 주문에 적힌 회원 번호가 명부에 없는 번호라면, 그 주문은 "누가 했는지 모르는 주문"이 됩니다. 이런 어긋남을 막는 것이 무결성을 지키는 일입니다.

제약조건 종류 한눈에 보기

PostgreSQL에서 자주 쓰는 제약조건은 다섯 가지입니다. 이 장에서 하나씩 자세히 다루므로, 먼저 이름과 역할만 표로 익혀 둡니다.

주요 제약조건 다섯 가지

제약조건	역할	이 장에서 배우는 곳
PRIMARY KEY	각 행을 유일하게 식별(중복·NULL 금지)	2절
FOREIGN KEY	다른 테이블의 기본 키를 가리켜 관계 보장	2절
UNIQUE	컬럼 값이 겹치지 않도록 강제	3절
NOT NULL	빈 값(NULL)을 허용하지 않음	3절
CHECK	값이 지정한 조건(범위·형식)을 만족하게 함	3절

절 요약

- 제약조건은 잘못된 데이터가 테이블에 들어오기 전에 막는 규칙입니다.
- 애플리케이션 검증과 별개로, 데이터베이스 자체가 마지막 문지기 역할을 합니다.
- 제약조건은 데이터 무결성, 즉 "믿을 수 있는 데이터" 상태를 기술적으로 보장합니다.
- 주요 제약은 PRIMARY KEY·FOREIGN KEY·UNIQUE·NOT NULL·CHECK 다섯 가지입니다.

기본 키·외래 키와 참조 무결성

도입

회원 명부에서 한 사람을 꼭 집어 가리키려면 겹치지 않는 번호가 필요합니다. 또 주문서에 적힌 "회원 번호"는 실제 명부에 있는 번호여야 의미가 있습니다. 이 절에서는 행을 유일하게 식별하는 **기본 키(primary key)** 와, 테이블 사이의 관계를 지키는 **외래 키(foreign key)** 를 배웁니다.

핵심 개념 설명

기본 키 (primary key)

기본 키(primary key) 란, 테이블에서 각 행을 유일하게 식별하는 컬럼(들)으로, 중복과 NULL(빈 값)을 허용하지 않습니다. 주민등록번호처럼 한 사람을 겹치지 않게 구분하는 고유 번호와 같습니다.

- **왜(why) 필요한가:** "3번 회원의 이메일을 바꿔라" 같은 작업을 하려면 행을 정확히 한 개만 골라낼 방법이 있어야 합니다. 기본 키가 그 역할을 합니다. 기본 키가 없으면 똑같이 생긴 행이 두 개일 때 어느 것을 가리키는지 알 수 없습니다.
- **언제(when) 쓰나:** 거의 모든 테이블에 하나씩 둡니다. 보통 `id` 라는 자동 증가 정수 컬럼을 기본 키로 삼습니다.
- **어떻게(how) 쓰나:** 컬럼 옆에 `PRIMARY KEY` 를 적습니다. 기본 키로 지정하면 자동으로 `NOT NULL`과 `UNIQUE`가 함께 적용되고, 빠른 조회를 위한 인덱스도 자동으로 만들어집니다.

입문자가 자주 하는 오해 두 가지가 있습니다. "기본 키는 꼭 숫자여야 한다"는 생각이지만, 이메일·주문번호 문자열도 기본 키가 될 수 있습니다(다만 짧고 변하지 않는 정수가 다루기 편해 관례로 많이 씁니다). 또 "한 테이블에 기본 키를 여러 개 둘 수 있다"는 오해도 흔한데, 기본 키는 테이블당 **딱 하나**입니다. 단, 그 하나의 기본 키가 여러 컬럼을 묶은 형태(복합 키)일 수는 있습니다.

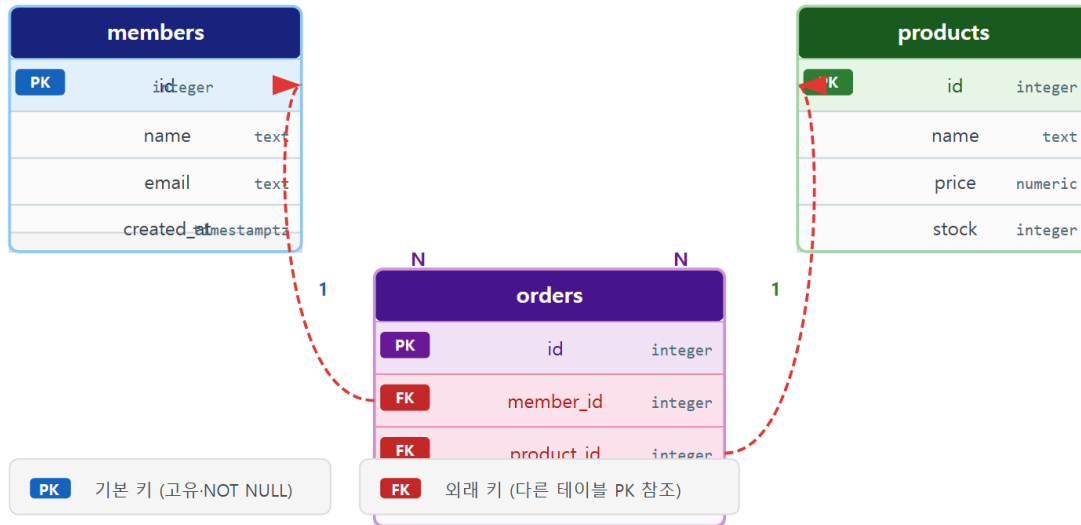
외래 키 (foreign key)

외래 키(foreign key)란, 한 테이블의 컬럼이 다른 테이블의 기본 키를 가리켜 두 테이블의 관계와 참조 무결성을 보장하는 제약입니다. 주문서에 적힌 회원 번호가 실제 회원 명부에 있는 번호여야만 하도록 강제하는 것과 같습니다.

여기서 **참조 무결성(referential integrity)**이란, "가리키는 대상이 반드시 존재한다"는 보장입니다. `orders.member_id`가 외래 키로 `members.id`를 가리킨다면, `members`에 없는 번호를 `orders`에 넣으려는 시도는 거절됩니다. "주인 없는 주문"이 생기지 않는 것입니다.

기본 키(PK)와 외래 키(FK) — ERD 참조 관계

orders가 members(1:N)-products(1:N) 양쪽을 외래 키로 참조합니다
ch_09_s02 · 기본 키-외래 키와 참조 무결성



- **왜(why) 필요한가:** 테이블은 서로 연결되어 의미를 가집니다. 주문은 "어느 회원이 어떤 상품을"을 가리켜야 완성됩니다. 외래 키는 그 연결이 항상 유효하도록 지켜, 끊어진 연결 (존재하지 않는 회원을 가리키는 주문)을 막습니다.
- **언제(when) 쓰나:** 한 테이블이 다른 테이블을 참조할 때입니다. 주문이 회원과 상품을 참조하는 관계가 대표적입니다.
- **어떻게(how) 쓰나:** 컬럼 옆에 `REFERENCES` 대stage테이블(대상컬럼) 을 적습니다.

오해 하나를 짚겠습니다. "외래 키가 없으면 JOIN(테이블 연결 조회)을 못 한다"는 생각인데, 사실이 아닙니다. JOIN은 외래 키가 없어도 가능합니다. 외래 키는 JOIN을 *가능하게* 하는 것이 아니라, 잘못된 참조 값이 애초에 들어오지 못하게 *막아* 주는 안전장치입니다.

이렇게 작성합니다

먼저 기본 키부터 봅시다. 이 장 도입부의 테이블 정의에서 이미 `id ... PRIMARY KEY` 로 기본 키를 지정했습니다. 기본 키가 제대로 동작하는지, 같은 `id` 를 두 번 넣어 보며 확인합니다.

```
-- 기본 키 컬럼에 같은 값을 두 번 넣으면 거절됩니다
INSERT INTO members (name, email) VALUES ('김민수', 'minsu@example.com');
INSERT INTO members (name, email) VALUES ('박지훈', 'jihoon@example.com');
```

기본 키 `id` 는 자동으로 1, 2가 매겨지므로 위 두 건은 정상입니다. 그런데 만약 같은 `id` 가 강제로 들어가는 상황이 되면 다음과 같은 오류가 납니다(개념 확인용 예상 화면입니다).

```
ERROR: duplicate key value violates unique constraint "members_pkey"
DETAIL: Key (id)=(1) already exists.
```

`members_pkey` 는 기본 키 제약의 이름이고, 메시지는 "이미 1번이 있어 중복은 허용되지 않는다"는 뜻입니다. 기본 키가 중복을 막아 주는 모습입니다.

이제 외래 키를 추가합니다. `orders` 의 `member_id` 와 `product_id` 가 각각 `members.id` 와 `products.id` 를 가리키도록 만듭니다.

```
-- orders에 외래 키 제약을 추가하기
ALTER TABLE orders
  ADD CONSTRAINT orders_member_fk
  FOREIGN KEY (member_id) REFERENCES members (id);

ALTER TABLE orders
  ADD CONSTRAINT orders_product_fk
  FOREIGN KEY (product_id) REFERENCES products (id);
```

예상 화면:

```
ALTER TABLE
ALTER TABLE
```

이제 존재하지 않는 회원 번호로 주문을 넣어 보면 외래 키가 막아 줍니다.

```
-- 999번 회원은 없는데 주문을 넣으려는 시도
INSERT INTO orders (member_id, product_id, quantity)
VALUES (999, 1, 2);
```

예상 화면:

```
ERROR: insert or update on table "orders" violates foreign key constraint "orders_member_fk"
DETAIL: Key (member_id)=(999) is not present in table "members".
```

999 번 회원이 `members` 에 없기 때문에, 그 회원을 가리키는 주문은 거절됩니다. 참조 무결성이 지켜지는 순간입니다.

구문 요소 설명

구문	의미
PRIMARY KEY	그 컬럼을 기본 키로 지정합니다(중복·NULL 금지)
ADD CONSTRAINT 이름	추가할 제약에 이름을 붙입니다(오류 메시지에 표시됨)
FOREIGN KEY (member_id)	외래 키가 될 이 테이블의 컬럼입니다
REFERENCES members (id)	가리킬 대상 테이블과 그 기본 키 컬럼입니다

주의: 외래 키가 가리키는 행은 함부로 지울 수 없습니다. 예를 들어 어떤 회원이 주문을 가지고 있는데 그 회원을 DELETE 하려 하면, 외래 키 때문에 거절됩니다(주인 없는 주문이 생기기 때문입니다). 자식 행을 함께 처리하려면 외래 키에 ON DELETE CASCADE (함께 삭제) 같은 옵션을 줄 수 있지만, 의도치 않은 대량 삭제를 부를 수 있으므로 신중히 선택합니다.

팁: 외래 키 컬럼(orders.member_id 등)에는 인덱스를 직접 만들어 두면 좋습니다. PostgreSQL은 기본 키에는 인덱스를 자동으로 만들지만, **외래 키 컬럼에는 자동으로 만들지 않습니다**. 부모 행을 지우거나 JOIN할 때 이 인덱스가 성능에 큰 도움이 됩니다(인덱스는 4절에서 배웁니다).

DBeaver 활용: 위와 같은 테이블 관계(ERD)는 DBeaver(1장)에서 **자동으로 그림으로 볼 수** 있습니다. 데이터베이스 탐색기에서 shop_lab 의 스키마나 테이블을 더블클릭한 뒤 **ER Diagram** 탭을 열면, 기본 키·외래 키를 따라 테이블이 선으로 연결된 다이어그램이 그려집니다. 외래 키를 제대로 걸어 두었는지 눈으로 확인하기 좋습니다.

절 요약

- 기본 키는 행을 유일하게 식별하며, 테이블당 하나만 두고 중복·NULL을 허용하지 않습니다.
- 외래 키는 다른 테이블의 기본 키를 가리켜 참조 무결성을 지킵니다.
- 외래 키 덕분에 "존재하지 않는 대상을 가리키는" 잘못된 데이터가 거절됩니다.
- 기본 키에는 인덱스가 자동 생성되지만, 외래 키 컬럼에는 직접 만들어 주는 것이 좋습니다.

UNIQUE·CHECK·NOT NULL 활용

도입

기본 키 하나로는 모든 규칙을 담을 수 없습니다. "이메일은 회원마다 겹치면 안 된다", "이름은 반드시 있어야 한다", "나이는 음수일 수 없다" 같은 규칙은 따로 걸어야 합니다. 이 절에서는 중복을 막는 **UNIQUE**, 빈 값을 막는 **NOT NULL**, 값의 범위·형식을 검사하는 **CHECK** 를 배웁니다.

핵심 개념 설명

고유 제약 (UNIQUE)

고유 제약(UNIQUE) 이란, 특정 컬럼의 값이 테이블 안에서 겹치지 않도록 강제하는 제약입니다. 같은 이메일로 두 번 가입하는 것을 막는 규칙이 대표적입니다.

기본 키와 비슷해 보이지만 차이가 있습니다. 기본 키는 테이블당 하나뿐이고 NULL을 허용하지 않지만, UNIQUE는 **여러 컬럼에 각각 걸 수 있고** NULL을 허용합니다(NULL은 "값 없음"이라 서로 같다고 보지 않으므로 여러 개 있어도 됩니다). 즉 기본 키가 "행의 대표 식별자"라면, UNIQUE는 "이 컬럼만큼은 겹치지 마라"는 별도의 규칙입니다.

빈 값 금지 (NOT NULL)

빈 값 금지(NOT NULL) 란, 컬럼에 NULL(값 없음)을 허용하지 않아 반드시 어떤 값이 들어가게 하는 제약입니다. 회원 이름처럼 비어 있으면 안 되는 필수 항목에 씁니다. 여기서 주의할 점은 NULL과 빈 문자열('')은 다르다는 것입니다. NOT NULL은 "값이 아예 없는 상태"만 막을 뿐, 빈 문자열은 "값이 있는 것"으로 보아 통과시킵니다.

검사 제약 (CHECK)

검사 제약(CHECK) 이란, 컬럼 값이 지정한 조건을 만족할 때만 저장을 허용하는 제약입니다. "가격은 0보다 커야 한다", "나이는 0 이상 150 이하" 같은 **범위·형식 규칙**을 직접 적을 수 있습니다. 조건식이 참(true)이어야 통과하고, 거짓이면 거절됩니다.

CHECK는 다섯 제약 중 가장 자유롭습니다. 비교(>, <, >=), 범위(BETWEEN), 목록(IN), 패턴 검사까지 SQL 조건식으로 표현할 수 있는 거의 모든 규칙을 담을 수 있습니다.

이렇게 작성합니다

먼저 `members.email` 에 UNIQUE를, `members.age` 에 CHECK를 추가합니다. 이미 만든 테이블에 `ALTER TABLE` 로 붙입니다.


```
-- 이메일 중복 금지, 나이 범위 검사 추가하기
ALTER TABLE members
  ADD CONSTRAINT members_email_unique UNIQUE (email);

ALTER TABLE members
  ADD CONSTRAINT members_age_check CHECK (age >= 0 AND age <= 150);
```

예상 화면:

```
ALTER TABLE
ALTER TABLE
```

이제 규칙을 어기는 값을 넣어 보겠습니다. 먼저 이미 있는 이메일을 다시 넣으면 UNIQUE가 막습니다.

```
-- minsu@example.com 은 이미 등록되어 있음
INSERT INTO members (name, email) VALUES ('홍길동', 'minsu@example.com');
```

예상 화면:

```
ERROR: duplicate key value violates unique constraint "members_email_unique"
DETAIL: Key (email)=(minsu@example.com) already exists.
```

다음으로 나이에 음수를 넣으면 CHECK가 막습니다.

```
-- 나이에 음수를 넣으려는 시도
INSERT INTO members (name, email, age)
VALUES ('이상한', 'weird@example.com', -5);
```

예상 화면:

```
ERROR: new row for relation "members" violates check constraint "members_age_check"
DETAIL: Failing row contains (4, 이상한, weird@example.com, -5).
```

마지막으로 NOT NULL입니다. `name` 은 테이블 정의에서 이미 `NOT NULL` 이므로, 이름을 비우면 거절됩니다.

```
-- 이름(name)을 주지 않으면 NOT NULL이 막습니다
INSERT INTO members (email) VALUES ('noname@example.com');
```

예상 화면:

```
ERROR: null value in column "name" of relation "members" violates not-null constraint
DETAIL: Failing row contains (5, null, noname@example.com, null).
```

세 제약 모두 잘못된 데이터를 입력 단계에서 정확히 걸러 냅니다.

구문 요소 설명

구문	의미
UNIQUE (email)	email 값이 테이블 안에서 겹치지 못하게 합니다
CHECK (age >= 0 AND age <= 150)	조건이 참일 때만 저장을 허용합니다
NOT NULL	컬럼에 빈 값(NULL)을 허용하지 않습니다
ADD CONSTRAINT 이름	제약에 이름을 붙여 관리·식별을 쉽게 합니다

팁: 제약에는 위 예시처럼 **이름을 직접 붙이는 습관**을 권합니다(`members_email_unique` 등). 이름을 주지 않으면 PostgreSQL이 자동으로 만들지만, 의미를 알기 어렵습니다. 이름이 명확하면 오류 메시지만 보고도 어떤 규칙을 어겼는지 바로 알 수 있고, 나중에 제약을 지울 때도 편합니다.

주의: 이미 데이터가 들어 있는 테이블에 제약을 추가할 때는, **기존 데이터가 그 규칙을 위반하면 추가 자체가 실패**합니다. 예를 들어 이미 중복된 이메일이 들어 있는데 UNIQUE를 걸려고 하면 오류가 납니다. 제약을 추가하기 전에 `SELECT email, count(*) FROM members GROUP BY email HAVING count(*) > 1;` 처럼 위반 데이터가 있는지 먼저 확인하고 정리합니다.

절 요약

- UNIQUE는 컬럼 값의 중복을 막고, 여러 컬럼에 각각 걸 수 있으며 NULL은 허용합니다.
- NOT NULL은 빈 값을 막아 필수 입력을 강제하며, 빈 문자열과는 다릅니다.
- CHECK는 범위·형식 등 자유로운 조건식으로 값을 검증합니다.
- 제약에 이름을 붙이면 오류 메시지와 관리가 쉬워집니다.

인덱스와 조회 성능

도입

지금까지는 "올바른 데이터를 지키는 일"을 다뤘습니다. 이제 방향을 바꿔, 쌓인 데이터를 **빠르게 꺼내는 일**을 봅니다. 두꺼운 사전에서 단어를 찾을 때 첫 장부터 한 장씩 넘기지 않고 색인을 보듯, 데이터베이스도 **인덱스(index)** 라는 색인을 두면 조회가 훨씬 빨라집니다. 이 절에서는 인덱스의 원리와, 실제로 인덱스가 쓰였는지 확인하는 방법을 배웁니다.

핵심 개념 설명

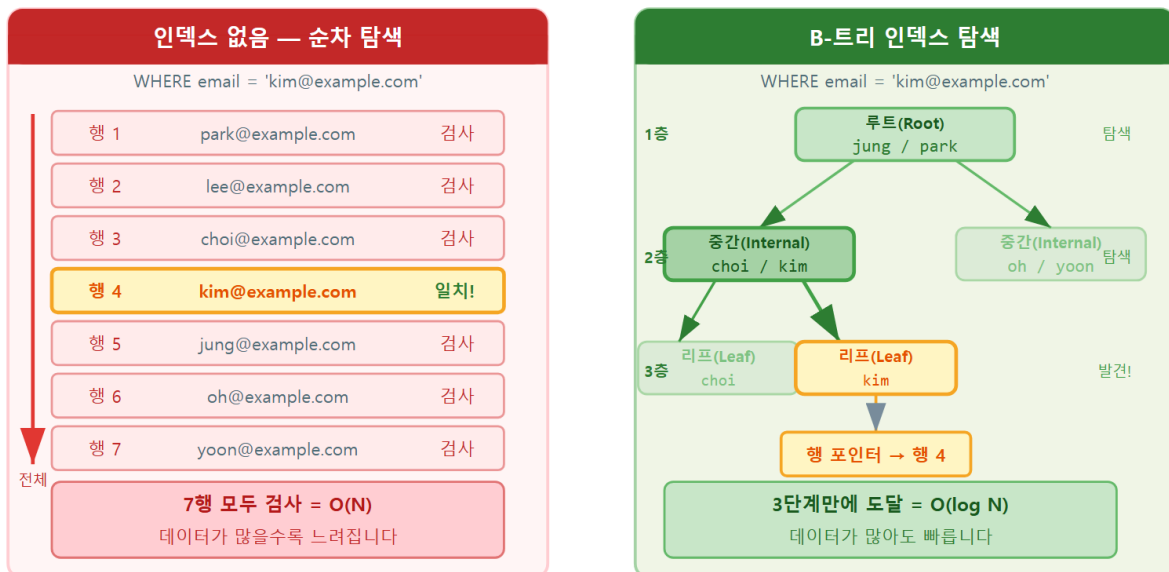
인덱스 (index)

인덱스(index) 란, 특정 컬럼 값을 빠르게 찾도록 미리 정렬·구조화해 둔 보조 자료구조로, 조회 성능을 높여 줍니다. 책 뒤의 색인을 보고 원하는 단어가 있는 쪽을 바로 찾는 것과 같습니다.

PostgreSQL이 기본으로 쓰는 인덱스는 **B-트리(B-tree)** 구조입니다. 값을 정렬된 트리(가지치기) 모양으로 보관해, 루트(맨 위)에서 시작해 몇 단계만 내려가면 목표 값에 도달합니다. 데이터가 100만 건이어도 처음부터 100만 번 비교하는 대신, 대략 20단계 안쪽이면 찾아냅니다.

순차 탐색(Full Scan) vs B-트리 인덱스 탐색

같은 행을 찾는 두 가지 방법 — 데이터 양이 많을수록 차이가 커집니다



EXPLAIN ANALYZE로 Seq Scan vs Index Scan 여부를 확인할 수 있습니다

ch_09_s04 · 인덱스와 조회 성능

- **왜(why) 필요한가:** 인덱스가 없으면 PostgreSQL은 조건에 맞는 행을 찾으려고 테이블을 처음부터 끝까지 한 행씩 훑습니다(이를 순차 탐색, full scan이라 합니다). 데이터가 많을수록 점점 느려집니다. 인덱스가 있으면 색인을 타고 바로 목표로 갑니다.
- **언제(when) 쓰나:** WHERE, JOIN, ORDER BY 에서 자주 쓰는 컬럼, 특히 조회가 빈번한 컬럼에 만듭니다.
- **어떻게(how) 쓰나:** CREATE INDEX 인덱스이름 ON 테이블 (컬럼); 으로 만듭니다.

오해 두 가지를 짚겠습니다. 첫째, "인덱스를 많이 만들수록 항상 빨라진다"는 생각인데, 그렇지 않습니다. 인덱스는 조회를 빠르게 하는 대신 쓰기를 느리게 하고 저장 공간을 더 씁니다(이 트레이드오프는 다음 절에서 다룹니다). 둘째, "인덱스는 쓰기 성능에 영향이 없다"는 오해입니다. 데이터를 넣고·고치고·지울 때마다 관련 인덱스도 함께 갱신해야 하므로, 쓰기에는 분명한 비용이 따릅니다.

조회 성능 (query performance)

조회 성능(query performance)이란, 질의(쿼리)가 결과를 돌려주기까지 걸리는 속도와 효율을 말합니다. 같은 결과라도 어떤 경로(순차 탐색이나, 인덱스 탐색이나)로 찾느냐에 따라 속도가 크게 달라집니다.

PostgreSQL에는 질의를 실제로 어떻게 처리할지 계획을 보여 주는 `EXPLAIN` 명령이 있습니다. 이 계획을 읽으면 "지금 인덱스를 타는지, 아니면 테이블 전체를 훑는지"를 눈으로 확인할 수 있습니다.

이렇게 작성합니다

먼저 인덱스 없이 이메일로 회원을 찾을 때 어떤 계획이 나오는지 `EXPLAIN` 으로 봅니다.

```
-- 인덱스가 없을 때의 조회 계획 확인하기
EXPLAIN SELECT * FROM members WHERE email = 'minsu@example.com';
```

예상 화면:

```

              QUERY PLAN
-----
Seq Scan on members (cost=0.00..1.06 rows=1 width=...)
  Filter: (email = 'minsu@example.com'::text)
```

Seq Scan (순차 탐색)은 테이블을 처음부터 한 행씩 훑는다는 뜻입니다. 데이터가 적으면 빠르지만, 수십만 건이 되면 느려집니다. 이제 `email` 에 인덱스를 만들고 다시 확인합니다.

```
-- email 컬럼에 인덱스 만들기
CREATE INDEX idx_members_email ON members (email);
```

예상 화면:

```
CREATE INDEX
```

```
-- 인덱스가 생긴 뒤의 조회 계획 다시 확인하기
EXPLAIN SELECT * FROM members WHERE email = 'minsu@example.com';
```

예상 화면:

```

QUERY PLAN
-----
Index Scan using idx_members_email on members (cost=0.15..8.17 rows=1 ...)
Index Cond: (email = 'minsu@example.com'::text)

```

이번에는 **Index Scan** (인덱스 탐색)으로 바뀌었습니다. PostgreSQL이 방금 만든 색인을 타고 목표 행을 바로 찾아가겠다는 계획입니다. 데이터가 많을수록 **Seq Scan** 과의 속도 차이가 커집니다.

EXPLAIN 결과 읽기

항목	의미
Seq Scan	테이블을 처음부터 끝까지 한 행씩 훑음(인덱스 미사용)
Index Scan	인덱스를 타고 목표 행으로 바로 접근(인덱스 사용)
cost=...	예상 처리 비용(작을수록 가벼움, 단위는 상대값)
rows=...	돌려줄 것으로 예상하는 행 수

팁: EXPLAIN 뒤에 ANALYZE 를 붙이면(EXPLAIN ANALYZE ...) 계획만 보여 주는 데 그치지 않고 **질의**를 실제로 실행해 걸린 시간까지 알려 줍니다. 다만 INSERT · UPDATE · DELETE 에 ANALYZE 를 붙이면 실제로 데이터가 바뀌므로, 쓰기 질의에는 주의해서 사용합니다.

주의: 데이터가 적으면 인덱스를 만들어도 EXPLAIN 에 여전히 Seq Scan 이 나올 수 있습니다. 이는 오류가 아닙니다. 행이 몇 개뿐이면 테이블을 통째로 읽는 편이 인덱스를 거치는 것보다 오히려 빠르다고 PostgreSQL이 판단하기 때문입니다. 인덱스의 효과는 데이터가 충분히 많을 때 분명해집니다.

절 요약

- 인덱스는 컬럼 값을 미리 정렬·구조화한 색인으로, 조회를 빠르게 합니다.
- PostgreSQL 기본 인덱스는 B-트리이며, 몇 단계 만에 목표 값에 도달합니다.
- 인덱스가 없으면 순차 탐색(Seq Scan)으로 테이블 전체를 훑습니다.

- `EXPLAIN` 으로 질의가 인덱스를 타는지(Index Scan) 눈으로 확인할 수 있습니다.

인덱스 설계 시 주의점

도입

"그럼 모든 컬럼에 인덱스를 걸면 되지 않나"라는 생각이 들 수 있습니다. 하지만 인덱스는 공짜가 아닙니다. 색인을 많이 둘수록 책을 새로 찍을 때마다 색인을 전부 다시 만들어야 하듯, 데이터를 바꿀 때마다 인덱스도 갱신해야 합니다. 이 절에서는 인덱스의 비용과, 어떤 컬럼에 인덱스를 걸지 판단하는 기준을 배웁니다.

핵심 개념 설명

인덱스 비용 (index cost)

인덱스 비용(index cost) 이란, 인덱스를 두면서 치르게 되는 대가를 말합니다. 인덱스는 조회를 빠르게 하지만, 그 대가로 다음 두 가지를 지불합니다.

- **쓰기 성능 저하:** `INSERT` · `UPDATE` · `DELETE` 로 데이터를 바꿀 때마다, 관련된 인덱스도 함께 갱신해야 합니다. 인덱스가 많은 테이블일수록 쓰기가 느려집니다.
- **저장 공간 증가:** 인덱스는 별도의 자료구조이므로 디스크 공간을 추가로 차지합니다. 컬럼이 클수록, 인덱스가 많을수록 공간이 늘어납니다.

즉 인덱스는 "읽기는 빠르게, 쓰기·공간은 손해"라는 **트레이드오프(trade-off, 맞바꿈)** 관계에 있습니다. 조회가 잦은 컬럼에는 이득이 크지만, 거의 조회하지 않는데 자주 바뀌는 컬럼에 인덱스를 걸면 손해만 봅니다.

선택적 인덱싱 (selective indexing)

선택적 인덱싱(selective indexing) 이란, 모든 컬럼이 아니라 **효과가 큰 컬럼만 골라** 인덱스를 만드는 설계 원칙입니다. 핵심은 "이 인덱스가 조회를 얼마나 자주, 얼마나 크게 빠르게 하는가"를 기준으로 판단하는 것입니다.

인덱스가 효과적인 컬럼의 특징은 다음과 같습니다.

- **값의 종류가 다양한 컬럼:** 이메일·주문번호처럼 값이 거의 다 다른 컬럼은 인덱스로 후보를 크게 좁힐 수 있어 효과가 큼니다. 반대로 "성별"처럼 값이 두세 종류뿐인 컬럼은, 인덱

스를 타도 절반을 골라내는 정도라 효과가 작습니다(이런 성질을 선택도, selectivity가 높다/낮다고 표현합니다).

- **WHERE·JOIN·ORDER BY에 자주 등장하는 컬럼:** 실제 조회에서 자주 조건으로 쓰이는 컬럼이어야 인덱스가 일을 합니다.
- **외래 키 컬럼:** 앞서 본 것처럼 JOIN과 부모 행 삭제 검사에서 자주 쓰이므로 인덱스를 만들어 두면 좋습니다.

이렇게 작성합니다

자주 조회하는 컬럼에 인덱스를 만드는 예시입니다. 주문에서 회원별·상품별 조회가 잦다면, 외래 키 컬럼에 인덱스를 둡니다.

```
-- 자주 조회·JOIN하는 외래 키 컬럼에 인덱스 만들기
CREATE INDEX idx_orders_member_id ON orders (member_id);
CREATE INDEX idx_orders_product_id ON orders (product_id);
```

예상 화면:

```
CREATE INDEX
CREATE INDEX
```

현재 테이블에 어떤 인덱스가 걸려 있는지는 psql의 \d 명령으로 확인합니다.

```
-- orders 테이블의 구조와 인덱스 목록 보기
\d orders
```

예상 화면:

```
Table "public.orders"
  Column | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
id       | integer       |           | not null | generated always as identity
member_id | integer       |           | not null |
product_id | integer       |           | not null |
quantity | integer       |           | not null |
Indexes:
    "orders_pkey" PRIMARY KEY, btree (id)
    "idx_orders_member_id" btree (member_id)
    "idx_orders_product_id" btree (product_id)
Foreign-key constraints:
    "orders_member_fk" FOREIGN KEY (member_id) REFERENCES members(id)
    "orders_product_fk" FOREIGN KEY (product_id) REFERENCES products(id)
```

Indexes: 항목에서 기본 키에 자동 생성된 `orders_pkey` 와, 우리가 만든 두 인덱스가 모두 `btree` (B-트리)임을 확인할 수 있습니다.

인덱스 설계 판단 기준

컬럼 특성	인덱스 효과	권장 여부
값 종류가 다양(이메일·주문번호)	큼	권장
WHERE·JOIN에 자주 등장	큼	권장
외래 키 컬럼	큼(JOIN·삭제 검사)	권장
값 종류가 적음(성별 등)	작음	신중
거의 조회 안 하고 자주 바뀜	손해만	비권장

베스트 프랙티스: 인덱스는 "필요해 보여서"가 아니라 **실제 느린 조회를 확인한 뒤** 추가하는 것이 좋습니다. 느린 질의를 `EXPLAIN` 으로 분석해 `Seq Scan` 이 병목인 컬럼을 찾고, 그 컬럼에 인덱스를 만든 다음 다시 `EXPLAIN` 으로 `Index Scan` 으로 바뀌었는지 확인하는 흐름을 권합니다. 추측이 아니라 측정에 근거해 인덱스를 설계합니다.

주의: 쓰기가 매우 잦은 테이블(예: 로그 기록)에 인덱스를 많이 걸면, 조회 이득보다 쓰기 부담이 더 클 수 있습니다. 또 쓰지 않는 인덱스는 공간과 갱신 비용만 잡아먹는 짐이 됩니다. 주기적으로 사용되지 않는 인덱스를 점검해 정리합니다.

절 요약

- 인덱스는 읽기를 빠르게 하는 대신 쓰기 성능과 저장 공간을 희생하는 트레이드오프입니다.
- 값 종류가 다양하고 조회에 자주 쓰이는 컬럼, 외래 키 컬럼이 인덱스 후보로 좋습니다.
- 값 종류가 적거나 거의 조회하지 않는 컬럼에는 인덱스를 신중히 결정합니다.
- 추측이 아니라 `EXPLAIN` 으로 측정해 인덱스를 설계합니다.

대용량 데이터로 인덱스 효과 실측

도입

앞 절에서 "데이터가 적으면 인덱스를 만들어도 Seq Scan 이 나올 수 있다"고 했습니다. 그렇다면 인덱스의 효과는 언제 분명해질까요. 지금까지의 실습 데이터는 수십 행 규모라 인덱스 차이가 거의 드러나지 않습니다. 이 절에서는 주문 데이터를 **100만 건**으로 늘려, 인덱스 유무가 실제 실행 시간을 어떻게 가르는지 EXPLAIN ANALYZE 로 직접 측정합니다.

핵심 개념 설명

대용량 데이터 준비 (seed_large.sql)

교재와 함께 제공되는 data/seed_large.sql 스크립트는 **대량 생성 함수(generate_series)** 로 회원 5만·상품 1천·주문 **100만 건**을 한 번에 만듭니다. 손으로 INSERT 를 100만 번 적을 수는 없으니, 일련번호로 값을 찍어 내는 방식입니다.

```
-- data/seed_large.sql 에서 주문 100만 건을 만드는 부분(발췌)
INSERT INTO orders (member_id, product_id, quantity)
SELECT 1 + (g % 50000), -- 회원 id를 1~5만에 고르게 분포
       1 + (g % 1000),  -- 상품 id를 1~1천에 고르게 분포
       1 + (g % 5)      -- 수량 1~5 순환
FROM generate_series(1, 1000000) AS g;
```

코드 설명

요소	의미
generate_series(1, 1000000) AS g	1부터 100만까지 번호를 한 행씩 만들어, 그 수만큼 행을 생성합니다
1 + (g % 50000)	나머지 연산으로 회원 id를 1~5만에 고르게 흩뿌립니다
난수(random) 미사용	번호 g 로만 값을 만들어 누가 실행해도 같은 데이터가 나옵니다(재현 가능)

스크립트 실행은 psql 에서 한 줄입니다. Windows 11의 PowerShell(파워셸)에서 shop_lab 데이터베이스를 대상으로 실행합니다.

```
psql -d shop_lab -f data/seed_large.sql
```

100만 행 생성에는 일반 PC 기준 수초~수십초가 걸립니다. 끝나면 행 수를 확인합니다.

```
SELECT count(*) FROM orders;
```

```

count
-----
1000000
(1 row)

```

인덱스 없이 — Seq Scan

이제 특정 회원의 주문을 찾아봅니다. `EXPLAIN ANALYZE` 는 실행 계획만이 아니라 **실제로 질의를 실행해 걸린 시간까지** 보여 줍니다.

```

EXPLAIN ANALYZE
SELECT * FROM orders WHERE member_id = 12345;

```

```

Seq Scan on orders (cost=0.00..18334.00 rows=20 width=20)
  (actual time=0.412..63.108 rows=20 loops=1)
  Filter: (member_id = 12345)
  Rows Removed by Filter: 999980
  Planning Time: 0.085 ms
  Execution Time: 63.214 ms      -- 환경에 따라 다름(수십 ms대)

```

인덱스가 없으니 PostgreSQL은 100만 행을 처음부터 끝까지 훑습니다(`Seq Scan`). 찾는 행이 20건뿐인데도 나머지 99만 9,980행을 일일이 걸러 냈음을(`Rows Removed by Filter`) 볼 수 있습니다. 데이터가 많을수록 느려지는 구조입니다.

인덱스 생성 후 — Index Scan

`member_id` 에 인덱스를 만들고 같은 질의를 다시 측정합니다.

```

CREATE INDEX idx_orders_member_id ON orders (member_id);

EXPLAIN ANALYZE
SELECT * FROM orders WHERE member_id = 12345;

```

```

Index Scan using idx_orders_member_id on orders
  (cost=0.42..8.61 rows=20 width=20)
  (actual time=0.039..0.061 rows=20 loops=1)
  Index Cond: (member_id = 12345)
  Planning Time: 0.121 ms
  Execution Time: 0.094 ms      -- 환경에 따라 다름(0.1 ms 안팎)

```

이번에는 B-트리(B-tree) 인덱스를 타고 해당 회원의 주문 20건만 곧장 찾아갑니다(`Index Scan`). 100만 행을 다 읽지 않으므로 비교 대상 자체가 사라졌습니다.

주의: 위 측정값은 환경(CPU·디스크·캐시·PostgreSQL 버전)에 따라 달라집니다. 절대 수치보다 **Seq Scan → Index Scan 전환**과 실행 시간의 **자릿수 차이**에 주목하십시오. 또 같은 질의를 두 번째 실행하면 캐시 덕분에 더 빨라질 수 있으므로, 비교는 같은 조건에서 합니다.

무엇이 달라졌나

구분	실행 계획	읽는 방식	실행 시간(예시)
인덱스 없음	Seq Scan	100만 행 전부 훑음	약 63 ms
인덱스 있음	Index Scan	B-트리로 직접 탐색	약 0.09 ms

같은 질의·같은 데이터인데 실행 시간이 수백 배 차이 납니다. 앞 절에서 20행으로는 보이지 않던 효과가 100만 행에서 극명하게 드러납니다. "인덱스의 가치는 데이터가 충분히 쌓였을 때 분명해진다"는 원리를 추측이 아니라 **실측**으로 확인한 셈입니다.

절 요약

- 인덱스 효과는 데이터가 많을 때(여기서는 주문 100만 건) 분명해집니다.
- EXPLAIN ANALYZE 로 Seq Scan → Index Scan 전환과 실행 시간 차이를 직접 측정합니다.
- 대용량 데이터는 generate_series 로 재현 가능하게 만듭니다(data/seed_large.sql).

실습 과제

해보기: 쇼핑몰 스키마에 제약·인덱스 입히기

실습 데이터베이스 shop_lab 에서 이 장의 제약조건과 인덱스를 직접 적용해 봅니다.

- 단계 1: members · products · orders 테이블을 만들고, 각 테이블에 기본 키(id PRIMARY KEY)를 둡니다.
- 단계 2: orders.member_id 와 orders.product_id 에 외래 키를 추가해, 각각 members.id · products.id 를 가리키게 합니다. 존재하지 않는 회원 번호로 주문을 넣어 보고 거절되는지 확인합니다.
- 단계 3: members.email 에 UNIQUE, members.age 에 CHECK (age >= 0 AND age <= 150) , products.price 에 CHECK (price > 0) 를 추가합니다. 규칙을 어기는 값을 넣어 보고 어떤 오류 메시지가 나오는지 적어 둡니다.
- 단계 4: members.email 과 orders.member_id 에 인덱스를 만듭니다.

- 점검: `EXPLAIN SELECT * FROM members WHERE email = '...';` 을 인덱스 생성 전후로 실행해, Seq Scan 이 Index Scan 으로 바뀌는지(또는 데이터가 적어 그대로인지) 결과를 기록합니다. \d orders 로 걸린 제약과 인덱스 목록을 확인합니다.

FAQ

Q1. 제약조건은 테이블을 만들 때만 걸 수 있나요? → A1. 아닙니다. CREATE TABLE 로 만들 때 컬럼 옆에 적을 수도 있고, 이미 있는 테이블에 ALTER TABLE ... ADD CONSTRAINT ... 로 나중에 추가할 수도 있습니다. 단, 나중에 추가할 때 기존 데이터가 그 규칙을 어기고 있으면 추가가 실패하므로, 먼저 위반 데이터를 정리해야 합니다.

Q2. 기본 키(PRIMARY KEY)와 UNIQUE는 뭐가 다른가요? → A2. 둘 다 중복을 막지만 차이가 있습니다. 기본 키는 테이블당 **하나만** 두며 NULL을 허용하지 않고, 그 테이블의 대표 식별자 역할을 합니다. UNIQUE는 **여러 컬럼에 각각** 걸 수 있고 NULL을 허용합니다. 보통 id 를 기본 키로 두고, 이메일처럼 추가로 겹치면 안 되는 컬럼에 UNIQUE를 겁니다.

Q3. 인덱스를 만들면 무조건 빨라지나요? → A3. 그렇지 않습니다. 조회는 빨라지지만, 데이터를 넣고·고치고·지울 때 인덱스도 함께 갱신해야 해서 쓰기는 느려지고 저장 공간도 더 씁니다. 또 데이터가 적으면 PostgreSQL이 인덱스 대신 순차 탐색을 택하기도 합니다. 자주 조회하는 컬럼에 선택적으로 만드는 것이 좋습니다.

Q4. 외래 키가 있으면 부모 행을 못 지우나요? → A4. 기본적으로는 자식 행이 가리키고 있는 부모 행을 지우려 하면 거절됩니다(참조 무결성 보호). 함께 처리하려면 외래 키에 ON DELETE CASCADE (부모와 함께 자식도 삭제)나 ON DELETE SET NULL (자식의 참조를 NULL로) 같은 옵션을 줄 수 있습니다. 다만 의도치 않은 대량 삭제를 부를 수 있어 신중히 선택합니다.

장 요약

이 장에서는 데이터를 올바르게 지키는 **제약조건**과, 빠르게 꺼내는 **인덱스**를 배웠습니다. 제약조건은 잘못된 데이터가 들어오기 전에 막는 규칙으로, 기본 키(PRIMARY KEY)는 행을 유일하게 식별하고, 외래 키(FOREIGN KEY)는 테이블 사이의 참조 무결성을 지키며, UNIQUE·NOT NULL·CHECK는 중복·빈 값·범위 위반을 각각 거릅니다. 인덱스는 B-트리 색인으로 조회를 빠르게 하지만, 쓰기 성능과 저장 공간을 희생하는 트레이드오프가 있으므로 효과가 큰 컬럼에 선택

적으로 만들어야 합니다. `EXPLAIN` 으로 인덱스 사용 여부를 측정해 추측이 아닌 근거로 설계하는 습관이 핵심입니다.

핵심 키워드

제약조건(constraint), 데이터 무결성(data integrity), 기본 키(primary key), 외래 키(foreign key), 참조 무결성(referential integrity), `UNIQUE`, `NOT NULL`, `CHECK`, 인덱스(index), B-트리(B-tree), `EXPLAIN`, 선택적 인덱싱(selective indexing)

다음 장 미리보기

다음 장에서는 여러 테이블에 흩어진 데이터를 하나로 연결해 보는 **JOIN(조인)** 을 배웁니다. 이 장에서 외래 키로 맺어 둔 회원·상품·주문의 관계가, JOIN을 통해 "누가 어떤 상품을 주문했는지"를 한 화면에 모아 보여 주는 실제 조회로 어떻게 이어지는지 살펴봅니다.

트랜잭션: CRUD의 원자성과 ACID

학습 목표 - 트랜잭션이 여러 작업을 하나로 묶는 단위임을 ACID와 함께 설명할 수 있습니다. - `BEGIN` · `COMMIT` · `ROLLBACK` 으로 작업을 확정하거나 되돌릴 수 있습니다. - 동시 접근에서 격리 수준(isolation level)이 필요한 이유를 이해합니다. - `SAVEPOINT` 로 트랜잭션 일부만 롤백할 수 있습니다.

앞 장들에서 우리는 데이터를 만들고(`INSERT`), 읽고(`SELECT`), 고치고(`UPDATE`), 지우는(`DELETE`) 네 가지 작업을 하나씩 배웠습니다. 그런데 현실의 일은 명령 한 줄로 끝나지 않는 경우가 많습니다. "A 계좌에서 출금하고 B 계좌에 입금한다"처럼 **여러 작업이 한 묶음으로 함께 성공하거나 함께 실패해야** 하는 일이 흔합니다.

이 장에서 배우는 **트랜잭션(transaction)** 은 바로 그 "한 묶음"을 만드는 장치입니다. 여러 SQL 을 하나의 단위로 묶어, 중간에 문제가 생기면 묶음 전체를 깔끔하게 되돌릴 수 있게 해 줍니다. 트랜잭션은 특정 언어의 기능이 아니라 데이터를 안전하게 다루기 위한 데이터베이스의 핵심 개념입니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 에서 `psql` 로 PostgreSQL에 접속한 상태를 기준으로 합니다. 화면 앞의 `shop_lab=#` 표시는 "지금 `shop_lab` 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다. 이 장에서는 새로운 표시 하나가 더 등장합니다. 트랜잭션이 진행 중일 때는 `=#` 이 `=*#` 처럼 별표가 붙어, "지금 묶음 작업 안에 있다"는 것을 알려 줍니다.

이 장의 예시는 아래 두 테이블을 기준으로 합니다. 계좌 이체 비유를 SQL로 다루기 위한 `accounts` (계좌)와, 재고를 다루기 위한 `products` (상품)입니다.

```
-- 이 장 예시가 기준으로 삼는 테이블 (참고용)
CREATE TABLE accounts (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  owner   varchar(50) NOT NULL,
  balance numeric(12, 2) NOT NULL DEFAULT 0
);

INSERT INTO accounts (owner, balance)
VALUES ('김민수', 100000), ('이서연', 50000);
```

`balance` 는 잔액이고, 처음에 김민수 계좌에 100,000원, 이서연 계좌에 50,000원이 들어 있다고 가정합니다.

트랜잭션(transaction)과 ACID

도입

은행에서 김민수 계좌의 돈을 이서연 계좌로 옮긴다고 떠올려 봅시다. 이 일은 두 단계로 이루어집니다. 첫째, 김민수 잔액을 줄인다. 둘째, 이서연 잔액을 늘린다. 그런데 첫 단계만 성공하고 둘째 단계에서 정전이 난다면, 돈은 김민수에게서 빠졌는데 이서연에게는 들어가지 않은 **사라진 돈**이 생깁니다. 이 절에서는 이런 사고를 막아 주는 트랜잭션과 그 약속인 ACID를 배웁니다.

핵심 개념 설명

트랜잭션 (transaction)

트랜잭션(transaction)이란, 여러 SQL 작업을 하나의 단위로 묶어 **전부 성공하거나 전부 취소 되도록** 보장하는 처리 단위입니다. 계좌 이체에서 출금과 입금이 둘 다 되거나 둘 다 안 되어야 하는 것과 똑같습니다.

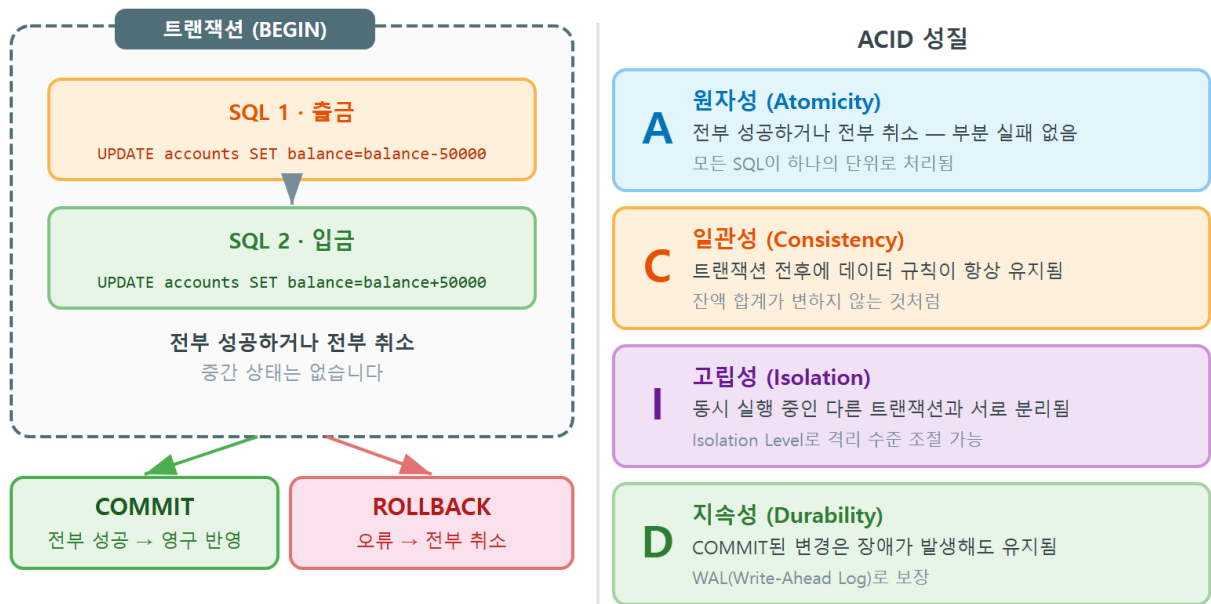
- **왜(why) 필요한가:** 여러 작업이 부분적으로만 적용되면 데이터가 앞뒤가 맞지 않는 상태가 됩니다(앞서 본 사라진 돈처럼). 트랜잭션은 "중간 상태로 멈추는 일"을 막아, 데이터를 언제나 말이 되는 상태로 유지합니다.
- **언제(when) 쓰나:** 두 개 이상의 변경이 논리적으로 한 묶음일 때입니다. 이체(출금+입금), 주문 처리(재고 차감+주문 생성+결제 기록), 회원 탈퇴(여러 테이블의 관련 행 삭제) 등이 대표적입니다.
- **어떻게(how) 쓰나:** `BEGIN` 으로 묶음을 시작하고, 작업을 마친 뒤 `COMMIT` 으로 확정하거나 `ROLLBACK` 으로 통째로 되돌립니다(다음 절에서 자세히 다룹니다).

여기서 입문자가 자주 하는 오해 두 가지를 짚겠습니다. 첫째, "각 SQL이 알아서 묶여 함께 취소된다"는 생각입니다. 그렇지 않습니다. `BEGIN` 으로 명시적으로 묶지 않으면, PostgreSQL은 SQL 한 줄 한 줄을 각각 독립된 트랜잭션으로 보고 즉시 확정합니다. 둘째, "트랜잭션은 큰 시스템에서만 필요하다"는 오해입니다. 데이터 한 건이라도 정확성이 중요하다면 규모와 상관없이 필요합니다.

ACID 성질 (ACID properties)

ACID 성질(ACID properties)이란, 트랜잭션이 지켜 주는 네 가지 약속의 머리글자를 모은 말입니다. 트랜잭션이 "믿을 수 있는 한 묶음"이 되기 위한 조건이라고 생각하면 됩니다.

트랜잭션과 ACID 성질



ch_10_s01 · 트랜잭션과 ACID

ACID 네 가지 약속

글자	이름	한 줄 의미	이체 예시
A	원자성(Atomicity)	전부 되거나 전부 안 됨	출금과 입금이 한 덩어리로 처리됨
C	일관성(Consistency)	규칙을 깨지 않음	잔액이 음수가 되는 등 제약 위반을 막음
I	고립성(Isolation)	동시 작업이 서로 간섭하지 않음	다른 사람의 이체가 끼어들어 섞이지 않음
D	지속성(Durability)	확정되면 사라지지 않음	정전이 나도 끝난 이체는 남아있음

네 가지 중 입문 단계에서 가장 먼저 와닿는 것은 **원자성**입니다. 원자(atom)는 더 쪼갤 수 없는 단위라는 뜻으로, 트랜잭션 안의 작업들이 더 쪼개지지 않고 한 덩어리로 처리된다는 의미입니다. 출금만 되고 입금이 안 되는 "절반의 처리"는 원자성 위반이며, 트랜잭션은 이를 ROLLBACK으로 막습니다.

이렇게 작성합니다

다음은 트랜잭션 없이 이체를 시도하다 사고가 나는 상황을 보여 주는 예시입니다. 두 명령 사이에 문제가 생기면 어떻게 되는지 봅니다.

```
-- 트랜잭션으로 묶지 않은 위험한 이체 (출금만 성공할 수 있음)
UPDATE accounts SET balance = balance - 30000 WHERE owner = '김민수';
-- 여기서 오류나 정전이 나면, 아래 입금은 실행되지 않습니다
UPDATE accounts SET balance = balance + 30000 WHERE owner = '이서연';
```

예상 화면:

```
shop_lab=# UPDATE accounts SET balance = balance - 30000 WHERE owner = '김민수';
UPDATE 1
shop_lab=# UPDATE accounts SET balance = balance + 30000 WHERE owner = '이서연';
UPDATE 1
```

위 예시는 묶이지 않았기 때문에, 첫 UPDATE 가 끝나는 순간 그 변경이 곧바로 확정됩니다. 만약 첫 명령 직후에 연결이 끊겼다면, 김민수의 30,000원만 사라진 상태로 남습니다. 이 문제를 트랜잭션으로 어떻게 막는지는 다음 절에서 직접 다룹니다.

팁: PostgreSQL에서 BEGIN 없이 실행한 SQL 한 줄은 그 자체로 하나의 트랜잭션으로 처리되어 자동으로 확정됩니다. 이를 자동 커밋(autocommit)이라고 합니다. 즉 트랜잭션이 "없는" 것이 아니라, 한 줄마다 작은 트랜잭션이 자동으로 열렸다 닫히는 것입니다.

절 요약

- 트랜잭션은 여러 SQL을 한 묶음으로 만들어 "전부 성공 또는 전부 취소"를 보장합니다.
- ACID는 트랜잭션의 네 가지 약속(원자성·일관성·고립성·지속성)입니다.
- BEGIN 없이 실행한 SQL은 한 줄마다 자동으로 확정(자동 커밋)됩니다.

BEGIN·COMMIT·ROLLBACK 사용하기

도입

앞 절에서 트랜잭션이 왜 필요한지 보았습니다. 이제 실제로 묶는 방법을 배웁니다. 문서 편집기에서 여러 군데를 고친 뒤 마음에 들면 '저장', 마음에 안 들면 '저장하지 않고 닫기'를 누르는 것과 같습니다. 여기서 '편집 시작'이 BEGIN, '저장'이 COMMIT, '저장 안 함'이 ROLLBACK 입니다.

핵심 개념 설명

트랜잭션 시작 (BEGIN)

트랜잭션 시작(BEGIN) 이란, `BEGIN` 명령으로 "지금부터 여러 SQL을 한 묶음으로 다루겠다"고 선언하는 것입니다. `BEGIN` 이후의 변경은 아직 임시 상태이며, 같은 트랜잭션 안에서는 보이지만 다른 사용자에게는 보이지 않습니다.

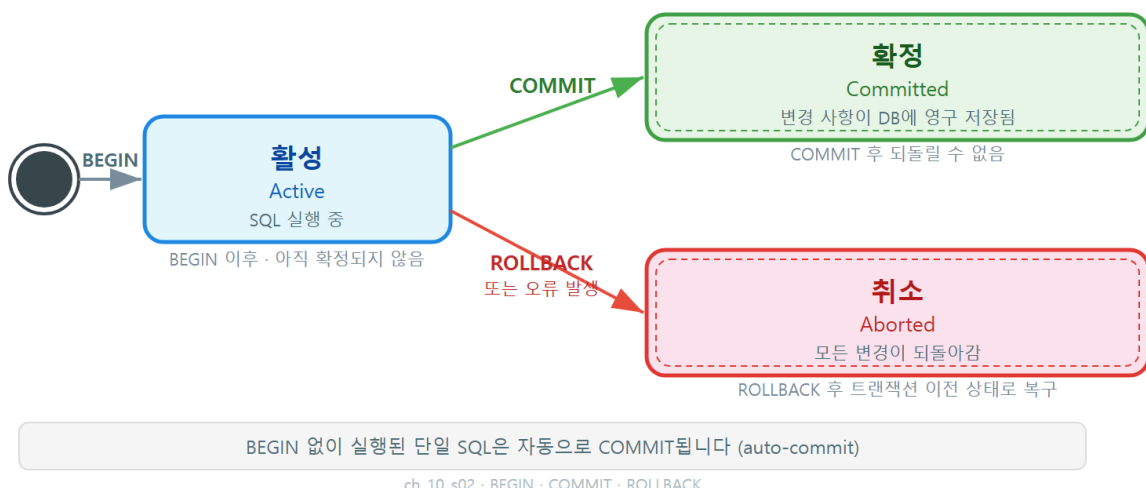
- **왜(why) 쓰나:** 자동 커밋을 잠시 멈추고, 여러 작업을 묶어 한꺼번에 확정하거나 취소할 수 있게 하기 위해서입니다.
- **언제(when) 쓰나:** 두 개 이상의 변경을 한 덩어리로 처리하고 싶을 때입니다.
- **어떻게(how) 쓰나:** `BEGIN;` 을 입력하면 프롬프트가 `shop_lab=#` 처럼 별표가 붙어 트랜잭션이 열렸음을 보여 줍니다.

확정·취소 (COMMIT/ROLLBACK)

확정·취소(COMMIT/ROLLBACK) 란, 트랜잭션의 변경을 영구히 확정하는 `COMMIT` 과, 변경을 모두 되돌려 취소하는 `ROLLBACK` 명령입니다. 문서를 '저장'해 확정하거나, '저장 안 함'으로 편집 내용을 버리는 것과 같습니다.

`COMMIT` 전까지의 변경은 임시 상태입니다. 여기서 입문자가 자주 하는 오해를 짚겠습니다. 첫째, "`COMMIT` 전에도 변경이 다른 사용자에게 보인다"는 생각입니다. 그렇지 않습니다. 확정 전 변경은 같은 트랜잭션 안에서만 보이고, 다른 접속에는 옛날 값이 보입니다. 둘째, "`ROLLBACK` 후에도 일부 변경이 남는다"는 오해입니다. `ROLLBACK` 은 `BEGIN` 이후의 모든 변경을 깔끔하게 되돌립니다(부분만 되돌리는 방법은 마지막 절의 `SAVEPOINT` 에서 다룹니다).

트랜잭션 상태 전이: BEGIN · COMMIT · ROLLBACK



트랜잭션은 세 가지 상태를 거칩니다. `BEGIN` 을 만나면 **활성(active)** 상태가 되고, `COMMIT` 이면 **확정(committed)**, `ROLLBACK` 이나 오류면 **취소(aborted)** 상태로 끝납니다.

이렇게 작성합니다

이번에는 앞 절의 위험한 이체를 트랜잭션으로 안전하게 묶어 봅니다. 두 `UPDATE` 를 `BEGIN` 과 `COMMIT` 사이에 넣습니다.

```
-- 출금과 입금을 한 트랜잭션으로 안전하게 묶어 확정하기
BEGIN;

UPDATE accounts SET balance = balance - 30000 WHERE owner = '김민수';
UPDATE accounts SET balance = balance + 30000 WHERE owner = '이서연';

COMMIT;
```

예상 화면:

```
shop_lab=# BEGIN;
BEGIN
shop_lab=# UPDATE accounts SET balance = balance - 30000 WHERE owner = '김민수';
UPDATE 1
shop_lab=# UPDATE accounts SET balance = balance + 30000 WHERE owner = '이서연';
UPDATE 1
shop_lab=# COMMIT;
COMMIT
```

`BEGIN` 직후부터 프롬프트가 `shop_lab=#` 로 바뀌어 트랜잭션이 열려 있음을 알려 줍니다. `COMMIT` 을 입력하면 두 변경이 동시에 확정되고 프롬프트가 다시 `shop_lab=#` 로 돌아옵니다. 이제 두 `UPDATE` 는 한 덩어리이므로, 둘 다 적용되거나 둘 다 적용되지 않습니다.

명령별 의미

명령	의미
<code>BEGIN;</code>	트랜잭션을 시작합니다(이후 변경은 임시 상태)
<code>UPDATE ...</code>	묶음 안의 작업입니다(아직 확정 전)
<code>COMMIT;</code>	묶음 전체를 영구히 확정합니다
<code>ROLLBACK;</code>	<code>BEGIN</code> 이후 모든 변경을 되돌립니다

이번에는 도중에 문제가 있음을 발견해 통째로 되돌리는 경우입니다. 출금은 했지만 입금 계좌를 잘못 골랐다고 가정하고 `ROLLBACK` 합니다.

```
-- 도중에 잘못을 발견하면 ROLLBACK으로 통째로 되돌리기
BEGIN;

UPDATE accounts SET balance = balance - 30000 WHERE owner = '김민수';
-- 입금 대상을 잘못 지정했음을 발견 -> 전부 취소
ROLLBACK;
```

예상 화면:

```
shop_lab=# BEGIN;
BEGIN
shop_lab=# UPDATE accounts SET balance = balance - 30000 WHERE owner = '김민수';
UPDATE 1
shop_lab=# ROLLBACK;
ROLLBACK
shop_lab=# SELECT owner, balance FROM accounts WHERE owner = '김민수';
 owner | balance
-----+-----
 김민수 | 100000.00
(1 row)
```

`UPDATE 1`로 잔액이 줄어든 것처럼 보였지만, `ROLLBACK` 후 다시 조회하면 김민수의 잔액이 원래의 `100000.00`으로 돌아와 있습니다. 임시 변경이 흔적 없이 되돌려진 것입니다.

주의: 트랜잭션을 열어 두고(`BEGIN`) `COMMIT`이나 `ROLLBACK` 없이 접속을 끊으면, 그동안의 변경은 **자동으로 ROLLBACK**되어 사라집니다. 또한 트랜잭션을 연 채로 방치하면 다른 작업이 기다리게 될 수 있으므로, 묶음 작업은 가능한 한 짧게 처리하고 곧바로 확정 또는 취소합니다.

위험: 트랜잭션 도중 SQL 하나가 오류를 내면, PostgreSQL은 트랜잭션 전체를 오류 상태로 만들고 이후 명령을 모두 거부합니다. 이때 화면에는 `ERROR: current transaction is aborted, commands ignored until end of transaction block`이 반복됩니다. 이 경우 `COMMIT`을 해도 실제로는 확정되지 않으니, `ROLLBACK`으로 트랜잭션을 닫고 처음부터 다시 시작합니다.

절 요약

- `BEGIN`으로 묶음을 시작하면 프롬프트에 별표(=*)가 붙습니다.
- `COMMIT`은 변경을 영구히 확정하고, `ROLLBACK`은 `BEGIN` 이후 변경을 모두 되돌립니다.
- 확정 전 변경은 같은 트랜잭션 안에서만 보이며, 오류가 나면 트랜잭션 전체가 막혀 `ROLLBACK`이 필요합니다.

동시성과 격리 수준(isolation level)

도입

지금까지는 나 혼자 데이터를 다룬다고 가정했습니다. 그러나 실제 서비스에서는 수많은 사용자가 **같은 데이터를 동시에** 읽고 씁니다. 인기 상품 하나를 여러 사람이 동시에 주문하면, 재고 숫자를 두고 작업이 서로 부딪칩니다. 이 절에서는 이런 충돌을 다루는 동시성과 격리 수준을 배웁니다.

핵심 개념 설명

동시성 (concurrency)

동시성(concurrency)이란, 여러 트랜잭션이 같은 데이터에 비슷한 시점에 접근하는 상황을 말합니다. 좁은 문 하나로 여러 사람이 동시에 들어가려는 것과 비슷해서, 순서를 잘 조정하지 않으면 서로 부딪칩니다.

- **왜(why) 중요한가:** 동시 접근을 방치하면 한 사람의 변경이 다른 사람의 변경을 덮어쓰거나, 아직 확정되지 않은 임시 값을 잘못 읽는 등 데이터가 어긋날 수 있습니다.
- **언제(when) 나타나나:** 여러 사용자가 같은 행을 동시에 수정할 때, 한쪽이 읽는 동안 다른 쪽이 그 값을 바꿀 때입니다.
- **어떻게(how) 다루나:** PostgreSQL은 각 트랜잭션이 일관된 데이터만 보도록 **격리 수준**으로 조절합니다.

동시성 문제는 몇 가지 전형적인 모습으로 나타납니다. 아직 확정되지 않은 값을 읽는 **더티 리드(dirty read)**, 같은 조회를 두 번 했는데 값이 달라지는 **반복 불가능 읽기(non-repeatable read)**, 조건에 맞는 행 수가 중간에 달라지는 **유령 읽기(phantom read)**가 대표적입니다. 용어가 어렵게 들리지만, 핵심은 "동시에 일하다 보면 읽은 값이 흔들릴 수 있다"는 한 가지입니다.

격리 수준 (isolation level)

격리 수준(isolation level)이란, 동시에 도는 트랜잭션들이 서로의 변경을 **얼마나 차단하고 방지** 정하는 단계입니다. 칸막이를 얼마나 높이 칠지 정하는 것과 같아서, 칸막이가 높을수록 서로 간섭이 줄지만 그만큼 동시 처리 속도는 떨어질 수 있습니다.

SQL 표준은 네 단계를 정의하며, 위로 갈수록 격리가 강해집니다.

격리 수준 4단계 비교

수준	더티 리드	반복 불가능 읽기	유령 읽기	비고
Read Uncommitted	(표준상 허용)	허용	허용	PostgreSQL에서는 Read Committed처럼 동작
Read Committed	막음	허용	허용	PostgreSQL의 기본값
Repeatable Read	막음	막음	막음 (PG)	트랜잭션 내내 같은 스냅샷을 봄
Serializable	막음	막음	막음	마치 한 번에 하나씩 실행한 것처럼 보장

표에서 알 수 있듯, PostgreSQL은 **Read Committed**를 기본값으로 씁니다. 이 수준에서는 각 SQL이 시작될 때 이미 확정된 값만 읽으므로, 적어도 남이 아직 확정하지 않은 임시 값(더티 리드)을 잘못 읽는 일은 없습니다. 대부분의 일반적인 CRUD 작업에는 이 기본값으로 충분합니다.

이렇게 설정합니다

격리 수준은 트랜잭션을 시작할 때 지정할 수 있습니다. 다음은 현재 격리 수준을 확인하고, 더 강한 수준으로 트랜잭션을 여는 예시입니다.

```
-- 현재 트랜잭션의 격리 수준 확인하기
BEGIN;
SHOW transaction_isolation;
COMMIT;
```

예상 화면:

```
shop_lab=# BEGIN;
BEGIN
shop_lab=# SHOW transaction_isolation;
transaction_isolation
-----
read committed
(1 row)
shop_lab=# COMMIT;
COMMIT
```

기본값이 `read committed`로 나옵니다. 더 강한 격리가 필요하면 `BEGIN` 시점에 수준을 명시합니다.

```
-- Repeatable Read로 트랜잭션 시작하기 (내내 같은 스냅샷을 봄)
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;

SELECT balance FROM accounts WHERE owner = '김민수';
-- 이 트랜잭션이 끝날 때까지, 다른 트랜잭션이 값을 바꿔 COMMIT해도
-- 여기서는 처음 본 값이 계속 동일하게 보입니다
SELECT balance FROM accounts WHERE owner = '김민수';

COMMIT;
```

예상 화면:

```
shop_lab=# BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN
shop_lab=# SELECT balance FROM accounts WHERE owner = '김민수';
  balance
-----
100000.00
(1 row)
shop_lab=# SELECT balance FROM accounts WHERE owner = '김민수';
  balance
-----
100000.00
(1 row)
shop_lab=# COMMIT;
COMMIT
```

REPEATABLE READ에서는 트랜잭션이 시작된 순간의 데이터 모습(스냅샷)을 끝까지 유지합니다. 그래서 두 번째 조회 사이에 다른 누군가가 잔액을 바꿔 확정하더라도, 이 트랜잭션 안에서는 처음 본 100000.00이 변함없이 보입니다.

팁: 입문 단계에서는 격리 수준을 직접 바꿀 일이 드뭅니다. 기본값 Read Committed로 시작하고, "조회한 값이 트랜잭션 도중 바뀌면 곤란한" 집계나 정산 작업처럼 일관된 스냅샷이 꼭 필요할 때만 Repeatable Read 이상을 고려합니다.

주의: 격리를 강하게 할수록 안전하지만, 트랜잭션끼리 충돌이 더 자주 감지됩니다. 특히 Serializable에서는 충돌이 발견되면 한쪽 트랜잭션이 직렬화 오류(serialization failure)로 거부될 수 있습니다. 이때 애플리케이션은 그 트랜잭션을 처음부터 다시 시도하도록 만들어야 합니다.

절 요약

- 동시성은 여러 트랜잭션이 같은 데이터에 동시에 접근하는 상황이며, 잘못 다루면 값이 흔들립니다.
- 격리 수준은 트랜잭션이 서로를 얼마나 차단할지 정하는 4단계이고, PostgreSQL 기본값은 Read Committed입니다.

- 강한 격리는 안전하지만 충돌과 재시도 부담이 늘어, 필요한 경우에만 올립니다.

SAVEPOINT로 부분 롤백하기

도입

ROLLBACK 은 강력하지만, BEGIN 이후 **모든** 변경을 통째로 되돌립니다. 그런데 긴 작업 묶음에서 "앞 단계는 멀쩡한데 마지막 한 단계만 잘못됐다"면, 전부 버리고 처음부터 다시 하는 것은 아깝습니다. 이 절에서는 트랜잭션 중간에 체크포인트를 꽂아 **일부만 되돌리는** SAVEPOINT 를 배웁니다.

핵심 개념 설명

세이브포인트 (SAVEPOINT)

세이브포인트(SAVEPOINT)란, 트랜잭션 도중에 이름을 붙여 저장해 두는 중간 지점입니다. 게임에서 중간 저장(체크포인트)을 해 두고, 실패하면 마지막 저장 지점으로만 돌아가는 것과 같습니다.

- **왜(why) 쓰나:** 긴 트랜잭션에서 일부 단계만 실패했을 때, 성공한 앞 단계를 살리면서 문제 구간만 되돌리기 위해서입니다.
- **언제(when) 쓰나:** 여러 단계로 이루어진 처리에서, 한 단계가 실패해도 나머지를 이어 가고 싶을 때입니다.
- **어떻게(how) 쓰나:** SAVEPOINT 이름 으로 체크포인트를 만들고, 문제가 생기면 ROLLBACK TO SAVEPOINT 이름 으로 그 지점까지만 되돌립니다.

부분 롤백 (partial rollback)

부분 롤백(partial rollback)이란, ROLLBACK TO SAVEPOINT 로 트랜잭션 전체가 아니라 **특정 세이브포인트 이후의 변경만** 되돌리는 것입니다. 되돌린 뒤에도 트랜잭션은 계속 열려 있어, 이어서 다른 작업을 하고 마지막에 COMMIT 할 수 있습니다.

여기서 주의할 점이 있습니다. ROLLBACK TO SAVEPOINT 는 트랜잭션을 끝내지 않습니다. 전체를 끝내려면 여전히 COMMIT 이나 ROLLBACK 이 필요합니다. 또한 세이브포인트로 되돌려도, 그 앞에서 확정 대기 중이던 변경은 그대로 살아 있습니다.

이렇게 작성합니다

상품 재고를 두 번 차감하는 작업을 한 트랜잭션으로 묶되, 두 번째 차감 전에 세이브포인트를 두는 예시입니다. 두 번째 차감이 잘못되면 그 부분만 되돌립니다.

```
-- 첫 차감은 살리고, 잘못된 두 번째 차감만 부분 롤백하기
BEGIN;

UPDATE accounts SET balance = balance - 10000 WHERE owner = '김민수';

SAVEPOINT after_first;  -- 여기까지를 체크포인트로 저장

UPDATE accounts SET balance = balance - 999999 WHERE owner = '김민수';
-- 두 번째 차감 금액이 잘못됨을 발견 -> 체크포인트로만 되돌림
ROLLBACK TO SAVEPOINT after_first;

COMMIT;  -- 첫 번째 차감만 확정됨
```

예상 화면:

```
shop_lab=# BEGIN;
BEGIN
shop_lab=# UPDATE accounts SET balance = balance - 10000 WHERE owner = '김민수';
UPDATE 1
shop_lab=# SAVEPOINT after_first;
SAVEPOINT
shop_lab=# UPDATE accounts SET balance = balance - 999999 WHERE owner = '김민수';
UPDATE 1
shop_lab=# ROLLBACK TO SAVEPOINT after_first;
ROLLBACK
shop_lab=# COMMIT;
COMMIT
shop_lab=# SELECT owner, balance FROM accounts WHERE owner = '김민수';
 owner | balance
-----+-----
 김민수 | 90000.00
(1 row)
```

처음 100,000원에서 첫 차감 10,000원만 반영되어 최종 잔액이 90000.00 이 되었습니다. 잘못된 두 번째 차감은 ROLLBACK TO SAVEPOINT 로 되돌려져 사라졌고, 트랜잭션은 그대로 이어져 COMMIT 으로 마무리되었습니다.

세이브포인트 관련 명령

명령	의미
SAVEPOINT 이름	현재 지점에 이름표를 붙여 체크포인트를 만듭니다
ROLLBACK TO SAVEPOINT 이름	그 체크포인트 이후 변경만 되돌립니다(트랜잭션은 계속)
RELEASE SAVEPOINT 이름	더 이상 필요 없는 체크포인트를 지웁니다
COMMIT / ROLLBACK	트랜잭션 전체를 확정하거나 전부 취소합니다

팁: 앞 절(위험 박스)에서 본 "오류로 트랜잭션 전체가 막히는" 상황도 세이브포인트로 완화할 수 있습니다. 실패할 가능성이 있는 SQL 앞에 세이브포인트를 두면, 그 SQL이 오류를 내도 ROLLBACK TO SAVEPOINT 로 직전 지점까지만 되돌려 트랜잭션을 살릴 수 있습니다.

주의: ROLLBACK TO SAVEPOINT 는 트랜잭션을 끝내지 않습니다. 되돌린 뒤에도 트랜잭션은 열린 채이므로, 반드시 마지막에 COMMIT 또는 ROLLBACK 으로 닫아야 변경이 확정되거나 정리됩니다.

절 요약

- SAVEPOINT 이름 으로 트랜잭션 중간에 체크포인트를 만듭니다.
- ROLLBACK TO SAVEPOINT 이름 은 그 지점 이후 변경만 되돌리고, 트랜잭션은 계속 진행됩니다.
- 부분 롤백 후에도 트랜잭션을 끝내려면 COMMIT 이나 ROLLBACK 이 필요합니다.

실습 과제

해보기: 주문 처리 트랜잭션 만들기

실습 데이터베이스 shop_lab 에서 재고 차감·주문 생성·결제 기록을 하나의 트랜잭션으로 묶어 봅니다.

- 단계 1: accounts (또는 products) 테이블을 준비하고 초기 잔액(또는 재고)을 확인합니다.
- 단계 2: BEGIN 으로 트랜잭션을 열고, 재고 차감 UPDATE 와 주문 생성 INSERT 를 차례로 실행합니다. 프롬프트가 ==*# 로 바뀌는지 확인합니다.
- 단계 3: 두 작업이 모두 정상이면 COMMIT 하고, 일부러 잘못된 값을 넣어 본 뒤에는 ROLLBACK 으로 전부 되돌려 봅니다.

- 단계 4: 결제 기록 INSERT 앞에 SAVEPOINT before_payment 를 두고, 결제만 실패했다고 가정해 ROLLBACK TO SAVEPOINT before_payment 로 결제 단계만 되돌린 뒤 나머지를 COMMIT 합니다.
- 점검: 각 단계 후 SELECT 로 잔액·재고·주문 행 수를 확인해, 의도한 변경만 남았는지 비교합니다. 특히 ROLLBACK 직후 값이 원래대로 돌아오는지 꼭 확인합니다.

FAQ

Q1. BEGIN 을 깜빡하고 SQL을 실행했는데 되돌릴 수 있나요? → A1. 어렵습니다. BEGIN 없이 실행한 SQL은 자동 커밋되어 즉시 확정되므로, ROLLBACK 으로 되돌릴 수 없습니다. 되돌릴 가능성이 있는 작업은 반드시 먼저 BEGIN 으로 묶은 뒤 실행하고, 확인이 끝난 뒤에 COMMIT 하는 습관을 들이는 것이 안전합니다.

Q2. 트랜잭션 도중에 오류가 나서 이후 명령이 전부 거부됩니다. 어떻게 해야 하나요? → A2. 그 트랜잭션은 오류 상태로 막힌 것입니다. ROLLBACK 을 입력해 트랜잭션을 닫고 처음부터 다시 시작하면 됩니다. 일부만 살리고 싶다면, 다음에는 위험한 SQL 앞에 SAVEPOINT 를 두고 ROLLBACK TO SAVEPOINT 로 그 구간만 되돌리는 방법을 씁니다.

Q3. COMMIT 과 END 는 같은 건가요? → A3. PostgreSQL에서 END 는 COMMIT 의 다른 이름으로, 둘 다 트랜잭션을 확정합니다. 다만 표준 SQL에서 널리 쓰이고 의미가 분명한 COMMIT 을 쓰는 편을 권합니다. 마찬가지로 BEGIN 은 START TRANSACTION 으로도 시작할 수 있습니다.

Q4. Read Committed면 충분한데 왜 더 강한 격리 수준이 있나요? → A4. 집계나 정산처럼 트랜잭션 도중 데이터가 바뀌면 결과가 어긋나는 작업이 있기 때문입니다. 이런 경우 Repeatable Read로 일관된 스냅샷을 보거나, 가장 엄격한 일관성이 필요하면 Serializable을 씁니다. 대신 강한 격리는 충돌 감지와 재시도 부담이 늘어, 일반 CRUD에는 기본값으로 충분합니다.

장 요약

이 장에서는 여러 SQL을 한 묶음으로 다루는 트랜잭션을 배웠습니다. 트랜잭션은 ACID(원자성·일관성·고립성·지속성)를 통해 "전부 성공하거나 전부 취소"를 보장합니다. BEGIN 으로 묶음을 시작하고 COMMIT 으로 확정하거나 ROLLBACK 으로 통째로 되돌렸으며, 동시에 여러 사용자가 데이터를 다룰 때 값이 흔들리지 않도록 격리 수준이 조절한다는 것을 보았습니다. 마지막으로

SAVEPOINT 와 ROLLBACK TO SAVEPOINT 로 긴 트랜잭션의 일부만 되돌리는 부분 롤백을 익혔습니다. 트랜잭션은 앞서 배운 INSERT·UPDATE·DELETE를 안전하게 묶어 주는 안전장치입니다.

핵심 키워드

트랜잭션(transaction), ACID, BEGIN, COMMIT, ROLLBACK, 동시성(concurrency), 격리 수준(isolation level), Read Committed, SAVEPOINT, 부분 롤백(partial rollback)

다음 장 미리보기

다음 장에서는 자주 쓰는 조회를 이름 붙여 재사용하는 뷰(view)와, 반복 처리를 묶어 두는 함수(function)를 배웁니다. 복잡한 CRUD를 더 간결하고 재사용 가능하게 다루는 방법을 익힙니다.

뷰·함수로 CRUD 재사용하기

학습 목표 - 뷰(view)로 복잡한 조회를 캡슐화하고 재사용할 수 있습니다. - 갱신 가능 뷰(updatable view)와 그 한계를 구분할 수 있습니다. - 함수(function)로 재사용 가능한 로직을 정의할 수 있습니다. - 트리거(trigger)로 CRUD 이벤트에 따른 자동화를 구성할 수 있습니다.

지금까지 우리는 `SELECT`, `INSERT`, `UPDATE`, `DELETE` 로 데이터를 직접 다루는 법을 익혔습니다. 그런데 실무에서는 똑같은 조회를 매번 길게 다시 적거나, 비슷한 처리를 여기저기서 반복하게 됩니다. 같은 일을 반복해서 적다 보면 실수가 끼어 들고, 규칙이 바뀌면 모든 곳을 고쳐야 합니다.

이 장에서는 자주 쓰는 SQL에 이름을 붙여 재사용하는 세 가지 도구를 배웁니다. 복잡한 조회에 이름을 붙이는 **뷰(view)**, 로직을 묶어 두는 **함수(function)**, 그리고 특정 사건이 생기면 자동으로 동작하는 **트리거(trigger)**입니다. 한 번 잘 만들어 두면 두고두고 짧고 안전하게 다시 쓸 수 있습니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 에서 `psql` 로 PostgreSQL에 접속한 상태를 기준으로 합니다. 화면 앞의 `shop_lab=#` 표시는 "지금 `shop_lab` 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 모두 아래 두 테이블을 기준으로 합니다. 앞 장들에서 다룬 회원·주문과 같은 모양이라고 생각하면 됩니다.

```
-- 이 장 예시가 기준으로 삼는 두 테이블 (참고용)
CREATE TABLE members (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        varchar(50) NOT NULL,
  email       varchar(100) NOT NULL,
  grade       varchar(10) DEFAULT 'BRONZE',
  updated_at  timestamp DEFAULT now()
);

CREATE TABLE orders (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  member_id   integer NOT NULL REFERENCES members(id),
  amount      numeric(10, 2) NOT NULL,
  status      varchar(20) DEFAULT 'PAID',
  ordered_at  timestamp DEFAULT now()
);
```

`members`에는 회원이, `orders`에는 그 회원이 낸 주문이 들어갑니다. `orders.member_id`는 `members.id`를 가리키는 외래 키(foreign key)로, "이 주문이 누구의 것인지"를 연결합니다.

뷰(view)로 복잡한 조회 재사용하기

도입

자주 찾는 가게의 검색 조건을 떠올려 봅시다. "우리 동네, 별점 4점 이상, 영업 중"을 매번 일일이 다시 고르는 대신, 한 번 즐겨찾기로 저장해 두면 다음부터는 한 번에 불러올 수 있습니다. **뷰(view)**가 SQL에서 하는 일이 바로 이것입니다. 이 절에서는 길고 복잡한 조회에 이름을 붙여 두고, 그 이름을 테이블처럼 간단히 조회하는 법을 배웁니다.

핵심 개념 설명

뷰 (view)

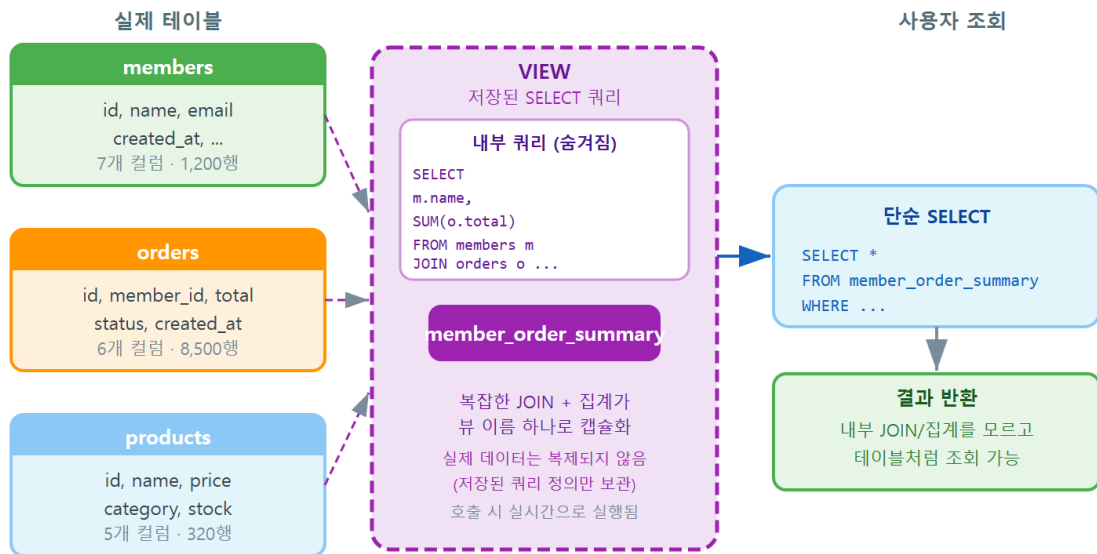
뷰(view)란, 이름을 붙여 저장한 `SELECT` 쿼리입니다. 실제 데이터를 따로 복제해 두는 것이 아니라, "이 이름을 조회하면 이 `SELECT`를 실행하라"는 약속만 저장합니다. 그래서 뷰를 조회하면 언제나 원본 테이블의 **현재 데이터**가 그대로 반영됩니다.

자주 쓰는 검색 조건을 즐겨찾기로 저장해 두고 한 번에 불러오는 것과 같습니다. 즐겨찾기 자체가 가게 정보를 복사해 가지고 있는 것이 아니라, "이 조건으로 다시 검색하라"는 바로 가기인 것과 똑같습니다.

- **왜(why) 필요한가:** 여러 테이블을 `JOIN`하고 집계하는 긴 조회를 매번 다시 적으면 길고, 틀리기 쉽고, 규칙이 바뀌면 모든 곳을 고쳐야 합니다. 뷰로 한 번 정의해 두면 그 정의 한 곳만 관리하면 됩니다.
- **언제(when) 쓰나:** 같은 조회를 여러 화면·여러 사람이 반복해서 쓸 때, 복잡한 집계 결과를 단순한 테이블처럼 보여 주고 싶을 때 유용합니다.
- **어떻게(how) 쓰나:** `CREATE VIEW 뷰이름 AS SELECT ...` 형태로 만들고, 이후에는 `SELECT * FROM 뷰이름`처럼 평범한 테이블을 조회하듯 사용합니다.

여기서 입문자가 자주 하는 오해 하나를 짚겠습니다. 뷰가 데이터를 따로 복사해 저장한다고 생각하는 경우입니다. 일반 뷰는 데이터를 **복사하지 않습니다**. 저장하는 것은 쿼리(질의문)뿐이고, 실제 값은 조회하는 순간 원본 테이블에서 가져옵니다. 그래서 원본이 바뀌면 뷰의 결과도 즉시 함께 바뀝니다.

뷰(VIEW) 캡슐화 — 복잡한 쿼리를 단순하게



뷰를 사용하면 복잡한 JOIN·집계를 매번 작성하지 않고, 간단한 테이블 조회처럼 재사용할 수 있습니다

ch_11_s01 · 뷰(VIEW)로 복잡한 조회 재사용하기

이렇게 작성합니다

먼저 뷰가 없을 때를 생각해 봅시다. "회원별로 주문 건수와 총 결제 금액을 보고 싶다"면 매번 다음과 같은 조회를 적어야 합니다.

```
-- 뷰 없이 매번 적어야 하는 회원별 주문 요약 조회
SELECT m.id,
       m.name,
       count(o.id)          AS order_count,
       coalesce(sum(o.amount), 0) AS total_amount
FROM members m
LEFT JOIN orders o ON o.member_id = m.id
GROUP BY m.id, m.name
ORDER BY m.id;
```

이 조회를 화면마다 다시 적는 대신, 뷰로 이름을 붙여 둡니다.

```
-- 회원별 주문 요약 조회에 이름을 붙여 뷰로 저장하기
CREATE VIEW member_order_summary AS
SELECT m.id,
       m.name,
       count(o.id)          AS order_count,
       coalesce(sum(o.amount), 0) AS total_amount
FROM members m
LEFT JOIN orders o ON o.member_id = m.id
GROUP BY m.id, m.name;
```

예상 화면:

```
shop_lab=# CREATE VIEW member_order_summary AS
shop_lab=# SELECT m.id, m.name, count(o.id) AS order_count,
shop_lab=#         coalesce(sum(o.amount), 0) AS total_amount
shop_lab=# FROM members m
shop_lab=# LEFT JOIN orders o ON o.member_id = m.id
shop_lab=# GROUP BY m.id, m.name;
shop_lab=# CREATE VIEW
```

CREATE VIEW 라는 응답은 뷰가 잘 만들어졌다는 뜻입니다. 이제부터는 긴 조회 대신 뷰 이름만으로 같은 결과를 얻습니다.

```
-- 뷰를 평범한 테이블처럼 조회하기
SELECT * FROM member_order_summary
ORDER BY total_amount DESC;
```

예상 화면:

```
id | name | order_count | total_amount
---+---+-----+-----
 2 | 이서연 |          3 |   142000.00
 1 | 김민수 |          2 |    53000.00
 3 | 박지훈 |          0 |         0.00
(3 rows)
```

긴 JOIN 과 집계를 매번 적지 않고도, 짧은 한 줄로 같은 결과를 얻었습니다. 게다가 뷰에는 WHERE 나 ORDER BY 를 평소처럼 덧붙일 수 있어, 일반 테이블과 거의 똑같이 다룰 수 있습니다.

구문 요소 설명

구문	의미
CREATE VIEW member_order_summary AS	이 이름으로 뒤의 SELECT 를 저장합니다
LEFT JOIN orders o	주문이 없는 회원도 결과에 포함시킵니다
count(o.id) AS order_count	회원별 주문 건수에 별칭을 붙입니다
coalesce(sum(...), 0)	주문이 없어 합계가 NULL이면 0으로 바꿉니다
SELECT * FROM member_order_summary	저장해 둔 조회를 테이블처럼 실행합니다

팁: 뷰 정의를 바꾸고 싶을 때는 `CREATE OR REPLACE VIEW` 뷰이름 `AS ...` 를 씁니다. 이미 있는 뷰를 지웠다 다시 만들지 않고 정의만 같아 끼웁니다. 단, 기존 컬럼의 이름·순서·자료형은 그대로 두고 **뒤에 컬럼을 추가**하는 식의 변경만 허용됩니다.

주의: 뷰는 조회할 때마다 안에 저장된 `SELECT` 를 **그때그때 다시 실행**합니다. 따라서 매우 무거운 집계를 담은 뷰를 자주 조회하면 그만큼 비용이 듭니다. 결과를 실제로 저장해 두고 빠르게 읽고 싶다면 구체화 뷰(materialized view)라는 별도 기능을 쓰지만, 그쪽은 데이터가 자동으로 최신화되지 않으므로 입문 단계에서는 일반 뷰부터 익히는 것을 권합니다.

절 요약

- 뷰는 이름을 붙여 저장한 `SELECT` 이며, 데이터를 복사하지 않고 조회 시 원본을 그대로 반영합니다.
- `CREATE VIEW 이름 AS SELECT ...` 로 만들고, `SELECT * FROM 이름` 으로 테이블처럼 조회합니다.
- 복잡한 `JOIN` 집계를 한 곳에 정의해 두면 재사용이 쉽고 관리가 편합니다.

뷰 갱신과 제약 이해하기

도입

뷰를 테이블처럼 조회할 수 있다면, "그럼 뷰에 `UPDATE` 나 `INSERT` 도 할 수 있을까?"라는 질문이 자연스럽게 떠오릅니다. 답은 "어떤 뷰냐에 따라 다릅니다"입니다. 이 절에서는 뷰를 통한 수정이 되는 경우와 안 되는 경우를 구분하고, 안 되는 이유를 이해합니다.

핵심 개념 설명

갱신 가능 뷰 (updatable view)

갱신 가능 뷰(updatable view)란, 그 뷰를 통해 `INSERT` · `UPDATE` · `DELETE` 를 하면 원본 테이블의 데이터가 실제로 바뀌는 뷰를 말합니다. PostgreSQL은 뷰가 충분히 **단순할 때만** 자동으로 갱신을 허용합니다.

- **언제(when) 갱신이 되나:** 뷰가 **하나의 테이블만** 대상으로 하고, `GROUP BY` · 집계 함수 · `DISTINCT` · `JOIN` 같은 복잡한 가공이 없을 때입니다. 즉, 단순히 일부 컬럼이나 일부 행만 골라 보여 주는 뷰입니다.

- **왜(why) 그런 제한이 있나:** 뷰를 통한 수정은 결국 "이 뷰의 한 행이 원본 테이블의 어느 한 행에 해당하는가"가 분명해야 가능합니다. 단순 뷰는 이 대응이 1:1로 명확하지만, 집계나 조인이 들어가면 대응이 모호해집니다.

뷰의 한계 (view limitation)

뷰의 한계(view limitation)란, 집계·조인·중복 제거가 들어간 뷰는 그대로는 갱신할 수 없다는 제약을 말합니다. 모든 뷰를 통해 자유롭게 수정할 수 있다고 오해하기 쉽지만, 그렇지 않습니다.

예를 들어 앞 절의 `member_order_summary` 뷰에는 `total_amount` 라는 컬럼이 있습니다. 이 값은 여러 주문 금액을 `sum` 으로 더한 **합계**입니다. 만약 누군가 이 합계를 `150000` 으로 바꾸려 한다면, 데이터베이스는 난감해집니다. 합계가 150000이 되려면 원본 주문들의 금액을 각각 얼마로 바꿔야 할지 정답이 하나로 정해지지 않기 때문입니다. 그래서 이런 뷰는 갱신이 거부됩니다.

이렇게 작성합니다

먼저 갱신이 되는 단순 뷰를 봅시다. `members` 에서 골드 등급 회원만 골라 보여 주는 뷰입니다.

```
-- 단일 테이블에서 일부 행만 고른 단순 뷰 (갱신 가능)
CREATE VIEW gold_members AS
SELECT id, name, email, grade
FROM members
WHERE grade = 'GOLD';
```

이 뷰는 한 테이블만 대상으로 하고 집계가 없으므로, 뷰를 통한 `UPDATE` 가 원본 `members` 에 그대로 반영됩니다.

```
-- 단순 뷰를 통한 UPDATE는 원본 테이블에 반영됩니다
UPDATE gold_members
SET email = 'vip@example.com'
WHERE id = 2;
```

예상 화면:

```
UPDATE 1
```

`UPDATE 1` 은 한 행이 실제로 수정되었다는 뜻입니다. `members` 테이블을 직접 조회해 보면 `id` 가 2인 회원의 이메일이 바뀐 것을 확인할 수 있습니다.

반면, 집계가 들어간 뷰를 갱신하려 하면 거부됩니다.

```
-- 집계 뷰를 통한 UPDATE는 거부됩니다
UPDATE member_order_summary
SET total_amount = 150000
WHERE id = 1;
```

예상 화면:

```
ERROR: cannot update view "member_order_summary"
DETAIL: Views containing GROUP BY are not automatically updatable.
HINT: To enable updating the view, provide an INSTEAD OF UPDATE
      trigger or an unconditional ON UPDATE DO INSTEAD rule.
```

오류 메시지가 "GROUP BY 가 들어간 뷰는 자동으로 갱신되지 않는다"고 분명히 알려 줍니다. 즉, 이것은 실수가 아니라 데이터베이스가 모호함을 막기 위한 의도된 동작입니다.

갱신 가능 여부 판단 기준

뷰의 형태	갱신 가능?	이유
단일 테이블 + 일부 컬럼/행 선택	가능	원본 행과 1:1로 대응됩니다
JOIN 이 포함된 뷰	보통 불가	어느 테이블을 바꿀지 모호합니다
GROUP BY · 집계 함수	불가	한 값이 여러 원본 행을 합친 것입니다
DISTINCT 포함	불가	어떤 원본 행인지 특정할 수 없습니다

팁: 복잡한 뷰도 꼭 갱신해야 한다면 INSTEAD OF 트리거를 달아 "뷰를 수정하면 실제로는 이렇게 원본을 바꿔라"는 규칙을 직접 정의할 수 있습니다. 다만 이는 한 단계 더 높은 주제이므로, 입문 단계에서는 "복잡한 뷰는 조회 전용으로 쓰고, 수정은 원본 테이블에 직접 한다"는 원칙만 기억해도 충분합니다.

위험: 갱신 가능 뷰라도 WHERE 조건을 가진 뷰를 통해 INSERT 하면, 조건에 맞지 않는 행이 들어가 뷰에서는 보이지 않는데 원본에는 존재하는 행이 생길 수 있습니다. 이를 막으려면 뷰 정의 끝에 WITH CHECK OPTION 을 붙여, 뷰 조건을 벗어나는 삽입·수정을 거부하게 합니다.

절 요약

- 단일 테이블 기반의 단순 뷰는 갱신이 가능해 원본에 반영됩니다.
- JOIN · GROUP BY · 집계 · DISTINCT 가 든 뷰는 대응이 모호해 자동 갱신이 거부됩니다.
- 복잡한 뷰는 조회 전용으로 쓰고, 수정은 원본 테이블에 직접 하는 것이 안전합니다.

함수(function)로 로직 묶기

도입

같은 계산이나 처리를 여러 곳에서 반복한다면, 그 처리에 이름을 붙여 한 번 정의해 두고 싶어 집니다. 계산기에 자주 쓰는 공식을 단축 버튼으로 저장해 두는 것과 같습니다. SQL에서는 이런 일을 **함수(function)** 가 말합니다. 이 절에서는 매개변수를 받아 값을 돌려주는 함수를 만들고 호출하는 법을 배웁니다.

핵심 개념 설명

SQL 함수 (SQL function)

SQL 함수(SQL function) 란, 입력값(매개변수)을 받아 정해진 처리를 한 뒤 결과를 돌려주도록 이름 붙여 저장한 로직입니다. 우리가 이미 써 온 `count()`, `sum()`, `now()` 같은 것도 PostgreSQL 이 미리 만들어 둔 함수이며, `CREATE FUNCTION` 으로 우리만의 함수를 직접 만들 수 있습니다.

- **왜(why) 필요한가:** 같은 계산·조회를 여러 곳에서 반복하면 로직이 흩어집니다. 함수로 묶어 두면 규칙이 바뀌어도 함수 한 곳만 고치면 됩니다.
- **언제(when) 쓰나:** "회원 등급에 따른 할인을 계산", "특정 회원의 총 주문액 구하기"처럼 입력에 따라 결과가 정해지는 처리를 여러 번 쓸 때 유용합니다.
- **어떻게(how) 쓰나:** `CREATE FUNCTION 이름(매개변수) RETURNS 반환형 AS $$ ... $$ LANGUAGE ...` 형태로 정의하고, `SELECT 함수이름(인자)` 로 호출합니다.

재사용 로직 (reusable logic)

재사용 로직(reusable logic) 이란, 한 번 정의해 여러 곳에서 불러 쓰는 처리 단위를 말합니다. 함수의 핵심 가치가 바로 이것입니다. 로직을 함수 안에 담아 두면, 부르는 쪽은 "어떻게 계산하는지" 몰라도 이름과 입력값만 알면 결과를 얻습니다. 이를 **캡슐화(encapsulation)** 라고 합니다.

이렇게 작성합니다

다음은 특정 회원의 총 주문 금액을 돌려주는 함수입니다. `member_id` 를 입력받아, 그 회원의 주문 금액 합계를 반환합니다.

```
-- 회원 id를 받아 그 회원의 총 주문 금액을 돌려주는 함수
CREATE FUNCTION total_spent(p_member_id integer)
RETURNS numeric AS $$
    SELECT coalesce(sum(amount), 0)
    FROM orders
    WHERE member_id = p_member_id;
$$ LANGUAGE sql;
```

예상 화면:

```
CREATE FUNCTION
```

이제 이 함수를 일반 함수처럼 호출합니다. 긴 집계 조회 대신 이름과 회원 번호만 넘기면 됩니다.

```
-- 정의한 함수 호출하기
SELECT total_spent(2) AS 이서연_총주문액;
```

예상 화면:

```
이서연_총주문액
-----
          142000.00
(1 row)
```

매번 `SELECT sum(amount) FROM orders WHERE ...` 를 적는 대신, `total_spent(2)` 라는 짧은 표현으로 같은 결과를 얻었습니다. 함수는 `SELECT` 의 컬럼 자리에서도 쓸 수 있어, 다른 조회와 자연스럽게 어울립니다.

```
-- 조회 결과의 각 행에 함수를 적용하기
SELECT id, name, total_spent(id) AS total_amount
FROM members
ORDER BY total_amount DESC;
```

예상 화면:

```
id | name | total_amount
---+-----+-----
  2 | 이서연 |      142000.00
  1 | 김민수 |       53000.00
  3 | 박지훈 |           0.00
(3 rows)
```

CREATE FUNCTION 함수이름(매개변수 자료형) RETURNS 반환형 AS \$\$ 본문 \$\$ LANGUAGE 언어;

구문 요소 설명

구문	의미
<code>total_spent(p_member_id integer)</code>	함수 이름과, 정수형 매개변수 하나를 받습니다
<code>RETURNS numeric</code>	이 함수가 숫자(합계)를 돌려준다고 선언합니다
<code>\$\$... \$\$</code>	함수 본문을 감싸는 구분 기호(달러 인용)입니다
<code>coalesce(sum(amount), 0)</code>	주문이 없으면 NULL 대신 0을 돌려줍니다
<code>LANGUAGE sql</code>	본문을 SQL로 작성했음을 알립니다

팁: 함수 본문을 작은따옴표 대신 `$$` 로 감싸는 이유는, 본문 안에 작은따옴표가 들어가도 충돌하지 않게 하기 위해서입니다. 이름을 붙인 `$func$ ... $func$` 처럼 써도 됩니다. 입문 단계에서는 `$$ ... $$` 만 일관되게 써도 충분합니다.

주의: CREATE FUNCTION 은 같은 이름·같은 매개변수의 함수가 이미 있으면 오류를 냅니다. 정의를 바꿔 다시 만들고 싶다면 CREATE OR REPLACE FUNCTION 을 씁니다. 다만 반환형을 바꾸는 변경은 기존 함수를 먼저 DROP FUNCTION 으로 지운 뒤에야 가능합니다.

절 요약

- 함수는 매개변수를 받아 처리한 결과를 돌려주는, 이름 붙인 재사용 로직입니다.
- CREATE FUNCTION ... RETURNS ... AS \$\$... \$\$ LANGUAGE sql 로 정의하고 SELECT 함수(인자) 로 호출합니다.
- 반복되는 계산·조회를 함수로 묶으면 한 곳만 고쳐 전체에 반영할 수 있습니다.

트리거(trigger)로 CRUD 자동화 맛보기

도입

문이 열리면 자동으로 켜지는 센서 등을 떠올려 봅시다. 사람이 스위치를 누르지 않아도, "문이 열린다"는 사건이 생기면 알아서 동작합니다. 데이터베이스에도 이런 장치가 있습니다. 테이블에 INSERT · UPDATE · DELETE 같은 사건이 일어나면 **자동으로** 지정한 처리를 실행하는 **트리거(trigger)** 입니다. 이 절에서는 행이 수정될 때 갱신 시각을 자동으로 기록하는 트리거를 만들어 봅시다.

핵심 개념 설명

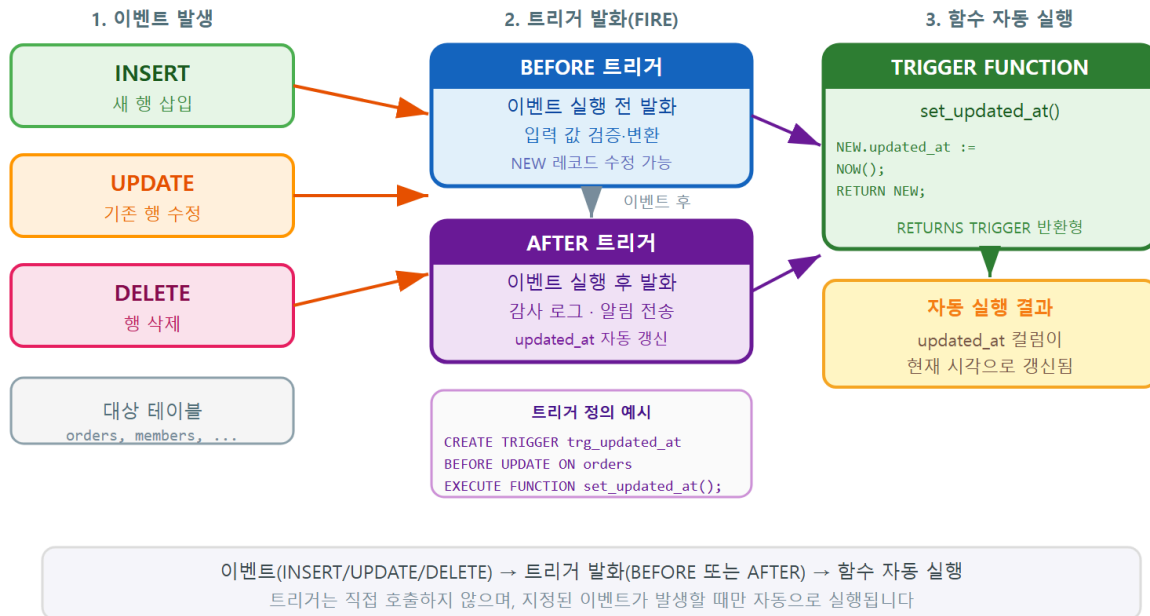
트리거(trigger)

트리거(trigger)란, 테이블에 특정 이벤트(INSERT·UPDATE·DELETE)가 일어날 때 자동으로 지정한 함수를 실행하게 하는 장치입니다. 사람이 매번 챙기지 않아도, 정해진 사건이 생기면 데이터베이스가 알아서 함수를 발화(fire)시킵니다.

- **왜(why) 필요한가:** "행이 수정될 때마다 갱신 시각을 기록한다"처럼 빠뜨리면 안 되는 처리를, 애플리케이션 코드가 매번 챙기는 대신 데이터베이스가 보장하게 할 수 있습니다.
- **언제(when) 쓰나:** 갱신 시각 자동 기록, 변경 이력 남기기, 입력값 자동 보정처럼 CRUD 사건에 딸려 항상 함께 일어나야 하는 처리에 씁니다.
- **어떻게(how) 쓰나:** 먼저 실행할 **트리거 함수**를 만들고, 그다음 CREATE TRIGGER 로 "어느 테이블의 어떤 사건에, 언제(BEFORE·AFTER) 그 함수를 실행할지" 연결합니다.

여기서 자주 하는 오해 두 가지를 짚겠습니다. 첫째, 트리거를 직접 호출해 실행한다고 생각하는 경우입니다. 트리거는 직접 부르는 것이 아니라 **사건이 생기면 자동으로** 동작합니다. 둘째, 트리거가 많을수록 좋다고 여기는 경우입니다. 트리거는 눈에 보이지 않게 동작하므로, 너무 많으면 "왜 이 값이 바뀌었는지" 추적하기 어려워집니다.

트리거(TRIGGER) 발화 흐름 — 이벤트 자동화



ch_11_s04 · 트리거(TRIGGER)로 CRUD 자동화 맞보기

이렇게 작성합니다

트리거를 만드는 일은 두 단계입니다. 먼저, 실행될 **트리거 함수**를 만듭니다. 이 함수는 수정되는 행의 `updated_at` 을 현재 시각으로 채워 돌려줍니다.

```
-- 1단계: 수정되는 행의 updated_at을 현재 시각으로 채우는 트리거 함수
CREATE FUNCTION set_updated_at()
RETURNS trigger AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

예상 화면:

```
CREATE FUNCTION
```

다음으로, 이 함수를 `members` 테이블의 `UPDATE` 사건에 연결합니다. 행이 수정되기 **직전 (BEFORE)** 에 함수를 실행하도록 지정합니다.

```
-- 2단계: members가 UPDATE될 때마다 위 함수를 자동 실행하도록 연결
CREATE TRIGGER trg_members_updated
BEFORE UPDATE ON members
FOR EACH ROW
EXECUTE FUNCTION set_updated_at();
```

예상 화면:

```
CREATE TRIGGER
```

이제 `updated_at` 을 직접 건드리지 않아도, 회원 정보를 수정하면 갱신 시각이 자동으로 최신화 됩니다.

```
-- updated_at은 손대지 않고 등급만 수정합니다
UPDATE members
SET grade = 'GOLD'
WHERE id = 1;

-- 결과 확인: updated_at이 자동으로 바뀌었습니다
SELECT id, name, grade, updated_at FROM members WHERE id = 1;
```

예상 화면:

```
UPDATE 1
 id | name  | grade | updated_at
-----+-----+-----+-----
  1 | 김민수 | GOLD  | 2026-06-16 15:22:09.481027
(1 row)
```

SET 절에는 `grade` 만 적었는데도 `updated_at` 이 현재 시각으로 바뀌었습니다. 트리거가 수정 직전에 자동으로 `updated_at` 을 채워 넣었기 때문입니다.

CREATE TRIGGER 이름 { BEFORE | AFTER } { INSERT | UPDATE | DELETE } ON 테이블 FOR EACH ROW EXECUTE FUNCTION 함수();

구문 요소 설명

구문	의미
<code>RETURNS trigger</code>	트리거 전용 함수임을 나타냅니다
<code>NEW</code>	새로 들어오거나 수정될 행을 가리키는 특수 변수입니다
<code>NEW.updated_at := now()</code>	그 행의 갱신 시각을 현재 시각으로 채웁니다
<code>LANGUAGE plpgsql</code>	절차적 로직을 쓰기 위한 PL/pgSQL 언어입니다
<code>BEFORE UPDATE ON members</code>	<code>members</code> 가 수정되기 직전에 발화합니다
<code>FOR EACH ROW</code>	영향받는 행마다 한 번씩 함수를 실행합니다

팁: BEFORE 는 사건이 일어나기 **직전**, AFTER 는 **직후**입니다. `updated_at` 처럼 저장될 값을 바꿔야 할 때는 BEFORE 를 씁니다. 저장이 끝난 뒤 다른 테이블에 이력을 남기는 처리는 AFTER 가 적합합니다. INSERT 에도 같은 트리거를 걸면, 새로 만들어질 때와 수정될 때 모두 시각이 채워집니다.

주의: 트리거는 눈에 띄지 않게 자동으로 동작하므로, 동작을 잊으면 "분명히 `updated_at` 을 건드리지 않았는데 값이 바뀌었다"며 혼란스러울 수 있습니다. 트리거에는 이름과 용도를 분명히 적고, 어떤 트리거가 걸려 있는지 `\d members` (테이블 구조 보기)로 수시로 확인하는 습관을 들입니다.

절 요약

- 트리거는 `INSERT · UPDATE · DELETE` 사건이 일어날 때 자동으로 지정한 함수를 실행합니다.
- 트리거 함수(`RETURNS trigger`)를 먼저 만들고, `CREATE TRIGGER` 로 테이블·사건·시점에 연결합니다.
- BEFORE 는 저장될 값을 바꿀 때, AFTER 는 저장 후 부수 처리에 적합합니다.

실습 과제

해보기: 주문 요약 뷰와 갱신 시각 트리거 만들기

실습 데이터베이스 `shop_lab` 에서 이 장의 뷰·함수·트리거를 직접 만들어 봅니다.

- 단계 1: 회원별 주문 건수와 총 결제 금액을 보여 주는 뷰 `member_order_summary` 를 만들고, `SELECT * FROM member_order_summary` 로 단순 조회가 되는지 확인합니다.
- 단계 2: 이 뷰에 `UPDATE` 를 시도해 보고, 집계 뷰가 갱신되지 않는다는 오류 메시지를 직접 확인합니다.
- 단계 3: 회원 한 명의 총 주문액을 돌려주는 함수 `total_spent(member_id)` 를 만들고, `SELECT name, total_spent(id) FROM members` 로 모든 회원에 적용해 봅니다.
- 단계 4: `set_updated_at()` 트리거 함수와 `trg_members_updated` 트리거를 만든 뒤, 회원의 `grade` 만 수정하고 `updated_at` 이 자동으로 바뀌는지 확인합니다.
- 점검: `\d members` 로 트리거가 걸려 있는지 확인하고, `UPDATE` 전후의 `updated_at` 값을 비교해 자동 갱신이 동작했는지 검증합니다.

FAQ

Q1. 뷰와 테이블은 무엇이 다른가요? → A1. 테이블은 실제 데이터를 저장하는 공간이고, 뷰는 데이터를 저장하지 않고 "이렇게 조회하라"는 `SELECT` 만 저장합니다. 그래서 뷰를 조회하면 그 순간 원본 테이블의 최신 데이터가 그대로 나옵니다. 디스크 공간을 거의 차지하지 않으며, 원본이 바뀌면 뷰 결과도 함께 바뀝니다.

Q2. 함수와 뷰는 언제 나눠 쓰나요? → A2. 입력값에 따라 결과가 달라지는 처리(예: 회원 번호로 총액 계산)는 매개변수를 받는 **함수**가 알맞습니다. 반면 고정된 조회 결과를 테이블처럼 재사용하고 싶을 때는 **뷰**가 알맞습니다. 즉, "입력을 받는가"가 둘을 나누는 기준입니다.

Q3. 트리거를 잘못 만들면 데이터가 망가지지 않나요? → A3. 트리거는 강력한 만큼 신중히 다뤄야 합니다. 잘못 만든 트리거가 `INSERT` 마다 엉뚱한 값을 넣을 수 있으므로, 운영 데이터에 걸기 전에 실습 데이터베이스에서 충분히 시험합니다. 더 이상 필요 없는 트리거는 `DROP TRIGGER` 이름 `ON` 테이블; 로 제거할 수 있습니다.

Q4. 뷰·함수·트리거를 지우려면 어떻게 하나요? → A4. 각각 `DROP VIEW` 이름; , `DROP FUNCTION` 이름(매개변수형); , `DROP TRIGGER` 이름 `ON` 테이블; 로 지웁니다. 다른 객체가 그 대상에 의존하고 있으면 오류가 날 수 있는데, 의존 객체까지 함께 지우려면 끝에 `CASCADE` 를 붙입니다(영향 범위를 반드시 확인한 뒤 사용합니다).

장 요약

이 장에서는 자주 쓰는 SQL에 이름을 붙여 재사용하는 세 가지 도구를 배웠습니다. **뷰(view)**로 복잡한 JOIN 집계 조회를 캡슐화해 테이블처럼 간단히 조회했고, 단순 뷰는 갱신이 되지만 집계·조인 뷰는 대응이 모호해 갱신이 거부된다는 **한계**를 이해했습니다. **함수(function)**로 매개변수를 받아 처리하는 재사용 로직을 정의했고, **트리거(trigger)**로 UPDATE 사건에 맞춰 갱신 시각을 자동으로 기록하는 자동화를 구성했습니다. 이 도구들은 반복을 줄이고 규칙을 한곳에 모아 주어, 데이터베이스를 더 안전하고 깔끔하게 관리하도록 돕습니다.

핵심 키워드

뷰(view), CREATE VIEW, 갱신 가능 뷰(updatable view), CREATE FUNCTION, 재사용 로직(reusable logic), 트리거(trigger), CREATE TRIGGER, BEFORE/AFTER

다음 장 미리보기

다음 장에서는 지금까지 배운 CRUD와 재사용 도구를 하나의 작은 프로젝트로 엮어, 실제 서비스처럼 데이터를 설계하고 다루는 흐름을 정리합니다.

실무 CRUD 패턴과 마무리

학습 목표 - OFFSET과 keyset 페이지네이션의 차이와 적합한 상황을 설명할 수 있습니다. - 감사 컬럼과 소프트 삭제로 이력을 남기고 안전하게 데이터를 숨길 수 있습니다. - 백업과 권한 분리 등 안전한 운영 습관을 적용할 수 있습니다. - CRUD 전체 흐름을 복습하고 다음 학습 방향을 정할 수 있습니다.

여기까지 우리는 데이터를 만들고(INSERT), 읽고(SELECT · JOIN), 고치고(UPDATE), 지우는(DELETE) 네 가지 기본 동작과 그것을 지켜 주는 제약조건.인덱스.트랜잭션.뷰까지 배웠습니다. 도구는 모두 손에 쥘 셈입니다.

이 마지막 장에서는 그 도구들을 **실제 서비스에서 쓰는 방식**으로 묶습니다. 목록 화면을 끊어 보여 주는 페이지네이션, 누가 언제 만들고 고쳤는지 남기는 감사 컬럼과 안전하게 숨기는 소프트 삭제, 그리고 사고를 막는 백업과 권한 분리를 차례로 다룹니다. 마지막 절에서는 책 전체의 CRUD 여정을 한 흐름으로 되짚고 다음 학습 방향을 안내합니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 에서 `psql` 로 PostgreSQL에 접속한 상태를 기준으로 합니다. PowerShell은 글자로 명령을 입력하는 검은 화면(터미널)이고, `psql` 은 그 안에서 데이터베이스와 대화하는 도구입니다. 화면 앞의 `shop_lab=#` 표시는 "지금 `shop_lab` 데이터베이스에 접속해 SQL을 기다리는 중"이라는 안내입니다.

이 장의 예시는 앞 장들에서 만든 `products` (상품)와 `orders` (주문) 테이블을 기준으로 합니다. 페이지네이션.소프트 삭제 설명을 위해 다음과 같은 모양이라고 생각하면 됩니다.

```
-- 이 장 예시가 기준으로 삼는 테이블 (참고용)
CREATE TABLE products (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        varchar(100) NOT NULL,
  category    varchar(30),
  price       numeric(10, 2) NOT NULL,
  created_at  timestamp DEFAULT now(),
  updated_at  timestamp DEFAULT now(),
  deleted_at  timestamp      -- 소프트 삭제 표시용 (NULL이면 살아 있는 행)
);
```

`created_at` · `updated_at` · `deleted_at` 세 컬럼이 이 장의 주인공입니다. 각각이 왜 필요한지는 절을 진행하며 하나씩 풀어 가겠습니다.

페이지네이션(pagination) 패턴

도입

상품이 10건이면 한 화면에 다 보여 줘도 됩니다. 하지만 10만 건이라면 한 번에 다 보내는 순간 화면도 서버도 멈춥니다. 그래서 긴 목록은 1페이지, 2페이지처럼 **일정 개수씩 끊어** 보여 줍니다. 이 절에서는 가장 단순한 `OFFSET` 방식과, 데이터가 많아도 빠른 `keyset` 방식을 비교하며 익힙니다.

핵심 개념 설명

페이지네이션 (pagination)

페이지네이션(pagination)이란, 큰 결과 집합을 일정 개수씩 여러 페이지로 나누어 조회하는 기법입니다. 긴 검색 결과를 1페이지, 2페이지로 끊어 보여 주는 것과 같습니다.

- **왜(why) 필요한가:** 수만 건을 한 번에 내려보내면 네트워크·메모리·화면이 모두 버겁습니다. 사용자도 한 화면에 20건 정도만 보면 충분합니다. 필요한 만큼만 잘라 보내는 것이 빠르고 가볍습니다.
- **언제(when) 쓰나:** 상품 목록, 게시판, 검색 결과처럼 "전체는 많지만 한 화면에는 일부만" 보여 주는 모든 화면입니다.
- **어떻게(how) 쓰나:** 4장에서 배운 `LIMIT` (가져올 개수)과 `OFFSET` (건너뛴 개수), 그리고 `ORDER BY` (정렬 기준)를 조합합니다.

여기서 입문자가 자주 하는 오해 하나를 먼저 짚겠습니다. **정렬(`ORDER BY`) 없이도 페이지가 일관되게 나뉜다고 생각하는 경우**입니다. 정렬 기준이 없으면 PostgreSQL은 행 순서를 보장하지 않으므로, 같은 쿼리를 두 번 실행해도 1페이지에 나오던 행이 2페이지로 밀릴 수 있습니다. **페이지네이션에는 반드시 안정적인 `ORDER BY`가 필요합니다.**

키셋 페이지네이션 (keyset pagination)

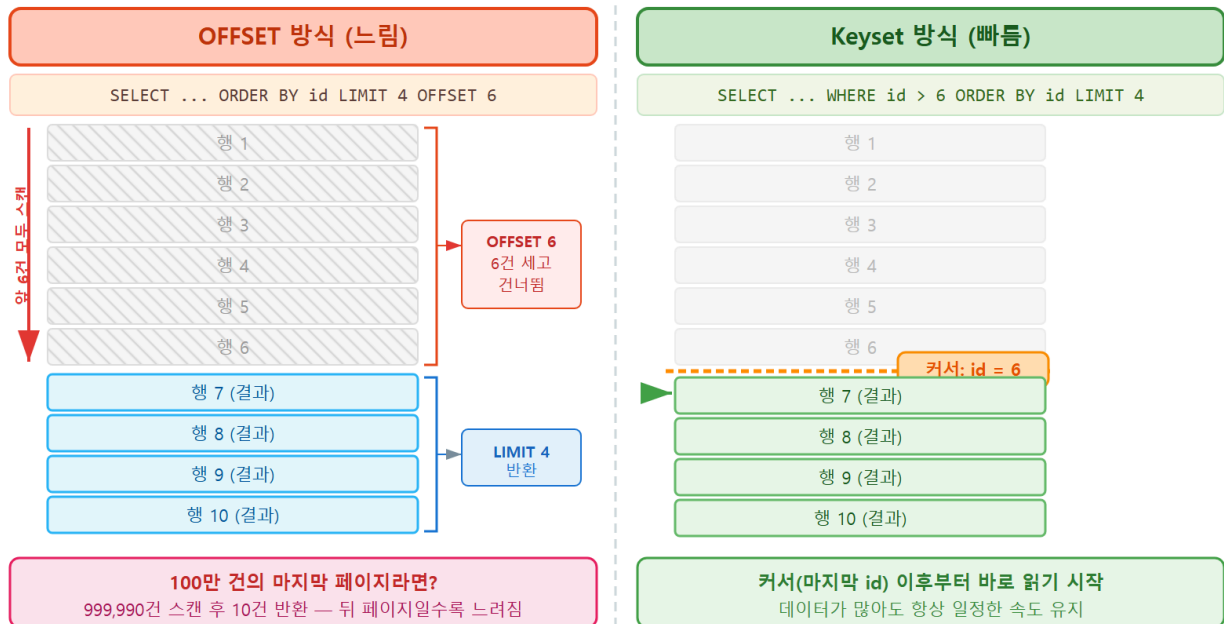
키셋 페이지네이션(keyset pagination)이란, "몇 개를 건너뛴지" 대신 "**마지막으로 본 행의 키(예: id) 다음부터**" 읽어 오는 방식입니다. 책갈피를 꽂아 두고 다음에 그 자리부터 펴 읽는 것과 같습니다.

`OFFSET` 방식은 100페이지를 보려면 앞의 99페이지치 행을 매번 세어 건너뛹니다. 데이터가 많을수록, 뒤 페이지일수록 느려집니다. 반면 `keyset` 방식은 "id가 직전 페이지 마지막 값보다 큰 것 중 앞에서 20개"만 읽으므로, 몇 페이지째든 속도가 거의 일정합니다.

여기서 또 하나 흔한 오해는 **OFFSET 방식이 데이터가 많아도 항상 빠르다고 생각하는 것**입니다. 작은 데이터에서는 차이가 없지만, 수십만 건에서 뒤쪽 페이지를 조회하면 OFFSET은 눈에 띄게 느려집니다.

OFFSET vs Keyset 페이지네이션 비교

대용량 데이터에서 성능 차이가 발생하는 원리



ch_12_s01 · 페이지네이션(pagination) 패턴

두 방식 비교

항목	OFFSET 방식	keyset 방식
구문	LIMIT n OFFSET m	WHERE id > 마지막값 LIMIT n
뒤 페이지 속도	페이지가 깊을수록 느려짐	어느 페이지든 거의 일정
임의 페이지 이동	쉬움(5페이지로 바로 점프)	어려움(순차 이동에 적합)
적합한 곳	데이터 적음, 페이지 번호 UI	데이터 많음, 무한 스크롤

이렇게 작성합니다

먼저 가장 익숙한 **OFFSET** 방식입니다. 한 페이지에 5건씩 보여 준다고 할 때, 3페이지는 앞의 10건을 건너뛰고 5건을 가져옵니다.

```
-- OFFSET 방식: 3페이지(한 페이지 5건) 조회
SELECT id, name, price
FROM products
ORDER BY id
LIMIT 5 OFFSET 10;    -- (3 - 1) * 5 = 10건 건너뛰기
```

예상 화면:

id	name	price
11	텀블러	15000.00
12	우산	9000.00
13	가습기	45000.00
14	탁상시계	23000.00
15	무선 충전기	29000.00

(5 rows)

OFFSET 10 은 정렬된 결과에서 앞 10건을 버리고, LIMIT 5 로 그다음 5건만 가져옵니다. 페이지 번호로 바로 점프할 수 있어 편리하지만, OFFSET 값이 커지면 버려질 행까지 매번 세어야 해서 느려집니다.

이제 같은 화면을 keyset 방식으로 바꿔 봅시다. 직전 페이지의 마지막 id 가 10 이었다고 가정합니다.

```
-- keyset 방식: 직전 페이지 마지막 id(10) 다음부터 5건
SELECT id, name, price
FROM products
WHERE id > 10          -- 마지막으로 본 키 이후부터
ORDER BY id
LIMIT 5;
```

예상 화면:

id	name	price
11	텀블러	15000.00
12	우산	9000.00
13	가습기	45000.00
14	탁상시계	23000.00
15	무선 충전기	29000.00

(5 rows)

결과는 같지만 동작이 다릅니다. keyset 방식은 앞의 10건을 세지 않고, 인덱스를 타고 id > 10 지점으로 바로 건너뛸니다(id 가 기본 키라 인덱스가 이미 있습니다). 다음 페이지는 이번 결과의 마지막 id 인 15 를 다음 조건 WHERE id > 15 에 넣어 이어 갑니다.

keyset 구문 요소 설명

구문	의미
<code>WHERE id > 10</code>	직전 페이지 마지막 키 이후 행만 고릅니다
<code>ORDER BY id</code>	키 순서로 정렬해 페이지 경계를 안정화합니다
<code>LIMIT 5</code>	이번 페이지에서 가져올 행 수입니다
다음 페이지	이번 결과의 마지막 id 를 다음 WHERE 에 넣습니다

정렬 기준이 `created_at` 처럼 중복될 수 있는 값이면, 같은 시각의 행끼리 순서가 흔들립니다. 이때는 `ORDER BY created_at, id` 처럼 **유일한 컬럼(id)**을 **마지막 정렬 기준으로 덧붙여** 경계를 확실히 합니다.

팁: 페이지 번호 버튼(1, 2, 3, ...)이 꼭 필요한 관리 화면이라면 `OFFSET` 방식이 직관적입니다. 반대로 "더 보기"-무한 스크롤처럼 아래로만 이어 읽는 화면이라면 keyset 방식이 훨씬 빠르고 안정적입니다. UI 형태에 맞춰 고릅니다.

주의: `OFFSET` 방식은 중간에 행이 삽입·삭제되면 페이지 경계가 어긋날 수 있습니다. 예를 들어 1페이지와 2페이지를 보는 사이 새 행이 추가되면, 이미 본 행이 2페이지에 다시 나오거나 어떤 행은 건너뛰어집니다. keyset 방식은 "본 키 이후"를 기준으로 하므로 이런 흔들림에 강합니다.

절 요약

- 페이지네이션은 `LIMIT · OFFSET · ORDER BY` 로 긴 목록을 일정 개수씩 끊어 조회합니다.
- 정렬 기준 없는 페이지네이션은 순서가 흔들리므로 안정적인 `ORDER BY` 가 필수입니다.
- `OFFSET` 방식은 페이지 점프에 편하지만 뒤 페이지일수록 느립니다.
- keyset 방식은 "마지막으로 본 키 이후"부터 읽어 데이터가 많아도 속도가 일정합니다.

감사 컬럼과 소프트 삭제(soft delete)

도입

실제 서비스에서는 "이 데이터가 언제 만들어졌고 마지막으로 언제 바뀌었는지", 그리고 "지운 데이터를 정말 영영 버려도 되는지"가 중요해집니다. 회계 장부를 함부로 지우지 않고 빨간 줄

만 곳듯, 데이터도 이력을 남기고 조심스럽게 다룹니다. 이 절에서는 감사 컬럼과 소프트 삭제 두 가지 실무 패턴을 배웁니다.

핵심 개념 설명

감사 컬럼 (audit column)

감사 컬럼(audit column)이란, 행이 언제 만들어지고(`created_at`) 언제 마지막으로 바뀌었는지(`updated_at`)를 기록해 두는 시각 컬럼입니다. 문서마다 작성일·수정일을 적어 두는 것과 같습니다.

- **왜(why) 필요한가:** 문제가 생겼을 때 "언제부터 그랬는지"를 추적하고, 최근에 바뀐 데이터를 골라 보는 데 쓰입니다. 운영의 기본 정보입니다.
- **언제(when) 쓰나:** 거의 모든 실무 테이블에 둡니다. 회원·주문·상품처럼 변화가 의미 있는 데이터일수록 필수입니다.
- **어떻게(how) 쓰나:** `created_at`은 `DEFAULT now()`로 삽입 시 자동 기록하고, `updated_at`은 갱신할 때마다 `now()`로 다시 채웁니다.

소프트 삭제 (soft delete)

소프트 삭제(soft delete)란, 행을 실제로 지우지 않고 `deleted_at` 같은 표시 컬럼에 삭제 시각을 적어 '삭제됨'을 기록만 하는 방식입니다. 서류를 파쇄하지 않고 '폐기 예정' 도장을 찍어 보관함에 따로 두는 것과 같습니다.

`DELETE`로 진짜 지우면(이것을 하드 삭제라 부릅니다) 되돌릴 수 없고, 그 데이터를 참조하던 주문·통계도 함께 깨질 수 있습니다. 소프트 삭제는 데이터를 남겨 두므로 실수로 지워도 복구할 수 있고, 과거 이력도 보존됩니다.

여기서 두 가지 오해를 짚겠습니다. 첫째, **소프트 삭제하면 저장 공간이 줄어든다고 생각하는 경우**입니다. 행을 지우지 않고 그대로 두므로 공간은 오히려 그대로 차지합니다. 둘째, **조회 쿼리에서 삭제 표시를 거르지 않아도 된다고 오해하는 경우**입니다. 소프트 삭제된 행도 테이블에 남아 있으므로, 평소 조회에서는 `WHERE deleted_at IS NULL`로 반드시 걸러 줘야 사용자에게 삭제한 데이터가 다시 보이지 않습니다.

이렇게 작성합니다

먼저 감사 컬럼입니다. 삽입할 때 `created_at`·`updated_at`은 기본값 `now()`로 자동 채워집니다.

```
-- 삽입 시 created_at·updated_at은 자동으로 현재 시각
INSERT INTO products (name, category, price)
VALUES ('블루투스 스피커', '전자기기', 49000)
RETURNING id, created_at, updated_at;
```

예상 화면:

```
id |          created_at          |          updated_at
---+-----+-----
21 | 2026-06-16 14:20:05.331842 | 2026-06-16 14:20:05.331842
(1 row)
INSERT 0 1
```

이제 가격을 수정할 때 `updated_at` 도 함께 갱신합니다. 7장에서 배운 `UPDATE` 에 `updated_at = now()` 를 한 줄 더 적으면 됩니다.

```
-- 값을 바꿀 때 updated_at도 함께 갱신
UPDATE products
SET price = 45000,
    updated_at = now()
WHERE id = 21;
```

예상 화면:

```
UPDATE 1
```

팁: `updated_at` 을 매번 손으로 적는 것을 깜빡하기 쉽습니다. 11장에서 배운 **트리거(trigger)** 로 "행이 수정될 때 자동으로 `updated_at` 을 `now()` 로 채우게" 만들면 빠뜨릴 염려가 없습니다. 실무에서는 감사 컬럼 갱신을 트리거에 맡기는 경우가 많습니다.

다음은 소프트 삭제입니다. `DELETE` 대신 `UPDATE` 로 `deleted_at` 에 현재 시각을 기록합니다.

```
-- 하드 삭제 대신 소프트 삭제: deleted_at에 시각 기록
UPDATE products
SET deleted_at = now()
WHERE id = 21;
```

예상 화면:

```
UPDATE 1
```

행은 여전히 테이블에 남아 있습니다. 그래서 평소 목록 조회에서는 살아 있는 행만 보이도록 `deleted_at IS NULL` 조건을 꼭 넣습니다.

```
-- 살아 있는(삭제되지 않은) 상품만 조회
SELECT id, name, price
FROM products
WHERE deleted_at IS NULL
ORDER BY id;
```

예상 화면:

```
id | name      | price
---+-----+-----
 1 | 무선 마우스 | 19900.00
 2 | 기계식 키보드 | 89000.00
...
(20 rows)
```

`id = 21` 인 블루투스 스피커는 `deleted_at` 이 채워졌으므로 이 목록에 나오지 않습니다. 하지만 테이블에는 그대로 있어, 필요하면 `deleted_at = NULL` 로 되돌려 복구할 수 있습니다.

세 가지 시각 컬럼의 역할

컬럼	채워지는 시점	조회에서 쓰는 법
<code>created_at</code>	삽입할 때 자동(<code>DEFAULT now()</code>)	가입일·등록일 표시, 기간 필터
<code>updated_at</code>	수정할 때마다 <code>now()</code> 로 갱신	최근 변경 데이터 추적
<code>deleted_at</code>	소프트 삭제할 때 <code>now()</code> 기록	<code>IS NULL</code> 로 살아 있는 행만 조회

주의: 소프트 삭제를 도입하면 **모든 조회에 `deleted_at IS NULL` 을 빠짐없이** 넣어야 합니다. 한 군데라도 빠지면 사용자에게 삭제한 데이터가 다시 보입니다. 매번 적기 번거롭다면 11장의 **뷰 (view)** 로 "살아 있는 행만 보이는 `active_products` 뷰"를 만들어 두고, 평소에는 그 뷰를 조회하는 방법이 안전합니다.

절 요약

- 감사 컬럼 `created_at` · `updated_at` 으로 행의 생성·수정 이력을 남깁니다.
- `updated_at` 갱신은 트리거에 맡기면 빠뜨릴 염려가 없습니다.
- 소프트 삭제는 `DELETE` 대신 `deleted_at` 에 시각을 기록해 데이터를 숨기되 보존합니다.

- 소프트 삭제 도입 시 모든 조회에 `deleted_at IS NULL` 을 넣거나 뷰로 감쌉니다.

안전한 운영 - 백업·권한·실수 방지

도입

데이터를 잘 다루는 것만큼 중요한 것이 **사고를 막는 일**입니다. 실수로 테이블을 통째로 지웠는데 백업이 없다면 끝입니다. 또 모든 사람이 무엇이든 지울 수 있는 권한을 가지면, 한 번의 실수가 큰 사고로 번집니다. 이 절에서는 백업과 권한 분리, 그리고 일상적인 실수 예방 습관을 배웁니다.

핵심 개념 설명

백업 (backup)

백업(backup)이란, 데이터베이스의 현재 상태를 따로 파일로 복사해 두어, 사고가 나도 그 시점으로 되돌릴 수 있게 하는 것입니다. 중요한 문서를 USB에 한 부 더 복사해 두는 것과 같습니다.

- **왜(why) 필요한가:** 잘못된 `DELETE` · `DROP`, 디스크 고장, 잘못된 배포 등 어떤 사고든 백업이 있으면 복구할 수 있습니다. 백업 없는 운영은 안전망 없는 외줄타기입니다.
- **언제(when) 하나:** 실서비스는 매일 정기적으로, 실습 환경이라면 큰 변경(대량 삭제·구조 변경) 직전에 받아 둡니다.
- **어떻게(how) 하나:** PostgreSQL이 제공하는 `pg_dump` 도구로 데이터베이스를 파일로 내보냅니다.

최소 권한 (least privilege)

최소 권한(least privilege)이란, 각 사용자(역할, role)에게 **꼭 필요한 만큼만** 권한을 주는 원칙입니다. 가게에서 아르바이트생에게 금고 열쇠까지 주지 않고, 계산대 권한만 주는 것과 같습니다.

조회만 하면 되는 분석용 계정에 삭제 권한까지 주면, 실수든 사고든 위험이 커집니다. 읽기만 필요한 역할에는 `SELECT` 만, 데이터를 다루는 역할에는 `SELECT` · `INSERT` · `UPDATE` · `DELETE` 를 주는 식으로 나눕니다. 관리자(슈퍼유저) 계정은 평소 작업에 쓰지 않습니다.

이렇게 설정/실행합니다

먼저 백업입니다. `pg_dump` 는 SQL 안에서 실행하는 명령이 아니라 PowerShell에서 직접 실행하는 도구이므로, `psql` 밖(파워셸 프롬프트)에서 입력합니다.

```
# PowerShell에서 실행 (psql 안이 아님)
# shop_lab 데이터베이스를 파일 하나로 백업하기
pg_dump -U postgres -d shop_lab -f C:\Users\내이름\backup\shop_lab_20260616.sql
```

예상 화면:

```
Password for user postgres:
PS C:\Users\내이름>
```

비밀번호를 입력하면 화면에는 아무 메시지 없이 프롬프트로 돌아오고, 지정한 경로에 `.sql` 파일이 생깁니다. 이 파일 안에는 테이블을 다시 만들고 데이터를 채우는 SQL이 모두 담겨 있어, 나중에 그대로 복원할 수 있습니다.

pg_dump 옵션 설명

옵션	의미
<code>-U postgres</code>	접속할 사용자(여기서는 슈퍼유저 postgres)
<code>-d shop_lab</code>	백업할 데이터베이스 이름
<code>-f 경로\파일.sql</code>	백업 내용을 저장할 파일 경로
(파일명에 날짜)	<code>..._20260616.sql</code> 처럼 날짜를 넣어 언제 받은 백업인지 구분

복원이 필요하다면, 비어 있는 데이터베이스에 이 파일을 `psql` 로 흘려 넣습니다.

```
# 백업 파일로 복원하기 (먼저 빈 데이터베이스를 만들어 둔 상태)
psql -U postgres -d shop_lab_restored -f C:\Users\내이름\backup\shop_lab_20260616.sql
```

다음은 권한 분리입니다. 조회만 가능한 읽기 전용 역할을 만들어 봅니다. 이 SQL은 `psql` 안에서 실행합니다.

```
-- 읽기 전용 역할(role) 만들기
CREATE ROLE analyst LOGIN PASSWORD 'a-strong-password';

-- shop_lab의 모든 테이블에 SELECT만 허용
GRANT CONNECT ON DATABASE shop_lab TO analyst;
GRANT USAGE ON SCHEMA public TO analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst;
```

예상 화면:

```
CREATE ROLE
GRANT
GRANT
GRANT
```

이제 `analyst` 계정은 데이터를 읽을 수는 있지만 `INSERT` · `UPDATE` · `DELETE` 는 권한이 없어 거부됩니다. 분석가가 실수로 데이터를 바꿀 위험이 사라집니다.

권한 부여 구문 설명

구문	의미
<code>CREATE ROLE analyst LOGIN</code>	접속 가능한 사용자(역할)를 만듭니다
<code>GRANT CONNECT ON DATABASE</code>	해당 데이터베이스에 접속을 허용합니다
<code>GRANT USAGE ON SCHEMA public</code>	public 스키마 사용을 허용합니다
<code>GRANT SELECT ON ALL TABLES</code>	모든 테이블에 조회 권한만 부여합니다

위험: 대량 삭제·구조 변경(`DELETE` · `TRUNCATE` · `DROP`) 전에는 반드시 백업을 먼저 받습니다. 특히 `WHERE` 없는 `DELETE` 나 `UPDATE` 는 테이블 전체를 바꿉니다. 실행 전 같은 조건으로 `SELECT count(*)` 를 돌려 **영향받을 행 수를 눈으로 확인**하고, 가능하면 8장·10장에서 배운 트랜잭션 (`BEGIN` ... 확인 ... `COMMIT`) 안에서 수행합니다.

베스트 프랙티스: 평소 작업은 슈퍼유저(postgres)가 아니라 권한이 제한된 일반 역할로 합니다. 운영·개발·분석용 계정을 용도별로 나누고, 비밀번호는 코드에 직접 적지 않습니다. 정기 백업을 자동화하고, 백업 파일이 실제로 복원되는지 가끔 점검합니다. "지울 수 있는 사람"을 줄이는 것이 사고를 줄이는 가장 확실한 방법입니다.

절 요약

- `pg_dump` 로 데이터베이스를 파일로 백업하고, 파일명에 날짜를 넣어 관리합니다.
- 복원은 빈 데이터베이스에 백업 `.sql` 파일을 `psql` 로 흘려 넣습니다.
- 최소 권한 원칙에 따라 역할별로 꼭 필요한 권한만 `GRANT` 합니다.
- 대량 변경 전 백업·건수 확인·트랜잭션으로 사고를 예방합니다.

마무리 - CRUD 전체 흐름 복습과 다음 단계

도입

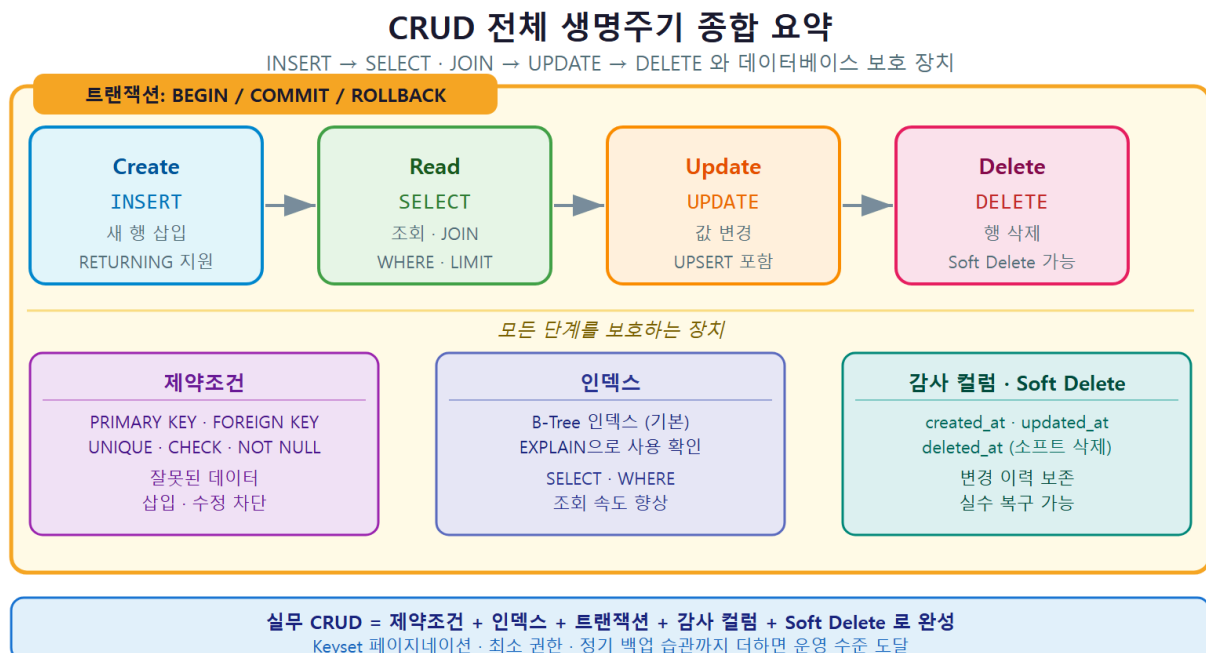
긴 여정의 마지막 절입니다. 지금까지 배운 것을 한 데이터의 한살이로 꿰어 보고, 이 책 다음에 무엇을 배우면 좋을지 방향을 잡습니다. 흩어진 지식이 하나의 그림으로 연결되는 순간입니다.

핵심 개념 설명

CRUD 종합 (CRUD recap)

CRUD 종합(CRUD recap) 이란, 이 책에서 배운 생성·조회·수정·삭제와 그것을 둘러싼 보호 장치들이 어떻게 한 흐름으로 맞물리는지 전체를 되짚는 것입니다.

상품 한 건의 한살이를 따라가 보겠습니다. 이 한 건이 이 책 전체를 관통합니다.



1. **생성(Create)**: INSERT 로 새 상품을 등록합니다(3장). RETURNING 으로 생성된 id 를 바로 받습니다.
2. **조회(Read)**: SELECT 로 읽고(4장), GROUP BY 로 집계하며(5장), JOIN 으로 주문·회원과 묶어 봅니다(6장). 목록은 페이지네이션으로 끊어 보여 줍니다(이 장).
3. **수정(Update)**: UPDATE 로 가격·재고를 바꾸고(7장), updated_at 을 함께 갱신합니다(이 장).
4. **삭제>Delete)**: DELETE 로 지우거나(8장), 소프트 삭제로 deleted_at 에 표시만 합니다(이 장).

그리고 이 네 동작은 다음 장치들의 보호를 받습니다.

- **제약조건·인덱스(9장)**: 잘못된 데이터를 막고, 조회를 빠르게 합니다.
- **트랜잭션(10장)**: 여러 작업을 하나로 묶어 "전부 성공 아니면 전부 취소"를 보장합니다.
- **뷰·함수·트리거(11장)**: 복잡한 조회를 재사용하고, 감사 컬럼 갱신 같은 일을 자동화합니다.

CRUD와 이 책의 지도

단계	SQL	배운 곳	실무에서 함께 쓰는 것
Create	INSERT	3장	RETURNING, 감사 컬럼
Read	SELECT · JOIN	4~6장	페이지네이션, 뷰, 인덱스
Update	UPDATE	7장	updated_at 갱신, UPSERT
Delete	DELETE	8장	소프트 삭제, 트랜잭션
보호	제약·트랜잭션	9~10장	백업, 권한 분리

다음 학습 (next steps)

다음 학습(next steps) 이란, 이 입문서를 마친 뒤 한 단계 더 나아가기 위한 방향을 말합니다. CRUD는 데이터베이스의 기본기이고, 그 위에 쌓을 것이 많습니다.

- **쿼리 성능 튜닝**: EXPLAIN ANALYZE 로 실행 계획을 읽고, 인덱스를 더 깊이 설계합니다.
- **고급 SQL**: 윈도우 함수(window function), 공통 테이블 식(CTE, WITH), 재귀 쿼리로 복잡한 분석을 다룹니다.
- **애플리케이션 연동**: 백엔드 프레임워크에서 ORM(객체-관계 매핑)으로 CRUD를 코드로 다룹니다.
- **운영 심화**: 복제(replication), 모니터링, 자동 백업·복구 전략을 익힙니다.

가장 확실한 다음 걸음은 **PostgreSQL 공식 문서**입니다. 한국어 자료보다 양이 많지만, 가장 정확하고 최신입니다. 이 책에서 배운 키워드(`SELECT` , `INSERT` , `pg_dump` 등)로 검색하면 깊이 있는 설명을 바로 찾을 수 있습니다.

팁: 가장 좋은 복습은 직접 만들어 보는 것입니다. 관심 있는 주제(가계부, 독서 기록, 일정 관리 등)로 작은 테이블 두세 개를 설계하고, 이 책에서 배운 CRUD를 처음부터 끝까지 적용해 보면 흠어진 지식이 단단하게 자리잡습니다.

절 요약

- 한 데이터는 `INSERT` 로 생성 → `SELECT` · `JOIN` 으로 조회 → `UPDATE` 로 수정 → `DELETE` 로 삭제되는 한살이를 거칩니다.
- 이 흐름은 제약조건·인덱스·트랜잭션·뷰의 보호를 받습니다.
- 다음 단계로 성능 튜닝, 고급 SQL, 애플리케이션 연동, 운영 심화가 있습니다.
- 공식 문서와 작은 실습 프로젝트가 가장 좋은 다음 걸음입니다.

실습 과제

해보기: 실무형 상품 목록 API 쿼리 세트 완성하기

`shop_lab` 의 `products` 테이블에 감사 컬럼(`created_at` · `updated_at` · `deleted_at`)이 있다고 보고, 실무에서 쓸 법한 쿼리 세트를 완성합니다.

- 단계 1: keyset 페이지네이션으로 상품 목록을 5건씩 조회합니다. 1페이지를 본 뒤 마지막 `id` 를 다음 조건 `WHERE id > 마지막값` 에 넣어 2페이지를 가져옵니다. `deleted_at IS NULL` 조건도 함께 넣습니다.
- 단계 2: 상품 한 건의 가격을 `UPDATE` 로 바꾸면서 `updated_at = now()` 를 함께 갱신합니다. 바뀐 행을 `RETURNING` 으로 확인합니다.
- 단계 3: 상품 한 건을 하드 삭제 대신 소프트 삭제(`UPDATE ... SET deleted_at = now()`)로 숨긴 뒤, 단계 1의 목록 쿼리에서 그 상품이 사라졌는지 확인합니다.
- 단계 4: PowerShell에서 `pg_dump` 로 `shop_lab` 전체를 날짜가 들어간 `.sql` 파일로 백업합니다.
- 점검: 소프트 삭제한 상품이 `WHERE deleted_at IS NULL` 조건에서는 보이지 않지만, 조건을 떼면 여전히 테이블에 남아 있는지 `SELECT` 로 확인합니다. 백업 파일이 지정한 경로에 생겼는지 확인합니다.

FAQ

Q1. keyset 페이지네이션은 항상 OFFSET보다 좋은가요? → A1. 아닙니다. 데이터가 적거나, "5 페이지로 바로 점프" 같은 임의 페이지 이동이 필요한 화면에서는 OFFSET 방식이 더 단순하고 직관적입니다. keyset 방식은 데이터가 많고 아래로만 이어 읽는(무한 스크롤) 화면에서 진가를 발휘합니다. 화면의 동작 방식에 맞춰 고르는 것이 정답입니다.

Q2. 소프트 삭제와 하드 삭제(DELETE) 중 무엇을 써야 하나요? → A2. 복구 가능성과 이력 보존이 중요하면 소프트 삭제, 정말 흔적도 없이 지워야 하면 하드 삭제를 씁니다. 다만 소프트 삭제는 데이터가 계속 쌓여 공간을 차지하고 모든 조회에 `deleted_at IS NULL` 을 붙여야 하는 부담이 있으므로, 개인정보처럼 법적으로 완전 삭제가 필요한 데이터는 하드 삭제(또는 일정 기간 후 영구 삭제)를 함께 고려합니다.

Q3. `pg_dump` 로 받은 백업은 어떻게 복원하나요? → A3. 비어 있는 데이터베이스를 먼저 만든 뒤 (`CREATE DATABASE`), `psql -U postgres -d 새DB -f 백업파일.sql` 로 백업 파일을 흘려 넣으면 테이블과 데이터가 그대로 되살아납니다. 백업은 받아 두는 것만으로 끝이 아니라, 실제로 복원이 되는지 가끔 시험해 보는 것까지가 안전한 운영입니다.

Q4. `updated_at` 을 매번 직접 적기 번거로운데 자동으로 할 수 없나요? → A4. 11장에서 배운 트리거(trigger)를 쓰면 됩니다. "행이 UPDATE 될 때마다 `updated_at` 을 `now()` 로 채우는" 트리거 함수를 테이블에 연결해 두면, UPDATE 문에서 `updated_at` 을 빠뜨려도 자동으로 갱신됩니다. 실무에서 매우 흔히 쓰는 패턴입니다.

장 요약

이 마지막 장에서는 CRUD를 실제 서비스에서 쓰는 방식으로 묶었습니다. 긴 목록을 끊어 보여주는 페이지네이션에서 OFFSET 방식과 keyset 방식의 차이를 배웠고, `created_at` · `updated_at` 감사 컬럼과 `deleted_at` 을 이용한 소프트 삭제로 이력을 남기고 안전하게 데이터를 숨기는 법을 익혔습니다. `pg_dump` 백업과 최소 권한 원칙으로 사고를 예방하는 운영 습관도 다루었습니다. 마지막으로 `INSERT` → `SELECT` → `UPDATE` → `DELETE` 로 이어지는 한 데이터의 한살이가 제약조건·트랜잭션·뷰의 보호를 받는 전체 그림을 되짚으며, 책을 마무리했습니다. 이제 여러분은 데이터를 만들고 읽고 고치고 지우며, 그 과정을 안전하게 지킬 수 있는 기본기를 갖추었습니다.

핵심 키워드

페이지네이션(pagination), 키셋 페이지네이션(keyset pagination), OFFSET, 감사 컬럼(audit column), 소프트 삭제(soft delete), deleted_at, pg_dump, 최소 권한(least privilege), GRANT, CRUD 종합(CRUD recap)

다음 장 미리보기

이 책의 마지막 장입니다. 다음 걸음은 책 밖에 있습니다. 직접 작은 프로젝트의 테이블을 설계해 이 책에서 배운 CRUD를 처음부터 끝까지 적용해 보고, PostgreSQL 공식 문서에서 원도 함수·성능 튜닝 같은 다음 주제로 나아가 보기 바랍니다. 데이터를 다루는 즐거운 여정이 이제 막 시작되었습니다.

부록 - 대용량 테스트 데이터 만들기

학습 목표 - 소량 데이터로는 드러나지 않는 성능 문제를 설명하고, 대용량 테스트 데이터의 필요성을 압니다. - 대량 생성 함수(generate_series)로 회원·상품·주문 데이터를 재현 가능하게 만들 수 있습니다. - 데이터 규모·날짜·값 분포를 조정하고 복합 인덱스 실습용 데이터를 만들 수 있습니다. - seed 스크립트를 psql 로 실행·검증하고 흔한 오류에 대응할 수 있습니다.

9장에서 우리는 인덱스를 만들었는데도 EXPLAIN 에 Seq Scan 이 그대로 나오는 경우를 봤습니다. 행이 몇십 개뿐이면 PostgreSQL이 "통째로 읽는 편이 더 빠르다"고 판단하기 때문입니다. 즉 **인덱스의 효과는 데이터가 충분히 많아야 보입니다**. 이 부록은 그 "충분히 많은" 데이터를 손으로 한 줄씩 적지 않고 한 번에 만드는 방법을 다룹니다. 교재와 함께 제공되는 `data/seed_large.sql` 스크립트가 그 결과물입니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 에서 psql 로 PostgreSQL에 접속한 상태를 기준으로 합니다.

왜 대용량 테스트 데이터가 필요한가

도입

지금까지 우리가 쓴 실습 데이터는 회원 4명·상품 8개·주문 8건, 합쳐서 스무 행 남짓이었습니다. 개념을 손으로 검증하기에는 좋지만, 이 규모로는 성능을 이야기할 수 없습니다. 이 절에서는 왜 작은 데이터가 성능 판단을 가리는지 짚습니다.

핵심 개념 설명

소량 데이터로는 보이지 않는 것들

테스트 데이터(test data) 란, 실제 운영 데이터를 흉내 내어 기능과 성능을 검증하려고 만든 가짜 데이터입니다. 데이터가 적으면 다음 문제들이 **숨습니다**.

- 인덱스가 없어 느린 조회: 20행은 Seq Scan 도 즉시 끝나므로 느린 줄 모릅니다.
- 잘못 잡힌 인덱스: 효과가 0에 가까워 있으나 마나 똑같아 보입니다.

- 정렬·집계의 비용: `ORDER BY` 나 `GROUP BY` 가 수백만 행에서야 부담이 됩니다.

20행에서는 모든 질의가 1밀리초 안에 끝나기 때문에, "**빠르다**"는 착각에 빠지기 쉽습니다.

실무 투입 전 성능 점검

성능 검증(performance validation) 은 운영에 가까운 규모로 미리 질의 속도를 재 보는 일입니다. 운영 데이터베이스의 주문 테이블은 흔히 수십만~수백만 행입니다. 개발 단계에서 그 규모를 재현해 두면, 느린 질의·인덱스 누락을 **사용자가 겪기 전에** 발견할 수 있습니다.

베스트 프랙티스: 새 인덱스나 쿼리를 운영에 올리기 전에, 운영과 비슷한 규모의 테스트 데이터에서 `EXPLAIN ANALYZE` 로 한 번 측정하십시오. "작은 DB에서 빨랐으니 괜찮겠지"가 가장 흔한 성능 사고의 원인입니다.

절 요약

- 스무 행 규모로는 인덱스 효과·느린 질의가 모두 숨어 성능을 판단할 수 없습니다.
- 운영에 가까운 대용량 테스트 데이터로 미리 측정해야 문제를 앞당겨 발견합니다.

generate_series로 대량 생성하기

도입

100만 건을 손으로 `INSERT` 할 수는 없습니다. PostgreSQL의 `generate_series` 함수가 번호를 한 행씩 찍어 주면, 그 번호를 값으로 바꿔 한 번에 삽입할 수 있습니다. 이 절에서 `data/seed_large.sql` 의 핵심을 뜯어봅니다.

핵심 개념 설명

generate_series 기본 패턴

대량 생성 함수(generate_series) 는 시작값부터 끝값까지 정수를 한 행씩 만들어 돌려주는 함수입니다. `generate_series(1, 5)` 는 1·2·3·4·5 다섯 행을 만듭니다. 이 행들을 `SELECT` 로 가공해 `INSERT INTO ... SELECT` 로 넣는 것이 기본 골격입니다.

```
-- 1부터 5까지 행을 만들어, 번호를 이름으로 바꿔 넣는 최소 예시
INSERT INTO members (name, email, age)
SELECT '회원' || g,
       'member' || g || '@example.com',
       20 + (g % 50)
FROM generate_series(1, 5) AS g;
```

`g` 는 `generate_series`가 돌려주는 번호입니다. '회원' || `g` 로 이름을, 나머지 연산 `g % 50` 으로 나이를 만듭니다. `generate_series(1, 5)` 의 `5` 를 `50000` 으로 바꾸면 그대로 회원 5만 명이 됩니다.

seed_large.sql의 세 테이블 생성

`data/seed_large.sql` 은 같은 패턴으로 회원 5만·상품 1천·주문 100만을 만듭니다. 주문 생성 부분이 핵심입니다.

```
-- 주문 100만 건: member_id 1~5만, product_id 1~1천에 고르게 분포
INSERT INTO orders (member_id, product_id, quantity)
SELECT 1 + (g % 50000), -- 회원 id를 1~5만에 순환 분포
       1 + (g % 1000),  -- 상품 id를 1~1천에 순환 분포
       1 + (g % 5)      -- 수량 1~5 순환
FROM generate_series(1, 1000000) AS g;
```

코드 설명

요소	의미
<code>generate_series(1, 1000000)</code>	100만 개의 번호를 만들어 그 수만큼 주문 행을 생성합니다
<code>1 + (g % 50000)</code>	번호를 5만으로 나눈 나머지에 1을 더해, member_id를 1~50000에 고르게 분포시킵니다
<code>1 + (g % 5)</code>	수량을 1~5 사이로 순환시켜 단조롭지 않게 만듭니다

재현 가능성과 ANALYZE

위 코드는 난수(`random()`)를 쓰지 않고 번호 `g` 로만 값을 만듭니다. 그래서 **결정론적 생성 (deterministic generation)** 입니다. 누가 언제 실행해도 똑같은 데이터가 나오므로, "내 결과가 책과 다른데요?" 같은 혼선이 없습니다.

데이터를 다 넣은 뒤에는 반드시 통계를 갱신합니다.

```
ANALYZE members;
ANALYZE products;
ANALYZE orders;
```

ANALYZE 는 각 테이블의 행 수·값 분포 통계를 다시 수집합니다. PostgreSQL의 계획 수립기 (planner)는 이 통계를 보고 "인덱스를 탈지, 통째로 읽을지"를 결정합니다. 대량 삽입 직후 통계가 낡아 있으면 잘못된 계획을 세울 수 있으므로, 측정 전에 ANALYZE 로 통계를 최신화하는 것이 중요합니다.

절 요약

- generate_series(1, N) 로 N개의 번호를 만들고, INSERT ... SELECT 로 값을 가공해 넣습니다.
- 난수 대신 번호로만 값을 만들면 결과가 재현 가능합니다.
- 대량 삽입 후에는 ANALYZE 로 통계를 갱신해야 계획 수립기가 올바른 판단을 합니다.

규모 조정과 응용 - 날짜·분포·복합 인덱스

도입

seed_large.sql 은 출발점일 뿐입니다. 실습 목적에 따라 규모를 키우거나, 날짜·분포를 더 현실적으로 바꿀 수 있습니다. 이 절은 자주 쓰는 변형을 모았습니다.

핵심 개념 설명

100만에서 500만으로 늘리기

규모 조정(scaling) 은 generate_series의 끝값과 분포 계수만 바꾸면 됩니다. 주문을 500만 건으로 늘리려면 끝값을 바꾸고, 회원·상품도 비례해 늘립니다.

```
-- 주문 500만 건으로 확대 (회원 20만·상품 5천에 맞춰 분포 계수도 조정)
INSERT INTO orders (member_id, product_id, quantity)
SELECT 1 + (g % 200000),
       1 + (g % 5000),
       1 + (g % 5)
FROM generate_series(1, 5000000) AS g;
```

분포 계수(% 200000)를 회원 수와 맞춰야 존재하지 않는 member_id를 가리키지 않습니다.

날짜·난수로 현실적인 분포 만들기

기본 스크립트에는 주문 날짜가 없습니다. 기간별 조회나 정렬 인덱스를 실습하려면 **날짜 생성 (date generation)** 컬럼을 더합니다. 먼저 orders에 컬럼을 추가하고, 최근 1년에 흩뿌립니다.

```
ALTER TABLE orders ADD COLUMN ordered_at date;

-- 0~364일 전 사이로 주문일을 분산
UPDATE orders
SET ordered_at = CURRENT_DATE - (id % 365);
```

더 자연스러운 분포를 원하면 **값 분포(value distribution)**에 난수를 씁니다. 단, 난수를 쓰면 재현성이 사라지므로, `setseed`로 시드를 고정하면 재현성과 무작위성을 함께 얻습니다.

```
SELECT setseed(0.42); -- 시드를 고정하면 같은 난수 수열이 반복됨

INSERT INTO orders (member_id, product_id, quantity)
SELECT 1 + floor(random() * 50000)::int,
       1 + floor(random() * 1000)::int,
       1 + floor(random() * 5)::int
FROM generate_series(1, 1000000) AS g;
```

복합 인덱스 실습용 데이터

(member_id, ordered_at) 같은 **복합 인덱스(composite index)**를 실습하려면 두 컬럼 모두 값이 다양해야 합니다. 위에서 member_id를 5만 가지로, ordered_at을 365가지로 분포시켰으므로, "특정 회원의 최근 주문"을 빠르게 찾는 복합 인덱스를 실습할 수 있습니다.

```
CREATE INDEX idx_orders_member_date ON orders (member_id, ordered_at);

EXPLAIN ANALYZE
SELECT * FROM orders
WHERE member_id = 12345 AND ordered_at >= CURRENT_DATE - 30
ORDER BY ordered_at DESC;
```

절 요약

- generate_series의 끝값과 분포 계수를 바꿔 규모를 조정합니다(분포 계수는 부모 테이블 행 수에 맞춤).
- 날짜 컬럼·난수로 현실적인 분포를 만들되, 재현성이 필요하다면 `setseed`로 시드를 고정합니다.
- 두 컬럼을 다양하게 분포시키면 복합 인덱스 실습이 가능합니다.

실행·검증과 트러블슈팅

도입

스크립트를 만들었으면 실행하고, 의도대로 들어갔는지 확인하고, 막힐 때 대응하는 법까지 알아야 합니다. 이 절에서 마무리합니다.

핵심 개념 설명

psql로 실행하고 시간 재기

스크립트 실행(script execution)은 `psql`의 `-f` 옵션으로 파일을 통째로 실행하는 것입니다. PowerShell에서 `shop_lab` 데이터베이스를 대상으로 실행합니다.

```
psql -d shop_lab -f data/seed_large.sql
```

스크립트 첫머리의 `\timing on` 덕분에 각 명령이 걸린 시간이 함께 출력됩니다. 어느 단계가 오래 걸리는지 한눈에 보입니다.

행 수·통계 확인

생성이 끝나면 행 수로 검증합니다. 스크립트 끝에 들어 있는 확인 질의를 직접 실행해도 됩니다.

```
SELECT count(*) FROM orders;
```

```
count
-----
1000000
(1 row)
```

느림·메모리·IDENTITY 충돌 대응

주의 - 흔한 문제와 대응

- **삽입이 너무 느리다:** 인덱스를 *먼저* 만들고 데이터를 넣으면 매 행마다 인덱스를 갱신해 느려집니다. 데이터를 *먼저* 넣고 인덱스를 *나중에* 만드십시오(`seed_large.sql`도 인덱스 없이 데이터만 만듭니다).

- **IDENTITY 컬럼에 값을 못 넣는다:** `GENERATED ALWAYS AS IDENTITY` 컬럼(id)에는 값을 직접 줄 수 없습니다. `INSERT` 컬럼 목록에서 id를 빼면 자동으로 채워집니다.
- **다시 채우고 싶다:** 테이블을 비우고 id를 1부터 다시 시작하려면 `TRUNCATE orders RESTART IDENTITY;` 를 씁니다. `seed_large.sql`은 아예 `DROP TABLE` 후 재생성하므로 그대로 다시 실행하면 됩니다.
- **디스크가 부담된다:** 500만 행 이상은 디스크와 시간을 많이 씁니다. 노트북 환경이라면 100만 행으로 시작하는 것을 권합니다.

절 요약

- `psql -d <db> -f <파일>` 로 스크립트를 실행하고, `\timing` 으로 단계별 시간을 봅니다.
- `count(*)` 로 의도한 행 수가 들어갔는지 검증합니다.
- 인덱스는 데이터 적재 뒤에 만들고, IDENTITY 충돌은 컬럼 목록 조정·`TRUNCATE ... RESTART IDENTITY` 로 해결합니다.

실습 과제

해보기 - 내 실습 DB에 100만 행 주문 데이터 채우기

- 단계 1: `psql -d shop_lab -f data/seed_large.sql` 로 스크립트를 실행하고, 출력된 시간 (`\timing`)을 기록합니다.
- 단계 2: `SELECT count(*) FROM orders;` 로 100만 행이 들어갔는지 확인합니다.
- 단계 3: 9장의 `EXPLAIN ANALYZE SELECT * FROM orders WHERE member_id = 12345;` 를 인덱스 생성 전후로 실행해 `Seq Scan` → `Index Scan` 전환과 실행 시간을 비교합니다.
- 도전: 스크립트의 주문 끝값을 500만으로 바꾸거나 `ordered_at` 날짜 컬럼을 추가하도록 수정한 뒤 다시 실행해 봅니다.

FAQ

Q1. `generate_series`로 만든 데이터는 실제 운영 데이터처럼 써도 되나요? → A1. 아닙니다. 어디까지나 기능·성능 검증용 가짜 데이터입니다. 값의 분포가 규칙적이라 실제 사용자 행동을 반영하지 못하므로, 통계·분석이 아니라 "규모에 따른 성능"을 보는 용도로만 씁니다.

Q2. 난수(`random`)를 쓰면 매번 결과가 달라지나요? → A2. 그렇습니다. 그래서 재현성이 중요하면 `setseed(0.42)` 처럼 시드를 고정하십시오. 같은 시드에서는 같은 난수 수열이 나와, 무작위처럼 보이면서도 재현 가능한 데이터가 됩니다.

Q3. 100만 행을 넣었더니 인덱스를 만들 때도 오래 걸립니다. 정상인가요? → A3. 정상입니다. 100만 행에 인덱스를 만들면 전체를 정렬해 B-트리를 구성하므로 수초가 걸릴 수 있습니다. 그래서 "데이터 먼저, 인덱스 나중" 순서가 전체적으로 빠릅니다.

Q4. 실행 중 디스크가 가득 차면 어떻게 하나요? → A4. 행 수를 줄여 다시 실행하십시오. 스크립트는 `DROP TABLE` 로 시작하므로, 끝값을 100만에서 10만으로 낮춰 재실행하면 이전 데이터가 정리되고 작은 데이터로 다시 채워집니다.

장 요약

대용량 테스트 데이터는 소량 데이터로는 숨는 성능 문제(느린 조화·무효한 인덱스·정렬 비용)를 드러내기 위한 도구입니다. 손으로 만들 수 없는 규모는 `generate_series` 로 번호를 찍어 `INSERT ... SELECT` 로 가공해 넣으며, 난수 대신 번호로만 값을 만들면 결과가 재현 가능합니다. 대량 삽입 뒤에는 `ANALYZE` 로 통계를 갱신해야 계획 수립기가 올바른 판단을 합니다. 규모는 `generate_series`의 끝값과 분포 계수로 조정하고, 날짜·복합 인덱스 실습을 위해 컬럼과 분포를 더할 수 있습니다. 실행은 `psql -f` 한 줄이며, 인덱스는 데이터 적재 뒤에 만드는 것이 빠릅니다. 교재의 `data/seed_large.sql` 이 이 모든 패턴을 담은 출발점입니다.

핵심 키워드

테스트 데이터(test data), 성능 검증(performance validation), 대량 생성 함수(`generate_series`), 결정론적 생성(deterministic generation), `ANALYZE`, 규모 조정(scaling), 날짜 생성(date generation), 값 분포(value distribution), 복합 인덱스(composite index), `setseed`, `TRUNCATE RESTART IDENTITY`